

OMB Portfolio Builder Curated File Guides

Book-style explanations for selected important files. This first curated pass covers `server.mjs`, `script.js`, and `index.html`.

How This Curated Set Is Different

The generated source references are useful as raw maps, but they are not how a beginner learns a system. This curated set starts each file with a long overview, then explains code by responsibility. It avoids line-number references because source lines will move as the application evolves.

The first covered files are `server.mjs`, `script.js`, and `index.html` because they form the local backend, public runtime, and public shell of the portfolio website.

Curated File Guide: server.mjs

The local backend that saves drafts, runs compilers, checks GitHub publishing access, imports/pushes site files, handles updates, and relays AI.

Before The Code: File Overview

Abstract

server.mjs is the local nervous system of the OMB Portfolio Builder. It is not the public website and it is not the Electron shell. It is the private backend that runs on the owner's machine and gives the builder the powers a browser page should not have by itself: reading and writing local files, saving drafts, accepting uploads, running compilers, using Git, checking publishing authorization, importing a target site, launching installers, and relaying AI requests. The file is intentionally broad because the builder is local-first. Instead of depending on a hosted database and a remote application server, the desktop app starts this Node server beside the user-owned workspace and talks to it through local API endpoints.

The most important idea is boundary control. The frontend can draw buttons and collect input, but it cannot safely decide where files are written, which Git repository receives a push, whether a command should run, or whether a request came from the local computer. server.mjs owns those decisions. It is the layer that turns a UI action into a guarded filesystem, compiler, Git, installer, or AI operation.

Introduction

When the builder starts, Electron loads a local page. That page needs catalog data, templates, upload handling, compile actions, publishing actions, and update checks. A static HTML page cannot perform those operations directly without either exposing dangerous privileges or running into browser security walls. server.mjs solves that by becoming a small local application server. It serves the builder files, exposes API endpoints under /api, and keeps privileged work in Node where the filesystem, process runner, and Git commands are available.

A useful mental model is to treat this file like a workshop manager. The frontend is the workbench surface where the user sees projects, editors, buttons, and dialogs. server.mjs is the person behind the counter who knows where the tools are, where files belong, whether a customer has permission to ship something, and whether the requested operation is safe. If the frontend says 'save this PDF under this project,' server.mjs sanitizes the path and writes it under the correct docs folder. If the frontend says 'compile this C++ program,' server.mjs finds g++, creates a temporary run folder, captures terminal output, and returns a result object. If the frontend says 'apply to site,' server.mjs checks Git authorization first, then synchronizes publishable files and pushes them.

Background Information

The file uses built-in Node modules rather than a large web framework. createServer from node:http accepts requests, node:fs/promises handles files, node:child_process runs compilers and Git, node:path normalizes paths, node:crypto hashes authorization cache scopes, and node:os identifies the current machine. This keeps the backend predictable and package-light. The tradeoff is that routing and request handling are written manually. That is why handleApi is large: it plays the role that Express or another framework would normally play.

There are three important roots. root is the builder workspace that contains the site files and local draft catalog. portfolioRoot is the publish mirror, which may be separate from root so generated website files can be pushed without mixing in builder-only files. compileRoot is where project-local code workspaces and temporary compiler run folders live. Most path-related functions exist to make sure a request stays inside one of these roots.

The backend is also intentionally local-sensitive. Some endpoints are read-only, but write endpoints reject non-local callers. That is why functions such as `isLocalRequest` matter: the builder may be opened in a browser or accessed on a LAN during testing, but operations that change files, compile code, install tools, or publish should be performed only from the owner's computer.

Purpose Of The File

The purpose of `server.mjs` is to make the builder operational instead of decorative. Without it, `template-preview.html` and `template-preview.js` would still be able to render a screen, but they would not be able to reliably save drafts, upload project evidence, run compilers, authenticate GitHub targets, import an existing website, launch updates, or call private AI providers. The file therefore holds the application's real side effects.

The file is also a protection layer. It validates filenames, normalizes repository URLs, keeps publish authentication scoped to a machine and target, prevents stageable publishing paths from wandering outside the publish mirror, blocks remote write requests, caps text fetched from external sources, and strips terminal output into UI-safe strings. Those decisions are not cosmetic. They prevent the portfolio builder from becoming a general-purpose remote command runner or a tool that accidentally pushes the wrong files.

Primary Responsibilities

The first responsibility is serving local files. The same server that handles `/api` requests also serves `index.html`, `styles.css`, `script.js`, `template-preview.html`, project catalogs, assets, and uploaded docs during local preview. The static-serving logic checks that the requested path stays inside the workspace before reading the file. That simple check is what prevents a URL from escaping into arbitrary folders on the machine.

The second responsibility is compilation. `Compile Code` works because the backend can write source files to a project workspace, discover tools such as `node`, `python`, `gcc`, `g++`, `javac`, `iverilog`, `vvp`, and `LTspice`, run those tools with timeouts, collect stdout and stderr, cache successful builds, and return terminal output immediately to the frontend. The backend treats each language differently because JavaScript, Python, C, C++, Java, Verilog, SystemVerilog, LTspice, and HTML do not build the same way.

The third responsibility is publishing. Publishing is not just 'write `projects.json`.' The backend must verify that the publish mirror is a Git repository, know its origin remote and branch, check compatibility with a static portfolio site, synchronize with the remote to avoid non-fast-forward errors, stage only approved publish paths, commit if there are changes, and push to the selected branch. This is why the publishing code is careful and verbose.

The fourth responsibility is intelligence. For local AI, the backend can use an OpenAI API key or a local Ollama model. It also enriches portfolio context by reading public project links, same-site files, GitHub repositories, GitHub profiles, README files, and selected source files when the question calls for it. The browser cannot safely hold private API keys, so `server.mjs` keeps that path private.

The fifth responsibility is application maintenance. The update functions check GitHub Releases, download an installer, write a PowerShell handoff script, close the running app, run the installer, and relaunch the updated executable. This is backend work because the browser cannot replace the running desktop app.

Important Data Objects And Why They Matter

`compileLanguageProfiles` is the backend's language map. It tells the compiler system what default filename to use, which file extensions belong to each language, what human label to show, which tools are required, and which Winget package can install the tool if automatic installation exists. Without this object, `compileAndRunCode` would need hardcoded language assumptions scattered throughout the file.

`compileToolCandidates` is the tool-discovery map. It lists command names and likely Windows install paths for Git-adjacent and compiler-adjacent executables. `findTool` uses this object so the app can work even when a tool

is installed but not perfectly exposed on PATH. The caches `compileToolCache` and `compileToolVersionCache` exist so repeated compiles do not waste time rediscovering the same executables.

`publishPaths` is the publishing allowlist. It lists the site files and folders the backend is allowed to copy into the publish mirror and stage in Git. This is one of the most important safety objects in the file. Without an allowlist, an Apply to site operation could accidentally stage builder-only files, private drafts, temporary compile output, or unrelated local files.

`portfolioAiInstructions` is the system behavior contract for the portfolio assistant. It tells the model how to distinguish general conversation, general engineering, general knowledge, and portfolio-specific questions. It also tells the model when to use context, when not to invent, and how to handle public GitHub source excerpts. That string is long because it is the policy boundary for the AI feature.

Top-Level Objects And Variables

This section explains the long-lived objects and variables before the function walkthrough. These are the values that give the file memory, configuration, API handles, DOM anchors, path boundaries, cache state, and behavior maps. They are explained by meaning, not by line number.

Filesystem Location

root

root is the absolute folder that contains server.mjs. Every local static-file lookup starts from this folder unless the app is deliberately publishing or compiling somewhere else. It is calculated from import.meta.url instead of the current shell directory so the backend behaves the same whether Electron, PowerShell, or another launcher starts it. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include installCompilerTools, resolveInsideRoot, getPublishTargetInfo, runGit, runOptionalCommand, writeTargetCustomDomain, publishAuthCacheScope, syncFromPublishTarget, syncPortfolioPublishFiles, handleApi.

portfolioRoot

portfolioRoot is the folder treated as the publishable website mirror. It normally equals root, but OMB_PORTFOLIO_WORKSPACE can redirect it to a separate portfolio folder. Publishing functions use this boundary when copying and staging public website files. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include resolveInsidePortfolioRoot, getPublishTargetInfo, runPublishGit, runGitWithInput, storeGitCredentials, writeTargetCustomDomain, ensureGitRepository, publishAuthCacheScope, authenticateGitHubForTarget, syncPortfolioPublishFiles.

compileRoot

compileRoot is the separate workspace for Compile Code projects. It is deliberately outside the public site folder so temporary source files, binaries, VCD waveforms, and compiler output do not get mixed into the portfolio website. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include runProcess, compileToolStatus, resolveInsideCompileRoot.

draftPath

draftPath is the private local draft catalog path. It stores builder-side edits that have not necessarily been applied to the public projects.json file. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include syncFromPublishTarget, handleApi.

catalogPath

catalogPath is the public portfolio catalog path. The website runtime reads this file to render the public project list, profile, sections, fun facts, links, and rich content. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include handleApi.

publishAuthCachePath

publishAuthCachePath is the local file that remembers recent publishing authorization. It is intentionally local to the machine and target so GitHub verification does not have to repeat on every Apply to site click. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include readPublishAuthCache, writePublishAuthCache.

publishPaths

`publishPaths` is the allowlist of files and folders that can be copied and staged during publishing. This is a critical safety object: it prevents builder-only files, temp compiler output, credentials, and internal docs from being pushed as website content. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Functions that reference it include `syncPortfolioPublishFiles`, `publishSiteChanges`.

Temporary Calculation

port

`port` is the TCP port used by the local HTTP server. `PORT` can override it for development or packaging, while the default supports the local builder preview without requiring a user to choose a port. Functions that reference it include `handleApi`.

host

`host` is the network interface binding for the server. The default allows the local runtime to listen broadly, but write operations still rely on `isLocalRequest` so privilege is not granted merely because a browser can reach the server. Functions that reference it include `parseGitHubSourceUrl`, `callOllamaPortfolioAi`, `storeGitCredentials`, `handleApi`.

execFileAsync

`execFileAsync` is the Promise-based wrapper around Node's `execFile`. It lets Git, version checks, and other short commands use `async/await` instead of nested callbacks. Functions that reference it include `findTool`, `runGit`, `runOptionalCommand`.

packageJson

`packageJson` holds metadata from `package.json`, especially the current builder version used by update checks, release comparisons, and status messages. Functions that reference it include `getGitStatus`, `getUpdateInfo`.

types

`types` maps file extensions to HTTP Content-Type headers. Static serving uses it so browsers interpret CSS as CSS, JavaScript as JavaScript, PDFs as PDFs, icons as icons, and uploaded assets as the right media type. Visible object fields include `.css`, `.csv`, `.html`, `.ico`, `.js`, `.json`, `.md`, `.pdf`, `.png`, `.jpg`, `.jpeg`, `.svg`, `.txt`, `.webp`, `.xml`, `.xlsx`, `.zip`. Those fields form the contract consumed by related functions. Functions that reference it include `handleApi`.

defaultSiteRepository

`defaultSiteRepository` stores the optional repository configured through the environment. It gives packaged or developer-launched builds a default target without hardcoding one into the code. Functions that reference it include `getPublishTargetInfo`, `assertPublishAccess`.

Runtime State Or Cache

publishAuthCacheTtlMs

`publishAuthCacheTtlMs` is the normal one-day authorization window. It implements the rule that GitHub publishing should not ask again every time the app opens, but also should not trust an old session forever. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include `writePublishAuthCache`, `publishAuthCacheExpiresAt`.

publishAuthHistoryWindowMs

`publishAuthHistoryWindowMs` is the seven-day window used to count successful authorizations. The cache logic uses it to decide whether the user qualifies for extended trust. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include `writePublishAuthCache`.

compileToolCache

compileToolCache remembers the resolved path for compiler tools. It prevents every compile click from scanning the filesystem again. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Functions that reference it include findTool, installCompilerTools.

compileToolVersionCache

compileToolVersionCache remembers version strings for tools after they have been checked. This makes status panels faster and keeps repeated diagnostics from slowing the UI. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Functions that reference it include toolVersionLine.

Git Publishing And Authorization State

publishAuthExtendedTtlMs

publishAuthExtendedTtlMs is the thirty-day extended authorization window. It is used after repeated successful authorizations prove the same device and target are consistently owned by the same publisher. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include writePublishAuthCache.

publishAuthExtendedThreshold

publishAuthExtendedThreshold is the number of successful authorizations required before the builder extends trust. It turns the user's requested security rule into a concrete number. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include writePublishAuthCache.

publishAuthorizationHelp

publishAuthorizationHelp is the human-readable troubleshooting text shown when publishing is blocked. It explains the sequence the user should follow: configure target, authenticate, load if needed, then apply. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include publishAccessError, handleApi.

gitCandidates

gitCandidates lists command names and likely Windows Git paths. Git operations use this list to find Git even when PATH is incomplete, which is common on Windows installs. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include runGit, runGitWithInput.

githubTextFilePattern

githubTextFilePattern tells the AI source reader which GitHub files are worth fetching as text. It includes code, README, HDL, schematic-like text formats, scripts, and engineering project files. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include sourceUrlAllowed, scoreGitHubFile, fetchGitHubRepositorySource.

githubSkipFilePattern

githubSkipFilePattern tells the AI source reader which GitHub files to skip because they are binary, huge, media-oriented, or not useful as prompt text. It affects publishing, where mistakes can change the public website or expose the wrong files. Functions that reference it include sourceUrlAllowed, fetchGitHubRepositorySource.

Lookup Collection

blockedAppUpdateVersions

blockedAppUpdateVersions is a Set of release versions the updater must skip. It exists so a known bad release can be blocked without deleting every release artifact from GitHub. Functions that reference it include getUpdateInfo.

Portfolio Data State

compileLanguageProfiles

compileLanguageProfiles is the language rulebook for the IDE feature. Each language entry defines the default filename, valid extensions, display label, required tools, and installer hints. Compile, build, run, simulate, and beautify operations all depend on this map. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Visible object fields include c, defaultFile, extensions, label, primaryTools, winget, cpp, verilog, systemverilog, ltspice, java, javascript, python, html. Those fields form the contract consumed by related functions. Functions that reference it include normalizeCodeLanguage, languageFromFileName, safeCodeFileName, compileToolStatus, hdlFilesFromPayload, compileWorkspaceFilesFromPayload, compileAndRunCode, installCompilerTools.

Compiler Or Language Configuration

compileToolCandidates

compileToolCandidates maps compiler tool names to possible executable locations. It lets the app find gcc, g++, javac, java, iverilog, vvp, python, node, and LTspice even when a normal terminal cannot. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Visible object fields include gcc, g++, clang, clang++, cl, javac, java, node, python, iverilog, vvp, ltspice. Those fields form the contract consumed by related functions. Functions that reference it include findTool.

Ai Configuration Or Conversation State

portfolioAiInstructions

portfolioAiInstructions is the system prompt for the portfolio assistant. It defines how the model should distinguish greetings, general electronics questions, portfolio-specific questions, public GitHub code requests, and uncertain answers. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include callOllamaPortfolioAi, handlePortfolioAi.

Code Walkthrough

This walkthrough is function-by-function. Each section now follows a textbook rhythm: first the idea of the function, then the syntax, then how to read that syntax, then the implementation and the local objects that make the implementation work. Line numbers are deliberately avoided because the source will keep changing.

HTTP API surface

The functions in this group all support http api surface. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function securityHeaders(extra = {})

Begin with the role of securityHeaders. securityHeaders belongs to http api surface. This function belongs to the local HTTP/API layer. It either shapes responses, reads requests, or decides whether a request is allowed to perform local work. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function securityHeaders(extra = {}) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: extra. Read those names as the contract

between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `extra` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `securityHeaders` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `securityHeaders`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `securityHeaders`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `sendJson(response, status, data)`

Begin with the role of `sendJson`. `sendJson` belongs to `http api` surface. This function belongs to the local HTTP/API layer. It either shapes responses, reads requests, or decides whether a request is allowed to perform local work. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sendJson(response, status, data) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `response`, `status`, `data`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `response` is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals. `status` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `data` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: serializes structured data before sending or saving it; writes the HTTP response directly.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `securityHeaders`. Those calls show that `sendJson` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `sendJson`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `sendJson`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function isLocalRequest(request)

Begin with the role of `isLocalRequest`. `isLocalRequest` is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

The syntax of the function is:

```
function isLocalRequest(request) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `request`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `request` is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `isLocalRequest` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `isLocalRequest`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `isLocalRequest` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

address: `address` stores a computed value for temporary calculation. It is initialized from `request.socket.remoteAddress || ""`. Within the function it is returned to the caller; assigned or updated later in the function.

async function readRequestJson(request)

The best entry point for `readRequestJson` is its responsibility in the surrounding workflow. `readRequestJson` reads `RequestJson` from a request, file, cache, or generated artifact. It gives the caller parsed data rather than raw bytes or untrusted text, which keeps parsing rules centralized.

The syntax of the function is:

```
async function readRequestJson(request) { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `request`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `request` is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: parses JSON rather than treating incoming text as already trusted data; throws explicit errors when a required safety or compatibility condition fails.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `readRequestJson` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `readRequestJson`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The easiest way to follow the body of `readRequestJson` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

chunks: `chunks` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `push`; passed into `parse`; assigned or updated later in the function.

size: `size` stores a numeric value for temporary calculation. It is initialized from `0`. Within the function it is assigned or updated later in the function.

chunk: `chunk` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `length`; passed into `await`, `push`.

async function handleApi(request, response, url)

Read `handleApi` first as a named idea, not as syntax. `handleApi` is the backend router. It receives the parsed URL and decides which local API feature should run: project catalog reads, uploads, draft saves, compile actions, compiler installation, target authentication, load from target, apply to site, update checks, security reports, AI calls, or builder download reports. In a framework such as Express this would be spread across route handlers. Here it is one explicit switch-like controller. The function protects the boundary between a browser click and local machine power. Before write operations, it checks whether the request is local. It reads request JSON only when needed, calls the specific operation function, and returns a normalized JSON response. Without `handleApi`, the server could still serve files, but the builder would have no organized command surface.

The syntax of the function is:

```
async function handleApi(request, response, url) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `request`, `response`, `url`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `request` is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals. `response` is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: serializes structured data before sending or saving it; uses Node path utilities so Windows path separators and relative paths are handled consistently; writes the HTTP response directly; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation; reads data from disk as part of the operation.

The function also has to be read through the helpers it calls. It works with `readJsonFile`, `sendJson`, `isLocalRequest`, `getPublishTargetInfo`, `getGitStatus`, `getUpdateInfo`, `getSecurityReport`, `compileToolStatus`. Those calls show that `handleApi` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `handleApi`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside `handleApi` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

catalog: `catalog` stores a resolved filesystem or route value. It is initialized from `await readJsonFile(draftPath)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `categories`, `projects`; passed into `isArray`, `sendJson`, `writeFile`; assigned or updated later in the function.

body: `body` stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods `catalog`, `code`, `data`, `fileName`, `language`, `projectId`, `section`; passed into `String`, `authenticateGitHubForTarget`, `compileAndRunCode`, `end`, `installCompilerTools`, `normalizeCodeLanguage`, `safeFileName`, `safeSegment`; assigned or updated later in the function.

language: `language` stores a computed value for compiler or language configuration. It is initialized from `normalizeCodeLanguage(body.language || detectCodeLanguageFromSource(body.code, body.fileName))`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `installCompilerTools`, `normalizeCodeLanguage`, `sendJson`; assigned or updated later in the function.

body: `body` stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods `catalog`, `code`, `data`,

fileName, language, projectId, section; passed into String, authenticateGitHubForTarget, compileAndRunCode, end, installCompilerTools, normalizeCodeLanguage, safeFileName, safeSegment; assigned or updated later in the function.

saved: saved stores the completed result of an awaited operation. It is initialized from await saveCompileSource(body). Within the function it is passed into sendJson; assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from await readRequestJson(request). Within the function it is accesses properties/methods catalog, code, data, fileName, language, projectId, section; passed into String, authenticateGitHubForTarget, compileAndRunCode, end, installCompilerTools, normalizeCodeLanguage, safeFileName, safeSegment; assigned or updated later in the function.

result: result stores the completed result of an awaited operation. It is initialized from await compileAndRunCode(body). Within the function it is accesses properties/methods ok; passed into sendJson; assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from await readRequestJson(request). Within the function it is accesses properties/methods catalog, code, data, fileName, language, projectId, section; passed into String, authenticateGitHubForTarget, compileAndRunCode, end, installCompilerTools, normalizeCodeLanguage, safeFileName, safeSegment; assigned or updated later in the function.

result: result stores the completed result of an awaited operation. It is initialized from await installCompilerTools(body.language || ""). Within the function it is accesses properties/methods ok; passed into sendJson; assigned or updated later in the function.

install: install stores the completed result of an awaited operation. It is initialized from await installGitForWindows(). Within the function it is passed into sendJson; assigned or updated later in the function.

update: update stores the completed result of an awaited operation. It is initialized from await downloadAndLaunchAppUpdate(). Within the function it is passed into sendJson; assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from await readRequestJson(request). Within the function it is accesses properties/methods catalog, code, data, fileName, language, projectId, section; passed into String, authenticateGitHubForTarget, compileAndRunCode, end, installCompilerTools, normalizeCodeLanguage, safeFileName, safeSegment; assigned or updated later in the function.

auth: auth stores the completed result of an awaited operation. It is initialized from await authenticateGitHubForTarget(body || {}). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into sendJson; assigned or updated later in the function.

sync: sync stores the completed result of an awaited operation. It is initialized from await syncFromPublishTarget(). Within the function it is passed into sendJson; assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from await readRequestJson(request). Within the function it is accesses properties/methods catalog, code, data, fileName, language, projectId, section; passed into String, authenticateGitHubForTarget,

compileAndRunCode, end, installCompilerTools, normalizeCodeLanguage, safeFileName, safeSegment; assigned or updated later in the function.

catalog: catalog stores a computed value for portfolio data state. It is initialized from body.catalog. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods categories, projects; passed into isArray, sendJson, writeFile; assigned or updated later in the function.

applyingToSite: applyingToSite stores a computed value for temporary calculation. It is initialized from url.pathname === "/api/apply-catalog". Within the function it is assigned or updated later in the function.

publishAccess: publishAccess stores a computed value for Git publishing and authorization state. It is initialized from null. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into publishSiteChanges, sendJson; assigned or updated later in the function.

target: target stores a resolved filesystem or route value. It is initialized from applyingToSite ? catalogPath : draftPath. Within the function it is passed into sendJson, writeFile; assigned or updated later in the function.

publish: publish stores the completed result of an awaited operation. It is initialized from await publishSiteChanges(publishAccess). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is assigned or updated later in the function.

body: body stores the completed result of an awaited operation. It is initialized from await readRequestJson(request). Within the function it is accesses properties/methods catalog, code, data, fileName, language, projectId, section; passed into String, authenticateGitHubForTarget, compileAndRunCode, end, installCompilerTools, normalizeCodeLanguage, safeFileName, safeSegment; assigned or updated later in the function.

projectId: projectId stores a computed value for portfolio data state. It is initialized from safeSegment(body.projectId, "project"). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into resolveInsideRoot, safeSegment; assigned or updated later in the function.

section: section stores a computed value for portfolio data state. It is initialized from safeSegment(body.section, "documents"). Within the function it is passed into resolveInsideRoot, safeSegment; assigned or updated later in the function.

fileName: fileName stores a computed value for temporary calculation. It is initialized from safeFileName(body.fileName). Within the function it is passed into normalizeCodeLanguage, resolveInsideRoot, safeFileName; assigned or updated later in the function.

base64: base64 stores a computed value for temporary calculation. It is initialized from String(body.data || "").replace(/^(data:[^).replace(/^(data:[^

folder: folder stores a computed value for filesystem location. It is initialized from resolveInsideRoot("docs", projectId, section). Within the function it is passed into mkdir; assigned or updated later in the function.

filePath: filePath stores a computed value for filesystem location. It is initialized from resolveInsideRoot("docs", projectId, section, fileName). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods startsWith; passed into extname, readFile, sendJson, writeFile; assigned or updated later in the function.

url: url stores a constructed object for network or routing value. It is initialized from `new URL(request.url || "/", `http://${request.headers.host}`)`. Within the function it is accesses properties/methods `pathname`; passed into `URL`, `decodeURIComponent`, `handleApi`, `sendJson`; assigned or updated later in the function.

pathname: `pathname` stores a computed value for filesystem location. It is initialized from `url.pathname === "/" ? "/index.html" : decodeURIComponent(url.pathname)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods `startsWith`; passed into `decodeURIComponent`, `normalize`; assigned or updated later in the function.

filePath: `filePath` stores a resolved filesystem or route value. It is initialized from `path.normalize(path.join(root, pathname))`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods `startsWith`; passed into `extname`, `readFile`, `sendJson`, `writeFile`; assigned or updated later in the function.

body: `body` stores a resolved filesystem or route value. It is initialized from `await readFile(filePath)`. Within the function it is accesses properties/methods `catalog`, `code`, `data`, `fileName`, `language`, `projectId`, `section`; passed into `String`, `authenticateGitHubForTarget`, `compileAndRunCode`, `end`, `installCompilerTools`, `normalizeCodeLanguage`, `safeFileName`, `safeSegment`; assigned or updated later in the function.

Backend utility

The functions in this group all support backend utility. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function clampText(value, maxLength = 12000)

Read `clampText` first as a named idea, not as syntax. `clampText` belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function clampText(value, maxLength = 12000) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`, `maxLength`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `maxLength` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `clampText` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `clampText`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `clampText`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function stripHtmlToText(value = "")

The best entry point for `stripHtmlToText` is its responsibility in the surrounding workflow. `stripHtmlToText` belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function stripHtmlToText(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `stripHtmlToText` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `stripHtmlToText`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `stripHtmlToText`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function normalizeCodeLanguage(value = "")

Begin with the role of `normalizeCodeLanguage`. `normalizeCodeLanguage` standardizes `CodeLanguage` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeCodeLanguage(value = "") { ... }
```


There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `normalizeCodeLanguage` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `normalizeCodeLanguage`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `normalizeCodeLanguage` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(value || "").trim().toLowerCase().replace(/[ _s-]+/g, "")`. Within the function it is assigned or updated later in the function.

aliases: `aliases` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; assigned or updated later in the function.

function indentBraceCode(code = "")

Read `indentBraceCode` first as a named idea, not as syntax. `indentBraceCode` belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function indentBraceCode(code = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `code`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a

value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means indentBraceCode performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without indentBraceCode, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside indentBraceCode show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

depth: depth stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is passed into max, repeat; assigned or updated later in the function.

line: line stores a computed value for temporary calculation. It is initialized from rawLine.trim(). Within the function it is accesses properties/methods match; passed into test; assigned or updated later in the function.

formatted: formatted stores a string value for temporary calculation. It is initialized from `\${" ".repeat(depth)}\${line}`. Within the function it is returned to the caller; assigned or updated later in the function.

opens: opens stores a function or callback value for temporary calculation. It is initialized from (line.match(/[([/g] || []).length. Within the function it is passed into max; assigned or updated later in the function.

closes: closes stores a function or callback value for temporary calculation. It is initialized from (line.match(/[)])/g || []).length. Within the function it is passed into max; assigned or updated later in the function.

function beautifyCode(code = "", language = "javascript")

Begin with the role of beautifyCode. beautifyCode belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function beautifyCode(code = "", language = "javascript") { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: code, language. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. code is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. language names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `normalizeCodeLanguage`, `indentBraceCode`. Those calls show that `beautifyCode` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `beautifyCode`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `beautifyCode` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

normalized: `normalized` stores a computed value for temporary calculation. It is initialized from `String(code || "").replace(/\r\n?/g, "\n").replace(/\t/g, " ")`. Within the function it is returned to the caller; passed into `indentBraceCode`; assigned or updated later in the function.

lang: `lang` stores a computed value for temporary calculation. It is initialized from `normalizeCodeLanguage(language)`. Within the function it is assigned or updated later in the function.

async function findExecutableUnder(folder, names = [], maxDepth = 5)

To understand `findExecutableUnder`, start with the problem it owns. `findExecutableUnder` belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function findExecutableUnder(folder, names = [], maxDepth = 5) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `folder`, `names`, `maxDepth`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `folder` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `names` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `maxDepth` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and reads or writes files. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently.

Next, trace the helper calls that surround the function. It works with `pathExists`. Those calls show that `findExecutableUnder` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `findExecutableUnder`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

After the main flow, the working values inside `findExecutableUnder` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

entries: `entries` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is assigned or updated later in the function.

entry: `entry` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `isDirectory`, `isFile`, `name`; passed into `findExecutableUnder`, `join`.

fullPath: `fullPath` stores a resolved filesystem or route value. It is initialized from `path.join(folder, entry.name)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is returned to the caller; assigned or updated later in the function.

entry: `entry` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `isDirectory`, `isFile`, `name`; passed into `findExecutableUnder`, `join`.

found: `found` stores a resolved filesystem or route value. It is initialized from `await findExecutableUnder(path.join(folder, entry.name), names, maxDepth - 1)`. Within the function it is returned to the caller; assigned or updated later in the function.

function terminalLine(label, text = "")

Begin with the role of `terminalLine`. `terminalLine` belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function terminalLine(label, text = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `label`, `text`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `label` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `text` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `terminalLine` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `terminalLine`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `terminalLine`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function cachedBuildLine(type, outputPath)

Begin with the role of `cachedBuildLine`. `cachedBuildLine` belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function cachedBuildLine(type, outputPath) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `type`, `outputPath`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `type` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `outputPath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `cachedBuildLine` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `cachedBuildLine`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `cachedBuildLine`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function fetchLimitedText(url, maxLength = 12000)

The best entry point for `fetchLimitedText` is its responsibility in the surrounding workflow. `fetchLimitedText` retrieves `LimitedText` from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

The syntax of the function is:

```
async function fetchLimitedText(url, maxLength = 12000) { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `url`, `maxLength`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. `maxLength` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and calls a network resource. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: performs a fetch and therefore must handle network delay and failure.

Its connection to the rest of the file matters as much as its local code. It works with `gitHubHeaders`, `clampText`, `stripHtmlToText`. Those calls show that `fetchLimitedText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `fetchLimitedText`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The easiest way to follow the body of `fetchLimitedText` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

response: `response` stores data returned from a network call. It is initialized from `await fetch(url, {`. Within the function it is accesses properties/methods `headers`, `ok`, `text`; passed into `clampText`; assigned or updated later in the function.

rawText: `rawText` stores the completed result of an awaited operation. It is initialized from `clampText(await response.text(), maxLength)`. Within the function it is passed into `stripHtmlToText`; assigned or updated later in the function.

contentType: `contentType` stores a computed value for temporary calculation. It is initialized from `response.headers.get("Content-Type") || ""`. Within the function it is passed into `test`; assigned or updated later in the function.

async function runOptionalCommand(command, args = [], options = {})

Read `runOptionalCommand` first as a named idea, not as syntax. `runOptionalCommand` executes an operation with side effects. It is separated from callers so process execution, Git execution, optional command behavior, or status collection remains consistent.

The syntax of the function is:

```
async function runOptionalCommand(command, args = [], options = {}) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `command`, `args`, `options`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `command` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `args` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work and runs an external command. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `runOptionalCommand` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `runOptionalCommand`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside `runOptionalCommand` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

result: `result` stores the completed result of an awaited operation. It is initialized from `await execFileAsync(command, args, {`. Within the function it is accesses properties/methods `stderr`, `stdout`; assigned or updated later in the function.

function powershellSingleQuoted(value = "")

To understand `powershellSingleQuoted`, start with the problem it owns. `powershellSingleQuoted` belongs to backend utility. This is a backend helper that normalizes data or supports a larger server operation. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function powershellSingleQuoted(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and

generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `powershellSingleQuoted` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `powershellSingleQuoted`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `powershellSingleQuoted`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

Compiler and IDE workspace

The functions in this group all support compiler and ide workspace. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function sourceLooksCpp(source = "")

Begin with the role of `sourceLooksCpp`. `sourceLooksCpp` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sourceLooksCpp(source = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `source`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `sourceLooksCpp` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `sourceLooksCpp`, the IDE-style compile workspace would become less reliable. The app might misidentify a

language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The local state inside `sourceLooksCpp` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

text: text stores a computed value for temporary calculation. It is initialized from `String(source || "")`. Within the function it is passed into test; assigned or updated later in the function.

function sourceLooksC(source = "")

Begin with the role of `sourceLooksC`. `sourceLooksC` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sourceLooksC(source = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `source`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `sourceLooksC` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `sourceLooksC`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The local state inside `sourceLooksC` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

text: text stores a computed value for temporary calculation. It is initialized from `String(source || "")`. Within the function it is passed into test; assigned or updated later in the function.

function detectCodeLanguageFromSource(code = "", fileName = "")

To understand `detectCodeLanguageFromSource`, start with the problem it owns.

`detectCodeLanguageFromSource` makes an educated classification from incomplete evidence. It looks at names, extensions, source text, repository state, or runtime state and returns the app's best decision so the next operation can choose the correct path.

The syntax of the function is:

```
function detectCodeLanguageFromSource(code = "", fileName = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `code`, `fileName`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `fileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Next, trace the helper calls that surround the function. It works with `languageFromFileName`, `sourceLooksCpp`, `sourceLooksC`. Those calls show that `detectCodeLanguageFromSource` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `detectCodeLanguageFromSource`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

After the main flow, the working values inside `detectCodeLanguageFromSource` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

byName: `byName` stores a computed value for temporary calculation. It is initialized from `languageFromFileName(fileName, code)`. Within the function it is returned to the caller; assigned or updated later in the function.

source: `source` stores a computed value for temporary calculation. It is initialized from `String(code || "")`. Within the function it is passed into `sourceLooksC`, `sourceLooksCpp`, `test`; assigned or updated later in the function.

async function findTool(toolName)

The best entry point for `findTool` is its responsibility in the surrounding workflow. `findTool` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function findTool(toolName) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `toolName`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `toolName` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and runs an external command. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently.

Its connection to the rest of the file matters as much as its local code. It works with `pathExists`, `findExecutableUnder`. Those calls show that `findTool` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `findTool`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The easiest way to follow the body of `findTool` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

candidates: `candidates` stores a function or callback value for temporary calculation. It is initialized from `(compileToolCandidates[toolName] || [toolName]).filter(Boolean)`. Within the function it is assigned or updated later in the function.

candidate: `candidate` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is returned to the caller; passed into `set`.

command: `command` stores a computed value for temporary calculation. It is initialized from `process.platform === "win32" ? "where" : "command"`. Within the function it is passed into `execFileAsync`; assigned or updated later in the function.

args: `args` stores a computed value for temporary calculation. It is initialized from `process.platform === "win32" ? [candidate] : ["-v", candidate]`. Within the function it is passed into `execFileAsync`; assigned or updated later in the function.

result: `result` stores the completed result of an awaited operation. It is initialized from `await execFileAsync(command, args, { timeout: 5000, windowsHide: true })`. Within the function it is accesses properties/methods `stdout`; passed into `String`; assigned or updated later in the function.

found: `found` stores a function or callback value for lookup collection. It is initialized from `String(result.stdout || "").split(/\r?\n/).map((line) => line.trim()).find(Boolean)`. Within the function it is returned to the caller; passed into `set`; assigned or updated later in the function.

found: `found` stores the completed result of an awaited operation. It is initialized from `await findExecutableUnder`. Within the function it is returned to the caller; passed into `set`; assigned or updated later in the function.

found: `found` stores the completed result of an awaited operation. It is initialized from `await findExecutableUnder("C:\\Program Files\\Eclipse Adoptium", [`${toolName}.exe`], 5)`. Within the function it is returned to the caller; passed into `set`; assigned or updated later in the function.

function processTerminalText(result = {})

Begin with the role of processTerminalText. processTerminalText belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function processTerminalText(result = {}) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: result. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. result is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means processTerminalText performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without processTerminalText, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The local state inside processTerminalText explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

elapsedSeconds: elapsedSeconds stores a computed value for temporary calculation. It is initialized from `Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : ""`. Within the function it is assigned or updated later in the function.

elapsed: elapsed stores a computed value for temporary calculation. It is initialized from `elapsedSeconds ? ` in ${elapsedSeconds}s` : ""`. Within the function it is assigned or updated later in the function.

status: status stores a computed value for temporary calculation. It is initialized from `result.timedOut`. Within the function it is assigned or updated later in the function.

output: output stores a list for lookup collection. It is initialized from `[result.stdout, result.stderr].map((part) => String(part || "").trimEnd()).filter(Boolean).join("\n")`. Within the function it is assigned or updated later in the function.

function processTerminalTextWithPaths(result = {}, replacements = [])

Begin with the role of processTerminalTextWithPaths. processTerminalTextWithPaths belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function processTerminalTextWithPaths(result = {}, replacements = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `result`, `replacements`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `result` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `replacements` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `replacePathReferences`, `processTerminalText`. Those calls show that `processTerminalTextWithPaths` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `processTerminalTextWithPaths`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Inside `processTerminalTextWithPaths`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function cFamilyCompileProfile(language = "c", fileName = "main.c")`

The best entry point for `cFamilyCompileProfile` is its responsibility in the surrounding workflow. `cFamilyCompileProfile` belongs to `compiler` and `ide workspace`. This function supports the `Compile Code workspace`: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function cFamilyCompileProfile(language = "c", fileName = "main.c") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `language`, `fileName`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `fileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app

does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `cFamilyCompileProfile` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `cFamilyCompileProfile`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The easiest way to follow the body of `cFamilyCompileProfile` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

isCpp: `isCpp` stores a computed value for temporary calculation. It is initialized from `language === "cpp"`. Within the function it is assigned or updated later in the function.

ext: `ext` stores a resolved filesystem or route value. It is initialized from `path.extname(String(fileName || "").toLowerCase())`. Within the function it is passed into `includes`; assigned or updated later in the function.

header: `header` stores a list for temporary calculation. It is initialized from `[".h", ".hpp", ".hh", ".hxx"].includes(ext)`. Within the function it is assigned or updated later in the function.

function cFamilyBinaryName(fileName = "program")

The best entry point for `cFamilyBinaryName` is its responsibility in the surrounding workflow. `cFamilyBinaryName` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function cFamilyBinaryName(fileName = "program") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `fileName`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `fileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Its connection to the rest of the file matters as much as its local code. It works with `safeSegment`. Those calls show that `cFamilyBinaryName` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `cFamilyBinaryName`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The easiest way to follow the body of `cFamilyBinaryName` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

parsed: `parsed` stores a resolved filesystem or route value. It is initialized from `path.parse(String(fileName || "program"))`. Within the function it is accessed properties/methods `name`; passed into `safeSegment`; assigned or updated later in the function.

async function toolVersionLine(toolPath = "")

Begin with the role of `toolVersionLine`. `toolVersionLine` belongs to `compiler` and `ide workspace`. This function supports the `Compile Code workspace`: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function toolVersionLine(toolPath = "") { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `toolPath`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `toolPath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work and runs an external command. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: delegates command execution to `runProcess` so timeout and terminal capture remain consistent.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `runProcess`. Those calls show that `toolVersionLine` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `toolVersionLine`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The local state inside `toolVersionLine` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

result: result stores a resolved filesystem or route value. It is initialized from `await runProcess(toolPath, ["--version"], { timeoutMs: 7000 })`. Within the function it is accesses properties/methods `stderr`, `stdout`; assigned or updated later in the function.

version: version stores a list for temporary calculation. It is initialized from `[result.stdout, result.stderr]`. Within the function it is returned to the caller; passed into `runProcess`, set; assigned or updated later in the function.

function cFamilyRunOutput(result = {})

The best entry point for `cFamilyRunOutput` is its responsibility in the surrounding workflow. `cFamilyRunOutput` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function cFamilyRunOutput(result = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `result`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `result` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `cFamilyRunOutput` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `cFamilyRunOutput`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The easiest way to follow the body of `cFamilyRunOutput` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

elapsedSeconds: `elapsedSeconds` stores a computed value for temporary calculation. It is initialized from `Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : ""`. Within the function it is assigned or updated later in the function.

status: `status` stores a computed value for temporary calculation. It is initialized from `result.timedOut`. Within the function it is assigned or updated later in the function.

output: `output` stores a list for lookup collection. It is initialized from `[result.stdout, result.stderr].map((part) => String(part || "").trimEnd()).filter(Boolean).join("\n")`. Within the function it is assigned or updated later in the function.

function cleanHdlSimulationOutput(result = {})

To understand `cleanHdlSimulationOutput`, start with the problem it owns. `cleanHdlSimulationOutput` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function cleanHdlSimulationOutput(result = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `result`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `result` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `cleanHdlSimulationOutput` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `cleanHdlSimulationOutput`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

After the main flow, the working values inside `cleanHdlSimulationOutput` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

raw: `raw` stores a list for lookup collection. It is initialized from `[result.stdout, result.stderr].map((part) => String(part || "").trimEnd()).filter(Boolean).join("\n")`. Within the function it is accesses properties/methods `split`; assigned or updated later in the function.

lines: `lines` stores a function or callback value for lookup collection. It is initialized from `raw.split(/\r?\n/).map((line) => line.trimEnd()).filter(Boolean)`. Within the function it is assigned or updated later in the function.

finishLines: `finishLines` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `length`, `push`; assigned or updated later in the function.

signalLines: `signalLines` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `length`, `push`; assigned or updated later in the function.

line: line starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods replace, trimEnd; passed into map, push.

elapsedSeconds: elapsedSeconds stores a computed value for temporary calculation. It is initialized from `Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : ""`. Within the function it is assigned or updated later in the function.

status: status stores a computed value for temporary calculation. It is initialized from `result.timedOut`. Within the function it is assigned or updated later in the function.

body: body stores a computed value for temporary calculation. It is initialized from `signalLines.length`. Within the function it is assigned or updated later in the function.

suffix: suffix stores a resolved filesystem or route value. It is initialized from `signalLines.length && finishLines.length ? `\\n${finishLines.join("\\n")}` : ""`. Within the function it is assigned or updated later in the function.

async function runProcess(command, args = [], options = {})

Begin with the role of runProcess. runProcess is the safe wrapper around external commands. Compilers, Git, Winget, Java, vvp, and installer helpers all eventually need child processes. This function starts the command with controlled options, captures stdout and stderr, enforces a timeout, kills process trees on Windows when necessary, and resolves to a structured result instead of letting raw process behavior leak through the API. Without this wrapper, every compiler and Git function would need to repeat timeout logic, stdout/stderr collection, and error handling. More importantly, failures would be inconsistent and the frontend terminal would receive unreliable output.

The syntax of the function is:

```
async function runProcess(command, args = [], options = {}) { ... }
```

The async keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: command, args, options. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. command is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. args is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. Inside the body, it runs an external command and uses a timer to avoid blocking or to wait for another process. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `runProcess` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `runProcess`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The local state inside `runProcess` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

timeoutMs: `timeoutMs` stores a computed value for temporary calculation. It is initialized from `options.timeoutMs || 20000`. Within the function it is assigned or updated later in the function.

cwd: `cwd` stores a computed value for temporary calculation. It is initialized from `options.cwd || compileRoot`. Within the function it is passed into `spawn`; assigned or updated later in the function.

startedAt: `startedAt` stores a computed value for temporary calculation. It is initialized from `Date.now()`. Within the function it is assigned or updated later in the function.

child: `child` stores a computed value for temporary calculation. It is initialized from `spawn(command, args, {`. Within the function it is accesses properties/methods `kill`, `on`, `pid`, `stderr`, `stdin`, `stdout`; passed into `execFile`; assigned or updated later in the function.

stdout: `stdout` stores a string value for temporary calculation. It is initialized from `""`. Within the function it is passed into `resolve`; assigned or updated later in the function.

stderr: `stderr` stores a string value for temporary calculation. It is initialized from `""`. Within the function it is passed into `resolve`; assigned or updated later in the function.

timedOut: `timedOut` stores a boolean value for temporary calculation. It is initialized from `false`. Within the function it is passed into `resolve`; assigned or updated later in the function.

timer: `timer` stores a function or callback value for temporary calculation. It is initialized from `setTimeout(() => {`. Within the function it is passed into `clearTimeout`; assigned or updated later in the function.

async function compileToolStatus()

The best entry point for `compileToolStatus` is its responsibility in the surrounding workflow. `compileToolStatus` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function compileToolStatus() { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Its connection to the rest of the file matters as much as its local code. It works with `findTool`. Those calls show that `compileToolStatus` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `compileToolStatus`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The easiest way to follow the body of `compileToolStatus` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

tools: `tools` stores a structured object for compiler or language configuration. It is initialized from `{}`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is assigned or updated later in the function.

uniqueTools: `uniqueTools` stores a list for compiler or language configuration. It is initialized from `[...new Set(Object.values(compileLanguageProfiles).flatMap((profile) => profile.primaryTools))]`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is assigned or updated later in the function.

tool: `tool` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into every, `findTool`, `fromEntries`.

languages: `languages` stores a structured object for compiler or language configuration. It is initialized from `{}`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is assigned or updated later in the function.

destructured [id, profile]: `destructured [id, profile]` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent.

required: `required` stores a computed value for temporary calculation. It is initialized from `profile.primaryTools`. Within the function it is iterated or transformed as a collection; accesses properties/methods every, `map`; passed into `fromEntries`; assigned or updated later in the function.

function `isHdlLanguage(language = "")`

Begin with the role of `isHdlLanguage`. `isHdlLanguage` is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

The syntax of the function is:

```
function isHdlLanguage(language = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `language`. Read those names as the contract

between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `normalizeCodeLanguage`. Those calls show that `isHdlLanguage` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `isHdlLanguage`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Inside `isHdlLanguage`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function inferCompileFileRole(fileName = "", code = "", language = "")`

The best entry point for `inferCompileFileRole` is its responsibility in the surrounding workflow.

`inferCompileFileRole` makes an educated classification from incomplete evidence. It looks at names, extensions, source text, repository state, or runtime state and returns the app's best decision so the next operation can choose the correct path.

The syntax of the function is:

```
function inferCompileFileRole(fileName = "", code = "", language = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `fileName`, `code`, `language`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `fileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller

can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Its connection to the rest of the file matters as much as its local code. It works with `isHdlLanguage`. Those calls show that `inferCompileFileRole` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `inferCompileFileRole`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The easiest way to follow the body of `inferCompileFileRole` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

haystack: haystack stores a string value for runtime state or cache. It is initialized from ``${fileName}\n${code}`.toLowerCase()`. Within the function it is passed into test; assigned or updated later in the function.

function normalizeCompileFileRole(value = "", language = "")

Begin with the role of `normalizeCompileFileRole`. `normalizeCompileFileRole` standardizes `CompileFileRole` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeCompileFileRole(value = "", language = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`, `language`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `isHdlLanguage`. Those calls show that `normalizeCompileFileRole` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `normalizeCompileFileRole`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The local state inside `normalizeCompileFileRole` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "")`. Within the function it is passed into `includes`; assigned or updated later in the function.

async function `saveCompileSource`{ `projectId`, `fileId`, `title`, `fileName`, `language`, `role` = "", `code`, `stdin` = "" }

To understand `saveCompileSource`, start with the problem it owns. `saveCompileSource` belongs to `compiler` and `ide workspace`. This function supports the `Compile Code workspace`: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function saveCompileSource({ projectId, fileId, title, fileName, language, role = "", code,
stdin = "" }) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The braces in the parameter list mean the caller passes one object and the function pulls named fields out of it. In this case the important fields are `projectId`, `fileId`, `title`, `fileName`, `language`, `role`, `code`, `stdin`. That style is useful when a function needs several related values but the call site should still be readable.

Those syntax pieces become practical once you connect them to the data being passed in. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `fileId` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. `title` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `fileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. `role` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. `stdin` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: serializes structured data before sending or saving it; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation.

Next, trace the helper calls that surround the function. It works with `normalizeCodeLanguage`, `detectCodeLanguageFromSource`, `safeSegment`, `safeCodeFileName`, `resolveInsideCompileRoot`, `clampText`, `normalizeCompileFileRole`, `inferCompileFileRole`. Those calls show that `saveCompileSource` is not isolated; it is

one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `saveCompileSource`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

After the main flow, the working values inside `saveCompileSource` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

lang: `lang` stores a computed value for temporary calculation. It is initialized from `normalizeCodeLanguage(language || detectCodeLanguageFromSource(code, fileName))`. Within the function it is passed into `normalizeCompileFileRole`, `safeCodeFileName`; assigned or updated later in the function.

projectFolder: `projectFolder` stores a computed value for filesystem location. It is initialized from `safeSegment(projectId, "project")`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into `resolveInsideCompileRoot`, `writeFile`; assigned or updated later in the function.

codeFileName: `codeFileName` stores a computed value for temporary calculation. It is initialized from `safeCodeFileName(fileName, lang)`. Within the function it is passed into `clampText`, `normalizeCompileFileRole`, `parse`, `resolveInsideCompileRoot`, `safeSegment`; assigned or updated later in the function.

fileFolder: `fileFolder` stores a resolved filesystem or route value. It is initialized from `safeSegment(fileId || path.parse(codeFileName).name, path.parse(codeFileName).name)`. Within the function it is passed into `resolveInsideCompileRoot`, `writeFile`; assigned or updated later in the function.

folder: `folder` stores a computed value for filesystem location. It is initialized from `resolveInsideCompileRoot(projectFolder, fileFolder)`. Within the function it is passed into `mkdir`; assigned or updated later in the function.

sourcePath: `sourcePath` stores a computed value for filesystem location. It is initialized from `resolveInsideCompileRoot(projectFolder, fileFolder, codeFileName)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `writeFile`; assigned or updated later in the function.

metadata: `metadata` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; passed into `stringify`; assigned or updated later in the function.

function javaMainClassName(code = "", fileName = "Main.java")

Read `javaMainClassName` first as a named idea, not as syntax. `javaMainClassName` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function javaMainClassName(code = "", fileName = "Main.java") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `code`, `fileName`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. code is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. fileName points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means javaMainClassName performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without javaMainClassName, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The local objects and variables inside javaMainClassName show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

source: source stores a computed value for temporary calculation. It is initialized from `String(code || "")`. Within the function it is accesses properties/methods match; assigned or updated later in the function.

publicMatch: publicMatch stores a computed value for temporary calculation. It is initialized from `source.match(/bpublic\s+class\s+([A-Za-z_$][\w$]*)/)`. Within the function it is returned to the caller; assigned or updated later in the function.

classMatch: classMatch stores a computed value for appearance configuration. It is initialized from `source.match(/bclass\s+([A-Za-z_$][\w$]*)/)`. Within the function it is returned to the caller; assigned or updated later in the function.

function compileCacheKey(parts = {})

Read compileCacheKey first as a named idea, not as syntax. compileCacheKey belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function compileCacheKey(parts = {}) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: parts. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. parts is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: serializes structured data before sending or saving it.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `compileCacheKey` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `compileCacheKey`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Inside `compileCacheKey`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function compileCacheDirectory(projectId, language, key)

Read `compileCacheDirectory` first as a named idea, not as syntax. `compileCacheDirectory` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function compileCacheDirectory(projectId, language, key) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `projectId`, `language`, `key`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. `key` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. It works with `resolveInsideCompileRoot`, `safeSegment`. Those calls show that `compileCacheDirectory` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `compileCacheDirectory`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Inside `compileCacheDirectory`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function hdlModuleNames(code = "")

Read `hdlModuleNames` first as a named idea, not as syntax. `hdlModuleNames` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function hdlModuleNames(code = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `code`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `hdlModuleNames` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `hdlModuleNames`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Inside `hdlModuleNames`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function hdlHasWaveDump(code = "")

To understand `hdlHasWaveDump`, start with the problem it owns. `hdlHasWaveDump` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function hdlHasWaveDump(code = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `code`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. code is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means hdlHasWaveDump performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without hdlHasWaveDump, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

After the main flow, the working values inside hdlHasWaveDump reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

source: source stores a computed value for temporary calculation. It is initialized from `String(code || "")`. Within the function it is passed into test; assigned or updated later in the function.

function hdlFilesFromPayload(payload = {}, activeFileName = "", activeLanguage = "verilog")

To understand hdlFilesFromPayload, start with the problem it owns. hdlFilesFromPayload belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function hdlFilesFromPayload(payload = {}, activeFileName = "", activeLanguage = "verilog") { ...
}
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `payload`, `activeFileName`, `activeLanguage`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `payload` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `activeFileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `activeLanguage` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and

returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Next, trace the helper calls that surround the function. It works with `normalizeCodeLanguage`, `detectCodeLanguageFromSource`, `isHdlLanguage`, `safeCodeFileName`, `safeSegment`, `clampText`, `normalizeCompileFileRole`, `inferCompileFileRole`. Those calls show that `hdlFilesFromPayload` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `hdlFilesFromPayload`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

After the main flow, the working values inside `hdlFilesFromPayload` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

incoming: `incoming` stores a computed value for temporary calculation. It is initialized from `Array.isArray(payload.workspaceFiles) ? payload.workspaceFiles : []`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

byId: `byId` stores a constructed object for lookup collection. It is initialized from `new Map()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `set`, `values`; assigned or updated later in the function.

addFile: `addFile` stores a function or callback value for temporary calculation. It is initialized from `(item = {}, fallbackId = "") => {}`. Within the function it is assigned or updated later in the function.

code: `code` stores a computed value for temporary calculation. It is initialized from `String(item.code || "")`. Within the function it accesses properties/methods `trim`; passed into `String`, `addFile`, `normalizeCodeLanguage`, `normalizeCompileFileRole`; assigned or updated later in the function.

language: `language` stores a computed value for compiler or language configuration. It is initialized from `normalizeCodeLanguage(item.language || detectCodeLanguageFromSource(code, item.fileName))`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `addFile`, `normalizeCodeLanguage`, `normalizeCompileFileRole`, `safeCodeFileName`; assigned or updated later in the function.

fileName: `fileName` stores a computed value for temporary calculation. It is initialized from `safeCodeFileName(item.fileName || activeFileName || compileLanguageProfiles[language].defaultFile, language)`. Within the function it accesses properties/methods `localeCompare`; passed into `addFile`, `localeCompare`, `normalizeCodeLanguage`, `normalizeCompileFileRole`, `safeCodeFileName`, `safeSegment`, `set`; assigned or updated later in the function.

id: `id` stores a resolved filesystem or route value. It is initialized from `safeSegment(item.id || fallbackId || fileName, path.parse(fileName).name)`. Within the function it is passed into `addFile`, `safeSegment`, `set`; assigned or updated later in the function.

function compileActionFromPayload(value = "", language = "")

To understand `compileActionFromPayload`, start with the problem it owns. `compileActionFromPayload` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function compileActionFromPayload(value = "", language = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`, `language`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `isHdlLanguage`. Those calls show that `compileActionFromPayload` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `compileActionFromPayload`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

After the main flow, the working values inside `compileActionFromPayload` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(value || "").trim().toLowerCase().replace(/[ _s-]+/g, "-")`. Within the function it is returned to the caller; assigned or updated later in the function.

function compileActionLabel(action = "run", language = "")

The best entry point for `compileActionLabel` is its responsibility in the surrounding workflow. `compileActionLabel` belongs to `compiler` and `ide` workspace. This function supports the `Compile Code` workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function compileActionLabel(action = "run", language = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `action`, `language`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `action` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets

the function fail softly or produce a predictable empty result when the caller omits that field. language names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Its connection to the rest of the file matters as much as its local code. It works with `isHdlLanguage`. Those calls show that `compileActionLabel` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `compileActionLabel`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Inside `compileActionLabel`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function compileWorkspaceFilesFromPayload(payload = {}, activeFileName = "", activeLanguage = "javascript")

To understand `compileWorkspaceFilesFromPayload`, start with the problem it owns.

`compileWorkspaceFilesFromPayload` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function compileWorkspaceFilesFromPayload(payload = {}, activeFileName = "", activeLanguage = "javascript") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `payload`, `activeFileName`, `activeLanguage`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `payload` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `activeFileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `activeLanguage` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Next, trace the helper calls that surround the function. It works with `normalizeCodeLanguage`, `detectCodeLanguageFromSource`, `safeCodeFileName`, `safeSegment`, `clampText`, `normalizeCompileFileRole`, `inferCompileFileRole`. Those calls show that `compileWorkspaceFilesFromPayload` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `compileWorkspaceFilesFromPayload`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

After the main flow, the working values inside `compileWorkspaceFilesFromPayload` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

incoming: `incoming` stores a computed value for temporary calculation. It is initialized from `Array.isArray(payload.workspaceFiles) ? payload.workspaceFiles : []`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

byId: `byId` stores a constructed object for lookup collection. It is initialized from `new Map()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `set`, `values`; assigned or updated later in the function.

addFile: `addFile` stores a function or callback value for temporary calculation. It is initialized from `(item = {}, fallbackId = "") => {}`. Within the function it is assigned or updated later in the function.

code: `code` stores a computed value for temporary calculation. It is initialized from `String(item.code || "")`. Within the function it accesses properties/methods `trim`; passed into `String`, `addFile`, `normalizeCodeLanguage`, `normalizeCompileFileRole`; assigned or updated later in the function.

language: `language` stores a computed value for compiler or language configuration. It is initialized from `normalizeCodeLanguage(item.language || detectCodeLanguageFromSource(code, item.fileName))`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `addFile`, `normalizeCodeLanguage`, `normalizeCompileFileRole`, `safeCodeFileName`; assigned or updated later in the function.

fileName: `fileName` stores a computed value for temporary calculation. It is initialized from `safeCodeFileName(item.fileName || activeFileName || compileLanguageProfiles[language]?.defaultFile || "source.txt", language)`. Within the function it accesses properties/methods `localeCompare`; passed into `addFile`, `localeCompare`, `normalizeCodeLanguage`, `normalizeCompileFileRole`, `safeCodeFileName`, `safeSegment`, `set`; assigned or updated later in the function.

id: `id` stores a resolved filesystem or route value. It is initialized from `safeSegment(item.id || fallbackId || fileName, path.parse(fileName).name)`. Within the function it is passed into `addFile`, `safeSegment`, `set`; assigned or updated later in the function.

async function writeCompileWorkspaceSources(files = [], targetDir = "", options = {})

The best entry point for `writeCompileWorkspaceSources` is its responsibility in the surrounding workflow. `writeCompileWorkspaceSources` writes `CompileWorkspaceSources` to a controlled destination. Write helpers matter because a wrong path, stale format, or missing directory can corrupt local drafts, compile workspaces, publishing state, or generated website files.

The syntax of the function is:

```
async function writeCompileWorkspaceSources(files = [], targetDir = "", options = {}) { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `files`, `targetDir`, `options`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `files` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `targetDir` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation.

Its connection to the rest of the file matters as much as its local code. It works with `normalizeCodeLanguage`. Those calls show that `writeCompileWorkspaceSources` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `writeCompileWorkspaceSources`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The easiest way to follow the body of `writeCompileWorkspaceSources` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

seen: `seen` stores a constructed object for lookup collection. It is initialized from `new Map()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `get`, `set`; assigned or updated later in the function.

written: `written` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `push`; assigned or updated later in the function.

file: `file` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `code`, `fileName`, `language`; passed into `get`, `parse`, `push`, `set`, `writeFile`.

parsed: `parsed` stores a resolved filesystem or route value. It is initialized from `path.parse(file.fileName)`. Within the function it is accesses properties/methods `ext`, `name`; assigned or updated later in the function.

count: `count` stores a computed value for temporary calculation. It is initialized from `seen.get(file.fileName) || 0`. Within the function it is passed into `set`; assigned or updated later in the function.

uniqueName: uniqueName stores a computed value for temporary calculation. It is initialized from `count ? `${parsed.name}_${count}${parsed.ext}` : file.fileName`. Within the function it is passed into `join`, `push`; assigned or updated later in the function.

sourcePath: sourcePath stores a resolved filesystem or route value. It is initialized from `path.join(targetDir, uniqueName)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `push`, `writeFile`; assigned or updated later in the function.

function sourceDisplayName(file = {})

Begin with the role of `sourceDisplayName`. `sourceDisplayName` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sourceDisplayName(file = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `file`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `file` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `sourceDisplayName` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `sourceDisplayName`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

Inside `sourceDisplayName`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function cFamilyWorkspaceSources(files = [], language = "c", activeFileName = "")

Begin with the role of `cFamilyWorkspaceSources`. `cFamilyWorkspaceSources` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function cFamilyWorkspaceSources(files = [], language = "c", activeFileName = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `files`, `language`, `activeFileName`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `files` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `activeFileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `cFamilyWorkspaceSources` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `cFamilyWorkspaceSources`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The local state inside `cFamilyWorkspaceSources` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

exts: `exts` stores a computed value for temporary calculation. It is initialized from `language === "cpp"`. Within the function it is accessed via `properties/methods` `has`; assigned or updated later in the function.

selected: `selected` stores a function or callback value for temporary calculation. It is initialized from `files.filter((file) => exts.has(path.extname(file.fileName).toLowerCase()))`. Within the function it is returned to the caller; mutated as a collection or DOM container; accessed via `properties/methods` `push`, `some`; assigned or updated later in the function.

async function writeHdlSimulationSources(files = [], cacheDir = "")

Begin with the role of `writeHdlSimulationSources`. `writeHdlSimulationSources` writes `HdlSimulationSources` to a controlled destination. Write helpers matter because a wrong path, stale format, or missing directory can corrupt local drafts, compile workspaces, publishing state, or generated website files.

The syntax of the function is:

```
async function writeHdlSimulationSources(files = [], cacheDir = "") { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `files`, `cacheDir`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `files` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `cacheDir` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `writeHdlSimulationSources` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `writeHdlSimulationSources`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The local state inside `writeHdlSimulationSources` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

sourceDir: `sourceDir` stores a resolved filesystem or route value. It is initialized from `path.join(cacheDir, "sources")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `join`, `mkdir`, `rm`; assigned or updated later in the function.

seen: `seen` stores a constructed object for lookup collection. It is initialized from `new Map()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `get`, `set`; assigned or updated later in the function.

written: `written` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `push`; assigned or updated later in the function.

file: `file` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `code`, `fileName`; passed into `get`, `parse`, `push`, `set`, `writeFile`.

parsed: `parsed` stores a resolved filesystem or route value. It is initialized from `path.parse(file.fileName)`. Within the function it is accesses properties/methods `ext`, `name`; assigned or updated later in the function.

count: `count` stores a computed value for temporary calculation. It is initialized from `seen.get(file.fileName) || 0`. Within the function it is passed into `set`; assigned or updated later in the function.

uniqueName: uniqueName stores a computed value for temporary calculation. It is initialized from `count ? `${parsed.name}_${count}${parsed.ext}` : file.fileName`. Within the function it is passed into `join`, `push`; assigned or updated later in the function.

sourcePath: sourcePath stores a resolved filesystem or route value. It is initialized from `path.join(sourceDir, uniqueName)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `push`, `writeFile`; assigned or updated later in the function.

function normalizeVcdValue(raw = "")

The best entry point for `normalizeVcdValue` is its responsibility in the surrounding workflow. `normalizeVcdValue` standardizes `VcdValue` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeVcdValue(raw = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `raw`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `raw` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `normalizeVcdValue` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `normalizeVcdValue`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The easiest way to follow the body of `normalizeVcdValue` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(raw || "").trim()`.

Within the function it is returned to the caller; accesses properties/methods `slice`, `toLowerCase`; assigned or updated later in the function.

function parseVcdScopeText(text = "", source = "waveform.vcd")

The best entry point for `parseVcdScopeText` is its responsibility in the surrounding workflow. `parseVcdScopeText` reads VCD waveform text from HDL simulation and converts it into signals, time points, values, and scope metadata that a frontend scope viewer can draw. VCD files are compact event logs, not friendly UI data. This

parser turns symbol changes over time into understandable signal histories. The function is essential for the SystemVerilog/Verilog simulator feature. A simulation that produces only text is useful, but a scope view requires structured waveform data. Without this parser, the scope icon would have no signal timeline to show.

The syntax of the function is:

```
function parseVcdScopeText(text = "", source = "waveform.vcd") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `text`, `source`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `text` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Its connection to the rest of the file matters as much as its local code. It works with `normalizeVcdValue`. Those calls show that `parseVcdScopeText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `parseVcdScopeText`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The easiest way to follow the body of `parseVcdScopeText` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

lines: `lines` stores a computed value for temporary calculation. It is initialized from `String(text || "").split(/\r?\n/)`. Within the function it is assigned or updated later in the function.

scopes: `scopes` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `pop`, `push`; passed into `set`; assigned or updated later in the function.

signalsByCode: `signalsByCode` stores a constructed object for lookup collection. It is initialized from `new Map()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `get`, `has`, `set`, `size`, `values`; assigned or updated later in the function.

timeScaleParts: `timeScaleParts` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

inTimescale: `inTimescale` stores a boolean value for temporary calculation. It is initialized from `false`. Within the function it is assigned or updated later in the function.

time: time stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is passed into max, push; assigned or updated later in the function.

maxTime: maxTime stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is passed into max; assigned or updated later in the function.

definitionDone: definitionDone stores a boolean value for temporary calculation. It is initialized from false. Within the function it is assigned or updated later in the function.

rawLine: rawLine starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods trim.

line: line stores a computed value for temporary calculation. It is initialized from rawLine.trim(). Within the function it is accesses properties/methods includes, match, replace, slice, startsWith; passed into Number, normalizeVcdValue, push; assigned or updated later in the function.

inline: inline stores a computed value for temporary calculation. It is initialized from line.replace("\$timescale", "").replace("\$end", "").trim(). Within the function it is passed into push; assigned or updated later in the function.

scopeMatch: scopeMatch stores a computed value for temporary calculation. It is initialized from line.match(/^\sscope\s+\S+\s+(\.+?)\s+\\$end\$/). Within the function it is passed into push; assigned or updated later in the function.

varMatch: varMatch stores a computed value for temporary calculation. It is initialized from line.match(/^\svar\s+\S+\s+(\d+)\s+(\S+)\s+(\.+?)\s+\\$end\$/). Within the function it is assigned or updated later in the function.

destructured [, width, code, reference]: destructured [, width, code, reference] stores a computed value for imported or unpacked API handles. It is initialized from varMatch. It unpacks API handles from another object, giving the rest of the file short direct names for those APIs.

cleanReference: cleanReference stores a computed value for temporary calculation. It is initialized from reference.replace(/s+\[([^\]]+)\]/, ""). Within the function it is passed into set; assigned or updated later in the function.

code: code stores a string value for temporary calculation. It is initialized from "". Within the function it is passed into get, set; assigned or updated later in the function.

value: value stores a string value for temporary calculation. It is initialized from "". Within the function it is passed into push; assigned or updated later in the function.

vectorMatch: vectorMatch stores a computed value for temporary calculation. It is initialized from line.match(/^(
[01xz_]+)\s+(\S+)\\$/i). Within the function it is passed into normalizeVcdValue; assigned or updated later in the function.

signal: signal stores a computed value for temporary calculation. It is initialized from signalsByCode.get(code). Within the function it is accesses properties/methods changes; passed into filter; assigned or updated later in the function.

previous: previous stores a computed value for temporary calculation. It is initialized from signal.changes[signal.changes.length - 1]. Within the function it is accesses properties/methods time, value; assigned or updated later in the function.

signals: signals stores a list for temporary calculation. It is initialized from [...signalsByCode.values()]. Within the function it is assigned or updated later in the function.

async function readHdlWaveform(cacheDir = "")

The best entry point for readHdlWaveform is its responsibility in the surrounding workflow. readHdlWaveform reads HdlWaveform from a request, file, cache, or generated artifact. It gives the caller parsed data rather than raw bytes or untrusted text, which keeps parsing rules centralized.

The syntax of the function is:

```
async function readHdlWaveform(cacheDir = "") { ... }
```

The async keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: cacheDir. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. cacheDir is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: reads data from disk as part of the operation.

Its connection to the rest of the file matters as much as its local code. It works with findFilesByExtension, parseVcdScopeText. Those calls show that readHdlWaveform is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without readHdlWaveform, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The easiest way to follow the body of readHdlWaveform is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

vcdFiles: vcdFiles stores the completed result of an awaited operation. It is initialized from await findFilesByExtension(cacheDir, ".vcd", 4). Within the function it is iterated or transformed as a collection; accesses properties/methods length, sort; assigned or updated later in the function.

selected: selected stores a function or callback value for temporary calculation. It is initialized from vcdFiles.sort((a, b) => a.length - b.length)[0]. Within the function it is passed into parseVcdScopeText, readFile; assigned or updated later in the function.

text: text stores the completed result of an awaited operation. It is initialized from await readFile(selected, "utf8"). Within the function it is accesses properties/methods length, slice; assigned or updated later in the function.

limited: limited stores a computed value for temporary calculation. It is initialized from `text.length > 6_000_000 ? text.slice(0, 6_000_000) : text`. Within the function it is passed into `parseVcdScopeText`; assigned or updated later in the function.

async function clearHdlWaveforms(cacheDir = "")

To understand `clearHdlWaveforms`, start with the problem it owns. `clearHdlWaveforms` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function clearHdlWaveforms(cacheDir = "") { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `cacheDir`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `cacheDir` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and reads or writes files. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Next, trace the helper calls that surround the function. It works with `findFilesByExtension`. Those calls show that `clearHdlWaveforms` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `clearHdlWaveforms`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

After the main flow, the working values inside `clearHdlWaveforms` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

vcdFiles: `vcdFiles` stores the completed result of an awaited operation. It is initialized from `await findFilesByExtension(cacheDir, ".vcd", 4)`. Within the function it is iterated or transformed as a collection; accesses properties/methods map; passed into `all`; assigned or updated later in the function.

async function compileAndRunCode(payload = {})

Begin with the role of `compileAndRunCode`. This is the central Compile Code brain. It receives the project id, active source file, language, action requested by the user, optional stdin, and any workspace files that belong to the project. From that one payload it decides whether the user is only beautifying code, saving code, compiling, building, running, simulating HDL, or reading waveform output. It is `async` because every serious action here touches the filesystem, discovers tools, writes temporary workspaces, runs external compiler processes, and waits for terminal output. The function is needed because the builder treats each project like a small IDE workspace. JavaScript, Python, Java, C, C++, Verilog, SystemVerilog, LTspice, and HTML all have different

compile/run rules. This function keeps those rules behind one API endpoint so the frontend can ask for a compile operation without knowing every compiler command. Without it, Compile Code would degrade into a text box with no reliable build, run, simulate, cache, or output behavior. Important internal helpers include `ensureRunSource`, `append`, and `missing`. `ensureRunSource` writes the active source into the compile workspace before a run. `append` converts command results into terminal-style output. `missing` returns a helpful response when a required tool is unavailable. Those small local functions matter because they keep the larger language branches readable.

The syntax of the function is:

```
async function compileAndRunCode(payload = {}) { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `payload`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `payload` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work, runs an external command and reads or writes files. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; delegates command execution to `runProcess` so timeout and terminal capture remain consistent; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation; reads data from disk as part of the operation.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `normalizeCodeLanguage`, `detectCodeLanguageFromSource`, `safeCodeFileName`, `saveCompileSource`, `safeSegment`, `resolveInsideCompileRoot`, `compileActionFromPayload`, `compileWorkspaceFilesFromPayload`. Those calls show that `compileAndRunCode` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `compileAndRunCode`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The local state inside `compileAndRunCode` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

requestedLanguage: `requestedLanguage` stores a computed value for compiler or language configuration. It is initialized from `String(payload.language || "").trim().toLowerCase()`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `normalizeCodeLanguage`; assigned or updated later in the function.

language: `language` stores a computed value for compiler or language configuration. It is initialized from `requestedLanguage && requestedLanguage !== "auto"`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into

String, cFamilyCompileProfile, cFamilyWorkspaceSources, compileActionFromPayload, compileCacheDirectory, compileCacheKey, compileWorkspaceFilesFromPayload, hdlFilesFromPayload; assigned or updated later in the function.

profile: profile stores a computed value for portfolio data state. It is initialized from compileLanguageProfiles[language] || compileLanguageProfiles.javascript. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods label; passed into push; assigned or updated later in the function.

fileName: fileName stores a computed value for temporary calculation. It is initialized from safeCodeFileName(payload.fileName, language). Within the function it is passed into cFamilyBinaryName, cFamilyCompileProfile, cFamilyWorkspaceSources, compileCacheKey, compileWorkspaceFilesFromPayload, detectCodeLanguageFromSource, hdlFilesFromPayload; assigned or updated later in the function.

saved: saved stores the completed result of an awaited operation. It is initialized from await saveCompileSource({ ...payload, language, fileName }). Within the function it is accesses properties/methods fileName, id, role, sourcePath, title; passed into cFamilyBinaryName, cFamilyCompileProfile, cFamilyWorkspaceSources, compileCacheKey, compileWorkspaceFilesFromPayload, hdlFilesFromPayload, javaMainClassName; assigned or updated later in the function.

projectFolder: projectFolder stores a computed value for filesystem location. It is initialized from safeSegment(payload.projectId, "project"). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into resolveInsideCompileRoot; assigned or updated later in the function.

sourcePath: sourcePath stores a computed value for filesystem location. It is initialized from resolveInsideCompileRoot(projectFolder, safeSegment(saved.id), saved.fileName). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into cp, push, runProcess; assigned or updated later in the function.

sourceCode: sourceCode stores a computed value for temporary calculation. It is initialized from String(payload.code || ""). Within the function it is passed into push, writeFile; assigned or updated later in the function.

action: action stores a computed value for temporary calculation. It is initialized from compileActionFromPayload(payload.action, language). Within the function it is passed into compileActionFromPayload, compileCacheKey, push; assigned or updated later in the function.

allWorkspaceFiles: allWorkspaceFiles stores a computed value for temporary calculation. It is initialized from compileWorkspaceFilesFromPayload(payload, saved.fileName, language). Within the function it is iterated or transformed as a collection; accesses properties/methods filter; passed into cFamilyWorkspaceSources, ensureRunSource, writeCompileWorkspaceSources; assigned or updated later in the function.

runDir: runDir stores a string value for filesystem location. It is initialized from "". It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into join, mkdir, runProcess, writeCompileWorkspaceSources; assigned or updated later in the function.

runSourcePath: runSourcePath stores a string value for filesystem location. It is initialized from "". It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods replace; passed into cp, push, runProcess; assigned or updated later in the function.

runWorkspaceSources: runWorkspaceSources stores a list for temporary calculation. It is initialized from []. Within the function it is mutated as a collection or DOM container; accesses properties/methods find, push; assigned or updated later in the function.

ensureRunSource: ensureRunSource stores a function or callback value for temporary calculation. It is initialized from async (options = {}) => {}. Within the function it is assigned or updated later in the function.

runId: runId stores a string value for temporary calculation. It is initialized from `\${Date.now()}-\${Math.random().toString(16).slice(2)}`. Within the function it is passed into resolveInsideCompileRoot; assigned or updated later in the function.

filesToWrite: filesToWrite stores a computed value for temporary calculation. It is initialized from Array.isArray(options.files) && options.files.length ? options.files : allWorkspaceFiles. Within the function it is passed into writeCompileWorkspaceSources; assigned or updated later in the function.

active: active stores a function or callback value for temporary calculation. It is initialized from runWorkspaceSources.find((file) => file.id === saved.id || file.fileName === saved.fileName). Within the function it is assigned or updated later in the function.

stdin: stdin stores a computed value for temporary calculation. It is initialized from String(payload.stdin || ""). Within the function it is passed into String; assigned or updated later in the function.

forceRebuild: forceRebuild stores a computed value for temporary calculation. It is initialized from payload.forceRebuild === true. Within the function it is assigned or updated later in the function.

terminal: terminal stores a list for temporary calculation. It is initialized from []. Within the function it is mutated as a collection or DOM container; accesses properties/methods join, push; assigned or updated later in the function.

append: append stores a function or callback value for temporary calculation. It is initialized from (label, result) => {}. Within the function it is assigned or updated later in the function.

missing: missing stores a function or callback value for temporary calculation. It is initialized from async (tools) => {}. Within the function it is assigned or updated later in the function.

found: found stores a structured object for temporary calculation. It is initialized from {}. Within the function it is accesses properties/methods iverilog, java, javac, vvp; passed into append, push, runProcess, toolVersionLine; assigned or updated later in the function.

tool: tool starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into filter, findTool.

missingTools: missingTools stores a function or callback value for compiler or language configuration. It is initialized from tools.filter((tool) => !found[tool]). It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is accesses properties/methods join, length; assigned or updated later in the function.

openTags: openTags stores a function or callback value for temporary calculation. It is initialized from (String(payload.code || "").match(/<([a-z][\w:-]*)b[^\>]*>/gi) || []).length. Within the function it is assigned or updated later in the function.

closeTags: closeTags stores a function or callback value for temporary calculation. It is initialized from (String(payload.code || "").match(/<\/([a-z][\w:-]*)>/gi) || []).length. Within the function it is assigned or updated later in the function.

ok: ok stores a computed value for temporary calculation. It is initialized from openTags >= closeTags. Within the function it is accesses properties/methods txt; passed into join, push; assigned or updated later in the function.

stdinOptions: stdinOptions stores a computed value for temporary calculation. It is initialized from stdin ? { input: stdin } : {}. Within the function it is passed into runProcess; assigned or updated later in the function.

ok: ok stores a boolean value for temporary calculation. It is initialized from false. Within the function it is accesses properties/methods txt; passed into join, push; assigned or updated later in the function.

node: node stores the completed result of an awaited operation. It is initialized from await findTool("node"). Within the function it is passed into append, findTool, push, runProcess; assigned or updated later in the function.

jsFiles: jsFiles stores a function or callback value for temporary calculation. It is initialized from allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "javascript"). Within the function it is accesses properties/methods length; passed into ensureRunSource; assigned or updated later in the function.

runSource: runSource stores the completed result of an awaited operation. It is initialized from await ensureRunSource({ files: jsFiles.length ? jsFiles : allWorkspaceFiles, languages: ["javascript"] }). Within the function it is accesses properties/methods runDir, runSourcePath, writtenSources; passed into runProcess; assigned or updated later in the function.

targets: targets stores a computed value for temporary calculation. It is initialized from action === "build". Within the function it is accesses properties/methods length; assigned or updated later in the function.

target: target starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods sourcePath; passed into runProcess, sourceDisplayName.

check: check stores a resolved filesystem or route value. It is initialized from await runProcess(node, ["--check", target.sourcePath], { cwd: runSource.runDir, timeoutMs: 15000 }). Within the function it is accesses properties/methods ok; passed into processTerminalText, push, runProcess, writeFile; assigned or updated later in the function.

run: run stores a resolved filesystem or route value. It is initialized from await runProcess(node, [runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs: 20000, ...stdinOptions }). Within the function it is accesses properties/methods ok; passed into cleanHdlSimulationOutput, push; assigned or updated later in the function.

python: python stores the completed result of an awaited operation. It is initialized from await findTool("python"). Within the function it is passed into append, ensureRunSource, findTool, push, runProcess; assigned or updated later in the function.

pyFiles: pyFiles stores a function or callback value for temporary calculation. It is initialized from allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "python"). Within the function it is accesses properties/methods length; passed into ensureRunSource; assigned or updated later in the function.

runSource: runSource stores the completed result of an awaited operation. It is initialized from await ensureRunSource({ files: pyFiles.length ? pyFiles : allWorkspaceFiles, languages: ["python"] }). Within the function it is accesses properties/methods runDir, runSourcePath, writtenSources; passed into runProcess; assigned or updated later in the function.

targets: targets stores a computed value for temporary calculation. It is initialized from action === "build". Within the function it is accesses properties/methods length; assigned or updated later in the function.

target: target starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods sourcePath; passed into runProcess, sourceDisplayName.

check: check stores a resolved filesystem or route value. It is initialized from await runProcess(python, ["-m", "py_compile", target.sourcePath], { cwd: runSource.runDir, timeoutMs: 15000 }). Within the function it is accesses properties/methods ok; passed into processTerminalText, push, runProcess, writeFile; assigned or updated later in the function.

run: run stores a resolved filesystem or route value. It is initialized from await runProcess(python, [runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs: 20000, ...stdinOptions }). Within the function it is accesses properties/methods ok; passed into cleanHdlSimulationOutput, push; assigned or updated later in the function.

cProfile: cProfile stores a computed value for portfolio data state. It is initialized from cFamilyCompileProfile(language, saved.fileName). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods compiler, flags, header, label, languageFlag, standard; passed into push, writeFile; assigned or updated later in the function.

toolName: toolName stores a computed value for compiler or language configuration. It is initialized from cProfile.compiler. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into missing, push, runProcess, toolVersionLine; assigned or updated later in the function.

destructured { found, missingTools }: destructured { found, missingTools } stores the completed result of an awaited operation. It is initialized from await missing([toolName]). It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output.

workspaceSources: workspaceSources stores a computed value for temporary calculation. It is initialized from cFamilyWorkspaceSources(allWorkspaceFiles, language, saved.fileName). Within the function it is iterated or transformed as a collection; accesses properties/methods filter; passed into writeCompileWorkspaceSources; assigned or updated later in the function.

buildTargets: buildTargets stores a computed value for temporary calculation. It is initialized from action === "build". Within the function it is iterated or transformed as a collection; accesses properties/methods map; passed into compileCacheKey; assigned or updated later in the function.

compilerVersion: compilerVersion stores the completed result of an awaited operation. It is initialized from await toolVersionLine(found[toolName]). It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into push; assigned or updated later in the function.

cacheKey: cacheKey stores a computed value for runtime state or cache. It is initialized from compileCacheKey({. Within the function it is passed into compileCacheDirectory; assigned or updated later in the function.

cacheDir: cacheDir stores a computed value for filesystem location. It is initialized from compileCacheDirectory(payload.projectId, language, cacheKey). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the

function it is passed into `clearHdlWaveforms`, `join`, `mkdir`, `readHdlWaveform`, `runProcess`, `writeHdlSimulationSources`; assigned or updated later in the function.

workspaceDir: `workspaceDir` stores a resolved filesystem or route value. It is initialized from `path.join(cacheDir, "sources")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `rm`, `runProcess`, `writeCompileWorkspaceSources`; assigned or updated later in the function.

writtenSources: `writtenSources` stores the completed result of an awaited operation. It is initialized from `await writeCompileWorkspaceSources(workspaceSources, workspaceDir)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `find`, `length`, `map`; passed into `processTerminalTextWithPaths`, `push`; assigned or updated later in the function.

activeWritten: `activeWritten` stores a function or callback value for temporary calculation. It is initialized from `writtenSources.find((file) => file.fileName === saved.fileName) || writtenSources[0]`. Within the function it is assigned or updated later in the function.

binaryName: `binaryName` stores a computed value for temporary calculation. It is initialized from `cFamilyBinaryName(saved.fileName)`. Within the function it is passed into `join`, `push`; assigned or updated later in the function.

output: `output` stores a resolved filesystem or route value. It is initialized from `path.join(cacheDir, binaryName)`. Within the function it is passed into `pathExists`, `push`, `runProcess`; assigned or updated later in the function.

outputMarker: `outputMarker` stores a resolved filesystem or route value. It is initialized from `path.join(cacheDir, "syntax-ok.txt")`. Within the function it is passed into `pathExists`, `push`, `writeFile`; assigned or updated later in the function.

cached: `cached` stores the completed result of an awaited operation. It is initialized from `!forceRebuild && await pathExists(output)`. Within the function it is assigned or updated later in the function.

cachedHeader: `cachedHeader` stores the completed result of an awaited operation. It is initialized from `cProfile.header && !forceRebuild && await pathExists(outputMarker)`. Within the function it is assigned or updated later in the function.

args: `args` stores a computed value for temporary calculation. It is initialized from `cProfile.header`. Within the function it is passed into `runProcess`; assigned or updated later in the function.

compile: `compile` stores the completed result of an awaited operation. It is initialized from `await runProcess(found[toolName], args, { cwd: workspaceDir, timeoutMs: 30000 })`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is accesses properties/methods `ok`; passed into `processTerminalTextWithPaths`, `push`; assigned or updated later in the function.

replacements: `replacements` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is passed into `processTerminalTextWithPaths`; assigned or updated later in the function.

toolPath: `toolPath` stores a computed value for filesystem location. It is initialized from `found[toolName]`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `runProcess`; assigned or updated later in the function.

run: `run` stores the completed result of an awaited operation. It is initialized from `await runProcess(output, [], {`. Within the function it is accesses properties/methods `ok`; passed into `cleanHdlSimulationOutput`, `push`; assigned or updated later in the function.

destructured { found, missingTools }: destructured { found, missingTools } stores the completed result of an awaited operation. It is initialized from `await missing(["javac", "java"])`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output.

className: className stores a computed value for appearance configuration. It is initialized from `javaMainClassName(payload.code, saved.fileName)`. Within the function it is passed into `join`, `runProcess`; assigned or updated later in the function.

javaFiles: javaFiles stores a function or callback value for temporary calculation. It is initialized from `allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "java")`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; passed into `compileCacheKey`, `writeCompileWorkspaceSources`; assigned or updated later in the function.

cacheKey: cacheKey stores a computed value for runtime state or cache. It is initialized from `compileCacheKey({}`. Within the function it is passed into `compileCacheDirectory`; assigned or updated later in the function.

cacheDir: cacheDir stores a computed value for filesystem location. It is initialized from `compileCacheDirectory(payload.projectId, language, cacheKey)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `clearHdlWaveforms`, `join`, `mkdir`, `readHdlWaveform`, `runProcess`, `writeHdlSimulationSources`; assigned or updated later in the function.

classPath: classPath stores a resolved filesystem or route value. It is initialized from `path.join(cacheDir, `${className}.class`)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `pathExists`, `push`; assigned or updated later in the function.

sourceDir: sourceDir stores a resolved filesystem or route value. It is initialized from `path.join(cacheDir, "sources")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `join`, `rm`, `runProcess`, `writeCompileWorkspaceSources`; assigned or updated later in the function.

writtenSources: writtenSources stores the completed result of an awaited operation. It is initialized from `await writeCompileWorkspaceSources(javaFiles.length ? javaFiles : allWorkspaceFiles, sourceDir, { languages: ["java"] })`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `find`, `length`, `map`; passed into `processTerminalTextWithPaths`, `push`; assigned or updated later in the function.

javaPath: javaPath stores a function or callback value for filesystem location. It is initialized from `writtenSources.find((file) => file.fileName === saved.fileName)?.sourcePath || path.join(sourceDir, `${className}.java`)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `basename`, `writeFile`; assigned or updated later in the function.

cached: cached stores a resolved filesystem or route value. It is initialized from `!forceRebuild && await pathExists(classPath)`. Within the function it is assigned or updated later in the function.

compileTargets: compileTargets stores a function or callback value for compiler or language configuration. It is initialized from `action === "build" ? writtenSources.map((file) => file.sourcePath) : [javaPath]`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `runProcess`; assigned or updated later in the function.

compile: compile stores the completed result of an awaited operation. It is initialized from `await runProcess(found.javac, ["-d", cacheDir, ...compileTargets], { cwd: sourceDir, timeoutMs: 30000 })`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is accesses properties/methods `ok`; passed into `processTerminalTextWithPaths`, `push`; assigned or updated later in the function.

run: run stores the completed result of an awaited operation. It is initialized from `await runProcess(found.java, ["-cp", cacheDir, className], { cwd: cacheDir, timeoutMs: 20000, ...stdinOptions })`. Within the function it is accesses properties/methods `ok`; passed into `cleanHdlSimulationOutput`, `push`; assigned or updated later in the function.

destructured { found, missingTools }: destructured { found, missingTools } stores the completed result of an awaited operation. It is initialized from `await missing(["iverilog", "vvp"])`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output.

hdlFiles: hdlFiles stores a computed value for temporary calculation. It is initialized from `hdlFilesFromPayload(payload, saved.fileName, language)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `map`, `some`; passed into `compileCacheKey`, `writeHdlSimulationSources`; assigned or updated later in the function.

testbenchFiles: testbenchFiles stores a function or callback value for temporary calculation. It is initialized from `hdlFiles.filter((file) => file.role === "testbench")`. Within the function it is accesses properties/methods `flatMap`, `length`, `some`; passed into `push`; assigned or updated later in the function.

usesSystemVerilog: usesSystemVerilog stores a function or callback value for temporary calculation. It is initialized from `hdlFiles.some((file) => normalizeCodeLanguage(file.language) === "systemverilog")`. Within the function it is passed into `compileCacheKey`; assigned or updated later in the function.

flags: flags stores a computed value for temporary calculation. It is initialized from `usesSystemVerilog ? ["-g2012", "-Wall"] : ["-g2005-sv", "-Wall"]`. Within the function it is accesses properties/methods `join`; assigned or updated later in the function.

topModule: topModule stores a function or callback value for lookup collection. It is initialized from `testbenchFiles.flatMap((file) => hdlModuleNames(file.code)).find(Boolean) || ""`. Within the function it is passed into `push`; assigned or updated later in the function.

cacheKey: cacheKey stores a computed value for runtime state or cache. It is initialized from `compileCacheKey({}`. Within the function it is passed into `compileCacheDirectory`; assigned or updated later in the function.

cacheDir: cacheDir stores a computed value for filesystem location. It is initialized from `compileCacheDirectory(payload.projectId, language, cacheKey)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `clearHdlWaveforms`, `join`, `mkdir`, `readHdlWaveform`, `runProcess`, `writeHdlSimulationSources`; assigned or updated later in the function.

output: output stores a resolved filesystem or route value. It is initialized from `path.join(cacheDir, "simulation.vvp")`. Within the function it is passed into `pathExists`, `push`, `runProcess`; assigned or updated later in the function.

sourceFiles: sourceFiles stores the completed result of an awaited operation. It is initialized from `await writeHdlSimulationSources(hdlFiles, cacheDir)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `map`; assigned or updated later in the function.

designCount: designCount stores a function or callback value for temporary calculation. It is initialized from `sourceFiles.filter((file) => file.role !== "testbench").length`. Within the function it is passed into `push`; assigned or updated later in the function.

cached: cached stores the completed result of an awaited operation. It is initialized from `!forceRebuild && await pathExists(output)`. Within the function it is assigned or updated later in the function.

compileArgs: compileArgs stores a list for compiler or language configuration. It is initialized from `[]`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `runProcess`; assigned or updated later in the function.

compile: compile stores the completed result of an awaited operation. It is initialized from `await runProcess(found.iverilog, compileArgs, { cwd: cacheDir, timeoutMs: 30000 })`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is accesses properties/methods `ok`; passed into `processTerminalTextWithPaths`, `push`; assigned or updated later in the function.

replacements: replacements stores a function or callback value for lookup collection. It is initialized from `sourceFiles.map((file) => ({ from: file.sourcePath, to: file.uniqueName }))`. Within the function it is passed into `processTerminalTextWithPaths`; assigned or updated later in the function.

waveform: waveform stores a computed value for temporary calculation. It is initialized from `null`. Within the function it is accesses properties/methods `maxTime`, `signals`, `source`, `timeScale`, `vcd`; passed into `Boolean`, `dumpfile`, `push`; assigned or updated later in the function.

run: run stores the completed result of an awaited operation. It is initialized from `await runProcess(found.vvp, [output], { cwd: cacheDir, timeoutMs: 20000 })`. Within the function it is accesses properties/methods `ok`; passed into `cleanHdlSimulationOutput`, `push`; assigned or updated later in the function.

Itspice: Itspice stores the completed result of an awaited operation. It is initialized from `await findTool("Itspice")`. Within the function it is passed into `append`, `findTool`, `runProcess`; assigned or updated later in the function.

runSource: runSource stores the completed result of an awaited operation. It is initialized from `await ensureRunSource()`. Within the function it is accesses properties/methods `runDir`, `runSourcePath`, `writtenSources`; passed into `runProcess`; assigned or updated later in the function.

run: run stores a resolved filesystem or route value. It is initialized from `await runProcess(Itspice, ["-b", runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs: 45000 })`. Within the function it is accesses properties/methods `ok`; passed into `cleanHdlSimulationOutput`, `push`; assigned or updated later in the function.

logPath: logPath stores a resolved filesystem or route value. It is initialized from `runSource.runSourcePath.replace(/\.([^.]+)$/, ".log")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `push`; assigned or updated later in the function.

async function installCompilerTools(language = "")

The best entry point for `installCompilerTools` is its responsibility in the surrounding workflow. `installCompilerTools` installs or prepares external tooling. It belongs in the backend because installation touches the machine, not just the browser page.

The syntax of the function is:

```
async function installCompilerTools(language = "") { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: language. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and runs an external command. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: delegates command execution to `runProcess` so timeout and terminal capture remain consistent.

Its connection to the rest of the file matters as much as its local code. It works with `normalizeCodeLanguage`, `findTool`, `runProcess`, `terminalLine`, `processTerminalText`, `compileToolStatus`. Those calls show that `installCompilerTools` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `installCompilerTools`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The easiest way to follow the body of `installCompilerTools` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

lang: `lang` stores a computed value for temporary calculation. It is initialized from `normalizeCodeLanguage(language)`. Within the function it is assigned or updated later in the function.

profile: `profile` stores a computed value for portfolio data state. It is initialized from `compileLanguageProfiles[lang] || compileLanguageProfiles.javascript`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `label`, `winget`; assigned or updated later in the function.

winget: `winget` stores the completed result of an awaited operation. It is initialized from `await findTool("winget") || "winget"`. Within the function it is passed into `findTool`, `push`, `runProcess`; assigned or updated later in the function.

terminal: `terminal` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

ok: `ok` stores a boolean value for temporary calculation. It is initialized from `true`. Within the function it is assigned or updated later in the function.

packageld: `packageld` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function.

args: `args` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is accesses properties/methods `join`; passed into `push`, `runProcess`; assigned or updated later in the function.

result: result stores the completed result of an awaited operation. It is initialized from `await runProcess(winget, args, { cwd: root, timeoutMs: 600000 })`. Within the function it is accessed properties/methods ok; passed into `processTerminalText`; assigned or updated later in the function.

function sourceLooksTextual(source = {})

Begin with the role of `sourceLooksTextual`. `sourceLooksTextual` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sourceLooksTextual(source = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `source`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `sourceLooksTextual` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `sourceLooksTextual`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The local state inside `sourceLooksTextual` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

url: url stores a computed value for network or routing value. It is initialized from `String(source.url || "")`. Within the function it is passed into `String`; assigned or updated later in the function.

kind: kind stores a computed value for temporary calculation. It is initialized from `String(source.kind || "").toLowerCase()`. Within the function it is passed into `String`; assigned or updated later in the function.

function sourceUrlAllowed(url)

Begin with the role of `sourceUrlAllowed`. `sourceUrlAllowed` belongs to compiler and ide workspace. This function supports the Compile Code workspace: detecting languages, writing files, finding tools, running compilers, parsing terminal output, or preparing simulation data. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sourceUrlAllowed(url) { ... }
```


There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `url`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `sourceUrlAllowed` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `sourceUrlAllowed`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The local state inside `sourceUrlAllowed` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

parsed: `parsed` stores a constructed object for temporary calculation. It is initialized from `new URL(url)`. Within the function it is accesses properties/methods `hostname`; assigned or updated later in the function.

hostName: `hostName` stores a computed value for temporary calculation. It is initialized from `parsed.hostname.toLowerCase()`. Within the function it is passed into `includes`; assigned or updated later in the function.

githubTextFilePattern: `githubTextFilePattern` stores a computed value for Git publishing and authorization state. It is initialized from `/(txt|md|markdown|csv|json|xml|log|c|h|cpp|hpp|cc|hh|py|js|mjs|ts|tsx|v|sv|vhdl?|spice|cir|net|asc|sch|kicad_sch|kicad_pcb|ino|xdc|sdc|tcl|yaml|yml)$/i`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is assigned or updated later in the function.

githubSkipFilePattern: `githubSkipFilePattern` stores a computed value for Git publishing and authorization state. It is initialized from `/(.png|jpe?g|gif|webp|svg|pdf|zip|7z|rar|exe|dll|bin|obj|o|a|so|dylib|mp4|mov|avi|mp3|wav|xls|x?|pptx?|docx?)/i`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is assigned or updated later in the function.

function parseGitHubSourceUrl(url)

Read `parseGitHubSourceUrl` first as a named idea, not as syntax. `parseGitHubSourceUrl` interprets `GitHubSourceUrl` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

The syntax of the function is:

```
function parseGitHubSourceUrl(url) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `url`. Read those names as the contract

between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `parseGitHubSourceUrl` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `parseGitHubSourceUrl`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The local objects and variables inside `parseGitHubSourceUrl` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

parsed: `parsed` stores a constructed object for temporary calculation. It is initialized from `new URL(url)`. Within the function it is accesses properties/methods `hostname`, `pathname`; assigned or updated later in the function.

host: `host` stores a computed value for temporary calculation. It is initialized from `parsed.hostname.toLowerCase()`. Within the function it is assigned or updated later in the function.

parts: `parts` stores a computed value for temporary calculation. It is initialized from `parsed.pathname.split("/").filter(Boolean)`. Within the function it is accesses properties/methods `length`, `slice`; assigned or updated later in the function.

base: `base` stores a structured object for temporary calculation. It is initialized from `{ owner: parts[0], repo: parts[1] }`. Within the function it is assigned or updated later in the function.

async function fetchGitHubProfileSource(source = {}, parsed = {})

The best entry point for `fetchGitHubProfileSource` is its responsibility in the surrounding workflow. `fetchGitHubProfileSource` retrieves `GitHubProfileSource` from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

The syntax of the function is:

```
async function fetchGitHubProfileSource(source = {}, parsed = {}) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `source`, `parsed`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `parsed` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Its connection to the rest of the file matters as much as its local code. It works with `fetchGitHubJson`, `wantsGitHubCode`, `scoreGitHubRepo`, `fetchGitHubRepositorySource`, `clampText`. Those calls show that `fetchGitHubProfileSource` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `fetchGitHubProfileSource`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The easiest way to follow the body of `fetchGitHubProfileSource` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

owner: `owner` stores a computed value for temporary calculation. It is initialized from `parsed.owner`. Within the function it is passed into `all`, `fetchGitHubJson`, `fetchGitHubRepositorySource`; assigned or updated later in the function.

destructured [profile, repos]: `destructured [profile, repos]` stores the completed result of an awaited operation. It is initialized from `await Promise.all([`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent.

repoList: `repoList` stores a computed value for temporary calculation. It is initialized from `Array.isArray(repos) ? repos : []`. Within the function it is accesses properties/methods `slice`; assigned or updated later in the function.

includeCode: `includeCode` stores a computed value for temporary calculation. It is initialized from `wantsGitHubCode(source.question || "")`. Within the function it is assigned or updated later in the function.

selectedRepos: `selectedRepos` stores a computed value for temporary calculation. It is initialized from `includeCode`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; assigned or updated later in the function.

repoBlocks: `repoBlocks` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `flatMap`, `push`; assigned or updated later in the function.

repo: `repo` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `default_branch`, `description`, `full_name`, `html_url`, `language`, `name`, `updated_at`; passed into `fetchGitHubRepositorySource`, `map`.

repoSource: repoSource stores the completed result of an awaited operation. It is initialized from await fetchGitHubRepositorySource({}). Within the function it is accesses properties/methods text; passed into push; assigned or updated later in the function.

lines: lines stores a list for temporary calculation. It is initialized from []. Within the function it is accesses properties/methods join; passed into clampText; assigned or updated later in the function.

async function fetchGitHubRepositorySource(source = {}, parsed = {})

Read fetchGitHubRepositorySource first as a named idea, not as syntax. fetchGitHubRepositorySource retrieves GitHubRepositorySource from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

The syntax of the function is:

```
async function fetchGitHubRepositorySource(source = {}, parsed = {}) { ... }
```

The async keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: source, parsed. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. source is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. parsed is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

The function also has to be read through the helpers it calls. It works with fetchGitHubJson, wantsGitHubCode, fetchLimitedText, clampText, scoreGitHubFile. Those calls show that fetchGitHubRepositorySource is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without fetchGitHubRepositorySource, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The local objects and variables inside fetchGitHubRepositorySource show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

destructured { owner, repo }: destructured { owner, repo } stores a computed value for imported or unpacked API handles. It is initialized from parsed. It unpacks API handles from another object, giving the rest of the file short direct names for those APIs.

metadata: metadata stores the completed result of an awaited operation. It is initialized from `await fetchGitHubJson(`https://api.github.com/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}`). Within the function it is accesses properties/methods default_branch, description, html_url, language; assigned or updated later in the function.`

branch: branch stores a computed value for Git publishing and authorization state. It is initialized from `parsed.branch || metadata?.default_branch || "main"`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `encodeURIComponent`, `fetchLimitedText`; assigned or updated later in the function.

question: question stores a computed value for temporary calculation. It is initialized from `source.question || ""`. Within the function it is passed into `Number`, `scoreGitHubFile`, `wantsGitHubCode`; assigned or updated later in the function.

requestedMaxCodeFiles: requestedMaxCodeFiles stores a computed value for temporary calculation. It is initialized from `Number(source.maxCodeFiles || (wantsGitHubCode(question) ? 6 : 3))`. Within the function it is passed into `max`; assigned or updated later in the function.

maxCodeFiles: maxCodeFiles stores a computed value for temporary calculation. It is initialized from `Math.max(1, Math.min(8, Number.isFinite(requestedMaxCodeFiles) ? requestedMaxCodeFiles : 3))`. Within the function it is passed into `Number`, `slice`; assigned or updated later in the function.

requestedFileTextLength: requestedFileTextLength stores a computed value for temporary calculation. It is initialized from `Number(source.maxFileTextLength || (wantsGitHubCode(question) ? 7000 : 3000))`. Within the function it is passed into `max`; assigned or updated later in the function.

maxFileTextLength: maxFileTextLength stores a computed value for temporary calculation. It is initialized from `Math.max(1000, Math.min(10000, Number.isFinite(requestedFileTextLength) ? requestedFileTextLength : 3000))`. Within the function it is passed into `Number`, `fetchLimitedText`; assigned or updated later in the function.

requestedRepoTextLength: requestedRepoTextLength stores a computed value for temporary calculation. It is initialized from `Number(source.maxRepositoryTextLength || (wantsGitHubCode(question) ? 22000 : 14000))`. Within the function it is passed into `max`; assigned or updated later in the function.

maxRepositoryTextLength: maxRepositoryTextLength stores a computed value for temporary calculation. It is initialized from `Math.max(5000, Math.min(26000, Number.isFinite(requestedRepoTextLength) ? requestedRepoTextLength : 14000))`. Within the function it is passed into `Number`; assigned or updated later in the function.

lines: lines stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `push`; passed into `clampText`; assigned or updated later in the function.

rawUrl: rawUrl stores a string value for network or routing value. It is initialized from ``https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${parsed.filePath}``. Within the function it is passed into `fetchLimitedText`; assigned or updated later in the function.

text: text stores the completed result of an awaited operation. It is initialized from `await fetchLimitedText(rawUrl, 14000)`. Within the function it is accesses properties/methods `trim`; passed into `push`; assigned or updated later in the function.

tree: tree stores the completed result of an awaited operation. It is initialized from `await fetchGitHubJson(`https://api.github.com/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}`). Within the function it is accesses properties/methods default_branch, description, html_url, language; assigned or updated later in the function.`

`{encodeURIComponent(repo)}/git/trees/${encodeURIComponent(branch)}?recursive=1``). Within the function it is accesses properties/methods `tree`; passed into `isArray`; assigned or updated later in the function.

files: `files` stores a computed value for temporary calculation. It is initialized from `Array.isArray(tree?.tree)`. Within the function it is accesses properties/methods `find`, `length`, `slice`; passed into `push`; assigned or updated later in the function.

readmePath: `readmePath` stores a function or callback value for filesystem location. It is initialized from `files.find((file) => /^(^|V)readme\.md$/i.test(file.path))?.path || "README.md"`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `fetchLimitedText`, `push`; assigned or updated later in the function.

readmeText: `readmeText` stores a function or callback value for temporary calculation. It is initialized from `await fetchLimitedText(`https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${readmePath}`, 9000).catch(() => "")`. Within the function it is accesses properties/methods `trim`; passed into `push`; assigned or updated later in the function.

includeCode: `includeCode` stores a computed value for temporary calculation. It is initialized from `wantsGitHubCode(question)`. Within the function it is assigned or updated later in the function.

selectedFiles: `selectedFiles` stores a computed value for temporary calculation. It is initialized from `files`. Within the function it is assigned or updated later in the function.

file: `file` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `path`; passed into `filter`, `find`, `map`, `push`, `test`.

rawUrl: `rawUrl` stores a string value for network or routing value. It is initialized from ``https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${file.path}``. Within the function it is passed into `fetchLimitedText`; assigned or updated later in the function.

fileText: `fileText` stores a function or callback value for temporary calculation. It is initialized from `await fetchLimitedText(rawUrl, maxFileTextLength).catch(() => "")`. Within the function it is accesses properties/methods `trim`; passed into `push`; assigned or updated later in the function.

async function fetchGitHubSourceText(source = {})

The best entry point for `fetchGitHubSourceText` is its responsibility in the surrounding workflow. `fetchGitHubSourceText` retrieves `GitHubSourceText` from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

The syntax of the function is:

```
async function fetchGitHubSourceText(source = {}) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `source`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Its connection to the rest of the file matters as much as its local code. It works with `parseGitHubSourceUrl`, `fetchGitHubProfileSource`, `fetchGitHubRepositorySource`. Those calls show that `fetchGitHubSourceText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `fetchGitHubSourceText`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The easiest way to follow the body of `fetchGitHubSourceText` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

parsed: `parsed` stores a computed value for temporary calculation. It is initialized from `parseGitHubSourceUrl(source.url || "")`. Within the function it is accessed properties/methods type; passed into `fetchGitHubProfileSource`, `fetchGitHubRepositorySource`; assigned or updated later in the function.

async function readLocalSourceText(url)

To understand `readLocalSourceText`, start with the problem it owns. `readLocalSourceText` reads `LocalSourceText` from a request, file, cache, or generated artifact. It gives the caller `parsed` data rather than raw bytes or untrusted text, which keeps parsing rules centralized.

The syntax of the function is:

```
async function readLocalSourceText(url) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `url`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and reads or writes files. It returns the awaited result of another asynchronous operation, so its caller receives the completed value rather than a pending `Promise`. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: reads data from disk as part of the operation.

Next, trace the helper calls that surround the function. It works with `resolveInsideRoot`. Those calls show that `readLocalSourceText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `readLocalSourceText`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

After the main flow, the working values inside `readLocalSourceText` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

parsed: `parsed` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `hostname`, `pathname`; passed into `decodeURIComponent`; assigned or updated later in the function.

localHosts: `localHosts` stores a constructed object for lookup collection. It is initialized from `new Set(["localhost", "127.0.0.1"])`. Within the function it is accesses properties/methods `has`; assigned or updated later in the function.

pathname: `pathname` stores a computed value for filesystem location. It is initialized from `decodeURIComponent(parsed.pathname.replace(/^V+/, ""))`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods `includes`, `replace`; passed into `decodeURIComponent`, `extname`, `readFile`; assigned or updated later in the function.

ext: `ext` stores a resolved filesystem or route value. It is initialized from `path.extname(pathname).toLowerCase()`. Within the function it is assigned or updated later in the function.

async function fetchSourceText(source = {})

Begin with the role of `fetchSourceText`. `fetchSourceText` retrieves `SourceText` from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

The syntax of the function is:

```
async function fetchSourceText(source = {}) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `source`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work and calls a network resource. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: performs a fetch and therefore must handle network delay and failure.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `sourceLooksTextual`, `sourceUriAllowed`, `fetchGitHubSourceText`, `readLocalSourceText`, `clampText`, `stripHtmlToText`. Those calls show that `fetchSourceText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `fetchSourceText`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

The local state inside `fetchSourceText` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

url: `url` stores a computed value for network or routing value. It is initialized from `String(source.url || "")`. Within the function it is passed into `String`, `fetch`, `readLocalSourceText`, `sourceUrlAllowed`; assigned or updated later in the function.

githubText: `githubText` stores the completed result of an awaited operation. It is initialized from `await fetchGitHubSourceText(source)`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is returned to the caller; assigned or updated later in the function.

localText: `localText` stores the completed result of an awaited operation. It is initialized from `await readLocalSourceText(url)`. Within the function it is accesses properties/methods `replace`, `trim`; passed into `clampText`; assigned or updated later in the function.

response: `response` stores data returned from a network call. It is initialized from `await fetch(url, {`. Within the function it is accesses properties/methods `headers`, `ok`, `text`; passed into `clampText`; assigned or updated later in the function.

contentType: `contentType` stores a computed value for temporary calculation. It is initialized from `response.headers.get("Content-Type") || ""`. Within the function it is passed into `test`; assigned or updated later in the function.

rawText: `rawText` stores the completed result of an awaited operation. It is initialized from `clampText(await response.text(), 12000)`. Within the function it is passed into `stripHtmlToText`; assigned or updated later in the function.

text: `text` stores a computed value for temporary calculation. It is initialized from `/html/i.test(contentType) ? stripHtmlToText(rawText) : rawText`. Within the function it is accesses properties/methods `trim`; passed into `clampText`, `fetch`; assigned or updated later in the function.

function resolveInsideCompileRoot(...segments)

To understand `resolveInsideCompileRoot`, start with the problem it owns. `resolveInsideCompileRoot` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

The syntax of the function is:

```
function resolveInsideCompileRoot(...segments) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `segments`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `segments` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; throws explicit errors when a required safety or compatibility condition fails.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `resolveInsideCompileRoot` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `resolveInsideCompileRoot`, the IDE-style compile workspace would become less reliable. The app might misidentify a language, lose a file role, fail to find a compiler, show confusing terminal output, skip waveform data, or run code in the wrong folder.

After the main flow, the working values inside `resolveInsideCompileRoot` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

target: target stores a resolved filesystem or route value. It is initialized from `path.resolve(compileRoot, ...segments)`. Within the function it is returned to the caller; passed into `relative`; assigned or updated later in the function.

relative: relative stores a resolved filesystem or route value. It is initialized from `path.relative(compileRoot, target)`. Within the function it is accesses properties/methods `startsWith`; passed into `isAbsolute`; assigned or updated later in the function.

Filesystem safety

The functions in this group all support filesystem safety. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function languageFromFileName(fileName = "", code = "")

To understand `languageFromFileName`, start with the problem it owns. `languageFromFileName` belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function languageFromFileName(fileName = "", code = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `fileName`, `code`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `fileName` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Next, trace the helper calls that surround the function. It works with `sourceLooksCpp`, `sourceLooksC`. Those calls show that `languageFromFileName` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `languageFromFileName`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

After the main flow, the working values inside `languageFromFileName` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

ext: `ext` stores a resolved filesystem or route value. It is initialized from `path.extname(String(fileName || "").toLowerCase())`. Within the function it is passed into `includes`; assigned or updated later in the function.

function `safeCodeFileName(value = "", language = "javascript")`

The best entry point for `safeCodeFileName` is its responsibility in the surrounding workflow. `safeCodeFileName` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

The syntax of the function is:

```
function safeCodeFileName(value = "", language = "javascript") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`, `language`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Its connection to the rest of the file matters as much as its local code. It works with `normalizeCodeLanguage`, `safeSegment`. Those calls show that `safeCodeFileName` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `safeCodeFileName`, the specific rule it owns would disappear from the named workflow.

Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The easiest way to follow the body of `safeCodeFileName` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

profile: `profile` stores a computed value for portfolio data state. It is initialized from `compileLanguageProfiles[normalizeCodeLanguage(language)] || compileLanguageProfiles.javascript`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `defaultFile`, `extensions`; assigned or updated later in the function.

fallback: `fallback` stores a computed value for temporary calculation. It is initialized from `profile.defaultFile`. Within the function it is passed into `String`, `extname`, `parse`, `safeSegment`; assigned or updated later in the function.

parsed: `parsed` stores a resolved filesystem or route value. It is initialized from `path.parse(String(value || fallback))`. Within the function it is accesses properties/methods `ext`, `name`; passed into `String`, `safeSegment`; assigned or updated later in the function.

safeName: `safeName` stores a resolved filesystem or route value. It is initialized from `safeSegment(parsed.name, path.parse(fallback).name)`. Within the function it is assigned or updated later in the function.

ext: `ext` stores a resolved filesystem or route value. It is initialized from `String(parsed.ext || path.extname(fallback)).toLowerCase()`. Within the function it is passed into `String`; assigned or updated later in the function.

function pathVariantsForReplacement(value = "")

Begin with the role of `pathVariantsForReplacement`. `pathVariantsForReplacement` belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function pathVariantsForReplacement(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `pathVariantsForReplacement` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `pathVariantsForReplacement`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `pathVariantsForReplacement` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(value || "")`. Within the function it is accessed; properties/methods `replace`; passed into `from`, `normalize`; assigned or updated later in the function.

function replacePathReferences(text = "", replacements = [])

The best entry point for `replacePathReferences` is its responsibility in the surrounding workflow. `replacePathReferences` belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function replacePathReferences(text = "", replacements = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `text`, `replacements`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `text` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `replacements` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. It works with `pathVariantsForReplacement`. Those calls show that `replacePathReferences` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `replacePathReferences`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The easiest way to follow the body of `replacePathReferences` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

output: `output` stores a computed value for temporary calculation. It is initialized from `String(text || "")`. Within the function it is returned to the caller; accesses properties/methods `split`; assigned or updated later in the function.

replacement: replacement starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function.

from: from stores a computed value for temporary calculation. It is initialized from `replacement?.from || ""`. Within the function it is assigned or updated later in the function.

to: to stores a computed value for temporary calculation. It is initialized from `replacement?.to || ""`. Within the function it is passed into join; assigned or updated later in the function.

variant: variant starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is passed into split.

async function findFilesByExtension(folder = "", extension = ".vcd", maxDepth = 3)

Begin with the role of `findFilesByExtension`. `findFilesByExtension` belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function findFilesByExtension(folder = "", extension = ".vcd", maxDepth = 3) { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `folder`, `extension`, `maxDepth`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `folder` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `extension` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `maxDepth` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work and reads or writes files. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `pathExists`. Those calls show that `findFilesByExtension` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `findFilesByExtension`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `findFilesByExtension` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

entries: `entries` stores a function or callback value for temporary calculation. It is initialized from `await readdir(folder, { withFileTypes: true }).catch(() => [])`. Within the function it is assigned or updated later in the function.

matches: `matches` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `push`; assigned or updated later in the function.

entry: `entry` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `isDirectory`, `isFile`, `name`; passed into `join`.

fullPath: `fullPath` stores a resolved filesystem or route value. It is initialized from `path.join(folder, entry.name)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `push`; assigned or updated later in the function.

async function readJsonFile(filePath)

Read `readJsonFile` first as a named idea, not as syntax. `readJsonFile` reads `JsonFile` from a request, file, cache, or generated artifact. It gives the caller parsed data rather than raw bytes or untrusted text, which keeps parsing rules centralized.

The syntax of the function is:

```
async function readJsonFile(filePath) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `filePath`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `filePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: parses JSON rather than treating incoming text as already trusted data; reads data from disk as part of the operation.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `readJsonFile` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `readJsonFile`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `readJsonFile`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function safeSegment(value, fallback = "item")

Read `safeSegment` first as a named idea, not as syntax. `safeSegment` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

The syntax of the function is:

```
function safeSegment(value, fallback = "item") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`, `fallback`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `fallback` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `safeSegment` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `safeSegment`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside `safeSegment` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

segment: segment stores a computed value for temporary calculation. It is initialized from `String(value || fallback)`. Within the function it is returned to the caller; assigned or updated later in the function.

function safeFileName(value)

Begin with the role of `safeFileName`. `safeFileName` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

The syntax of the function is:

```
function safeFileName(value) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract

between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `safeSegment`. Those calls show that `safeFileName` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `safeFileName`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `safeFileName` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

`parsed`: `parsed` stores a resolved filesystem or route value. It is initialized from `path.parse(String(value || "file"))`. Within the function it is accesses properties/methods `ext`, `name`; passed into `safeSegment`; assigned or updated later in the function.

`name`: `name` stores a computed value for temporary calculation. It is initialized from `safeSegment(parsed.name, "file")`. Within the function it is passed into `safeSegment`; assigned or updated later in the function.

`ext`: `ext` stores a computed value for temporary calculation. It is initialized from `safeSegment(parsed.ext.replace(".", ""), "")`. Within the function it is returned to the caller; accesses properties/methods `replace`; passed into `safeSegment`; assigned or updated later in the function.

function `resolveInsideRoot(...segments)`

Begin with the role of `resolveInsideRoot`. `resolveInsideRoot` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

The syntax of the function is:

```
function resolveInsideRoot(...segments) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `segments`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `segments` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value

used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; throws explicit errors when a required safety or compatibility condition fails.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `resolveInsideRoot` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `resolveInsideRoot`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `resolveInsideRoot` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

target: target stores a resolved filesystem or route value. It is initialized from `path.normalize(path.join(root, ...segments))`. Within the function it is returned to the caller; accesses properties/methods `startsWith`; assigned or updated later in the function.

function `resolveInsidePortfolioRoot(...segments)`

Read `resolveInsidePortfolioRoot` first as a named idea, not as syntax. `resolveInsidePortfolioRoot` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

The syntax of the function is:

```
function resolveInsidePortfolioRoot(...segments) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `segments`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `segments` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; throws explicit errors when a required safety or compatibility condition fails.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `resolveInsidePortfolioRoot` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `resolveInsidePortfolioRoot`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside `resolveInsidePortfolioRoot` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

target: `target` stores a resolved filesystem or route value. It is initialized from `path.resolve(portfolioRoot, ...segments)`. Within the function it is returned to the caller; passed into `relative`; assigned or updated later in the function.

relative: `relative` stores a resolved filesystem or route value. It is initialized from `path.relative(portfolioRoot, target)`. Within the function it accesses properties/methods `startsWith`; passed into `isAbsolute`; assigned or updated later in the function.

function `samePath(left, right)`

The best entry point for `samePath` is its responsibility in the surrounding workflow. `samePath` belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function samePath(left, right) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `left`, `right`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `left` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `right` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `samePath` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `samePath`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `samePath`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function `pathExists(filePath)`

To understand `pathExists`, start with the problem it owns. `pathExists` belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function pathExists(filePath) { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `filePath`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `filePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `pathExists` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `pathExists`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `pathExists`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`async function workspaceHasCompatibleSiteFiles(baseRoot = portfolioRoot)`

The best entry point for `workspaceHasCompatibleSiteFiles` is its responsibility in the surrounding workflow. `workspaceHasCompatibleSiteFiles` belongs to filesystem safety. This helper prevents unsafe names and paths from escaping the intended workspace, portfolio mirror, or compile workspace. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function workspaceHasCompatibleSiteFiles(baseRoot = portfolioRoot) { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `baseRoot`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `baseRoot` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands,

compiler runs, or model responses have finished. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; reads data from disk as part of the operation.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `workspaceHasCompatibleSiteFiles` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `workspaceHasCompatibleSiteFiles`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The easiest way to follow the body of `workspaceHasCompatibleSiteFiles` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

fileName: `fileName` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is passed into `readFile`.

Public source and AI context

The functions in this group all support public source and ai context. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function githubHeaders(accept = "application/vnd.github+json")

The best entry point for `githubHeaders` is its responsibility in the surrounding workflow. `githubHeaders` belongs to public source and ai context. This function helps the assistant read public sources, GitHub repositories, local site files, or source snippets so answers can be based on actual portfolio evidence. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function githubHeaders(accept = "application/vnd.github+json") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `accept`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `accept` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `githubHeaders` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `githubHeaders`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Inside `githubHeaders`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function fetchGitHubJson(url)

The best entry point for `fetchGitHubJson` is its responsibility in the surrounding workflow. `fetchGitHubJson` retrieves `GitHubJson` from outside the immediate function. It wraps network or repository access so callers do not need to know URL construction, headers, limits, or error behavior. This matters because external data is slow, failure-prone, and must be bounded before it is trusted.

The syntax of the function is:

```
async function fetchGitHubJson(url) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `url`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and calls a network resource. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: performs a fetch and therefore must handle network delay and failure.

Its connection to the rest of the file matters as much as its local code. It works with `githubHeaders`. Those calls show that `fetchGitHubJson` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `fetchGitHubJson`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The easiest way to follow the body of `fetchGitHubJson` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

response: response stores data returned from a network call. It is initialized from `await fetch(url, { headers: githubHeaders() })`. Within the function it is returned to the caller; accesses properties/methods `json`, `ok`; assigned or updated later in the function.

function githubQuestionTokens(question = "")

The best entry point for `githubQuestionTokens` is its responsibility in the surrounding workflow. `githubQuestionTokens` belongs to public source and ai context. This function helps the assistant read public sources, GitHub repositories, local site files, or source snippets so answers can be based on actual portfolio evidence. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function githubQuestionTokens(question = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `githubQuestionTokens` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `githubQuestionTokens`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Inside `githubQuestionTokens`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function scoreGitHubFile(filePath = "", question = "")

The best entry point for `scoreGitHubFile` is its responsibility in the surrounding workflow. `scoreGitHubFile` ranks a candidate against a question or search term. Scoring functions are what make search and AI source selection feel relevant instead of random.

The syntax of the function is:

```
function scoreGitHubFile(filePath = "", question = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `filePath`, `question`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `filePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. It works with `githubQuestionTokens`. Those calls show that `scoreGitHubFile` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `scoreGitHubFile`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The easiest way to follow the body of `scoreGitHubFile` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

cleanPath: `cleanPath` stores a resolved filesystem or route value. It is initialized from `filePath.toLowerCase()`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accessed; properties/methods are included; passed into tests; assigned or updated later in the function.

tokens: `tokens` stores a computed value for temporary calculation. It is initialized from `githubQuestionTokens(question)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

score: `score` stores a resolved filesystem or route value. It is initialized from `githubTextFilePattern.test(cleanPath) ? 10 : 0`. Within the function it is returned to the caller; assigned or updated later in the function.

function wantsGitHubCode(question = "")

The best entry point for `wantsGitHubCode` is its responsibility in the surrounding workflow. `wantsGitHubCode` belongs to public source and AI context. This function helps the assistant read public sources, GitHub repositories, local site files, or source snippets so answers can be based on actual portfolio evidence. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function wantsGitHubCode(question = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `wantsGitHubCode` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `wantsGitHubCode`, the assistant would either answer with less context, display unrelated

links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Inside `wantsGitHubCode`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `scoreGitHubRepo(repo = {}, question = "")`

To understand `scoreGitHubRepo`, start with the problem it owns. `scoreGitHubRepo` ranks a candidate against a question or search term. Scoring functions are what make search and AI source selection feel relevant instead of random.

The syntax of the function is:

```
function scoreGitHubRepo(repo = {}, question = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `repo`, `question`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `repo` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `githubQuestionTokens`. Those calls show that `scoreGitHubRepo` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `scoreGitHubRepo`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

After the main flow, the working values inside `scoreGitHubRepo` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

tokens: `tokens` stores a computed value for temporary calculation. It is initialized from `githubQuestionTokens(question)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

haystack: `haystack` stores a list for runtime state or cache. It is initialized from `[]`. Within the function it is accessed; properties/methods included; passed into `test`; assigned or updated later in the function.

score: `score` stores a computed value for temporary calculation. It is initialized from `repo.fork ? -10 : 12`. Within the function it is returned to the caller; assigned or updated later in the function.

async function enrichPortfolioContext(context = {})

Begin with the role of `enrichPortfolioContext`. `enrichPortfolioContext` belongs to public source and ai context. This function helps the assistant read public sources, GitHub repositories, local site files, or source snippets so answers can be based on actual portfolio evidence. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function enrichPortfolioContext(context = {}) { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `context`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `context` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `enrichPortfolioContext` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `enrichPortfolioContext`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local state inside `enrichPortfolioContext` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

question: `question` stores a computed value for temporary calculation. It is initialized from `String(context.question || "")`. Within the function it is passed into `String`; assigned or updated later in the function.

sources: `sources` stores a computed value for temporary calculation. It is initialized from `Array.isArray(context.sourceFetches)`. Within the function it is iterated or transformed as a collection; accesses properties/methods map; passed into `all`; assigned or updated later in the function.

sourceExcerpts: `sourceExcerpts` stores a function or callback value for lookup collection. It is initialized from `(await Promise.all(sources.map(fetchSourceText))).filter(Boolean)`. Within the function it is assigned or updated later in the function.

function parseGitHubRemote(remoteUrl = "")

Read `parseGitHubRemote` first as a named idea, not as syntax. `parseGitHubRemote` interprets `GitHubRemote` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

The syntax of the function is:

```
function parseGitHubRemote(remoteUrl = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `remoteUrl`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `remoteUrl` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `parseGitHubRemote` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `parseGitHubRemote`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local objects and variables inside `parseGitHubRemote` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

trimmed: `trimmed` stores a computed value for temporary calculation. It is initialized from `String(remoteUrl || "").trim()`. Within the function it is accesses properties/methods `match`, `replace`; passed into `URL`; assigned or updated later in the function.

sshMatch: `sshMatch` stores a computed value for temporary calculation. It is initialized from `trimmed.match(/^git@github\.com:([^\s]+)\.(\.+)?:\.git)?$/i)`. Within the function it is assigned or updated later in the function.

parsed: `parsed` stores a constructed object for temporary calculation. It is initialized from `new URL(trimmed.replace(/^git\+/, ""))`. Within the function it is accesses properties/methods `hostname`, `pathname`; assigned or updated later in the function.

parts: `parts` stores a computed value for temporary calculation. It is initialized from `parsed.pathname.replace(/^\/+/, "").split("/")`. Within the function it is accesses properties/methods `length`; assigned or updated later in the function.

async function authenticateGitHubForTarget(options = {})

Begin with the role of `authenticateGitHubForTarget`. `authenticateGitHubForTarget` is the interactive setup path for a publishing target. It validates the target remote, initializes or fixes the local Git repository when necessary, stores credentials when credentials are supplied, verifies write access, saves the target configuration, and records the authorization cache. It is the correct first step before `Load from target` or `Apply to site`. The function exists because merely typing a GitHub URL does not prove the builder can read from or write to that repository. Authentication must be completed, verified, and remembered before the target is considered usable.

The syntax of the function is:

```
async function authenticateGitHubForTarget(options = {}) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: options. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `validatePublishRemoteUrl`, `validateCustomDomain`, `validateCredentialPair`, `publishAccessError`, `getPublishTargetInfo`, `ensureGitRepository`, `capturePublishTargetState`, `setOriginRemote`. Those calls show that `authenticateGitHubForTarget` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `authenticateGitHubForTarget`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local state inside `authenticateGitHubForTarget` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

customDomainProvided: `customDomainProvided` stores a computed value for temporary calculation. It is initialized from `Object.prototype.hasOwnProperty.call(options, "customDomain")`. Within the function it is passed into `writeTargetCustomDomain`; assigned or updated later in the function.

remoteUrl: `remoteUrl` stores a computed value for network or routing value. It is initialized from `validatePublishRemoteUrl(repositoryUrl)`. Within the function it is passed into `detectRemoteDefaultBranch`, `setOriginRemote`, `storeGitCredentials`; assigned or updated later in the function.

domain: `domain` stores a computed value for temporary calculation. It is initialized from `validateCustomDomain(customDomain)`. Within the function it is passed into `writeTargetCustomDomain`; assigned or updated later in the function.

credentials: `credentials` stores a computed value for Git publishing and authorization state. It is initialized from `validateCredentialPair(authUsername, authPassword)`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods `cleanUsername`; assigned or updated later in the function.

snapshot: `snapshot` stores the completed result of an awaited operation. It is initialized from `await capturePublishTargetState()`. Within the function it is passed into `restorePublishTargetState`; assigned or updated later in the function.

status: `status` stores the completed result of an awaited operation. It is initialized from `await getGitStatus()`. Within the function it is accesses properties/methods `credentialManager`, `git`; passed into `publishAccessError`; assigned or updated later in the function.

preliminaryTarget: preliminaryTarget stores the completed result of an awaited operation. It is initialized from await getPublishTargetInfo(). Within the function it is passed into publishAccessError; assigned or updated later in the function.

targetBranch: targetBranch stores the completed result of an awaited operation. It is initialized from await detectRemoteDefaultBranch(remoteUrl) || "main". It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into checkoutPublishBranch; assigned or updated later in the function.

targetBeforeAuth: targetBeforeAuth stores the completed result of an awaited operation. It is initialized from await getPublishTargetInfo(). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods branch, remote, repository; passed into publishAccessError; assigned or updated later in the function.

cached: cached stores the completed result of an awaited operation. It is initialized from await readPublishAuthCache(). Within the function it is accesses properties/methods checkedAt, extendedTrust, successCountLastWeek, trustDays; passed into Boolean, publishAuthCacheExpiresAt; assigned or updated later in the function.

cachedAccess: cachedAccess stores a structured object for runtime state or cache. It is initialized from {. Within the function it is accesses properties/methods branch; assigned or updated later in the function.

target: target stores the completed result of an awaited operation. It is initialized from await getPublishTargetInfo(). Within the function it is accesses properties/methods branch; passed into publishAccessError; assigned or updated later in the function.

storedCredentials: storedCredentials stores the completed result of an awaited operation. It is initialized from await storeGitCredentials(remoteUrl, authUsername, authPassword). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods stored, username; assigned or updated later in the function.

access: access stores the completed result of an awaited operation. It is initialized from await assertPublishAccess({ force: true, requireCompatible: false }). Within the function it is accesses properties/methods branch; passed into publishAccessError; assigned or updated later in the function.

target: target stores the completed result of an awaited operation. It is initialized from await getPublishTargetInfo(). Within the function it is accesses properties/methods branch; passed into publishAccessError; assigned or updated later in the function.

AI backend

The functions in this group all support ai backend. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function cleanConversationHistory(value)

Begin with the role of cleanConversationHistory. cleanConversationHistory belongs to ai backend. This function is part of the portfolio AI path. It prepares messages, reads model responses, maintains conversation shape, or creates a fallback answer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function cleanConversationHistory(value) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: value. Read those names as the contract

between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `clampText`. Those calls show that `cleanConversationHistory` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `cleanConversationHistory`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Inside `cleanConversationHistory`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function extractOpenAiText(data)

Begin with the role of `extractOpenAiText`. `extractOpenAiText` belongs to ai backend. This function is part of the portfolio AI path. It prepares messages, reads model responses, maintains conversation shape, or creates a fallback answer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function extractOpenAiText(data) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `data`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `data` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `extractOpenAiText` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `extractOpenAiText`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local state inside `extractOpenAiText` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

output: output stores a computed value for temporary calculation. It is initialized from `Array.isArray(data?.output) ? data.output : []`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; passed into `isArray`; assigned or updated later in the function.

chunks: chunks stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

function extractOllamaText(data = {})

Read `extractOllamaText` first as a named idea, not as syntax. `extractOllamaText` belongs to ai backend. This function is part of the portfolio AI path. It prepares messages, reads model responses, maintains conversation shape, or creates a fallback answer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function extractOllamaText(data = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `data`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `data` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `extractOllamaText` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `extractOllamaText`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Inside `extractOllamaText`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch })

Read `callOllamaPortfolioAi` first as a named idea, not as syntax. `callOllamaPortfolioAi` belongs to ai backend. This function is part of the portfolio AI path. It prepares messages, reads model responses, maintains conversation shape, or creates a fallback answer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch })  
{ ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The braces in the parameter list mean the caller passes one object and the function pulls named fields out of it. In this case the important fields are `question`, `intent`, `conversation`, `context`, `allowWebSearch`. That style is useful when a function needs several related values but the call site should still be readable.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. `intent` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. `conversation` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `context` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. `allowWebSearch` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work, calls a network resource and uses a timer to avoid blocking or to wait for another process. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: serializes structured data before sending or saving it; performs a fetch and therefore must handle network delay and failure.

The function also has to be read through the helpers it calls. It works with `clampText`, `extractOllamaText`. Those calls show that `callOllamaPortfolioAi` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `callOllamaPortfolioAi`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local objects and variables inside `callOllamaPortfolioAi` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

host: `host` stores an environment-derived value for temporary calculation. It is initialized from `process.env.OLLAMA_HOST || "http://127.0.0.1:11434"`. Within the function it is accessed; properties/methods replaced; passed into `fetch`; assigned or updated later in the function.

model: `model` stores an environment-derived value for temporary calculation. It is initialized from `process.env.OLLAMA_MODEL || "llama3.2"`. Within the function it is passed into `stringify`; assigned or updated later in the function.

controller: `controller` stores a constructed object for temporary calculation. It is initialized from `new AbortController()`. Within the function it is accessed; properties/methods `abort`, `signal`; assigned or updated later in the function.

timeout: `timeout` stores a function or callback value for temporary calculation. It is initialized from `setTimeout(() => controller.abort(), Number(process.env.OLLAMA_TIMEOUT_MS || 45000))`. Within the function it is passed into `clearTimeout`; assigned or updated later in the function.

response: response stores data returned from a network call. It is initialized from `await fetch(`${host.replace(/V+$/, "")}/api/chat`, {`. Within the function it is accesses properties/methods `json`, `ok`, `status`; assigned or updated later in the function.

data: data stores a function or callback value for temporary calculation. It is initialized from `await response.json().catch(() => ({}))`. Within the function it is passed into `extractOllamaText`; assigned or updated later in the function.

answer: answer stores a computed value for temporary calculation. It is initialized from `extractOllamaText(data)`. Within the function it is returned to the caller; assigned or updated later in the function.

function ruleBasedConversationAnswer(question = "")

Read `ruleBasedConversationAnswer` first as a named idea, not as syntax. `ruleBasedConversationAnswer` belongs to ai backend. This function is part of the portfolio AI path. It prepares messages, reads model responses, maintains conversation shape, or creates a fallback answer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function ruleBasedConversationAnswer(question = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `ruleBasedConversationAnswer` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `ruleBasedConversationAnswer`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local objects and variables inside `ruleBasedConversationAnswer` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(question || "").toLowerCase()`. Within the function it is passed into `test`; assigned or updated later in the function.

async function handlePortfolioAi(request, response)

Begin with the role of `handlePortfolioAi`. `handlePortfolioAi` is the backend side of Ask My Portfolio. It reads the visitor's question, conversation history, intent, and context, enriches that context when public sources are

allowed, then chooses an AI provider path. It can call OpenAI when configured, call Ollama locally when available, or fall back to rule-based answers when no model is available. The function matters because the public script should not hold private API keys or perform privileged source fetching directly. It also gives the assistant one place to enforce prompt rules about portfolio-specific answers, general engineering answers, public source usage, and uncertainty.

The syntax of the function is:

```
async function handlePortfolioAi(request, response) { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `request`, `response`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `request` is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals. `response` is the Node HTTP object passed into the handler. The function reads request details or writes response details through this object instead of depending on browser globals.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work and calls a network resource. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: serializes structured data before sending or saving it; performs a fetch and therefore must handle network delay and failure.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `readRequestJson`, `clampText`, `cleanConversationHistory`, `sendJson`, `enrichPortfolioContext`, `callOllamaPortfolioAi`, `extractOpenAiText`. Those calls show that `handlePortfolioAi` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `handlePortfolioAi`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local state inside `handlePortfolioAi` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

body: body stores the completed result of an awaited operation. It is initialized from `await readRequestJson(request)`. Within the function it is accesses properties/methods `allowWebSearch`, `context`, `conversation`, `intent`, `question`; passed into `clampText`, `cleanConversationHistory`, `enrichPortfolioContext`, `fetch`, `has`; assigned or updated later in the function.

question: question stores a computed value for temporary calculation. It is initialized from `clampText(body.question, 1200).trim()`. Within the function it is passed into `callOllamaPortfolioAi`, `clampText`; assigned or updated later in the function.

conversation: conversation stores a computed value for temporary calculation. It is initialized from `cleanConversationHistory(body.conversation)`. Within the function it is passed into `callOllamaPortfolioAi`, `clampText`, `cleanConversationHistory`; assigned or updated later in the function.

validIntents: validIntents stores a constructed object for lookup collection. It is initialized from new Set(["general_conversation", "general_engineering", "general_knowledge", "portfolio_specific"]). Within the function it is accesses properties/methods has; assigned or updated later in the function.

intent: intent stores a computed value for temporary calculation. It is initialized from validIntents.has(body.intent) ? body.intent : "portfolio_specific". Within the function it is passed into callOllamaPortfolioAi, has; assigned or updated later in the function.

allowWebSearch: allowWebSearch stores a computed value for temporary calculation. It is initialized from body.allowWebSearch === true. Within the function it is passed into callOllamaPortfolioAi; assigned or updated later in the function.

context: context stores the completed result of an awaited operation. It is initialized from await enrichPortfolioContext({ ...(body.context || {}), question }). Within the function it is passed into callOllamaPortfolioAi, clampText, enrichPortfolioContext; assigned or updated later in the function.

apiKey: apiKey stores a environment-derived value for temporary calculation. It is initialized from process.env.OPENAI_API_KEY || "". Within the function it is passed into fetch; assigned or updated later in the function.

ollama: ollama stores the completed result of an awaited operation. It is initialized from await callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch }). It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Within the function it is accesses properties/methods answer, error, model, ok; passed into sendJson; assigned or updated later in the function.

model: model stores a environment-derived value for temporary calculation. It is initialized from process.env.OPENAI_MODEL || "gpt-5.4". Within the function it is passed into callModel, sendJson; assigned or updated later in the function.

fallbackModel: fallbackModel stores a environment-derived value for temporary calculation. It is initialized from process.env.OPENAI_FALLBACK_MODEL || "gpt-4.1". Within the function it is passed into callModel; assigned or updated later in the function.

webSearchMode: webSearchMode stores a environment-derived value for temporary calculation. It is initialized from String(process.env.OPENAI_ENABLE_WEB_SEARCH || "auto").toLowerCase(). Within the function it is assigned or updated later in the function.

enableWebSearch: enableWebSearch stores a computed value for temporary calculation. It is initialized from webSearchMode === "true" || (webSearchMode !== "false" && allowWebSearch). Within the function it is assigned or updated later in the function.

buildPayload: buildPayload stores a function or callback value for temporary calculation. It is initialized from (selectedModel) => {}. Within the function it is assigned or updated later in the function.

callModel: callModel stores a function or callback value for temporary calculation. It is initialized from async (selectedModel) => {}. Within the function it is assigned or updated later in the function.

payload: payload stores a computed value for temporary calculation. It is initialized from buildPayload(selectedModel). Within the function it is accesses properties/methods tools; passed into fetch; assigned or updated later in the function.

openAiResponse: openAiResponse stores data returned from a network call. It is initialized from await fetch("https://api.openai.com/v1/responses", {. It influences assistant behavior: conversation memory, source

display, model prompts, backend routing, or context selection. Within the function it is accesses properties/methods json, ok, status; passed into sendJson; assigned or updated later in the function.

data: data stores a function or callback value for temporary calculation. It is initialized from await openAiResponse.json().catch(() => ({})). Within the function it is passed into sendJson; assigned or updated later in the function.

destructured { data, openAiResponse, selectedModel }: destructured { data, openAiResponse, selectedModel } stores the completed result of an awaited operation. It is initialized from await callModel(model). It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection.

Publishing and GitHub authorization

The functions in this group all support publishing and github authorization. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function publishAccessError(message, details = "", extra = {})

The best entry point for publishAccessError is its responsibility in the surrounding workflow. publishAccessError belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function publishAccessError(message, details = "", extra = {}) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: message, details, extra. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. message is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. details is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. extra is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means publishAccessError performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without publishAccessError, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The easiest way to follow the body of `publishAccessError` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

error: error stores a constructed object for temporary calculation. It is initialized from `new Error(message)`. Within the function it is returned to the caller; accesses properties/methods code, details, `publishAccess`; assigned or updated later in the function.

function gitFailureText(error)

To understand `gitFailureText`, start with the problem it owns. `gitFailureText` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function gitFailureText(error) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `error`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `error` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `gitFailureText` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `gitFailureText`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Inside `gitFailureText`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function remoteUrlForDisplay(remoteUrl = "")

Begin with the role of `remoteUrlForDisplay`. `remoteUrlForDisplay` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function remoteUrlForDisplay(remoteUrl = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `remoteUrl`. Read those names as the contract

between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `remoteUrl` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `remoteUrlForDisplay` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `remoteUrlForDisplay`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Inside `remoteUrlForDisplay`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function validatePublishRemoteUrl(value = "")

Begin with the role of `validatePublishRemoteUrl`. `validatePublishRemoteUrl` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

The syntax of the function is:

```
function validatePublishRemoteUrl(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `validatePublishRemoteUrl` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `validatePublishRemoteUrl`, the publishing flow would lose one guardrail or one mechanical step. Depending on

the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The local state inside `validatePublishRemoteUrl` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

remoteUrl: `remoteUrl` stores a computed value for network or routing value. It is initialized from `String(value || "").trim()`. Within the function it is returned to the caller; passed into `URL`, `test`; assigned or updated later in the function.

parsed: `parsed` stores a constructed object for temporary calculation. It is initialized from `new URL(remoteUrl)`. Within the function it is accesses properties/methods `hostname`, `pathname`, `protocol`; assigned or updated later in the function.

parts: `parts` stores a computed value for temporary calculation. It is initialized from `parsed.pathname.replace(/^V+/, "").split("/")`. Within the function it is accesses properties/methods `length`; assigned or updated later in the function.

function validateCustomDomain(value = "")

To understand `validateCustomDomain`, start with the problem it owns. `validateCustomDomain` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

The syntax of the function is:

```
function validateCustomDomain(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `validateCustomDomain` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `validateCustomDomain`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

After the main flow, the working values inside `validateCustomDomain` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

domain: domain stores a computed value for temporary calculation. It is initialized from `String(value || "").trim().toLowerCase()`. Within the function it is returned to the caller; passed into `Error`, `test`; assigned or updated later in the function.

async function bumpPublishedAssetVersions(baseRoot = portfolioRoot)

To understand `bumpPublishedAssetVersions`, start with the problem it owns. `bumpPublishedAssetVersions` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function bumpPublishedAssetVersions(baseRoot = portfolioRoot) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `baseRoot`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `baseRoot` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; writes data to disk as part of the operation; reads data from disk as part of the operation.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `bumpPublishedAssetVersions` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `bumpPublishedAssetVersions`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

After the main flow, the working values inside `bumpPublishedAssetVersions` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

html: `html` stores a string value for temporary calculation. It is initialized from `""`. Within the function it is passed into `join`; assigned or updated later in the function.

indexPath: `indexPath` stores a resolved filesystem or route value. It is initialized from `path.join(baseRoot, "index.html")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `readFile`, `writeFile`; assigned or updated later in the function.

version: `version` stores a computed value for temporary calculation. It is initialized from `Date.now().toString()`. Within the function it is assigned or updated later in the function.

next: next stores a computed value for temporary calculation. It is initialized from `html`. Within the function it is passed into `writeFile`; assigned or updated later in the function.

async function getPublishTargetInfo()

Read `getPublishTargetInfo` first as a named idea, not as syntax. `getPublishTargetInfo` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function getPublishTargetInfo() { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: reads data from disk as part of the operation.

The function also has to be read through the helpers it calls. It works with `samePath`, `workspaceHasCompatibleSiteFiles`, `runPublishGit`, `remoteUrlForDisplay`, `parseGitHubRemote`, `resolveInsidePortfolioRoot`, `readPublishAuthCache`, `publishAuthCachelsFresh`. Those calls show that `getPublishTargetInfo` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `getPublishTargetInfo`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The local objects and variables inside `getPublishTargetInfo` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

info: `info` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; accesses properties/methods `authorizationCached`, `authorizationChecked`, `branch`, `checkedAt`, `customDomain`, `expiresAt`, `extendedTrust`, `gitBacked`; passed into `parseGitHubRemote`; assigned or updated later in the function.

parsedRemote: `parsedRemote` stores a computed value for network or routing value. It is initialized from `parseGitHubRemote(info.remote)`. Within the function it is accesses properties/methods `owner`, `repo`; assigned or updated later in the function.

accessShape: `accessShape` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is assigned or updated later in the function.

cached: cached stores the completed result of an awaited operation. It is initialized from `await readPublishAuthCache()`. Within the function it is accesses properties/methods `checkedAt`, `extendedTrust`, `successCountLastWeek`, `trustDays`; passed into `Boolean`, `publishAuthCacheExpiresAt`; assigned or updated later in the function.

async function runGit(args, options = {})

To understand `runGit`, start with the problem it owns. `runGit` executes an operation with side effects. It is separated from callers so process execution, Git execution, optional command behavior, or status collection remains consistent.

The syntax of the function is:

```
async function runGit(args, options = {}) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `args`, `options`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `args` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and runs an external command. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `runGit` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `runGit`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

After the main flow, the working values inside `runGit` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

lastError: `lastError` stores a computed value for temporary calculation. It is initialized from `null`. Within the function it is assigned or updated later in the function.

candidate: `candidate` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is passed into `execFileAsync`.

result: `result` stores the completed result of an awaited operation. It is initialized from `await execFileAsync(candidate, args, {`. Within the function it is assigned or updated later in the function.

async function runPublishGit(args, options = {})

Begin with the role of runPublishGit. runPublishGit executes an operation with side effects. It is separated from callers so process execution, Git execution, optional command behavior, or status collection remains consistent.

The syntax of the function is:

```
async function runPublishGit(args, options = {}) { ... }
```

The async keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: args, options. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. args is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

The surrounding workflow becomes clearer when you notice its collaborators. It works with runGit. Those calls show that runPublishGit is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without runPublishGit, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Inside runPublishGit, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function getGitStatus()

Begin with the role of getGitStatus. getGitStatus belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function getGitStatus() { ... }
```

The async keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `runGit`. Those calls show that `getGitStatus` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `getGitStatus`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The local state inside `getGitStatus` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

git: `git` stores a structured object for Git publishing and authorization state. It is initialized from `{ ok: false, stdout: "", stderr: "", error: "Git for Windows was not found." }`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accessed via `properties/methods` `error`, `ok`, `stderr`, `stdout`; assigned or updated later in the function.

gitResult: `gitResult` stores the completed result of an awaited operation. It is initialized from `await runGit(["--version"])`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accessed via `properties/methods` `stderr`, `stdout`; assigned or updated later in the function.

credentialManager: `credentialManager` stores a structured object for Git publishing and authorization state. It is initialized from `{ ok: false, version: "", error: "Git Credential Manager was not found." }`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is assigned or updated later in the function.

gcm: `gcm` stores a structured object for temporary calculation. It is initialized from `{ ok: false, stdout: "", stderr: "", error: "Git Credential Manager was not found." }`. Within the function it is accessed via `properties/methods` `error`, `ok`, `stderr`, `stdout`; assigned or updated later in the function.

gcmResult: `gcmResult` stores the completed result of an awaited operation. It is initialized from `await runGit(["credential-manager", "--version"])`. Within the function it is accessed via `properties/methods` `stderr`, `stdout`; assigned or updated later in the function.

async function publishPathIsStageable(relativePath)

Begin with the role of `publishPathIsStageable`. `publishPathIsStageable` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function publishPathIsStageable(relativePath) { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide:

relativePath. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. relativePath points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

The surrounding workflow becomes clearer when you notice its collaborators. It works with resolveInsidePortfolioRoot, runPublishGit. Those calls show that publishPathsStageable is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without publishPathsStageable, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Inside publishPathsStageable, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function stageablePublishPaths(pathsToCheck)

The best entry point for stageablePublishPaths is its responsibility in the surrounding workflow. stageablePublishPaths belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function stageablePublishPaths(pathsToCheck) { ... }
```

The async keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: pathsToCheck. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. pathsToCheck points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Its connection to the rest of the file matters as much as its local code. It works with publishPathsStageable. Those calls show that stageablePublishPaths is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `stageablePublishPaths`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The easiest way to follow the body of `stageablePublishPaths` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

stageable: `stageable` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods push; assigned or updated later in the function.

relativePath: `relativePath` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into push.

async function runGitWithInput(args, input, options = {})

To understand `runGitWithInput`, start with the problem it owns. `runGitWithInput` executes an operation with side effects. It is separated from callers so process execution, Git execution, optional command behavior, or status collection remains consistent.

The syntax of the function is:

```
async function runGitWithInput(args, input, options = {}) { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `args`, `input`, `options`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `args` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `input` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and runs an external command. It returns the awaited result of another asynchronous operation, so its caller receives the completed value rather than a pending Promise. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `runGitWithInput` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `runGitWithInput`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

After the main flow, the working values inside `runGitWithInput` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

lastError: `lastError` stores a computed value for temporary calculation. It is initialized from `null`. Within the function it is assigned or updated later in the function.

candidate: `candidate` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is passed into `resolve`, `spawn`.

child: `child` stores a computed value for temporary calculation. It is initialized from `spawn(candidate, args, {`. Within the function it is accesses properties/methods `once`, `stderr`, `stdin`, `stdout`; assigned or updated later in the function.

stdout: `stdout` stores a string value for temporary calculation. It is initialized from `""`. Within the function it is accesses properties/methods `on`; passed into `Error`, `resolve`; assigned or updated later in the function.

stderr: `stderr` stores a string value for temporary calculation. It is initialized from `""`. Within the function it is accesses properties/methods `on`; passed into `Error`, `resolve`; assigned or updated later in the function.

error: `error` stores a constructed object for temporary calculation. It is initialized from `new Error(stderr || stdout || `Git exited with code ${code}.`)`. Within the function it is accesses properties/methods `code`, `git`, `stderr`, `stdout`; passed into `catch`, `once`, `reject`; assigned or updated later in the function.

async function storeGitCredentials(remoteUrl, username, password)

The best entry point for `storeGitCredentials` is its responsibility in the surrounding workflow. `storeGitCredentials` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function storeGitCredentials(remoteUrl, username, password) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `remoteUrl`, `username`, `password`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `remoteUrl` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. `username` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `password` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

Its connection to the rest of the file matters as much as its local code. It works with `runPublishGit`, `runGitWithInput`. Those calls show that `storeGitCredentials` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `storeGitCredentials`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The easiest way to follow the body of `storeGitCredentials` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

cleanUsername: `cleanUsername` stores a computed value for temporary calculation. It is initialized from `String(username || "").trim()`. Within the function it is assigned or updated later in the function.

cleanPassword: `cleanPassword` stores a computed value for temporary calculation. It is initialized from `String(password || "")`. Within the function it is assigned or updated later in the function.

parsed: `parsed` stores a computed value for temporary calculation. It is initialized from `null`. Within the function it accesses properties/methods `hostname`, `pathname`, `protocol`; assigned or updated later in the function.

pathName: `pathName` stores a computed value for filesystem location. It is initialized from `parsed.pathname.replace(/^V+/, "")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is assigned or updated later in the function.

credentialInput: `credentialInput` stores a list for Git publishing and authorization state. It is initialized from `[]`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `runGitWithInput`; assigned or updated later in the function.

function validateCredentialPair(username, password)

To understand `validateCredentialPair`, start with the problem it owns. `validateCredentialPair` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

The syntax of the function is:

```
function validateCredentialPair(username, password) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `username`, `password`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `username` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `password` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses

instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `validateCredentialPair` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `validateCredentialPair`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

After the main flow, the working values inside `validateCredentialPair` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

cleanUsername: `cleanUsername` stores a computed value for temporary calculation. It is initialized from `String(username || "").trim()`. Within the function it is assigned or updated later in the function.

cleanPassword: `cleanPassword` stores a computed value for temporary calculation. It is initialized from `String(password || "").trim()`. Within the function it is assigned or updated later in the function.

function normalizeGitBranchName(value = "")

Begin with the role of `normalizeGitBranchName`. `normalizeGitBranchName` standardizes `GitBranchName` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeGitBranchName(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `normalizeGitBranchName` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `normalizeGitBranchName`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The local state inside `normalizeGitBranchName` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

branch: branch stores a computed value for Git publishing and authorization state. It is initialized from `String(value || "").trim()`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is returned to the caller; accesses properties/methods `endsWith`, `includes`, `length`, `startsWith`; passed into `test`; assigned or updated later in the function.

async function detectRemoteDefaultBranch(remoteUrl = "")

Begin with the role of `detectRemoteDefaultBranch`. `detectRemoteDefaultBranch` makes an educated classification from incomplete evidence. It looks at names, extensions, source text, repository state, or runtime state and returns the app's best decision so the next operation can choose the correct path.

The syntax of the function is:

```
async function detectRemoteDefaultBranch(remoteUrl = "") { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `remoteUrl`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `remoteUrl` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `runPublishGit`, `normalizeGitBranchName`. Those calls show that `detectRemoteDefaultBranch` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `detectRemoteDefaultBranch`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The local state inside `detectRemoteDefaultBranch` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

result: result stores the completed result of an awaited operation. It is initialized from `await runPublishGit(["is-remote", "--symref", remoteUrl, "HEAD"], { timeout: 45000 })`. Within the function it is accesses properties/methods `stdout`; passed into `String`; assigned or updated later in the function.

match: match stores a computed value for temporary calculation. It is initialized from `String(result.stdout || "").match(/^ref:\s+refs\heads\(.+\)\s+HEAD$/m)`. Within the function it is passed into `normalizeGitBranchName`; assigned or updated later in the function.

async function originRemoteExists()

Begin with the role of `originRemoteExists`. `originRemoteExists` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function originRemoteExists() { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `runPublishGit`. Those calls show that `originRemoteExists` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `originRemoteExists`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Inside `originRemoteExists`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function setOriginRemote(remoteUrl)

Begin with the role of `setOriginRemote`. `setOriginRemote` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function setOriginRemote(remoteUrl) { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `remoteUrl`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `remoteUrl` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `originRemoteExists`, `runPublishGit`. Those calls show that `setOriginRemote` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `setOriginRemote`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Inside `setOriginRemote`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`async function checkoutPublishBranch(branch)`

The best entry point for `checkoutPublishBranch` is its responsibility in the surrounding workflow. `checkoutPublishBranch` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function checkoutPublishBranch(branch) { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `branch`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `branch` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Its connection to the rest of the file matters as much as its local code. It works with `normalizeGitBranchName`, `runPublishGit`. Those calls show that `checkoutPublishBranch` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `checkoutPublishBranch`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The easiest way to follow the body of `checkoutPublishBranch` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

cleanBranch: cleanBranch stores a computed value for Git publishing and authorization state. It is initialized from `normalizeGitBranchName(branch) || "main"`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is returned to the caller; passed into `runPublishGit`; assigned or updated later in the function.

current: current stores a function or callback value for temporary calculation. It is initialized from `(await runPublishGit(["branch", "--show-current"]).stdout.trim())`. Within the function it is passed into `runPublishGit`; assigned or updated later in the function.

async function verifyPublishWriteAccess(access = {})

To understand `verifyPublishWriteAccess`, start with the problem it owns. `verifyPublishWriteAccess` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function verifyPublishWriteAccess(access = {}) { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `access`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `access` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

Next, trace the helper calls that surround the function. It works with `safeSegment`, `runPublishGit`, `publishAccessError`, `gitFailureText`. Those calls show that `verifyPublishWriteAccess` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `verifyPublishWriteAccess`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

After the main flow, the working values inside `verifyPublishWriteAccess` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

checkBranch: checkBranch stores a string value for Git publishing and authorization state. It is initialized from ``omb-auth-check-${safeSegment(os.hostname(), "device")}-${Date.now()}``. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `runPublishGit`; assigned or updated later in the function.

async function ensurePublishHeadForWriteCheck()

The best entry point for `ensurePublishHeadForWriteCheck` is its responsibility in the surrounding workflow. `ensurePublishHeadForWriteCheck` creates or repairs something only when it is needed. This pattern avoids duplicate DOM nodes, duplicate repositories, duplicate handlers, or missing folders while still letting later code assume the resource exists.

The syntax of the function is:

```
async function ensurePublishHeadForWriteCheck() { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Its connection to the rest of the file matters as much as its local code. It works with `runPublishGit`. Those calls show that `ensurePublishHeadForWriteCheck` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `ensurePublishHeadForWriteCheck`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Inside `ensurePublishHeadForWriteCheck`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function synchronizePublishBranchFromRemote(branch, access = {})

To understand `synchronizePublishBranchFromRemote`, start with the problem it owns.

`synchronizePublishBranchFromRemote` reconciles two states that can drift apart. In this project, sync functions connect local drafts, route state, Git branches, publish mirrors, or frontend state to the current truth.

The syntax of the function is:

```
async function synchronizePublishBranchFromRemote(branch, access = {}) { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `branch`, `access`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `branch` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `access` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer,

more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

Next, trace the helper calls that surround the function. It works with `normalizeGitBranchName`, `runPublishGit`, `checkoutPublishBranch`, `publishAccessError`, `gitFailureText`. Those calls show that `synchronizePublishBranchFromRemote` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `synchronizePublishBranchFromRemote`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

After the main flow, the working values inside `synchronizePublishBranchFromRemote` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

cleanBranch: `cleanBranch` stores a computed value for Git publishing and authorization state. It is initialized from `normalizeGitBranchName(branch) || "main"`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `checkoutPublishBranch`, `runPublishGit`; assigned or updated later in the function.

remoteBranch: `remoteBranch` stores the completed result of an awaited operation. It is initialized from `await runPublishGit(["ls-remote", "--heads", "origin", cleanBranch], {`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods `stdout`; assigned or updated later in the function.

async function capturePublishTargetState()

The best entry point for `capturePublishTargetState` is its responsibility in the surrounding workflow. `capturePublishTargetState` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function capturePublishTargetState() { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller.

The important point is that the caller does not repeat this rule elsewhere. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: reads data from disk as part of the operation.

Its connection to the rest of the file matters as much as its local code. It works with `runPublishGit`, `resolveInsidePortfolioRoot`. Those calls show that `capturePublishTargetState` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `capturePublishTargetState`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The easiest way to follow the body of `capturePublishTargetState` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

snapshot: snapshot stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; accesses properties/methods `branch`, `customDomainExists`, `customDomainText`, `remote`, `remoteExists`; assigned or updated later in the function.

async function restorePublishTargetState(snapshot = {})

The best entry point for `restorePublishTargetState` is its responsibility in the surrounding workflow. `restorePublishTargetState` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function restorePublishTargetState(snapshot = {}) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `snapshot`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `snapshot` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and reads or writes files. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: writes data to disk as part of the operation.

Its connection to the rest of the file matters as much as its local code. It works with `setOriginRemote`, `originRemoteExists`, `runPublishGit`, `checkoutPublishBranch`, `resolveInsidePortfolioRoot`. Those calls show that `restorePublishTargetState` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `restorePublishTargetState`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Inside `restorePublishTargetState`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function writeTargetCustomDomain(domain, customDomainProvided)

To understand `writeTargetCustomDomain`, start with the problem it owns. `writeTargetCustomDomain` writes `TargetCustomDomain` to a controlled destination. Write helpers matter because a wrong path, stale format, or missing directory can corrupt local drafts, compile workspaces, publishing state, or generated website files.

The syntax of the function is:

```
async function writeTargetCustomDomain(domain, customDomainProvided) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `domain`, `customDomainProvided`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `domain` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `customDomainProvided` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and reads or writes files. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: writes data to disk as part of the operation.

Next, trace the helper calls that surround the function. It works with `resolveInsidePortfolioRoot`, `samePath`, `resolveInsideRoot`. Those calls show that `writeTargetCustomDomain` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `writeTargetCustomDomain`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

After the main flow, the working values inside `writeTargetCustomDomain` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

targetPaths: `targetPaths` stores a list for filesystem location. It is initialized from `[resolveInsidePortfolioRoot("CNAME")]`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is mutated

as a collection or DOM container; accesses properties/methods push; assigned or updated later in the function.

targetPath: targetPath starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into rm, writeFile.

targetPath: targetPath starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into rm, writeFile.

async function ensureGitRepository()

Read ensureGitRepository first as a named idea, not as syntax. ensureGitRepository creates or repairs something only when it is needed. This pattern avoids duplicate DOM nodes, duplicate repositories, duplicate handlers, or missing folders while still letting later code assume the resource exists.

The syntax of the function is:

```
async function ensureGitRepository() { ... }
```

The async keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work and reads or writes files. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: creates folders before writing files so missing directories do not crash the workflow.

The function also has to be read through the helpers it calls. It works with runPublishGit. Those calls show that ensureGitRepository is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without ensureGitRepository, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The local objects and variables inside ensureGitRepository show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

insideWorkTree: insideWorkTree stores a function or callback value for temporary calculation. It is initialized from (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim(). Within the function it is assigned or updated later in the function.

branch: branch stores a function or callback value for Git publishing and authorization state. It is initialized from `(await runPublishGit(["branch", "--show-current"])).stdout.trim()`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `runPublishGit`; assigned or updated later in the function.

async function configurePublishTarget(options = {})

Begin with the role of `configurePublishTarget`. `configurePublishTarget` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function configurePublishTarget(options = {}) { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `options`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `validatePublishRemoteUrl`, `validateCustomDomain`, `validateCredentialPair`, `ensureGitRepository`, `setOriginRemote`, `detectRemoteDefaultBranch`, `checkoutPublishBranch`, `writeTargetCustomDomain`. Those calls show that `configurePublishTarget` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `configurePublishTarget`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The local state inside `configurePublishTarget` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

customDomainProvided: `customDomainProvided` stores a computed value for temporary calculation. It is initialized from `Object.prototype.hasOwnProperty.call(options, "customDomain")`. Within the function it is passed into `writeTargetCustomDomain`; assigned or updated later in the function.

remoteUrl: `remoteUrl` stores a computed value for network or routing value. It is initialized from `validatePublishRemoteUrl(repositoryUrl)`. Within the function it is passed into `detectRemoteDefaultBranch`, `setOriginRemote`; assigned or updated later in the function.

domain: `domain` stores a computed value for temporary calculation. It is initialized from `validateCustomDomain(customDomain)`. Within the function it is passed into `writeTargetCustomDomain`; assigned or updated later in the function.

defaultBranch: defaultBranch stores the completed result of an awaited operation. It is initialized from await detectRemoteDefaultBranch(remoteUrl). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into checkoutPublishBranch; assigned or updated later in the function.

currentRemote: currentRemote stores the completed result of an awaited operation. It is initialized from remoteUrl || (await getPublishTargetInfo()).remote. Within the function it is passed into storeGitCredentials; assigned or updated later in the function.

credentials: credentials stores the completed result of an awaited operation. It is initialized from await storeGitCredentials(currentRemote, authUsername, authPassword). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods stored, username; assigned or updated later in the function.

target: target stores the completed result of an awaited operation. It is initialized from await getPublishTargetInfo(). Within the function it is assigned or updated later in the function.

async function readPublishAuthCache()

To understand readPublishAuthCache, start with the problem it owns. readPublishAuthCache reads PublishAuthCache from a request, file, cache, or generated artifact. It gives the caller parsed data rather than raw bytes or untrusted text, which keeps parsing rules centralized.

The syntax of the function is:

```
async function readPublishAuthCache() { ... }
```

The async keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: parses JSON rather than treating incoming text as already trusted data; reads data from disk as part of the operation.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means readPublishAuthCache performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without readPublishAuthCache, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

Inside readPublishAuthCache, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function writePublishAuthCache(access = {})

Begin with the role of writePublishAuthCache. writePublishAuthCache writes PublishAuthCache to a controlled destination. Write helpers matter because a wrong path, stale format, or missing directory can corrupt local drafts, compile workspaces, publishing state, or generated website files.

The syntax of the function is:

```
async function writePublishAuthCache(access = {}) { ... }
```

The async keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: access. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. access is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work and reads or writes files. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: serializes structured data before sending or saving it; writes data to disk as part of the operation.

The surrounding workflow becomes clearer when you notice its collaborators. It works with readPublishAuthCache, parsePublishAuthTimestamp, publishAuthCacheScope. Those calls show that writePublishAuthCache is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without writePublishAuthCache, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The local state inside writePublishAuthCache explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

previousCache: previousCache stores the completed result of an awaited operation. It is initialized from await readPublishAuthCache(). Within the function it is accesses properties/methods branch, remote, repository, successHistory; passed into isArray; assigned or updated later in the function.

checkedAt: checkedAt stores a constructed object for temporary calculation. It is initialized from new Date(). Within the function it is accesses properties/methods getTime, toISOString; assigned or updated later in the function.

checkedAtMs: checkedAtMs stores a computed value for temporary calculation. It is initialized from checkedAt.getTime(). Within the function it is passed into Date; assigned or updated later in the function.

sameTarget: sameTarget stores a computed value for temporary calculation. It is initialized from previousCache. Within the function it is assigned or updated later in the function.

previousHistory: previousHistory stores a computed value for runtime state or cache. It is initialized from sameTarget && Array.isArray(previousCache.successHistory). Within the function it is assigned or updated later in the function.

successTimestamps: successTimestamps stores a list for temporary calculation. It is initialized from []. Within the function it is iterated or transformed as a collection; accesses properties/methods map; assigned or updated later in the function.

successHistory: successHistory stores a function or callback value for runtime state or cache. It is initialized from successTimestamps.map((timestamp) => new Date(timestamp).toISOString()). Within the function it is passed into isArray; assigned or updated later in the function.

successCountLastWeek: successCountLastWeek stores a computed value for temporary calculation. It is initialized from successTimestamps. Within the function it is assigned or updated later in the function.

extendedTrust: extendedTrust stores a computed value for temporary calculation. It is initialized from successCountLastWeek > publishAuthExtendedThreshold. Within the function it is assigned or updated later in the function.

ttlMs: ttlMs stores a computed value for temporary calculation. It is initialized from extendedTrust ? publishAuthExtendedTtlMs : publishAuthCacheTtlMs. Within the function it is passed into Date; assigned or updated later in the function.

cache: cache stores a structured object for runtime state or cache. It is initialized from {}. Within the function it is returned to the caller; passed into writeFile; assigned or updated later in the function.

function publishAuthCacheScope()

The best entry point for publishAuthCacheScope is its responsibility in the surrounding workflow. publishAuthCacheScope belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function publishAuthCacheScope() { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means publishAuthCacheScope performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `publishAuthCacheScope`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The easiest way to follow the body of `publishAuthCacheScope` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

username: `username` stores a environment-derived value for temporary calculation. It is initialized from `process.env.USERNAME || process.env.USER || ""`. Within the function it is assigned or updated later in the function.

function parsePublishAuthTimestamp(value = "")

Begin with the role of `parsePublishAuthTimestamp`. `parsePublishAuthTimestamp` interprets `PublishAuthTimestamp` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

The syntax of the function is:

```
function parsePublishAuthTimestamp(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `parsePublishAuthTimestamp` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `parsePublishAuthTimestamp`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The local state inside `parsePublishAuthTimestamp` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

direct: `direct` stores a computed value for filesystem location. It is initialized from `Date.parse(String(value || ""))`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is returned to the caller; assigned or updated later in the function.

normalized: `normalized` stores a computed value for temporary calculation. It is initialized from `String(value || "").replace(/(\\.\d{3})\d+(Z[+-]\d{2}:\d{2})$/i, "$1$2")`. Within the function it is passed into `parse`; assigned or updated later in the function.

function publishAuthCacheExpiresAt(cache)

Read `publishAuthCacheExpiresAt` first as a named idea, not as syntax. `publishAuthCacheExpiresAt` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function publishAuthCacheExpiresAt(cache) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `cache`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `cache` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The function also has to be read through the helpers it calls. It works with `parsePublishAuthTimestamp`. Those calls show that `publishAuthCacheExpiresAt` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `publishAuthCacheExpiresAt`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The local objects and variables inside `publishAuthCacheExpiresAt` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

explicitExpiresAt: `explicitExpiresAt` stores a computed value for temporary calculation. It is initialized from `parsePublishAuthTimestamp(cache?.expiresAt || "")`. Within the function it is passed into `Date`; assigned or updated later in the function.

checkedAt: `checkedAt` stores a computed value for temporary calculation. It is initialized from `parsePublishAuthTimestamp(cache?.checkedAt || "")`. Within the function it is passed into `Date`, `parsePublishAuthTimestamp`; assigned or updated later in the function.

function publishAuthCachelsFresh(cache, access)

To understand `publishAuthCachelsFresh`, start with the problem it owns. `publishAuthCachelsFresh` belongs to publishing and github authorization. This function protects and performs website publishing. It validates Git remotes, checks branches, caches authorization, stages approved files, or imports the target site. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function publishAuthCacheIsFresh(cache, access) { ... }
```


There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `cache`, `access`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `cache` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `access` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `publishAuthCacheScope`, `publishAuthCacheExpiresAt`. Those calls show that `publishAuthCachelsFresh` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `publishAuthCachelsFresh`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

After the main flow, the working values inside `publishAuthCachelsFresh` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

expiresAt: `expiresAt` stores a computed value for temporary calculation. It is initialized from `Date.parse(publishAuthCacheExpiresAt(cache))`. Within the function it is assigned or updated later in the function.

async function assertPublishAccess(options = {})

The best entry point for `assertPublishAccess` is its responsibility in the surrounding workflow. `assertPublishAccess` is the gatekeeper that decides whether the current machine and target are allowed to change the website. It checks the configured publishing target, compares cached authorization against the current remote and branch, respects the once-per-day authentication window, and can force a fresh verification when needed. It returns the access object that publishing functions use as proof. Without this function, Apply to site would either ask for GitHub authorization too often or push too freely. The cache logic is what makes the app usable day to day while still preventing an unauthenticated installation from silently modifying Maurice's website.

The syntax of the function is:

```
async function assertPublishAccess(options = {}) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `options`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state

together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

Its connection to the rest of the file matters as much as its local code. It works with `runPublishGit`, `publishAccessError`, `gitFailureText`, `remoteUrlForDisplay`, `workspaceHasCompatibleSiteFiles`, `parseGitHubRemote`, `readPublishAuthCache`, `publishAuthCachesFresh`. Those calls show that `assertPublishAccess` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assertPublishAccess`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The easiest way to follow the body of `assertPublishAccess` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

insideWorkTree: `insideWorkTree` stores a string value for temporary calculation. It is initialized from `""`. Within the function it is assigned or updated later in the function.

remote: `remote` stores a string value for network or routing value. It is initialized from `""`. Within the function it is passed into `parseGitHubRemote`, `publishAccessError`, `remoteUrlForDisplay`, `runPublishGit`; assigned or updated later in the function.

branch: `branch` stores a string value for Git publishing and authorization state. It is initialized from `""`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `publishAccessError`, `runPublishGit`, `synchronizePublishBranchFromRemote`; assigned or updated later in the function.

parsedRemote: `parsedRemote` stores a computed value for network or routing value. It is initialized from `parseGitHubRemote(remote)`. Within the function it is accesses properties/methods `owner`, `repo`; assigned or updated later in the function.

repository: `repository` stores a computed value for temporary calculation. It is initialized from `parsedRemote ? `${parsedRemote.owner}/${parsedRemote.repo}` : remoteUrlForDisplay(remote)`. Within the function it is passed into `publishAccessError`; assigned or updated later in the function.

access: `access` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is passed into `synchronizePublishBranchFromRemote`, `verifyPublishWriteAccess`, `writePublishAuthCache`; assigned or updated later in the function.

cached: `cached` stores the completed result of an awaited operation. It is initialized from `await readPublishAuthCache()`. Within the function it is accesses properties/methods `checkedAt`, `extendedTrust`, `successCountLastWeek`, `trustDays`; passed into `Boolean`, `publishAuthCacheExpiresAt`; assigned or updated later in the function.

remoteSync: remoteSync stores the completed result of an awaited operation. It is initialized from await synchronizePublishBranchFromRemote(branch, access). Within the function it is assigned or updated later in the function.

localHead: localHead stores the completed result of an awaited operation. It is initialized from await ensurePublishHeadForWriteCheck(). Within the function it is assigned or updated later in the function.

writeCheck: writeCheck stores the completed result of an awaited operation. It is initialized from await verifyPublishWriteAccess(access). Within the function it is assigned or updated later in the function.

authCache: authCache stores the completed result of an awaited operation. It is initialized from await writePublishAuthCache(access). It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it accesses properties/methods checkedAt, expiresAt, extendedTrust, successCountLastWeek, trustDays; passed into Boolean; assigned or updated later in the function.

async function syncFromPublishTarget()

The best entry point for syncFromPublishTarget is its responsibility in the surrounding workflow. syncFromPublishTarget is the import path. After a target is authenticated, this function pulls or fetches the latest public portfolio files from the selected repository and copies compatible website assets back into the local builder workspace. It is what lets a fresh machine reconstruct the current portfolio state from the website repository. This function is deliberately separate from authentication. Authenticate target proves access. Load from target reads the target. Keeping those concepts separate makes the UI clearer and reduces accidental overwrites.

The syntax of the function is:

```
async function syncFromPublishTarget() { ... }
```

The async keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and reads or writes files. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; throws explicit errors when a required safety or compatibility condition fails; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation; reads data from disk as part of the operation.

Its connection to the rest of the file matters as much as its local code. It works with assertPublishAccess, runPublishGit, runGit, publishAccessError, resolveInsideRoot, syncPortfolioPublishFiles. Those calls show that syncFromPublishTarget is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without syncFromPublishTarget, the publishing flow would lose one guardrail or one mechanical

step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

The easiest way to follow the body of `syncFromPublishTarget` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

publishAccess: `publishAccess` stores the completed result of an awaited operation. It is initialized from `await assertPublishAccess({ requireCompatible: false })`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods `branch`; passed into `publishAccessError`; assigned or updated later in the function.

branch: `branch` stores a computed value for Git publishing and authorization state. It is initialized from `publishAccess.branch || "main"`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `runGit`; assigned or updated later in the function.

remote: `remote` stores a function or callback value for network or routing value. It is initialized from `(await runPublishGit(["remote", "get-url", "origin"])).stdout.trim()`. Within the function it is passed into `runGit`, `runPublishGit`; assigned or updated later in the function.

importRoot: `importRoot` stores a resolved filesystem or route value. It is initialized from `await mkdtemp(path.join(os.tmpdir(), "omb-publish-target-"))`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `join`, `rm`, `runGit`; assigned or updated later in the function.

cloneRoot: `cloneRoot` stores a resolved filesystem or route value. It is initialized from `path.join(importRoot, "target")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `fsAccess`, `join`, `readFile`, `runGit`; assigned or updated later in the function.

importPaths: `importPaths` stores a list for filesystem location. It is initialized from `[]`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is assigned or updated later in the function.

availablePaths: `availablePaths` stores a list for filesystem location. It is initialized from `[]`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is mutated as a collection or DOM container; accesses properties/methods `includes`, `push`; assigned or updated later in the function.

importPath: `importPath` starts without an immediate value. In this file that usually means it is filled by a loop, `branch`, `callback`, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `cp`, `fsAccess`, `join`, `push`, `resolveInsideRoot`.

backupRoot: `backupRoot` stores a computed value for filesystem location. It is initialized from `resolveInsideRoot()`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `cp`, `mkdir`, `relative`; assigned or updated later in the function.

importPath: `importPath` starts without an immediate value. In this file that usually means it is filled by a loop, `branch`, `callback`, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `cp`, `fsAccess`, `join`, `push`, `resolveInsideRoot`.

sourcePath: `sourcePath` stores a resolved filesystem or route value. It is initialized from `path.join(cloneRoot, importPath)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads,

previews, compiler artifacts, or published assets. Within the function it is passed into `cp`; assigned or updated later in the function.

targetPath: `targetPath` stores a resolved filesystem or route value. It is initialized from `resolveInsideRoot(importPath)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `cp`, `fsAccess`, `rm`; assigned or updated later in the function.

importedCatalog: `importedCatalog` stores a resolved filesystem or route value. It is initialized from `await readFile(path.join(cloneRoot, "projects.json"), "utf8")`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into `writeFile`; assigned or updated later in the function.

portfolioSync: `portfolioSync` stores the completed result of an awaited operation. It is initialized from `await syncPortfolioPublishFiles({ removeMissing: false })`. Within the function it is assigned or updated later in the function.

async function installGitForWindows()

To understand `installGitForWindows`, start with the problem it owns. `installGitForWindows` installs or prepares external tooling. It belongs in the backend because installation touches the machine, not just the browser page.

The syntax of the function is:

```
async function installGitForWindows() { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and runs an external command. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

Next, trace the helper calls that surround the function. It works with `runOptionalCommand`. Those calls show that `installGitForWindows` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `installGitForWindows`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

After the main flow, the working values inside `installGitForWindows` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

winget: `winget` stores the completed result of an awaited operation. It is initialized from `await runOptionalCommand("winget", ["--version"])`. Within the function it is accesses properties/methods ok; passed into `runOptionalCommand`, `spawn`; assigned or updated later in the function.

downloadUrl: downloadUrl stores a string value for network or routing value. It is initialized from "https://git-scm.com/download/win". Within the function it is assigned or updated later in the function.

child: child stores a computed value for temporary calculation. It is initialized from spawn("winget", ["install", "--id", "Git.Git", "-e", "--source", "winget"], {). Within the function it is accesses properties/methods unref; assigned or updated later in the function.

async function syncPortfolioPublishFiles(options = {})

To understand syncPortfolioPublishFiles, start with the problem it owns. syncPortfolioPublishFiles reconciles two states that can drift apart. In this project, sync functions connect local drafts, route state, Git branches, publish mirrors, or frontend state to the current truth.

The syntax of the function is:

```
async function syncPortfolioPublishFiles(options = {}) { ... }
```

The async keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: options. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. options is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work and reads or writes files. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: creates folders before writing files so missing directories do not crash the workflow.

Next, trace the helper calls that surround the function. It works with samePath, resolveInsideRoot, resolveInsidePortfolioRoot, pathExists. Those calls show that syncPortfolioPublishFiles is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without syncPortfolioPublishFiles, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

After the main flow, the working values inside syncPortfolioPublishFiles reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

copied: copied stores a list for temporary calculation. It is initialized from []. Within the function it is mutated as a collection or DOM container; accesses properties/methods push; assigned or updated later in the function.

skipped: skipped stores a list for temporary calculation. It is initialized from []. Within the function it is mutated as a collection or DOM container; accesses properties/methods push; assigned or updated later in the function.

relativePath: relativePath starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into push, resolveInsidePortfolioRoot, resolveInsideRoot.

sourcePath: sourcePath stores a resolved filesystem or route value. It is initialized from resolveInsideRoot(relativePath). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into cp; assigned or updated later in the function.

targetPath: targetPath stores a resolved filesystem or route value. It is initialized from resolveInsidePortfolioRoot(relativePath). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into cp, mkdir, rm; assigned or updated later in the function.

async function publishSiteChanges(publishAccess = null)

To understand publishSiteChanges, start with the problem it owns. publishSiteChanges is the final publishing operation behind Apply to site. It assumes authorization has been proven or supplied, synchronizes publishable files into the portfolio mirror, checks Git status, stages only approved paths, creates a commit when there are changes, and pushes to the selected branch. It is async because Git commands and file synchronization are external operations that take time and can fail. This function is important because publishing is the one place where local draft work becomes public website content. It must therefore be strict. If it staged arbitrary files or pushed without checking state, builder internals, private drafts, or stale branches could leak to the website. The function works with assertPublishAccess, syncPortfolioPublishFiles, getGitStatus, runPublishGit, and stageablePublishPaths.

The syntax of the function is:

```
async function publishSiteChanges(publishAccess = null) { ... }
```

The async keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: publishAccess. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. publishAccess is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails.

Next, trace the helper calls that surround the function. It works with assertPublishAccess, runPublishGit, synchronizePublishBranchFromRemote, syncPortfolioPublishFiles, bumpPublishedAssetVersions, stageablePublishPaths. Those calls show that publishSiteChanges is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `publishSiteChanges`, the publishing flow would lose one guardrail or one mechanical step. Depending on the function, that could mean pushing to the wrong branch, forgetting authorization, accepting a bad remote, staging unsafe files, or failing to import the latest website state.

After the main flow, the working values inside `publishSiteChanges` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

access: `access` stores the completed result of an awaited operation. It is initialized from `publishAccess || await assertPublishAccess()`. Within the function it is accesses properties/methods `branch`; passed into `synchronizePublishBranchFromRemote`; assigned or updated later in the function.

branch: `branch` stores the completed result of an awaited operation. It is initialized from `access.branch || (await runPublishGit(["branch", "--show-current"]).stdout.trim())`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `runPublishGit`, `synchronizePublishBranchFromRemote`; assigned or updated later in the function.

remoteSync: `remoteSync` stores the completed result of an awaited operation. It is initialized from `await synchronizePublishBranchFromRemote(branch, access)`. Within the function it is assigned or updated later in the function.

portfolioSync: `portfolioSync` stores the completed result of an awaited operation. It is initialized from `await syncPortfolioPublishFiles({ removeMissing: true })`. Within the function it is assigned or updated later in the function.

stageablePaths: `stageablePaths` stores a resolved filesystem or route value. It is initialized from `await stageablePublishPaths(publishPaths)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods `length`; passed into `runPublishGit`; assigned or updated later in the function.

status: `status` stores a resolved filesystem or route value. It is initialized from `await runPublishGit(["status", "--porcelain", "--", ...stageablePaths])`. Within the function it is accesses properties/methods `stdout`; passed into `runPublishGit`; assigned or updated later in the function.

hasChanges: `hasChanges` stores a computed value for temporary calculation. It is initialized from `status.stdout.trim().length > 0`. Within the function it is assigned or updated later in the function.

commit: `commit` stores a computed value for temporary calculation. It is initialized from `null`. Within the function it is passed into `runPublishGit`; assigned or updated later in the function.

message: `message` stores a string value for temporary calculation. It is initialized from ``Update portfolio site ${new Date().toISOString().slice(0, 10)}``. Within the function it is passed into `runPublishGit`; assigned or updated later in the function.

pushArgs: `pushArgs` stores a computed value for temporary calculation. It is initialized from `branch ? ["push", "origin", branch] : ["push"]`. Within the function it is passed into `runPublishGit`; assigned or updated later in the function.

push: `push` stores the completed result of an awaited operation. It is initialized from `await runPublishGit(pushArgs)`. Within the function it is accesses properties/methods `stderr`, `stdout`; assigned or updated later in the function.

Application update and reporting

The functions in this group all support application update and reporting. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function compareVersions(left = "", right = "")

Begin with the role of compareVersions. compareVersions belongs to application update and reporting. This function supports release checks, installer handoff, version comparison, or security/download reporting. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function compareVersions(left = "", right = "") { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: left, right. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. left is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. right is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means compareVersions performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without compareVersions, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside compareVersions explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

leftParts: leftParts stores a function or callback value for lookup collection. It is initialized from `String(left || "0").split(/[.]/).map((part) => Number.parseInt(part, 10) || 0)`. Within the function it is accesses properties/methods length; passed into max; assigned or updated later in the function.

rightParts: rightParts stores a function or callback value for lookup collection. It is initialized from `String(right || "0").split(/[.]/).map((part) => Number.parseInt(part, 10) || 0)`. Within the function it is accesses properties/methods length; passed into max; assigned or updated later in the function.

length: length stores a computed value for temporary calculation. It is initialized from `Math.max(leftParts.length, rightParts.length)`. Within the function it is passed into max; assigned or updated later in the function.

index: index stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is assigned or updated later in the function.

delta: delta stores a function or callback value for temporary calculation. It is initialized from `(leftParts[index] || 0) - (rightParts[index] || 0)`. Within the function it is returned to the caller; assigned or updated later in the function.

async function getUpdateInfo()

The best entry point for `getUpdateInfo` is its responsibility in the surrounding workflow. `getUpdateInfo` belongs to application update and reporting. This function supports release checks, installer handoff, version comparison, or security/download reporting. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function getUpdateInfo() { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and calls a network resource. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails; performs a fetch and therefore must handle network delay and failure.

Its connection to the rest of the file matters as much as its local code. It works with `compareVersions`. Those calls show that `getUpdateInfo` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `getUpdateInfo`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The easiest way to follow the body of `getUpdateInfo` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

currentVersion: `currentVersion` stores an environment-derived value for temporary calculation. It is initialized from `process.env.OMB_APP_VERSION || packageJson.version || "0.0.0"`. Within the function it is passed into `compareVersions`; assigned or updated later in the function.

owner: `owner` stores a string value for temporary calculation. It is initialized from `"otienomaurice"`. Within the function it is passed into `fetch`; assigned or updated later in the function.

repo: `repo` stores a string value for temporary calculation. It is initialized from `"omb-portfolio-builder"`. Within the function it is passed into `fetch`; assigned or updated later in the function.

response: response stores data returned from a network call. It is initialized from `await fetch('https://api.github.com/repos/${owner}/${repo}/releases/latest', {`. Within the function it is accesses properties/methods `json, ok, status`; passed into `Error`; assigned or updated later in the function.

release: release stores the completed result of an awaited operation. It is initialized from `await response.json()`. Within the function it is accesses properties/methods `assets, html_url, tag_name`; passed into `String`; assigned or updated later in the function.

latestVersion: latestVersion stores a computed value for temporary calculation. It is initialized from `String(release.tag_name || "").replace(/^builder-v/i, "")`. Within the function it is passed into `compareVersions`, `has`; assigned or updated later in the function.

installer: installer stores a function or callback value for temporary calculation. It is initialized from `(release.assets || []).find((asset) => /Setup-.*\.exe$/i.test(asset.name))`. Within the function it is assigned or updated later in the function.

portable: portable stores a function or callback value for temporary calculation. It is initialized from `(release.assets || []).find((asset) => /Portable-.*\.exe$/i.test(asset.name))`. Within the function it is assigned or updated later in the function.

updateBlocked: updateBlocked stores a computed value for temporary calculation. It is initialized from `blockedAppUpdateVersions.has(latestVersion)`. Within the function it is assigned or updated later in the function.

async function getBuilderReleaseDownloadReport()

Begin with the role of `getBuilderReleaseDownloadReport`. `getBuilderReleaseDownloadReport` belongs to application update and reporting. This function supports release checks, installer handoff, version comparison, or security/download reporting. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function getBuilderReleaseDownloadReport() { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work and calls a network resource. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must `await` it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails; performs a `fetch` and therefore must handle network delay and failure.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `getBuilderReleaseDownloadReport` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `getBuilderReleaseDownloadReport`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `getBuilderReleaseDownloadReport` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

owner: owner stores a string value for temporary calculation. It is initialized from "otienomaurice". Within the function it is passed into `fetch`; assigned or updated later in the function.

repo: repo stores a string value for temporary calculation. It is initialized from "omb-portfolio-builder". Within the function it is passed into `fetch`; assigned or updated later in the function.

response: response stores data returned from a network call. It is initialized from `await fetch(`https://api.github.com/repos/${owner}/${repo}/releases/latest`, {`})`. Within the function it is accesses properties/methods `json`, `ok`, `status`; passed into `Error`; assigned or updated later in the function.

release: release stores the completed result of an awaited operation. It is initialized from `await response.json()`. Within the function it is accesses properties/methods `assets`, `html_url`, `name`, `tag_name`; assigned or updated later in the function.

assets: assets stores a function or callback value for lookup collection. It is initialized from `(release.assets || []).map((asset) => ({`})`. Within the function it is iterated or transformed as a collection; accesses properties/methods `reduce`; assigned or updated later in the function.

async function `getSecurityReport()`

Begin with the role of `getSecurityReport`. `getSecurityReport` belongs to application update and reporting. This function supports release checks, installer handoff, version comparison, or security/download reporting. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function getSecurityReport() { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

The body is where the contract above becomes behavior. Inside the body, it waits for asynchronous work. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `getBuilderReleaseDownloadReport`, `readPublishAuthCache`, `getPublishTargetInfo`, `publishAuthCacheIsFresh`, `publishAuthCacheExpiresAt`. Those calls show that `getSecurityReport` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `getSecurityReport`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `getSecurityReport` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

destructured [downloadResult, authCacheResult, targetResult]: destructured [downloadResult, authCacheResult, targetResult] stores the completed result of an awaited operation. It is initialized from `await Promise.allSettled([`. It affects publishing, where mistakes can change the public website or expose the wrong files.

authCache: `authCache` stores a computed value for runtime state or cache. It is initialized from `authCacheResult.status === "fulfilled" ? authCacheResult.value : null`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is passed into `Boolean`, `Number`, `publishAuthCacheExpiresAt`; assigned or updated later in the function.

target: `target` stores a computed value for temporary calculation. It is initialized from `targetResult.status === "fulfilled" ? targetResult.value : null`. Within the function it is passed into `Boolean`; assigned or updated later in the function.

function safeUpdateFileSegment(value = "")

To understand `safeUpdateFileSegment`, start with the problem it owns. `safeUpdateFileSegment` is a guardrail function. It checks or transforms caller input before the system uses that input as a filename, path, remote, domain, credential, or public value. This keeps unsafe input from becoming filesystem access, Git access, or broken public links.

The syntax of the function is:

```
function safeUpdateFileSegment(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `safeUpdateFileSegment` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `safeUpdateFileSegment`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `safeUpdateFileSegment`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

async function downloadAndLaunchAppUpdate()

Read `downloadAndLaunchAppUpdate` first as a named idea, not as syntax. `downloadAndLaunchAppUpdate` is the updater handoff. It checks release information, downloads the installer, writes a PowerShell script that can wait for the current process to exit, launches the installer or updater quietly, and schedules the current app to close. It must be `async` because it performs network download, file writes, and process launches. The function is needed because a running Electron app cannot safely replace all of its own files while those files are in use. The handoff script becomes a temporary helper that finishes the update after the current process gets out of the way.

The syntax of the function is:

```
async function downloadAndLaunchAppUpdate() { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work, calls a network resource, runs an external command, reads or writes files and uses a timer to avoid blocking or to wait for another process. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently; throws explicit errors when a required safety or compatibility condition fails; performs a fetch and therefore must handle network delay and failure; creates folders before writing files so missing directories do not crash the workflow; writes data to disk as part of the operation.

The function also has to be read through the helpers it calls. It works with `getUpdateInfo`, `safeUpdateFileSegment`, `powershellSingleQuoted`. Those calls show that `downloadAndLaunchAppUpdate` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `downloadAndLaunchAppUpdate`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside `downloadAndLaunchAppUpdate` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

update: update stores the completed result of an awaited operation. It is initialized from `await getUpdateInfo()`. Within the function it is accesses properties/methods `blockedReason`, `currentVersion`, `error`, `installerName`, `installerUrl`, `latestVersion`, `ok`, `updateAvailable`; passed into `Error`, `emit`, `fetch`, `join`, `safeUpdateFileSegment`, `test`; assigned or updated later in the function.

updateRoot: updateRoot stores a resolved filesystem or route value. It is initialized from path.join(os.tmpdir(), "omb-portfolio-builder-updates"). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into join, mkdir; assigned or updated later in the function.

installerName: installerName stores a computed value for temporary calculation. It is initialized from update.installerName && /\.exe\$/i.test(update.installerName). Within the function it is passed into join, test; assigned or updated later in the function.

installerPath: installerPath stores a resolved filesystem or route value. It is initialized from path.join(updateRoot, installerName). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into emit, powershellSingleQuoted, writeFile; assigned or updated later in the function.

response: response stores data returned from a network call. It is initialized from await fetch(update.installerUrl, {). Within the function it is accesses properties/methods arrayBuffer, ok, status; passed into Error, from; assigned or updated later in the function.

installerBuffer: installerBuffer stores the completed result of an awaited operation. It is initialized from Buffer.from(await response.arrayBuffer()). Within the function it is accesses properties/methods length; passed into writeFile; assigned or updated later in the function.

updateLogPath: updateLogPath stores a resolved filesystem or route value. It is initialized from path.join(updateRoot, `update-\${safeUpdateFileSegment(update.latestVersion)}.log`). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into emit, powershellSingleQuoted; assigned or updated later in the function.

currentExecutable: currentExecutable stores a resolved filesystem or route value. It is initialized from /OMB Portfolio Builder\.exe\$/i.test(process.execPath || "") ? process.execPath : "". Within the function it is passed into dirname, from; assigned or updated later in the function.

currentInstallDirectory: currentInstallDirectory stores a resolved filesystem or route value. It is initialized from currentExecutable ? path.dirname(currentExecutable) : "". It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into from; assigned or updated later in the function.

localAppDataExecutable: localAppDataExecutable stores a environment-derived value for temporary calculation. It is initialized from process.env.LOCALAPPDATA. Within the function it is passed into from; assigned or updated later in the function.

legacyManagedApplicationExecutable: legacyManagedApplicationExecutable stores a resolved filesystem or route value. It is initialized from path.join(os.homedir(), "OMB", "application", "OMB Portfolio Builder", "OMB Portfolio Builder.exe"). Within the function it is assigned or updated later in the function.

legacyFlatApplicationExecutable: legacyFlatApplicationExecutable stores a resolved filesystem or route value. It is initialized from path.join(os.homedir(), "OMB", "application", "OMB Portfolio Builder.exe"). Within the function it is assigned or updated later in the function.

relaunchCandidates: relaunchCandidates stores a computed value for lookup collection. It is initialized from Array.from(new Set([). Within the function it is iterated or transformed as a collection; accesses properties/methods map; assigned or updated later in the function.

relaunchCandidatesPs: relaunchCandidatesPs stores a string value for lookup collection. It is initialized from ``@($relaunchCandidates.map(powershellSingleQuoted).join(", "))``. Within the function it is assigned or updated later in the function.

launcherPath: launcherPath stores a resolved filesystem or route value. It is initialized from `path.join(updateRoot, `run-update-${safeUpdateFileSegment(update.latestVersion)}.ps1`)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `emit`, `writeFile`; assigned or updated later in the function.

launcherCommandPath: launcherCommandPath stores a resolved filesystem or route value. It is initialized from `path.join(updateRoot, `run-update-${safeUpdateFileSegment(update.latestVersion)}.cmd`)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `spawn`, `writeFile`; assigned or updated later in the function.

launcherScript: launcherScript stores a list for temporary calculation. It is initialized from `[]`. Within the function it is passed into `writeFile`; assigned or updated later in the function.

launcherCommandScript: launcherCommandScript stores a list for temporary calculation. It is initialized from `[]`. Within the function it is passed into `writeFile`; assigned or updated later in the function.

child: child stores a resolved filesystem or route value. It is initialized from `spawn("cmd.exe", ["/d", "/c", launcherCommandPath], {`. Within the function it is accesses properties/methods `unref`; assigned or updated later in the function.

Curated File Guide: script.js

The public website runtime that renders the portfolio, search, project windows, browser history, rich content, links, and AI chat behavior.

Before The Code: File Overview

Abstract

script.js is the public website runtime. It is the JavaScript that turns the static portfolio files into an interactive recruiter-facing site. The HTML page provides stable containers and the JSON catalog provides saved content, but script.js decides what appears, how projects are filtered, how rich text renders, how files become links, how project sections open into full-window views, how browser Back and Refresh interact with those views, how search behaves like a real search tool, and how the AI chat gathers useful context before asking the backend.

The important distinction is that script.js is public and read-only. It should never expose builder-only editing controls. Its job is to present what has already been saved and parsed by the builder. That is why much of the file is about rendering, normalization, routing, search indexing, assistant context selection, and UI state rather than file writes.

Introduction

When a visitor opens the portfolio, index.html loads styles.css, electronics-search.js, and script.js. script.js immediately grabs the DOM elements it will control: the project grid, search input, filters, profile fields, resume area, contact area, AI assistant, and footer. Then it waits for portfolio data. The data can come from an embedded window.__PORTFOLIO_CATALOG__ object in previews or from projects.json on the published website. Once the catalog is loaded, script.js normalizes profile data, renders the hero and contact information, creates category filters, renders project cards, builds the search index, and restores any deep-linked project section.

This file is the main reason the same portfolio can feel alive without requiring a database-backed website. It uses static files as the source of truth, then builds an application-like experience in the browser. That makes the public website easy to host on GitHub Pages or Cloudflare, while still giving visitors search, project windows, AI questions, and responsive navigation.

Background Information

The file works with a catalog generated by the builder parser. That catalog contains categories, projects, profile fields, site content, field styles, rich text blocks, fun facts, and optional site sections. The functions in script.js assume the catalog is already safe enough to display, but they still escape HTML where user-facing strings are inserted into the DOM. Rich text rendering is intentionally structured because portfolio content can include plain text, links, formulas, images, files, code blocks, captions, and nested project subsections.

The website also has a windowing model. Project cards appear inside categories. A project section button opens a full-screen dialog. Inside that dialog, subsections can be clicked like nested folders. Browser history is updated with the project id, section index, and resource path so the browser Back button can move through the same hierarchy. This makes project content feel more like a navigable portfolio than a long flat webpage.

Search is also broader than a normal text filter. The search index contains page sections, project names, category labels, tools, languages, rich text, files, URLs, captions, uploaded text files where fetchable, and electronics keyword expansions. The goal is that a recruiter can type an electronics phrase and land near the relevant artifact, even if the phrase appears inside a project section or file caption rather than on the top card.

Because the website is static, script.js must do work that a database-backed application might normally ask a server to do. It must decide which categories exist, which projects belong to each category, which sections are empty, which files should render as downloads, which links should open externally, and which project paths should be represented in browser history. The price of static hosting is that the browser runtime has to be disciplined. The benefit is that the public site remains easy to deploy, cache, mirror, and serve from GitHub Pages or Cloudflare without a long-running custom web server.

The file also sits between several other files. index.html gives it DOM anchors. styles.css gives visual meaning to the classes it writes. electronics-search.js can provide expanded electronics vocabulary. projects.json provides the portfolio catalog. The AI backend endpoint gives it remote intelligence when available. If one of those contracts changes, script.js is usually the first file that reveals the mismatch because rendering, search, or chat stops behaving correctly.

Purpose Of The File

The purpose of script.js is to convert portfolio data into a polished public experience. The file decides not only what to render, but also what not to render. Empty sections are hidden. Overview content appears as the first readable content when present. Links that look like websites open as websites rather than downloads. Local files become downloadable resources. Images render inline only when they are meant to be shown. AI links are only displayed when related to the question context.

Without script.js, index.html would be mostly a static shell with placeholders. There would be no dynamic project directory, no generated categories, no rich project windows, no browser-aware section routing, no searchable uploaded file text, no AI context gathering, no responsive project rendering, and no dynamic profile/contact population.

Primary Responsibilities

The first responsibility is state hydration. loadProjectCatalog reads the portfolio catalog, normalizes the data, stores it in module-level variables, and triggers rendering. This is the startup path for the public website.

The second responsibility is rendering. Functions such as renderProfile, renderSiteContent, renderFunFacts, renderSiteSections, projectCard, parsedSection, and parsedNodeContent turn structured catalog objects into actual HTML.

The third responsibility is interaction. The dialog functions create the full-window project section view, the routing functions keep browser history synchronized, the search functions build and display selectable results, and the AI functions manage the chat panel.

The fourth responsibility is interpretation. The assistant functions classify questions, gather relevant portfolio context, select public source links, decide whether a question is general or portfolio-specific, and prepare the request sent to the AI backend. This prevents the assistant from blindly dumping generic links for every question.

The fifth responsibility is restraint. Public visitors should see only finished, parsed, saved portfolio output. The file therefore hides empty content, avoids exposing builder edit controls, limits which sources are shown with AI answers, and uses normalized link behavior so a website link does not become a fake downloadable file. This restraint is what separates the public website from the local builder application.

The sixth responsibility is mobile and desktop continuity. The same project data must read cleanly on an iPhone and on a wide monitor. The responsive helper functions estimate project density and media presence, while the dialog and assistant functions keep windows and chat panels usable without letting one part of the page resize unrelated parts. script.js is therefore not only a renderer; it is also the public interaction policy for different devices.

Important Data Objects And Why They Matter

categories, projects, siteSections, profile, siteContent, funFacts, fieldStyles, and the rich variants of those fields are the hydrated catalog state. They are not hardcoded content. They are populated from projects.json or preview data, which is why the same script can run for Maurice's site and also for someone else's freshly installed builder output.

searchableEntries is the in-memory search index. Every item in it has a title, type, context, text, optional project id, optional section index, optional URL, and normalized text. The search functions score and display these entries.

assistantChatHistory and assistantSourceMap are the AI interface memory. assistantChatHistory keeps recent conversation so follow-up questions make sense. assistantSourceMap maps clickable source buttons in an answer back to portfolio search results so a visitor can jump to the relevant project or file.

currentSearchResults and searchResultLimit are the active search session. They remember what the latest query produced and how many items the user wants to see. Without these values, pressing Enter, selecting a result, or changing the result limit would not know which result list is current.

sectionRouteKey and sectionRouteParams are the browser-history contract for project windows. They keep the public URL connected to the currently opened project section and nested subsection. The values are small, but they are what let Refresh reopen the same project view and Back return to the previous view.

Top-Level Objects And Variables

This section explains the long-lived objects and variables before the function walkthrough. These are the values that give the file memory, configuration, API handles, DOM anchors, path boundaries, cache state, and behavior maps. They are explained by meaning, not by line number.

Dom Reference Or Ui State

grid

grid points at the project-grid container. renderProjects writes category sections and project cards into this element, making it the public project's visible mounting point. It anchors UI behavior to a specific browser element or window state. Functions that reference it include mediaGrid, renderSiteSections, parsedChildCards, projectCard, renderProjects, restoreSectionRouteAfterCatalogLoad, loadProjectCatalog.

searchInput

searchInput points at the project search box. Search handlers read its value, build dropdown results, apply highlights, and decide which projects remain visible. It anchors UI behavior to a specific browser element or window state. Functions that reference it include queueSearchHighlights, ensureSearchPanel, renderSearchResults, updateSearchDropdown, showSearchPanelIfNeeded, renderProjects, restoreSectionRouteAfterCatalogLoad.

filterButtons

filterButtons stores the current filter button elements. It is updated after dynamic filters are rendered because the original NodeList would not include newly created buttons. It anchors UI behavior to a specific browser element or window state. Functions that reference it include setActiveFilter, bindFilterButtons.

aiClearChatButton

aiClearChatButton is the explicit Clear chat control. It resets conversation history, source maps, and the visible log. It anchors UI behavior to a specific browser element or window state. Functions that reference it include updateAssistantPanelGrowth, restoreSectionRouteAfterCatalogLoad.

Portfolio Data State

projectFilters

projectFilters points at the category filter container. renderCategoryFilters fills it with filter buttons derived from the actual catalog categories. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderCategoryFilters.

projectCount

projectCount displays the number of visible or total projects. It is updated after catalog load and render operations so the summary band matches the data. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderProjects.

projectTrackCount

projectTrackCount displays how many project categories/tracks exist. It is derived from the saved categories rather than hardcoded. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderCategoryFilters.

projectTrackLabels

projectTrackLabels displays the category names. It helps the top summary reflect Analog, Digital, Embedded, or any custom categories the builder creates. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderCategoryFilters.

resumeSection

resumeSection is the public resume panel. renderProfile hides or shows it depending on whether a resume URL exists. Functions that reference it include renderProfile.

profilePhoto

profilePhoto is the large contact-section profile image. It is separate from the header avatar so desktop/mobile layouts can size them differently. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderProfile.

projects

projects is the runtime project array. Rendering, search, assistant context, and project windows all read from it. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include addSiteSectionSearchEntries, rebuildSearchIndex, assistantNamedProjectIds, assistantQuestionHasPortfolioIntent, assistantQuestionTokens, assistantNamedProjectTopicTokens, assistantContextForQuestion, assistantFallbackResults, assistantLocalAnswer, sectionRouteUrl.

siteSections

siteSections stores custom main-page sections such as personal profile, hobbies, sports, social media, or professional material. Functions that reference it include addSiteSectionSearchEntries, assistantPublicProfileLinks, assistantContextForQuestion, renderSiteSections, loadProjectCatalog.

profile

profile stores owner identity: name, email, phone, images, resume, GitHub, LinkedIn, and other public contact details. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderProfile, isWebsiteLinkItem, projectResponsiveClass, responsiveStyleValues, addSiteSectionSearchEntries, assistantUrlsFromTextValue, assistantPublicProfileLinks, assistantQuestionAllowsPublicLookup, assistantContextForQuestion, assistantFallbackResults.

profileRich

profileRich stores rich versions of profile/contact fields so styled text can survive from builder to website. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderProfile, loadProjectCatalog.

activeSectionDialogDrag

activeSectionDialogDrag stores the drag state while a project window is being moved. It anchors UI behavior to a specific browser element or window state. Functions that reference it include beginSectionDialogDrag, moveSectionDialogDrag, endSectionDialogDrag.

activeSectionDialogResize

activeSectionDialogResize stores the resize state while a project window edge is being dragged. It anchors UI behavior to a specific browser element or window state. Functions that reference it include beginSectionDialogResize, moveSectionDialogResize, endSectionDialogResize.

sectionDialogDragEnabled

sectionDialogDragEnabled prevents drag handlers from being installed more than once. It anchors UI behavior to a specific browser element or window state. Functions that reference it include enableSectionDialogDrag.

isApplyingSectionRoute

isApplyingSectionRoute prevents browser history updates from recursively triggering themselves while a saved route is being restored. Functions that reference it include syncSectionRouteToHistory, clearSectionRouteInHistory, applySectionRoute.

sectionRouteKey

sectionRouteKey identifies portfolio section entries in browser history state. Functions that reference it include sectionRouteState, sectionRouteFromState.

sectionRouteParams

sectionRouteParams defines the URL parameter names for project id, section index, and nested subsection path. Visible object fields include path, project, section. Those fields form the contract consumed by related functions. Functions that reference it include sectionBaseUrl, sectionRouteUrl, sectionRouteFromLocation.

defaultProfile

defaultProfile is the empty-owner fallback. It defines all expected profile fields so renderProfile can run even before the user enters contact details. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Visible object fields include brandImage, brandText, contactIntro, displayName, email, githubUrl, herolImage, linkedinUrl, phone, portfolioLabel, profileImage, resumeUrl, websiteUrl. Those fields form the contract consumed by related functions. Functions that reference it include normalizeProfile.

Temporary Calculation

year

year points at the footer year span. Startup code can set it from the current date so the footer remains current without manual HTML edits. Functions that reference it include renderProfile.

funFactsCallout

funFactsCallout is the near-top personality callout. renderFunFacts only reveals it when saved fun facts exist. Functions that reference it include renderFunFacts.

heroEyebrow

heroEyebrow is the small label above the hero title. renderSiteContent can replace it from the builder's front page settings. Functions that reference it include normalizeSiteContent, renderSiteContent, addSiteSectionSearchEntries.

heroTitle

heroTitle is the main public headline. It receives the owner-controlled front-page title instead of forcing a fixed sentence into every portfolio. Functions that reference it include normalizeSiteContent, renderSiteContent, addSiteSectionSearchEntries.

heroCopy

heroCopy is the supporting paragraph in the hero section. It is one of the fields affected by rich text and plain text styling choices. Functions that reference it include normalizeSiteContent, renderSiteContent, addSiteSectionSearchEntries.

brandName

brandName is the bold brand text in the header. renderProfile updates it to match the portfolio owner or site title. Functions that reference it include renderProfile.

brandSubtitle

brandSubtitle is the small subtitle beside the brand. It usually communicates local builder/public portfolio context or a tagline. Functions that reference it include renderProfile.

brandIcon

brandIcon is the image element for the OMB visual mark. Profile settings can change it when the builder is generalized for other users. Functions that reference it include renderProfile.

brandText

brandText is the OMB engraving text in the header logo area. It carries the small trademark-like label. Functions that reference it include normalizeProfile, renderProfile.

headerAvatar

headerAvatar is the clickable top-right profile image container. renderProfile reveals it only when a profile image exists. Functions that reference it include renderProfile.

headerAvatarImage

headerAvatarImage is the actual img element inside the avatar link. It receives the saved profile picture URL. Functions that reference it include renderProfile.

heroImage

heroImage is the main hero/background image element. Site content settings decide whether it is displayed. Functions that reference it include normalizeProfile, renderProfile.

contactBand

contactBand is the final dark contact area. It also contains the AI assistant panel, so layout code must keep chat growth from distorting contact information. Functions that reference it include renderProfile.

contactTitle

contactTitle is the heading for the contact panel. renderProfile updates it from the owner identity. Functions that reference it include renderProfile.

contactIntro

contactIntro is the contact-section introduction text. It can be empty, plain text, or rich content depending on saved profile data. Functions that reference it include normalizeProfile, renderProfile, addSiteSectionSearchEntries.

contactDetails

contactDetails is the list of direct contact methods such as email and phone. renderProfile builds clickable mailto and tel links inside it. Functions that reference it include renderProfile.

contactLinks

contactLinks is the container for external profile links such as GitHub, LinkedIn, resume, or custom websites. Functions that reference it include renderProfile.

footerOwner

footerOwner is the footer ownership text. It lets the footer follow the current profile owner rather than remaining generic. Functions that reference it include renderProfile.

searchWrap

searchWrap is the parent around the search box. The search dropdown attaches near it so results appear where the user is typing. Functions that reference it include ensureSearchPanel.

categories

categories is the runtime copy of project categories loaded from the catalog. It can include more than the original Analog, Digital, and Embedded groups. Functions that reference it include slugLabel, setActiveFilter,

renderCategoryFilters, addSiteSectionSearchEntries, assistantContextForQuestion, assistantFallbackResults, projectCard, renderProjects, openParsedSection, loadProjectCatalog.

funFacts

funFacts stores simple fun fact strings. renderFunFacts turns at most a few lines into a near-top callout. Functions that reference it include renderFunFacts, loadProjectCatalog.

funFactsRich

funFactsRich stores rich-format fun facts when the builder saves styled content instead of plain strings. Functions that reference it include renderFunFacts, loadProjectCatalog.

siteContent

siteContent stores front-page text and site-level settings such as hero copy and titles. Functions that reference it include renderSiteContent, loadProjectCatalog.

siteContentRich

siteContentRich stores rich versions of site-level text fields. Functions that reference it include renderSiteContent, loadProjectCatalog.

activeFilter

activeFilter stores the currently selected project category filter. Functions that reference it include projectVisible, setActiveFilter, renderCategoryFilters, renderProjects.

searchPanel

searchPanel stores the dropdown result panel created for search. Functions that reference it include ensureSearchPanel, renderSearchResults, goToSearchResult, restoreSectionRouteAfterCatalogLoad.

searchStatus

searchStatus stores the status element inside the search dropdown. Functions that reference it include ensureSearchPanel, setSearchMessage.

searchLimitSelect

searchLimitSelect stores the dropdown control that lets the visitor choose 5, 10, 15, or 20 results. Functions that reference it include ensureSearchPanel.

currentSearchResults

currentSearchResults is the current ranked search-result array. It lets click handlers navigate results without recalculating the search. Functions that reference it include ensureSearchPanel, renderSearchResults, restoreSectionRouteAfterCatalogLoad.

searchableEntries

searchableEntries is the full in-browser search index. It contains project titles, sections, files, captions, page sections, uploaded text, and electronics keyword expansions. Functions that reference it include rebuildSearchIndex, searchResultsFor.

searchResultLimit

searchResultLimit stores how many dropdown results to display. Functions that reference it include ensureSearchPanel, renderSearchResults.

searchHighlightTimer

searchHighlightTimer debounces highlight updates so typing does not repaint the page too aggressively. Functions that reference it include queueSearchHighlights.

searchHighlightName

searchHighlightName is the CSS class applied to highlighted search matches. Functions that reference it include clearSearchHighlights, applySearchHighlights.

defaultSiteContent

defaultSiteContent is the fallback front-page copy for a blank install. It keeps the site readable before the builder user customizes hero text and site messaging. Visible object fields include heroCopy, heroEyebrow, heroTitle. Those fields form the contract consumed by related functions. Functions that reference it include normalizeSiteContent.

Ai Configuration Or Conversation State

aiAssistantForm

aiAssistantForm is the chat form. Its submit handler starts answerAssistantQuestion. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include updateAssistantPanelGrowth, setAssistantBusy, restoreSectionRouteAfterCatalogLoad.

aiAssistantPanel

aiAssistantPanel is the whole Ask My Portfolio panel. Growth and layout code use it to keep the assistant right-justified and scrollable. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include assistantDesktopDockedHeight, updateAssistantPanelGrowth.

aiAssistantInput

aiAssistantInput is the question field. It is cleared/focused by chat handlers and intentionally does not keep long instructional text inside the typed value. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include setAssistantBusy, clearAssistantChat, answerAssistantQuestion, restoreSectionRouteAfterCatalogLoad.

aiAssistantLog

aiAssistantLog is the scrollable message history. appendAssistantMessage and replaceAssistantMessage write user and assistant messages into it. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include updateAssistantPanelGrowth, appendAssistantMessage, scrollAssistantAnswerToStart, clearAssistantChat, restoreSectionRouteAfterCatalogLoad.

aiAssistantStatus

aiAssistantStatus is the small status line used for ready/thinking/error states. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include setAssistantStatus.

assistantSourceCounter

assistantSourceCounter creates stable ids for assistant source buttons. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include assistantSourcesMarkup, clearAssistantChat.

assistantSourceMap

assistantSourceMap connects source button ids to source records so clicking a source opens the right place. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include assistantSourcesMarkup, clearAssistantChat, restoreSectionRouteAfterCatalogLoad.

Appearance Configuration

fieldStyles

fieldStyles stores plain-field font, size, and color choices. It lets front-page, profile, and contact fields render consistently with builder formatting. Functions that reference it include applyPlainFieldStyle, plainFieldStyleAttribute, loadProjectCatalog.

legacyTemplateSkins

legacyTemplateSkins maps older template ids to the newer appearance-only template ids. It keeps older saved projects rendering after template behavior changed. Visible object fields include skin-light-blue, skin-clean-white, skin-deep-navy, skin-red-warm, skin-emerald-instrument, skin-amber-power, skin-violet-mixed, skin-graphite-asic, analog-opamp-topology, analog-power-charger, analog-filter-front-end, analog-sensor-interface, analog-mixed-signal-timing, digital-fpga-pipeline, digital-asic-block, digital-verification-suite, digital-interface-controller, digital-signal-processing. Those fields form the contract consumed by related functions. Functions that reference it include canonicalTemplateId.

Runtime State Or Cache

assistantChatHistory

assistantChatHistory stores recent user and assistant messages. It gives the chatbot conversation memory. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include assistantRemember, clearAssistantChat, answerAssistantQuestion.

Compiler Or Language Configuration

supportedCodeLanguages

supportedCodeLanguages is the public language list for code rendering and detection. It names C, C++, Verilog/SystemVerilog, Java, JavaScript, Python, HTML, and related aliases. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Functions that reference it include normalizeCodeLanguage, codeLanguageLabel.

Code Walkthrough

This walkthrough is function-by-function. Each section now follows a textbook rhythm: first the idea of the function, then the syntax, then how to read that syntax, then the implementation and the local objects that make the implementation work. Line numbers are deliberately avoided because the source will keep changing.

Public runtime utility

The functions in this group all support public runtime utility. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function normalize(value)

To understand normalize, start with the problem it owns. normalize standardizes value data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalize(value) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `normalize` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `normalize`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `normalize`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function sourceLooksCpp(source = "")

Begin with the role of `sourceLooksCpp`. `sourceLooksCpp` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sourceLooksCpp(source = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `source`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `sourceLooksCpp` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `sourceLooksCpp`, the specific rule it owns would disappear from the named workflow. Callers would either skip

that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `sourceLooksCpp` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

text: text stores a computed value for temporary calculation. It is initialized from `String(source || "")`. Within the function it is passed into test; assigned or updated later in the function.

function sourceLooksC(source = "")

Begin with the role of `sourceLooksC`. `sourceLooksC` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sourceLooksC(source = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `source`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `sourceLooksC` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `sourceLooksC`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `sourceLooksC` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

text: text stores a computed value for temporary calculation. It is initialized from `String(source || "")`. Within the function it is passed into test; assigned or updated later in the function.

function textBlocksFromPlainText(text)

Begin with the role of `textBlocksFromPlainText`. `textBlocksFromPlainText` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function textBlocksFromPlainText(text) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `text`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `text` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `textBlocksFromPlainText` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `textBlocksFromPlainText`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `textBlocksFromPlainText`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function unwrapFormula(value)

The best entry point for `unwrapFormula` is its responsibility in the surrounding workflow. `unwrapFormula` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function unwrapFormula(value) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `unwrapFormula` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `unwrapFormula`, the specific rule it owns would disappear from the named workflow. Callers

would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The easiest way to follow the body of `unwrapFormula` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

text: text stores a computed value for temporary calculation. It is initialized from `String(value || "").trim()`. Within the function it is returned to the caller; accesses properties/methods match; assigned or updated later in the function.

wrappers: wrappers stores a list for temporary calculation. It is initialized from `[]`. Within the function it is assigned or updated later in the function.

destructured [pattern, group]: destructured [pattern, group] starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It unpacks API handles from another object, giving the rest of the file short direct names for those APIs.

match: match stores a computed value for temporary calculation. It is initialized from `text.match(pattern)`. Within the function it is returned to the caller; assigned or updated later in the function.

function looksLikeBareWebAddress(value = "")

The best entry point for `looksLikeBareWebAddress` is its responsibility in the surrounding workflow. `looksLikeBareWebAddress` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function looksLikeBareWebAddress(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `looksLikeBareWebAddress` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `looksLikeBareWebAddress`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The easiest way to follow the body of `looksLikeBareWebAddress` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(value || "").trim()`.

Within the function it is accessed via `properties/methods split`; passed into `test`; assigned or updated later in the function.

host: `host` stores a computed value for temporary calculation. It is initialized from `clean.split(/[/?#]/)`

`[0].toLowerCase()`. Within the function it is passed into `test`; assigned or updated later in the function.

function displayTitle(value, fallback = "Untitled item")

Begin with the role of `displayTitle`. `displayTitle` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function displayTitle(value, fallback = "Untitled item") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`, `fallback`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `fallback` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `displayTitle` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `displayTitle`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `displayTitle` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

title: `title` stores a computed value for temporary calculation. It is initialized from `String(value || fallback).trim()`.

Within the function it is assigned or updated later in the function.

replacements: `replacements` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; assigned or updated later in the function.

function slugLabel(value)

The best entry point for slugLabel is its responsibility in the surrounding workflow. slugLabel belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function slugLabel(value) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: value. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means slugLabel performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without slugLabel, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The easiest way to follow the body of slugLabel is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

category: category stores a function or callback value for portfolio data state. It is initialized from categories.find((item) => item.id === value). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is returned to the caller; accesses properties/methods label; assigned or updated later in the function.

function parsedItemTerms(item)

To understand parsedItemTerms, start with the problem it owns. parsedItemTerms interprets dItemTerms text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

The syntax of the function is:

```
function parsedItemTerms(item) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: item. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `parsedItemTerms` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `parsedItemTerms`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `parsedItemTerms`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function pillList(items, className = "")

To understand `pillList`, start with the problem it owns. `pillList` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function pillList(items, className = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `items`, `className`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `items` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `className` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `pillList` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `pillList`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

After the main flow, the working values inside `pillList` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

label: `label` stores a computed value for temporary calculation. It is initialized from `typeof item === "string" ? item : item.name || item.title || item.label || ""`. Within the function it is returned to the caller; assigned or updated later in the function.

function evidenceList(items, renderItem, emptyMessage)

The best entry point for `evidenceList` is its responsibility in the surrounding workflow. `evidenceList` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function evidenceList(items, renderItem, emptyMessage) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `items`, `renderItem`, `emptyMessage`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `items` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `renderItem` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `emptyMessage` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `evidenceList` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `evidenceList`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `evidenceList`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function detailBlock(title, className, content)

Read `detailBlock` first as a named idea, not as syntax. `detailBlock` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function detailBlock(title, className, content) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `title`, `className`, `content`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `title` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `className` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `content` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `detailBlock` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `detailBlock`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `detailBlock`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function mediaGrid(items)

Read `mediaGrid` first as a named idea, not as syntax. `mediaGrid` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function mediaGrid(items) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `items`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `items` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The function also has to be read through the helpers it calls. It works with `escapeHtml`, `normalizeLinkTarget`, `isWebsiteLinkItem`, `linkAttributes`. Those calls show that `mediaGrid` is not isolated; it is one piece of a larger

workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `mediaGrid`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `mediaGrid`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function itemLabel(item = {}, fallback = "Download file")

The best entry point for `itemLabel` is its responsibility in the surrounding workflow. `itemLabel` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function itemLabel(item = {}, fallback = "Download file") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `item`, `fallback`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `fallback` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. It works with `itemUrl`, `fileNameFromUrl`. Those calls show that `itemLabel` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `itemLabel`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The easiest way to follow the body of `itemLabel` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

url: `url` stores a computed value for network or routing value. It is initialized from `itemUrl(item)`. Within the function it is passed into `fileNameFromUrl`; assigned or updated later in the function.

function parsedChildCards(children, projectId, sectionIndex, basePath = [])

Begin with the role of `parsedChildCards`. `parsedChildCards` interprets `dChildCards` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

The syntax of the function is:

```
function parsedChildCards(children, projectId, sectionIndex, basePath = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `children`, `projectId`, `sectionIndex`, `basePath`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `children` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `basePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `nodeHasRenderableContent`, `nodeIsOverview`, `parsedNodeCard`. Those calls show that `parsedChildCards` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `parsedChildCards`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `parsedChildCards` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

visibleChildren: `visibleChildren` stores a function or callback value for temporary calculation. It is initialized from `children.filter((child) => nodeHasRenderableContent(child) && !child.url && !nodeIsOverview(child))`. Within the function it accesses properties/methods `length`; assigned or updated later in the function.

function escapeRegExp(value = "")

Begin with the role of `escapeRegExp`. `escapeRegExp` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function escapeRegExp(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `escapeRegExp` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `escapeRegExp`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `escapeRegExp`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function scrollToDomTarget(id)

To understand `scrollToDomTarget`, start with the problem it owns. `scrollToDomTarget` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function scrollToDomTarget(id) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `id`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `id` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser route/history. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: updates browser history so Back, Forward, and refresh can follow the same window state.

Next, trace the helper calls that surround the function. It works with `closeSectionDialogForRoute`, `queueSearchHighlights`. Those calls show that `scrollToDomTarget` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `scrollToDomTarget`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

After the main flow, the working values inside `scrollToDomTarget` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

target: target stores a DOM element reference. It is initialized from `id ? document.getElementById(id) : null`. Within the function it is accesses properties/methods `scrollIntoView`; assigned or updated later in the function.

Profile and site content

The functions in this group all support profile and site content. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function normalizeFunFacts(value)

The best entry point for `normalizeFunFacts` is its responsibility in the surrounding workflow. `normalizeFunFacts` standardizes FunFacts data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeFunFacts(value) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `normalizeFunFacts` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `normalizeFunFacts`, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

The easiest way to follow the body of `normalizeFunFacts` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

items: items stores a computed value for temporary calculation. It is initialized from `Array.isArray(value) ? value : String(value || "").split(/\r?\n/)`. Within the function it is returned to the caller; assigned or updated later in the function.

function normalizeSiteContent(content = {})

To understand `normalizeSiteContent`, start with the problem it owns. `normalizeSiteContent` standardizes `SiteContent` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeSiteContent(content = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `content`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `content` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `normalizeSiteContent` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `normalizeSiteContent`, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Inside `normalizeSiteContent`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function normalizeProfile(profileValue = {})

Read `normalizeProfile` first as a named idea, not as syntax. `normalizeProfile` standardizes `Profile` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeProfile(profileValue = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `profileValue`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `profileValue` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `normalizeProfile` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `normalizeProfile`, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Inside `normalizeProfile`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function mailComposeLink(email = "")

The best entry point for `mailComposeLink` is its responsibility in the surrounding workflow. `mailComposeLink` belongs to profile and site content. This function renders or normalizes owner identity, front-page text, contact links, fun facts, or custom main-page sections. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function mailComposeLink(email = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `email`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `email` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `mailComposeLink` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `mailComposeLink`, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

The easiest way to follow the body of `mailComposeLink` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(email || "").trim()`.

Within the function it is returned to the caller; passed into `encodeURIComponent`; assigned or updated later in the function.

function phoneLink(phone = "")

Begin with the role of phoneLink. phoneLink belongs to profile and site content. This function renders or normalizes owner identity, front-page text, contact links, fun facts, or custom main-page sections. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function phoneLink(phone = "") { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: phone. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. phone is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means phoneLink performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without phoneLink, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

The local state inside phoneLink explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(phone || "").trim()`. Within the function it is returned to the caller; accesses properties/methods replace; assigned or updated later in the function.

function looksLikeWebOrContactText(value = "")

To understand looksLikeWebOrContactText, start with the problem it owns. looksLikeWebOrContactText belongs to profile and site content. This function renders or normalizes owner identity, front-page text, contact links, fun facts, or custom main-page sections. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function looksLikeWebOrContactText(value = "") { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: value. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and

generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `looksLikeBareWebAddress`. Those calls show that `looksLikeWebOrContactText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `looksLikeWebOrContactText`, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

After the main flow, the working values inside `looksLikeWebOrContactText` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(value || "").trim()`.

Within the function it is returned to the caller; accesses properties/methods `split`; passed into `test`; assigned or updated later in the function.

function `isWebsiteLinkItem(item = {}, target = "")`

Begin with the role of `isWebsiteLinkItem`. `isWebsiteLinkItem` is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

The syntax of the function is:

```
function isWebsiteLinkItem(item = {}, target = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `item`, `target`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `target` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `isWebsiteLinkItem` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `isWebsiteLinkItem`, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

The local state inside `isWebsiteLinkItem` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(target || "").trim()`. Within the function it is passed into `test`; assigned or updated later in the function.

typeText: `typeText` stores a list for temporary calculation. It is initialized from `[item.kind, item.type, item.status, item.meta, item.label, item.title]`. Within the function it is passed into `test`; assigned or updated later in the function.

function inferResponsiveProfile(project)

The best entry point for `inferResponsiveProfile` is its responsibility in the surrounding workflow. `inferResponsiveProfile` makes an educated classification from incomplete evidence. It looks at names, extensions, source text, repository state, or runtime state and returns the app's best decision so the next operation can choose the correct path.

The syntax of the function is:

```
function inferResponsiveProfile(project) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `project`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Its connection to the rest of the file matters as much as its local code. It works with `sectionHasRenderableContent`, `responsiveNodeCount`, `responsiveNodeHasMedia`, `hasPublicTemplate`. Those calls show that `inferResponsiveProfile` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `inferResponsiveProfile`, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

The easiest way to follow the body of `inferResponsiveProfile` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

sections: `sections` stores a computed value for portfolio data state. It is initialized from `project?.portfolioView?.sections || []`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`; assigned or updated later in the function.

visibleSections: visibleSections stores a function or callback value for portfolio data state. It is initialized from `sections.filter((section) => section?.id !== "brief" && sectionHasRenderableContent(section))`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `reduce`, `some`; assigned or updated later in the function.

itemCount: itemCount stores a computed value for temporary calculation. It is initialized from `visibleSections.reduce()`. Within the function it is assigned or updated later in the function.

maxDepth: maxDepth stores a computed value for temporary calculation. It is initialized from `visibleSections.reduce()`. Within the function it is assigned or updated later in the function.

hasMedia: hasMedia stores a function or callback value for temporary calculation. It is initialized from `visibleSections.some((section) => responsiveNodeHasMedia(section))`. Within the function it is assigned or updated later in the function.

density: density stores a computed value for temporary calculation. It is initialized from `itemCount >= 18 || visibleSections.length >= 6 || maxDepth >= 4`. Within the function it is assigned or updated later in the function.

function projectResponsiveProfile(project)

Read `projectResponsiveProfile` first as a named idea, not as syntax. `projectResponsiveProfile` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

The syntax of the function is:

```
function projectResponsiveProfile(project) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `project`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

The function also has to be read through the helpers it calls. It works with `inferResponsiveProfile`. Those calls show that `projectResponsiveProfile` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `projectResponsiveProfile`, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

The local objects and variables inside `projectResponsiveProfile` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

savedProfile: savedProfile stores a computed value for portfolio data state. It is initialized from `project?.portfolioView?.responsive`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

Rich content rendering

The functions in this group all support rich content rendering. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function renderFunFacts()

Read `renderFunFacts` first as a named idea, not as syntax. `renderFunFacts` turns saved data for FunFacts into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

The syntax of the function is:

```
function renderFunFacts() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

The function also has to be read through the helpers it calls. It works with `normalizeFunFacts`, `renderRichContent`, `renderInlineMath`. Those calls show that `renderFunFacts` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `renderFunFacts`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

The local objects and variables inside `renderFunFacts` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

facts: facts stores a computed value for temporary calculation. It is initialized from `normalizeFunFacts(funFacts)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `join`, `length`, `map`; assigned or updated later in the function.

hasRichFacts: hasRichFacts stores a computed value for temporary calculation. It is initialized from `Boolean(funFactsRich?.blocks?.length)`. Within the function it is assigned or updated later in the function.

function escapeHtml(value)

Read `escapeHtml` first as a named idea, not as syntax. `escapeHtml` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function escapeHtml(value) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `escapeHtml` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `escapeHtml`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Inside `escapeHtml`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function normalizeCodeLanguage(value = "")

Begin with the role of `normalizeCodeLanguage`. `normalizeCodeLanguage` standardizes `CodeLanguage` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeCodeLanguage(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `normalizeCodeLanguage` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `normalizeCodeLanguage`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

The local state inside `normalizeCodeLanguage` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(value || "").trim().toLowerCase().replace(/[_-]+/g, " ")`. Within the function it is passed into `includes`; assigned or updated later in the function.

match: `match` stores a function or callback value for temporary calculation. It is initialized from `supportedCodeLanguages.find((language) => ...)`. Within the function it is returned to the caller; assigned or updated later in the function.

function `codeLanguageLabel(value = "")`

The best entry point for `codeLanguageLabel` is its responsibility in the surrounding workflow. `codeLanguageLabel` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function codeLanguageLabel(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. It works with `normalizeCodeLanguage`. Those calls show that `codeLanguageLabel` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `codeLanguageLabel`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Inside `codeLanguageLabel`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function normalizeCodePasteMode(value = "")`

Read `normalizeCodePasteMode` first as a named idea, not as syntax. `normalizeCodePasteMode` standardizes `CodePasteMode` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeCodePasteMode(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `normalizeCodePasteMode` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `normalizeCodePasteMode`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Inside `normalizeCodePasteMode`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function normalizeCodeText(value = "", pasteMode = "source")`

To understand `normalizeCodeText`, start with the problem it owns. `normalizeCodeText` standardizes `CodeText` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeCodeText(value = "", pasteMode = "source") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`, `pasteMode`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `pasteMode` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `normalizeCodePasteMode`. Those calls show that `normalizeCodeText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `normalizeCodeText`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

After the main flow, the working values inside `normalizeCodeText` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

normalized: `normalized` stores a computed value for temporary calculation. It is initialized from `String(value || "").replace(/r\n?/g, "\n").replace(/u00a0/g, " ")`. Within the function it is returned to the caller; accesses properties/methods `replace`; assigned or updated later in the function.

function detectCodeLanguage(value = "")

Begin with the role of `detectCodeLanguage`. `detectCodeLanguage` makes an educated classification from incomplete evidence. It looks at names, extensions, source text, repository state, or runtime state and returns the app's best decision so the next operation can choose the correct path.

The syntax of the function is:

```
function detectCodeLanguage(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `sourceLooksCpp`, `sourceLooksC`. Those calls show that `detectCodeLanguage` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `detectCodeLanguage`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

The local state inside `detectCodeLanguage` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

code: code stores a computed value for temporary calculation. It is initialized from `String(value || "")`. Within the function it is passed into `sourceLooksC`, `sourceLooksCpp`, `test`; assigned or updated later in the function.

function `tokenizedCodeHtml(code = "", language = "javascript")`

To understand `tokenizedCodeHtml`, start with the problem it owns. `tokenizedCodeHtml` is the lightweight syntax highlighter used by the public site. It escapes the code first, then wraps comments, strings, numbers, keywords, preprocessor directives, and other recognized tokens in spans with CSS classes. It supports the major languages that the builder exposes, including C, C++, SystemVerilog, LTspice-oriented text, Java, JavaScript, Python, and HTML-like code. The function exists so code added to a project reads like code rather than plain paragraph text. Without it, source examples in the portfolio would be visually flat, harder to scan, and less convincing to technical recruiters.

The syntax of the function is:

```
function tokenizedCodeHtml(code = "", language = "javascript") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `code`, `language`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `escapeHtml`, `normalizeCodeLanguage`. Those calls show that `tokenizedCodeHtml` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `tokenizedCodeHtml`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

After the main flow, the working values inside `tokenizedCodeHtml` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

html: html stores a computed value for temporary calculation. It is initialized from `escapeHtml(code)`. Within the function it is returned to the caller; accesses properties/methods replace; assigned or updated later in the function.

tokens: tokens stores a list for temporary calculation. It is initialized from []. Within the function it is mutated as a collection or DOM container; accesses properties/methods length, push; assigned or updated later in the function.

protect: protect stores a function or callback value for temporary calculation. It is initialized from (pattern, className) => {}. Within the function it is assigned or updated later in the function.

token: token stores a string value for temporary calculation. It is initialized from `@@CODE\_TOKEN\_\${tokens.length}@@`. Within the function it is returned to the caller; passed into protect; assigned or updated later in the function.

commentPattern: commentPattern stores a list for temporary calculation. It is initialized from ["python", "Itspice"].includes(normalizeCodeLanguage(language)). Within the function it is passed into protect; assigned or updated later in the function.

keywordMap: keywordMap stores a structured object for temporary calculation. It is initialized from {}. Within the function it is accesses properties/methods javascript; assigned or updated later in the function.

keywords: keywords stores a computed value for lookup collection. It is initialized from keywordMap[normalizeCodeLanguage(language)] || keywordMap.javascript. Within the function it is passed into replace; assigned or updated later in the function.

function renderRichCodeBlock(block = {})

Begin with the role of renderRichCodeBlock. renderRichCodeBlock turns saved data for RichCodeBlock into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

The syntax of the function is:

```
function renderRichCodeBlock(block = {}) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: block. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. block is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The surrounding workflow becomes clearer when you notice its collaborators. It works with normalizeCodeLanguage, detectCodeLanguage, normalizeCodeText, escapeHtml, codeLanguageLabel, tokenizedCodeHtml. Those calls show that renderRichCodeBlock is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without renderRichCodeBlock, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

The local state inside `renderRichCodeBlock` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

language: `language` stores a computed value for compiler or language configuration. It is initialized from `normalizeCodeLanguage(block.language || detectCodeLanguage(block.code || ""))`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `escapeHtml`, `normalizeCodeLanguage`, `tokenizedCodeHtml`; assigned or updated later in the function.

code: `code` stores a computed value for temporary calculation. It is initialized from `normalizeCodeText(block.code || "", block.pasteMode || "source")`. Within the function it is passed into `normalizeCodeLanguage`, `normalizeCodeText`, `tokenizedCodeHtml`; assigned or updated later in the function.

function renderPlainMultiline(value = "")

The best entry point for `renderPlainMultiline` is its responsibility in the surrounding workflow. `renderPlainMultiline` turns saved data for `PlainMultiline` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

The syntax of the function is:

```
function renderPlainMultiline(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. It works with `escapeHtml`. Those calls show that `renderPlainMultiline` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `renderPlainMultiline`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Inside `renderPlainMultiline`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderSiteContent()

Begin with the role of `renderSiteContent`. `renderSiteContent` turns saved data for `SiteContent` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

The syntax of the function is:

```
function renderSiteContent() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `normalizeSiteContent`, `renderRichFieldContent`, `applyPlainFieldStyle`. Those calls show that `renderSiteContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `renderSiteContent`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

The local state inside `renderSiteContent` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

content: content stores a computed value for temporary calculation. It is initialized from `normalizeSiteContent(siteContent || {})`. Within the function it is accesses properties/methods `heroCopy`, `heroEyebrow`, `heroTitle`; passed into `renderRichFieldContent`; assigned or updated later in the function.

function renderProfile()

To understand `renderProfile`, start with the problem it owns. `renderProfile` applies owner identity to the public page. It updates the displayed name, tagline, profile photo, brand/title text, email link, phone link, GitHub/resume links, contact cards, resume viewer, metadata hints, and social/contact links. It is the bridge between saved profile data and the visible recruiter-facing contact section. The function is important because the same website shell can serve Maurice's portfolio or a fresh user's portfolio. Profile data cannot be hardcoded if the builder is intended to be reusable. Without `renderProfile`, the page would either remain generic or require manual HTML editing for every owner.

The syntax of the function is:

```
function renderProfile() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser DOM. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Next, trace the helper calls that surround the function. It works with `normalizeProfile`, `mailComposeLink`, `phoneLink`, `normalizeLinkTarget`, `renderRichFieldContent`, `applyPlainFieldStyle`, `plainFieldStyleAttribute`. Those calls show that `renderProfile` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `renderProfile`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

After the main flow, the working values inside `renderProfile` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

current: `current` stores a computed value for temporary calculation. It is initialized from `normalizeProfile(profile || {})`. Within the function it is accesses properties/methods `brandImage`, `brandText`, `contactIntro`, `displayName`, `email`, `githubUrl`, `heroImage`, `linkedinUrl`; passed into `mailComposeLink`, `normalizeLinkTarget`, `phoneLink`, `renderRichFieldContent`; assigned or updated later in the function.

displayName: `displayName` stores a computed value for temporary calculation. It is initialized from `current.displayName || "Portfolio"`. Within the function it is accesses properties/methods `slice`; passed into `renderRichFieldContent`, `setAttribute`; assigned or updated later in the function.

label: `label` stores a computed value for temporary calculation. It is initialized from `current.portfolioLabel || "Portfolio"`. Within the function it is passed into `applyPlainFieldStyle`, `renderRichFieldContent`, `setAttribute`; assigned or updated later in the function.

emailLink: `emailLink` stores a computed value for temporary calculation. It is initialized from `mailComposeLink(current.email)`. Within the function it is assigned or updated later in the function.

callLink: `callLink` stores a computed value for temporary calculation. It is initialized from `phoneLink(current.phone)`. Within the function it is assigned or updated later in the function.

links: `links` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; passed into `renderRichFieldContent`; assigned or updated later in the function.

heroGithubLink: `heroGithubLink` stores a list for Git publishing and authorization state. It is initialized from `[...document.querySelectorAll(".hero-actions a")]`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods `hidden`, `href`; assigned or updated later in the function.

resumeObject: `resumeObject` stores a DOM element reference. It is initialized from `resumeSection.querySelector("object")`. Within the function it is accesses properties/methods `data`; assigned or updated later in the function.

fallbackLink: `fallbackLink` stores a DOM element reference. It is initialized from `resumeSection.querySelector("object a")`. Within the function it is accesses properties/methods `href`; assigned or updated later in the function.

styleId: styleId stores a computed value for appearance configuration. It is initialized from link.label === "GitHub". Within the function it is passed into plainFieldStyleAttribute; assigned or updated later in the function.

function renderInlineMath(text)

To understand renderInlineMath, start with the problem it owns. renderInlineMath turns saved data for InlineMath into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

The syntax of the function is:

```
function renderInlineMath(text) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: text. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. text is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Next, trace the helper calls that surround the function. It works with escapeHtml, normalizeLinkTarget. Those calls show that renderInlineMath is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without renderInlineMath, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

After the main flow, the working values inside renderInlineMath reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

escaped: escaped stores a computed value for temporary calculation. It is initialized from escapeHtml(text). Within the function it is accesses properties/methods replace; assigned or updated later in the function.

withMath: withMath stores a computed value for temporary calculation. It is initialized from escaped.replace(/\\$([^\\$]+)\\$/g, '\$1'). Within the function it is returned to the caller; accesses properties/methods replace; assigned or updated later in the function.

trailing: trailing stores a computed value for temporary calculation. It is initialized from match.match(/I),... Within the function it is accesses properties/methods length; passed into slice; assigned or updated later in the function.

clean: clean stores a computed value for temporary calculation. It is initialized from trailing ? match.slice(0, -trailing.length) : match. Within the function it is passed into normalizeLinkTarget; assigned or updated later in the function.

href: href stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(clean, { assumeWeb: true })`. Within the function it is assigned or updated later in the function.

function renderMultilineInlineText(text = "")

The best entry point for `renderMultilineInlineText` is its responsibility in the surrounding workflow.

`renderMultilineInlineText` turns saved data for `MultilineInlineText` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

The syntax of the function is:

```
function renderMultilineInlineText(text = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `text`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `text` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. It works with `renderInlineMath`. Those calls show that `renderMultilineInlineText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `renderMultilineInlineText`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Inside `renderMultilineInlineText`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function sanitizeRichInlineHtml(value = "")

To understand `sanitizeRichInlineHtml`, start with the problem it owns. `sanitizeRichInlineHtml` is the safety filter for rich inline HTML. Builder editors may store spans, links, bold text, colors, and other inline markup, but the public website should not blindly inject arbitrary HTML. This function removes unsafe tags and attributes while preserving the formatting the portfolio actually needs. The function prevents a content field from becoming a script injection surface. It works with `renderRichContent` and `renderRichFieldContent`, which need formatted HTML but should only receive controlled markup.

The syntax of the function is:

```
function sanitizeRichInlineHtml(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: creates DOM elements instead of relying only on static HTML; writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Next, trace the helper calls that surround the function. It works with `normalizeLinkTarget`, `cleanFontFamily`, `normalizeFontPx`, `normalizeTextColor`, `linkifyRichTextNodes`. Those calls show that `sanitizeRichInlineHtml` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `sanitizeRichInlineHtml`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

After the main flow, the working values inside `sanitizeRichInlineHtml` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

template: `template` stores a DOM element reference. It is initialized from `document.createElement("template")`. Within the function it is returned to the caller; accesses properties/methods `content`, `innerHTML`; passed into `createElement`, `linkifyRichTextNodes`; assigned or updated later in the function.

allowedTags: `allowedTags` stores a constructed object for lookup collection. It is initialized from `new Set(["A", "B", "BR", "EM", "I", "SPAN", "STRONG", "U"])`. Within the function it is accesses properties/methods `has`; assigned or updated later in the function.

href: `href` stores a computed value for temporary calculation. It is initialized from `node.tagName === "A" ? String(node.getAttribute("href") || "").trim() : ""`. Within the function it is passed into `String`, `normalizeLinkTarget`, `setAttribute`; assigned or updated later in the function.

normalizedHref: `normalizedHref` stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(href, { assumeWeb: true })`. Within the function it is passed into `test`; assigned or updated later in the function.

safeHref: `safeHref` stores a computed value for temporary calculation. It is initialized from `/(https?:|mailto:|tel:|#|V)/i.test(normalizedHref) ? normalizedHref : ""`. Within the function it is passed into `setAttribute`; assigned or updated later in the function.

fontFamily: `fontFamily` stores a computed value for temporary calculation. It is initialized from `cleanFontFamily(node.style.fontFamily || "")`. Within the function it is passed into `cleanFontFamily`; assigned or updated later in the function.

fontPx: `fontPx` stores a computed value for temporary calculation. It is initialized from `normalizeFontPx(parseFloat(node.style.fontSize || ""))`. Within the function it is assigned or updated later in the function.

color: `color` stores a computed value for appearance configuration. It is initialized from `normalizeTextColor(node.style.color || "")`. Within the function it is passed into `normalizeTextColor`; assigned or updated later in the function.

rawFontWeight: rawFontWeight stores a computed value for temporary calculation. It is initialized from `node.style.fontWeight || ""`. Within the function it is passed into `test`; assigned or updated later in the function.

fontWeight: fontWeight stores a computed value for temporary calculation. It is initialized from `/(bold|[6-9]00)/i.test(rawFontWeight)`. Within the function it is assigned or updated later in the function.

fontStyle: fontStyle stores a computed value for appearance configuration. It is initialized from `node.style.fontStyle === "italic" ? "italic" : ""`. Within the function it is assigned or updated later in the function.

textDecoration: textDecoration stores a computed value for temporary calculation. It is initialized from `node.style.textDecoration.includes("underline") ? "underline" : ""`. Within the function it accesses properties/methods `includes`; assigned or updated later in the function.

function richTextStyle(block = {})

Begin with the role of `richTextStyle`. `richTextStyle` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function richTextStyle(block = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `block`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `block` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `cleanFontFamily`, `normalizeFontPx`, `normalizeTextColor`, `escapeHtml`. Those calls show that `richTextStyle` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `richTextStyle`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

The local state inside `richTextStyle` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

styles: styles stores a list for appearance configuration. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `join`, `length`, `push`; passed into `escapeHtml`; assigned or updated later in the function.

fontFamily: fontFamily stores a computed value for temporary calculation. It is initialized from `cleanFontFamily(block.fontFamily || "Arial") || "Arial"`. Within the function it is passed into `cleanFontFamily`, `push`; assigned or updated later in the function.

fontPx: fontPx stores a computed value for temporary calculation. It is initialized from `normalizeFontPx(block.fontPx)`. Within the function it is passed into `normalizeFontPx`, push; assigned or updated later in the function.

color: color stores a computed value for appearance configuration. It is initialized from `normalizeTextColor(block.color || "")`. Within the function it is passed into `normalizeTextColor`, push; assigned or updated later in the function.

function richImageCropStyle(block = {})

Read `richImageCropStyle` first as a named idea, not as syntax. `richImageCropStyle` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function richImageCropStyle(block = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `block`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `block` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The function also has to be read through the helpers it calls. It works with `normalizeCropAspect`, `normalizeCropZoom`, `normalizeCropPosition`, `escapeHtml`. Those calls show that `richImageCropStyle` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `richImageCropStyle`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

The local objects and variables inside `richImageCropStyle` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

aspect: aspect stores a computed value for temporary calculation. It is initialized from `normalizeCropAspect(block.cropAspect)`. Within the function it is passed into `push`; assigned or updated later in the function.

styles: styles stores a list for appearance configuration. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `push`; passed into `escapeHtml`; assigned or updated later in the function.

function richImageDownloadLink(block = {})

The best entry point for `richImageDownloadLink` is its responsibility in the surrounding workflow. `richImageDownloadLink` belongs to rich content rendering. This function turns saved builder content into public

HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function richImageDownloadLink(block = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `block`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `block` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Its connection to the rest of the file matters as much as its local code. It works with `escapeHtml`, `cleanRichImageTitle`, `normalizeLinkTarget`, `isWebsiteLinkItem`, `linkAttributes`, `downloadAttribute`. Those calls show that `richImageDownloadLink` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `richImageDownloadLink`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

The easiest way to follow the body of `richImageDownloadLink` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

label: `label` stores a computed value for temporary calculation. It is initialized from `escapeHtml(cleanRichImageTitle(block) || block.caption || "Image file")`. Within the function it is assigned or updated later in the function.

url: `url` stores a computed value for network or routing value. It is initialized from `block.url || "#"`. Within the function it is passed into `normalizeLinkTarget`; assigned or updated later in the function.

target: `target` stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(block, url) })`. Within the function it is passed into `downloadAttribute`, `escapeHtml`, `linkAttributes`; assigned or updated later in the function.

function `cleanRichImageTitle(block = {})`

Read `cleanRichImageTitle` first as a named idea, not as syntax. `cleanRichImageTitle` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function cleanRichImageTitle(block = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `block`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `block` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `cleanRichImageTitle` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `cleanRichImageTitle`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

The local objects and variables inside `cleanRichImageTitle` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

title: `title` stores a computed value for temporary calculation. It is initialized from `String(block.title || "").trim()`. Within the function it is passed into `String, test`; assigned or updated later in the function.

function renderRichContent(rich, fallbackText = "")

To understand `renderRichContent`, start with the problem it owns. `renderRichContent` is the renderer for builder-created rich content. It accepts blocks representing paragraphs, images, formulas, files, links, code blocks, captions, and styled text, then converts those blocks into safe public HTML. It decides whether an image should appear inline, whether a file should be downloadable, how captions attach, and how text styles survive into the website. This function matters because the builder is not just saving plain text. It supports formatted explanations, pasted images, formulas, code, and files. Without `renderRichContent`, much of what the builder collects would be flattened or lost when the portfolio is published.

The syntax of the function is:

```
function renderRichContent(rich, fallbackText = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `rich`, `fallbackText`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `rich` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `fallbackText` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Next, trace the helper calls that surround the function. It works with `textBlocksFromPlainText`, `richImageDownloadLink`, `cleanRichImageTitle`, `normalizeLinkTarget`, `normalizeCropAspect`, `richImageCropStyle`, `escapeHtml`, `unwrapFormula`. Those calls show that `renderRichContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `renderRichContent`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

After the main flow, the working values inside `renderRichContent` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

blocks: `blocks` stores a computed value for temporary calculation. It is initialized from `rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText)`. Within the function it is iterated or transformed as a collection; accesses properties/methods map; assigned or updated later in the function.

align: `align` stores a list for temporary calculation. It is initialized from `["left", "center", "right"].includes(block.align) ? block.align : "left"`. Within the function it is passed into `includes`; assigned or updated later in the function.

title: `title` stores a computed value for temporary calculation. It is initialized from `cleanRichImageTitle(block)`. Within the function it is passed into `escapeHtml`; assigned or updated later in the function.

imageSrc: `imageSrc` stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(block.url, { assumeWeb: true })`. Within the function it is passed into `escapeHtml`; assigned or updated later in the function.

formula: `formula` stores a computed value for temporary calculation. It is initialized from `unwrapFormula(block.formula)`. Within the function it is passed into `escapeHtml`, `renderInlineMath`, `unwrapFormula`; assigned or updated later in the function.

size: `size` stores a list for temporary calculation. It is initialized from `["small", "normal", "large"].includes(block.fontSize) ? block.fontSize : "normal"`. Within the function it is assigned or updated later in the function.

content: `content` stores a computed value for temporary calculation. It is initialized from `block.html`. Within the function it is assigned or updated later in the function.

function renderRichFieldContent(rich, fallbackText = "")

To understand `renderRichFieldContent`, start with the problem it owns. `renderRichFieldContent` turns saved data for `RichFieldContent` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

The syntax of the function is:

```
function renderRichFieldContent(rich, fallbackText = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `rich`, `fallbackText`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `rich` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `fallbackText` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Next, trace the helper calls that surround the function. It works with `textBlocksFromPlainText`, `sanitizeRichInlineHtml`, `renderInlineMath`, `richTextStyle`. Those calls show that `renderRichFieldContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `renderRichFieldContent`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

After the main flow, the working values inside `renderRichFieldContent` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

blocks: `blocks` stores a computed value for temporary calculation. It is initialized from `rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText)`. Within the function it is returned to the caller; iterated or transformed as a collection; accesses properties/methods map; assigned or updated later in the function.

align: `align` stores a list for temporary calculation. It is initialized from `["left", "center", "right"].includes(block.align) ? block.align : "left"`. Within the function it is passed into `includes`; assigned or updated later in the function.

content: `content` stores a computed value for temporary calculation. It is initialized from `block.html`. Within the function it is assigned or updated later in the function.

function `richTextTerms(rich)`

Read `richTextTerms` first as a named idea, not as syntax. `richTextTerms` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function richTextTerms(rich) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `rich`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `rich` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `richTextTerms` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `richTextTerms`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Inside `richTextTerms`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderDynamicLinks(items = [])

Read `renderDynamicLinks` first as a named idea, not as syntax. `renderDynamicLinks` turns saved data for `DynamicLinks` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

The syntax of the function is:

```
function renderDynamicLinks(items = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `items`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `items` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The function also has to be read through the helpers it calls. It works with `resourceLink`. Those calls show that `renderDynamicLinks` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `renderDynamicLinks`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

The local objects and variables inside `renderDynamicLinks` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

links: links stores a function or callback value for temporary calculation. It is initialized from `items.filter((item) => item?.url && (item.label || item.title))`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; assigned or updated later in the function.

function collectRichDownloads(rich, fallbackType = "File", output = [])

The best entry point for `collectRichDownloads` is its responsibility in the surrounding workflow.

`collectRichDownloads` gathers scattered nested data into a shape that another function can loop over. This is important because portfolio data is hierarchical while rendering, search, and AI context often need a flat list.

The syntax of the function is:

```
function collectRichDownloads(rich, fallbackType = "File", output = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `rich`, `fallbackType`, `output`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `rich` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `fallbackType` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `output` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. It works with `cleanRichImageTitle`, `fileNameFromUrl`. Those calls show that `collectRichDownloads` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `collectRichDownloads`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Inside `collectRichDownloads`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderDownloadBlock(title, items = [])

The best entry point for `renderDownloadBlock` is its responsibility in the surrounding workflow.

`renderDownloadBlock` turns saved data for `DownloadBlock` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

The syntax of the function is:

```
function renderDownloadBlock(title, items = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `title`, `items`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `title` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `items` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Its connection to the rest of the file matters as much as its local code. It works with `uniqueDownloads`, `detailBlock`, `escapeHtml`, `resourceLink`, `renderMultilineInlineText`. Those calls show that `renderDownloadBlock` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `renderDownloadBlock`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

The easiest way to follow the body of `renderDownloadBlock` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

downloads: `downloads` stores a computed value for temporary calculation. It is initialized from `uniqueDownloads(items)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; assigned or updated later in the function.

function `renderParsedBriefBlock(section, fallbackSummary = "")`

Read `renderParsedBriefBlock` first as a named idea, not as syntax. `renderParsedBriefBlock` turns saved data for `ParsedBriefBlock` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

The syntax of the function is:

```
function renderParsedBriefBlock(section, fallbackSummary = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `section`, `fallbackSummary`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `fallbackSummary` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The function also has to be read through the helpers it calls. It works with `renderRichContent`. Those calls show that `renderParsedBriefBlock` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `renderParsedBriefBlock`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

The local objects and variables inside `renderParsedBriefBlock` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

briefItem: `briefItem` stores a computed value for temporary calculation. It is initialized from `section?.items?.[0] || {}`. Within the function it is accesses properties/methods `description`, `rich`; passed into `renderRichContent`; assigned or updated later in the function.

briefText: `briefText` stores a computed value for temporary calculation. It is initialized from `briefItem.description || fallbackSummary || ""`. Within the function it is passed into `renderRichContent`; assigned or updated later in the function.

function richHasRenderableContent(rich)

To understand `richHasRenderableContent`, start with the problem it owns. `richHasRenderableContent` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function richHasRenderableContent(rich) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `rich`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `rich` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `richHasRenderableContent` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `richHasRenderableContent`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Inside `richHasRenderableContent`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function richContainsMedia(rich)

To understand `richContainsMedia`, start with the problem it owns. `richContainsMedia` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function richContainsMedia(rich) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `rich`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `rich` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `richContainsMedia` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `richContainsMedia`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Inside `richContainsMedia`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderProjects()

Read `renderProjects` first as a named idea, not as syntax. `renderProjects` rebuilds the visible project directory from current state. It checks the active category filter and search query, groups visible projects by category, creates the category sections, and updates the summary count. It is called after catalog load, filter changes, and search changes. The function is the public directory refresh mechanism. Without it, filters and searches might update state internally but the page would not visually change.

The syntax of the function is:

```
function renderProjects() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local

constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

The function also has to be read through the helpers it calls. It works with `normalize`, `projectVisible`, `queueSearchHighlights`, `categorySection`. Those calls show that `renderProjects` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `renderProjects`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

The local objects and variables inside `renderProjects` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

query: `query` stores a computed value for temporary calculation. It is initialized from `normalize(searchInput.value).trim()`. Within the function it is passed into `projectVisible`; assigned or updated later in the function.

visible: `visible` stores a function or callback value for temporary calculation. It is initialized from `projects.filter((project) => projectVisible(project, query))`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `length`; assigned or updated later in the function.

visibleProjects: `visibleProjects` stores a function or callback value for portfolio data state. It is initialized from `visible.filter((project) => project.category === category.id)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is returned to the caller; accesses properties/methods `length`; passed into `categorySection`; assigned or updated later in the function.

shouldShowEmptyCategory: `shouldShowEmptyCategory` stores a computed value for portfolio data state. It is initialized from `!query && (activeFilter === "all" || activeFilter === category.id)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

Project windows and navigation

The functions in this group all support project windows and navigation. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function clampSectionDialogPosition(left, top, dialog)

The best entry point for `clampSectionDialogPosition` is its responsibility in the surrounding workflow. `clampSectionDialogPosition` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function clampSectionDialogPosition(left, top, dialog) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `left`, `top`, `dialog`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `left` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `top` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `clampSectionDialogPosition` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `clampSectionDialogPosition`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The easiest way to follow the body of `clampSectionDialogPosition` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

rect: `rect` stores a computed value for temporary calculation. It is initialized from `dialog.getBoundingClientRect()`. Within the function it is accesses properties/methods `height`, `width`; passed into `max`; assigned or updated later in the function.

margin: `margin` stores a numeric value for temporary calculation. It is initialized from `12`. Within the function it is passed into `max`, `min`; assigned or updated later in the function.

maxLeft: `maxLeft` stores a computed value for temporary calculation. It is initialized from `Math.max(margin, window.innerWidth - rect.width - margin)`. Within the function it is assigned or updated later in the function.

maxTop: `maxTop` stores a computed value for temporary calculation. It is initialized from `Math.max(margin, window.innerHeight - rect.height - margin)`. Within the function it is assigned or updated later in the function.

function anchorSectionDialog(dialog)

Begin with the role of `anchorSectionDialog`. `anchorSectionDialog` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function anchorSectionDialog(dialog) { ... }
```


There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `dialog`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `clampSectionDialogPosition`. Those calls show that `anchorSectionDialog` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `anchorSectionDialog`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local state inside `anchorSectionDialog` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

rect: `rect` stores a computed value for temporary calculation. It is initialized from `dialog.getBoundingClientRect()`. Within the function it is accesses properties/methods `left`, `top`; passed into `clampSectionDialogPosition`; assigned or updated later in the function.

position: `position` stores a computed value for temporary calculation. It is initialized from `clampSectionDialogPosition(rect.left, rect.top, dialog)`. Within the function it is accesses properties/methods `left`, `top`; assigned or updated later in the function.

function ensureSectionResizeHandles(dialog)

Begin with the role of `ensureSectionResizeHandles`. `ensureSectionResizeHandles` creates or repairs something only when it is needed. This pattern avoids duplicate DOM nodes, duplicate repositories, duplicate handlers, or missing folders while still letting later code assume the resource exists.

The syntax of the function is:

```
function ensureSectionResizeHandles(dialog) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `dialog`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing

history, or mutating controlled runtime state. A close reading of the body shows these implementation details: creates DOM elements instead of relying only on static HTML.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `ensureSectionResizeHandles` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `ensureSectionResizeHandles`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local state inside `ensureSectionResizeHandles` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

handle: handle stores a DOM element reference. It is initialized from `document.createElement("div")`. Within the function it is accesses properties/methods `className`, `dataset`, `setAttribute`; passed into `append`; assigned or updated later in the function.

function anchorSectionDialogForResize(dialog)

To understand `anchorSectionDialogForResize`, start with the problem it owns. `anchorSectionDialogForResize` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function anchorSectionDialogForResize(dialog) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `dialog`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

Next, trace the helper calls that surround the function. It works with `anchorSectionDialog`. Those calls show that `anchorSectionDialogForResize` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `anchorSectionDialogForResize`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

After the main flow, the working values inside `anchorSectionDialogForResize` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

rect: rect stores a computed value for temporary calculation. It is initialized from `dialog.getBoundingClientRect()`. Within the function it is accesses properties/methods height, width; assigned or updated later in the function.

function sectionDialogResizeBounds(state, event)

Read `sectionDialogResizeBounds` first as a named idea, not as syntax. `sectionDialogResizeBounds` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sectionDialogResizeBounds(state, event) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `state`, `event`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `state` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `event` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `sectionDialogResizeBounds` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `sectionDialogResizeBounds`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local objects and variables inside `sectionDialogResizeBounds` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

margin: margin stores a numeric value for temporary calculation. It is initialized from 12. Within the function it is passed into `min`; assigned or updated later in the function.

minWidth: `minWidth` stores a computed value for temporary calculation. It is initialized from `Math.min(320, Math.max(180, window.innerWidth - margin * 2))`. Within the function it is passed into `min`; assigned or updated later in the function.

minHeight: `minHeight` stores a computed value for temporary calculation. It is initialized from `Math.min(170, Math.max(120, window.innerHeight - margin * 2))`. Within the function it is passed into `min`; assigned or updated later in the function.

direction: `direction` stores a computed value for filesystem location. It is initialized from `state.direction`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler

artifacts, or published assets. Within the function it is accesses properties/methods includes; assigned or updated later in the function.

destructured { left, top, width, height }: destructured { left, top, width, height } stores a computed value for imported or unpacked API handles. It is initialized from state.rect. It unpacks API handles from another object, giving the rest of the file short direct names for those APIs.

dx: dx stores a computed value for temporary calculation. It is initialized from event.clientX - state.startX. Within the function it is passed into min; assigned or updated later in the function.

dy: dy stores a computed value for temporary calculation. It is initialized from event.clientY - state.startY. Within the function it is passed into min; assigned or updated later in the function.

right: right stores a computed value for temporary calculation. It is initialized from state.rect.left + state.rect.width. Within the function it is assigned or updated later in the function.

bottom: bottom stores a computed value for temporary calculation. It is initialized from state.rect.top + state.rect.height. Within the function it is assigned or updated later in the function.

function sectionDialogFromDragEvent(event)

To understand sectionDialogFromDragEvent, start with the problem it owns. sectionDialogFromDragEvent belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sectionDialogFromDragEvent(event) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: event. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. event is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means sectionDialogFromDragEvent performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without sectionDialogFromDragEvent, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

After the main flow, the working values inside sectionDialogFromDragEvent reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

handle: handle stores a computed value for temporary calculation. It is initialized from `event.target.closest("[data-section-dialog-drag='true']")`. Within the function it is accesses properties/methods `closest`; assigned or updated later in the function.

dialog: dialog stores a computed value for DOM reference or UI state. It is initialized from `handle.closest("dialog")`. It anchors UI behavior to a specific browser element or window state. Within the function it is returned to the caller; passed into `closest`; assigned or updated later in the function.

function beginSectionDialogDrag(event)

Begin with the role of `beginSectionDialogDrag`. `beginSectionDialogDrag` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function beginSectionDialogDrag(event) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `event`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `event` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `sectionDialogFromDragEvent`, `anchorSectionDialog`. Those calls show that `beginSectionDialogDrag` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `beginSectionDialogDrag`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local state inside `beginSectionDialogDrag` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

dialog: dialog stores a computed value for DOM reference or UI state. It is initialized from `sectionDialogFromDragEvent(event)`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods `classList`, `getBoundingClientRect`; passed into `add`, `anchorSectionDialog`; assigned or updated later in the function.

rect: rect stores a computed value for temporary calculation. It is initialized from `dialog.getBoundingClientRect()`. Within the function it is accesses properties/methods `left`, `top`; assigned or updated later in the function.

function moveSectionDialogDrag(event)

Read `moveSectionDialogDrag` first as a named idea, not as syntax. `moveSectionDialogDrag` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function moveSectionDialogDrag(event) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `event`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `event` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

The function also has to be read through the helpers it calls. It works with `clampSectionDialogPosition`. Those calls show that `moveSectionDialogDrag` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `moveSectionDialogDrag`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local objects and variables inside `moveSectionDialogDrag` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

next: next stores a computed value for temporary calculation. It is initialized from `clampSectionDialogPosition()`. Within the function it is accesses properties/methods `left`, `top`; assigned or updated later in the function.

function endSectionDialogDrag()

Read `endSectionDialogDrag` first as a named idea, not as syntax. `endSectionDialogDrag` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function endSectionDialogDrag() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `endSectionDialogDrag` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `endSectionDialogDrag`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `endSectionDialogDrag`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function beginSectionDialogResize(event)

Begin with the role of `beginSectionDialogResize`. `beginSectionDialogResize` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function beginSectionDialogResize(event) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `event`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `event` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `anchorSectionDialogForResize`. Those calls show that `beginSectionDialogResize` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `beginSectionDialogResize`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local state inside `beginSectionDialogResize` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

handle: handle stores a computed value for temporary calculation. It is initialized from `event.target.closest("[data-section-dialog-resize-handle]")`. Within the function it is accesses properties/methods `dataset`; passed into `closest`; assigned or updated later in the function.

dialog: dialog stores a computed value for DOM reference or UI state. It is initialized from `handle?.closest("dialog")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods `classList`, `getBoundingClientRect`; passed into `add`, `anchorSectionDialogForResize`, `closest`; assigned or updated later in the function.

function moveSectionDialogResize(event)

Read `moveSectionDialogResize` first as a named idea, not as syntax. `moveSectionDialogResize` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function moveSectionDialogResize(event) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `event`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `event` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

The function also has to be read through the helpers it calls. It works with `sectionDialogResizeBounds`. Those calls show that `moveSectionDialogResize` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `moveSectionDialogResize`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local objects and variables inside `moveSectionDialogResize` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

next: next stores a computed value for temporary calculation. It is initialized from `sectionDialogResizeBounds(activeSectionDialogResize, event)`. Within the function it is accesses properties/methods `height`, `left`, `top`, `width`; assigned or updated later in the function.

function endSectionDialogResize()

Read `endSectionDialogResize` first as a named idea, not as syntax. `endSectionDialogResize` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function endSectionDialogResize() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `endSectionDialogResize` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `endSectionDialogResize`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `endSectionDialogResize`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function updateSectionDialogMinimize(dialog)

Read `updateSectionDialogMinimize` first as a named idea, not as syntax. `updateSectionDialogMinimize` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function updateSectionDialogMinimize(dialog) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `dialog`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `updateSectionDialogMinimize` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `updateSectionDialogMinimize`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local objects and variables inside `updateSectionDialogMinimize` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

minimized: `minimized` stores a computed value for temporary calculation. It is initialized from `dialog.classList.contains("is-minimized-dialog")`. Within the function it is passed into `contains`, `setAttribute`; assigned or updated later in the function.

button: `button` stores a DOM element reference. It is initialized from `dialog.querySelector(".section-view-minimize")`. It anchors UI behavior to a specific browser element or window state. Within the function it accesses properties/methods `setAttribute`, `textContent`, `title`; assigned or updated later in the function.

function toggleSectionDialogMinimized(dialog)

To understand `toggleSectionDialogMinimized`, start with the problem it owns. `toggleSectionDialogMinimized` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function toggleSectionDialogMinimized(dialog) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `dialog`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

Next, trace the helper calls that surround the function. It works with `anchorSectionDialog`, `updateSectionDialogMinimize`. Those calls show that `toggleSectionDialogMinimized` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `toggleSectionDialogMinimized`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `toggleSectionDialogMinimized`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function enableSectionDialogDrag()

Read `enableSectionDialogDrag` first as a named idea, not as syntax. `enableSectionDialogDrag` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function enableSectionDialogDrag() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: attaches event handlers that turn user actions into behavior.

The function also has to be read through the helpers it calls. It works with `clampSectionDialogPosition`. Those calls show that `enableSectionDialogDrag` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `enableSectionDialogDrag`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local objects and variables inside `enableSectionDialogDrag` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

rect: `rect` stores a computed value for temporary calculation. It is initialized from `dialog.getBoundingClientRect()`. Within the function it is accesses properties/methods `left`, `top`; passed into `clampSectionDialogPosition`; assigned or updated later in the function.

next: `next` stores a computed value for temporary calculation. It is initialized from `clampSectionDialogPosition(rect.left, rect.top, dialog)`. Within the function it is accesses properties/methods `left`, `top`; assigned or updated later in the function.

function linkifyRichTextNodes(root)

Read `linkifyRichTextNodes` first as a named idea, not as syntax. `linkifyRichTextNodes` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function linkifyRichTextNodes(root) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `root`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `root` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: creates DOM elements instead of relying only on static HTML.

The function also has to be read through the helpers it calls. It works with `normalizeLinkTarget`. Those calls show that `linkifyRichTextNodes` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `linkifyRichTextNodes`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local objects and variables inside `linkifyRichTextNodes` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

walker: `walker` stores a DOM element reference. It is initialized from `document.createTreeWalker(root, NodeFilter.SHOW_TEXT, {`. Within the function it is accesses properties/methods `nextNode`; assigned or updated later in the function.

nodes: `nodes` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is iterated or transformed as a collection; mutated as a collection or DOM container; accesses properties/methods `forEach`, `push`; assigned or updated later in the function.

node: `node` stores a computed value for temporary calculation. It is initialized from `walker.nextNode()`. Within the function it is returned to the caller; accesses properties/methods `parentElement`, `textContent`; passed into `createTreeWalker`, `push`, `test`; assigned or updated later in the function.

text: `text` stores a computed value for temporary calculation. It is initialized from `textNode.textContent || ""`. Within the function it is accesses properties/methods `length`, `replace`, `slice`; passed into `append`; assigned or updated later in the function.

fragment: `fragment` stores a DOM element reference. It is initialized from `document.createDocumentFragment()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `append`; passed into `replaceWith`; assigned or updated later in the function.

cursor: `cursor` stores a numeric value for temporary calculation. It is initialized from `0`. Within the function it is passed into `append`; assigned or updated later in the function.

trailing: `trailing` stores a computed value for temporary calculation. It is initialized from `match.match(/I),..`. Within the function it is accesses properties/methods `length`; passed into `append`, `slice`; assigned or updated later in the function.

clean: clean stores a computed value for temporary calculation. It is initialized from `trailing ? match.slice(0, -trailing.length) : match`. Within the function it is passed into `normalizeLinkTarget`; assigned or updated later in the function.

link: link stores a DOM element reference. It is initialized from `document.createElement("a")`. Within the function it accesses properties/methods `href`, `rel`, `target`, `textContent`; passed into `append`; assigned or updated later in the function.

function siteSectionHasContent(section)

To understand `siteSectionHasContent`, start with the problem it owns. `siteSectionHasContent` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function siteSectionHasContent(section) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `section`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `siteSectionHasContent` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `siteSectionHasContent`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `siteSectionHasContent`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function siteSectionRenderable(section)

Read `siteSectionRenderable` first as a named idea, not as syntax. `siteSectionRenderable` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function siteSectionRenderable(section) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `section`. Read those names as the contract

between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. section is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. It works with `siteSectionHasContent`. Those calls show that `siteSectionRenderable` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `siteSectionRenderable`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `siteSectionRenderable`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderSiteSections()

To understand `renderSiteSections`, start with the problem it owns. `renderSiteSections` turns saved data for `SiteSections` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

The syntax of the function is:

```
function renderSiteSections() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser DOM. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Next, trace the helper calls that surround the function. It works with `escapeHtml`, `renderRichContent`, `renderDynamicLinks`. Those calls show that `renderSiteSections` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `renderSiteSections`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

After the main flow, the working values inside `renderSiteSections` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

mount: `mount` stores a DOM element reference. It is initialized from `document.querySelector("#dynamic-sections")`. Within the function it is accesses properties/methods `innerHTML`; assigned or updated later in the function.

visibleSections: `visibleSections` stores a function or callback value for portfolio data state. It is initialized from `(siteSections || []).filter(siteSectionRenderable)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `map`; assigned or updated later in the function.

style: `style` stores a computed value for appearance configuration. It is initialized from `section.backgroundImage ? `style="--section-bg: url('${escapeHtml(section.backgroundImage)}')'` : ""`. Within the function it is assigned or updated later in the function.

links: `links` stores a list for temporary calculation. It is initialized from `[...(section.links || []), ...(section.assets || [])]`. Within the function it is passed into `renderDynamicLinks`; assigned or updated later in the function.

subsections: `subsections` stores a function or callback value for portfolio data state. It is initialized from `(section.subsections || []).filter((item) => item.title || item.description || item.richDescription?.blocks?.length || (item.links || []).length)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; assigned or updated later in the function.

function customSectionBlocks(project)

Read `customSectionBlocks` first as a named idea, not as syntax. `customSectionBlocks` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function customSectionBlocks(project) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `project`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. It works with `detailBlock`, `evidenceList`, `resourceLink`. Those calls show that `customSectionBlocks` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `customSectionBlocks`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `customSectionBlocks`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderCollectedDownloadSections(project)

The best entry point for `renderCollectedDownloadSections` is its responsibility in the surrounding workflow. `renderCollectedDownloadSections` turns saved data for `CollectedDownloadSections` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

The syntax of the function is:

```
function renderCollectedDownloadSections(project) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `project`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. It works with `uniqueDownloads`, `collectDownloadsFromValue`, `renderDownloadBlock`, `displayTitle`. Those calls show that `renderCollectedDownloadSections` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `renderCollectedDownloadSections`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The easiest way to follow the body of `renderCollectedDownloadSections` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

blocks: `blocks` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

addBlock: `addBlock` stores a function or callback value for temporary calculation. It is initialized from `(title, source, fallbackType = "File") => {}`. Within the function it is assigned or updated later in the function.

downloads: `downloads` stores a computed value for temporary calculation. It is initialized from `uniqueDownloads(collectDownloadsFromValue(source, fallbackType, []))`. Within the function it is accesses properties/methods `length`; passed into `addBlock`, `push`; assigned or updated later in the function.

function pathToString(path = [])

The best entry point for pathToString is its responsibility in the surrounding workflow. pathToString belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function pathToString(path = []) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: path. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. path points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means pathToString performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without pathToString, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside pathToString, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function pathFromString(value = "")

Begin with the role of pathFromString. pathFromString belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function pathFromString(value = "") { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: value. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and

generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `pathFromString` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `pathFromString`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `pathFromString`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function sectionRouteState(projectId, sectionIndex, resourcePath = "")`

To understand `sectionRouteState`, start with the problem it owns. `sectionRouteState` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sectionRouteState(projectId, sectionIndex, resourcePath = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `projectId`, `sectionIndex`, `resourcePath`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `resourcePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `sectionRouteState` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `sectionRouteState`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `sectionRouteState`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function canUseSectionHistory()

Read `canUseSectionHistory` first as a named idea, not as syntax. `canUseSectionHistory` is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

The syntax of the function is:

```
function canUseSectionHistory() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser route/history. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: updates browser history so Back, Forward, and refresh can follow the same window state.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `canUseSectionHistory` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `canUseSectionHistory`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `canUseSectionHistory`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function sectionBaseUrl()

The best entry point for `sectionBaseUrl` is its responsibility in the surrounding workflow. `sectionBaseUrl` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sectionBaseUrl() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `sectionBaseUrl` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `sectionBaseUrl`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The easiest way to follow the body of `sectionBaseUrl` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

url: url stores a constructed object for network or routing value. It is initialized from new `URL(window.location.href)`. Within the function it is returned to the caller; accesses properties/methods `searchParams`; assigned or updated later in the function.

function `sectionRouteUrl(projectId, sectionIndex, resourcePath = "")`

The best entry point for `sectionRouteUrl` is its responsibility in the surrounding workflow. `sectionRouteUrl` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sectionRouteUrl(projectId, sectionIndex, resourcePath = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `projectId`, `sectionIndex`, `resourcePath`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `resourcePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. It works with `sectionBaseUrl`. Those calls show that `sectionRouteUrl` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `sectionRouteUrl`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The easiest way to follow the body of `sectionRouteUrl` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

url: url stores a computed value for network or routing value. It is initialized from `sectionBaseUrl()`. Within the function it is returned to the caller; accesses properties/methods `hash`, `searchParams`; assigned or updated later in the function.

function sectionRouteFromState(state)

To understand `sectionRouteFromState`, start with the problem it owns. `sectionRouteFromState` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sectionRouteFromState(state) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `state`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `state` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `sectionRouteFromState` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `sectionRouteFromState`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `sectionRouteFromState`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function sectionRouteFromLocation()

To understand `sectionRouteFromLocation`, start with the problem it owns. `sectionRouteFromLocation` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser

history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sectionRouteFromLocation() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser route/history. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: reads or writes URL query parameters so browser history can represent the current view.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `sectionRouteFromLocation` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `sectionRouteFromLocation`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

After the main flow, the working values inside `sectionRouteFromLocation` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

params: `params` stores a constructed object for temporary calculation. It is initialized from `new URLSearchParams(window.location.search)`. Within the function it is accessed properties/methods `get`; assigned or updated later in the function.

projectId: `projectId` stores a computed value for portfolio data state. It is initialized from `params.get(sectionRouteParams.project)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

sectionIndex: `sectionIndex` stores a computed value for portfolio data state. It is initialized from `params.get(sectionRouteParams.section)`. Within the function it is assigned or updated later in the function.

function sameSectionRoute(left, right)

Read `sameSectionRoute` first as a named idea, not as syntax. `sameSectionRoute` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sameSectionRoute(left, right) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `left`, `right`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `left` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `right` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `sameSectionRoute` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `sameSectionRoute`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `sameSectionRoute`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function syncSectionRouteToHistory(projectId, sectionIndex, resourcePath = "", mode = "push")`

Read `syncSectionRouteToHistory` first as a named idea, not as syntax. `syncSectionRouteToHistory` reconciles two states that can drift apart. In this project, sync functions connect local drafts, route state, Git branches, publish mirrors, or frontend state to the current truth.

The syntax of the function is:

```
function syncSectionRouteToHistory(projectId, sectionIndex, resourcePath = "", mode = "push")
{ ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `projectId`, `sectionIndex`, `resourcePath`, `mode`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `resourcePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `mode` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser route/history. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

The function also has to be read through the helpers it calls. It works with `canUseSectionHistory`, `sectionRouteState`, `sectionRouteUrl`, `sectionRouteFromState`, `sectionRouteFromLocation`, `sameSectionRoute`. Those calls show that `syncSectionRouteToHistory` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `syncSectionRouteToHistory`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local objects and variables inside `syncSectionRouteToHistory` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

route: route stores a resolved filesystem or route value. It is initialized from `sectionRouteState(projectId, sectionIndex, resourcePath)`. Within the function it is passed into `sameSectionRoute`; assigned or updated later in the function.

url: url stores a resolved filesystem or route value. It is initialized from `sectionRouteUrl(projectId, sectionIndex, resourcePath)`. Within the function it is assigned or updated later in the function.

currentRoute: `currentRoute` stores a computed value for temporary calculation. It is initialized from `sectionRouteFromState(history.state) || sectionRouteFromLocation()`. Within the function it is passed into `sameSectionRoute`; assigned or updated later in the function.

method: method stores a computed value for temporary calculation. It is initialized from `mode === "replace" || sameSectionRoute(currentRoute, route) ? "replaceState" : "pushState"`. Within the function it is assigned or updated later in the function.

function clearSectionRouteInHistory()

Begin with the role of `clearSectionRouteInHistory`. `clearSectionRouteInHistory` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function clearSectionRouteInHistory() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

The body is where the contract above becomes behavior. Inside the body, it updates browser route/history. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: updates browser history so Back, Forward, and refresh can follow the same window state.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `canUseSectionHistory`, `sectionBaseUrl`. Those calls show that `clearSectionRouteInHistory` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `clearSectionRouteInHistory`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `clearSectionRouteInHistory`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function initializeSectionHistoryState()

Read `initializeSectionHistoryState` first as a named idea, not as syntax. `initializeSectionHistoryState` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function initializeSectionHistoryState() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser route/history. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: updates browser history so Back, Forward, and refresh can follow the same window state.

The function also has to be read through the helpers it calls. It works with `canUseSectionHistory`, `sectionRouteFromLocation`, `sectionRouteFromState`, `sectionRouteState`, `sectionRouteUrl`. Those calls show that `initializeSectionHistoryState` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `initializeSectionHistoryState`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local objects and variables inside `initializeSectionHistoryState` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

route: `route` stores a computed value for temporary calculation. It is initialized from `sectionRouteFromLocation()` || `sectionRouteFromState(history.state)`. Within the function it accesses properties/methods `projectId`, `resourcePath`, `sectionIndex`; passed into `replaceState`, `sectionRouteUrl`; assigned or updated later in the function.

function nodeAtPath(section, path = [])

Begin with the role of nodeAtPath. nodeAtPath belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function nodeAtPath(section, path = []) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: section, path. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. section is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. path points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means nodeAtPath performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without nodeAtPath, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local state inside nodeAtPath explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

node: node stores a computed value for temporary calculation. It is initialized from section. Within the function it is returned to the caller; accesses properties/methods children; assigned or updated later in the function.

children: children stores a computed value for temporary calculation. It is initialized from section?.items || []. Within the function it is assigned or updated later in the function.

index: index starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function.

function nodeChildren(node)

The best entry point for nodeChildren is its responsibility in the surrounding workflow. nodeChildren belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function nodeChildren(node) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `node`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `nodeChildren` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `nodeChildren`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `nodeChildren`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function nodeHasRenderableContent(node)

To understand `nodeHasRenderableContent`, start with the problem it owns. `nodeHasRenderableContent` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function nodeHasRenderableContent(node) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `node`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `richHasRenderableContent`, `nodeChildren`. Those calls show that `nodeHasRenderableContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `nodeHasRenderableContent`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

After the main flow, the working values inside `nodeHasRenderableContent` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

children: `children` stores a computed value for temporary calculation. It is initialized from `nodeChildren(node)`. Within the function it is accesses properties/methods some; assigned or updated later in the function.

function sectionHasRenderableContent(section)

Begin with the role of `sectionHasRenderableContent`. `sectionHasRenderableContent` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sectionHasRenderableContent(section) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `section`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `richHasRenderableContent`. Those calls show that `sectionHasRenderableContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `sectionHasRenderableContent`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `sectionHasRenderableContent`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function responsiveNodeHasMedia(node)

Begin with the role of `responsiveNodeHasMedia`. `responsiveNodeHasMedia` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:


```
function responsiveNodeHasMedia(node) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `node`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `richContainsMedia`, `responsiveChildren`. Those calls show that `responsiveNodeHasMedia` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `responsiveNodeHasMedia`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local state inside `responsiveNodeHasMedia` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

url: url stores a computed value for network or routing value. It is initialized from `String(node.url || "")`. Within the function it is passed into `String`, `test`; assigned or updated later in the function.

function responsiveNodeCount(node)

To understand `responsiveNodeCount`, start with the problem it owns. `responsiveNodeCount` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function responsiveNodeCount(node) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `node`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `nodeHasRenderableContent`, `responsiveChildren`. Those calls show that `responsiveNodeCount` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `responsiveNodeCount`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `responsiveNodeCount`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function responsiveNodeDepth(node)

To understand `responsiveNodeDepth`, start with the problem it owns. `responsiveNodeDepth` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function responsiveNodeDepth(node) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `node`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `nodeHasRenderableContent`, `responsiveChildren`. Those calls show that `responsiveNodeDepth` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `responsiveNodeDepth`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

After the main flow, the working values inside `responsiveNodeDepth` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

children: `children` stores a computed value for temporary calculation. It is initialized from `responsiveChildren(node).filter(nodeHasRenderableContent)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; passed into `max`; assigned or updated later in the function.

function nodeSummary(title, rich, text, emptyMessage = "No summary has been added yet.")

Begin with the role of nodeSummary. nodeSummary belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function nodeSummary(title, rich, text, emptyMessage = "No summary has been added yet.") { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: title, rich, text, emptyMessage. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. title is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. rich is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. text is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. emptyMessage is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The surrounding workflow becomes clearer when you notice its collaborators. It works with escapeHtml, renderRichContent. Those calls show that nodeSummary is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without nodeSummary, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside nodeSummary, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function nodeIsOverview(item = {})

To understand nodeIsOverview, start with the problem it owns. nodeIsOverview belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function nodeIsOverview(item = {}) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: item. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `normalize`, `displayTitle`. Those calls show that `nodeOverview` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `nodeOverview`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

After the main flow, the working values inside `nodeOverview` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

title: `title` stores a computed value for temporary calculation. It is initialized from `normalize(displayTitle(item.title || item.label || ""))`. Within the function it is accessed properties/methods `endsWith`; passed into `normalize`; assigned or updated later in the function.

function nodeOverviewDetails(node, children = [])

The best entry point for `nodeOverviewDetails` is its responsibility in the surrounding workflow.

`nodeOverviewDetails` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function nodeOverviewDetails(node, children = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `node`, `children`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. `children` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Its connection to the rest of the file matters as much as its local code. It works with `renderRichContent`, `renderInlineSectionFiles`, `nodeChildren`. Those calls show that `nodeOverviewDetails` is not isolated; it is one

piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `nodeOverviewDetails`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The easiest way to follow the body of `nodeOverviewDetails` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

overviewItems: `overviewItems` stores a computed value for temporary calculation. It is initialized from `children.filter(nodesOverview)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

blocks: `blocks` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `length`, `push`; assigned or updated later in the function.

itemFiles: `itemFiles` stores a computed value for temporary calculation. It is initialized from `renderInlineSectionFiles(nodeChildren(item))`. Within the function it is passed into `push`; assigned or updated later in the function.

function `parsedNodeCard(node, projectId, sectionIndex, path)`

The best entry point for `parsedNodeCard` is its responsibility in the surrounding workflow. `parsedNodeCard` interprets `dNodeCard` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

The syntax of the function is:

```
function parsedNodeCard(node, projectId, sectionIndex, path) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `node`, `projectId`, `sectionIndex`, `path`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `path` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Its connection to the rest of the file matters as much as its local code. It works with `displayTitle`, `pathToString`, `escapeHtml`. Those calls show that `parsedNodeCard` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `parsedNodeCard`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The easiest way to follow the body of `parsedNodeCard` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

title: `title` stores a computed value for temporary calculation. It is initialized from `displayTitle(node.title)`. Within the function it is passed into `displayTitle`, `escapeHtml`; assigned or updated later in the function.

function `parsedNodeContent(node, projectId, sectionIndex, path = [])`

Begin with the role of `parsedNodeContent`. `parsedNodeContent` interprets `dNodeContent` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

The syntax of the function is:

```
function parsedNodeContent(node, projectId, sectionIndex, path = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `node`, `projectId`, `sectionIndex`, `path`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `path` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `renderInlineSectionFiles`, `nodeChildren`, `nodeOverview`, `nodeOverviewDetails`, `parsedChildCards`. Those calls show that `parsedNodeContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `parsedNodeContent`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local state inside `parsedNodeContent` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

isContainer: `isContainer` stores a computed value for temporary calculation. It is initialized from `!node.kind || node.kind === "subsection"`. Within the function it is assigned or updated later in the function.

children: children stores a computed value for temporary calculation. It is initialized from `nodeChildren(node)`. Within the function it is iterated or transformed as a collection; accesses properties/methods filter; passed into `nodeOverviewDetails`, `parsedChildCards`; assigned or updated later in the function.

contentChildren: `contentChildren` stores a function or callback value for temporary calculation. It is initialized from `children.filter((child) => !modelsOverview(child))`. Within the function it is passed into `renderInlineSectionFiles`; assigned or updated later in the function.

function parsedSectionContent(section, projectId, sectionIndex, path = [])

The best entry point for `parsedSectionContent` is its responsibility in the surrounding workflow. `parsedSectionContent` interprets `dSectionContent` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

The syntax of the function is:

```
function parsedSectionContent(section, projectId, sectionIndex, path = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `section`, `projectId`, `sectionIndex`, `path`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `path` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Its connection to the rest of the file matters as much as its local code. It works with `nodeAtPath`, `parsedNodeContent`. Those calls show that `parsedSectionContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `parsedSectionContent`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The easiest way to follow the body of `parsedSectionContent` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

node: node stores a resolved filesystem or route value. It is initialized from `path.length ? nodeAtPath(section, path)` : section. Within the function it is passed into `parsedNodeContent`; assigned or updated later in the function.

function parsedSection(section, index, project)

Begin with the role of `parsedSection`. `parsedSection` interprets `dSection` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

The syntax of the function is:

```
function parsedSection(section, index, project) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `section`, `index`, `project`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `index` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `escapeHtml`, `displayTitle`. Those calls show that `parsedSection` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `parsedSection`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local state inside `parsedSection` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

visibleItems: `visibleItems` stores a function or callback value for temporary calculation. It is initialized from `(section.items || []).filter(nodeHasRenderableContent)`. Within the function it is accesses properties/methods some; assigned or updated later in the function.

hasImages: `hasImages` stores a function or callback value for temporary calculation. It is initialized from `visibleItems.some((item) => item.kind === "image")`. Within the function it is assigned or updated later in the function.

function categorySection(category, visibleProjects)

The best entry point for `categorySection` is its responsibility in the surrounding workflow. `categorySection` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

The syntax of the function is:

```
function categorySection(category, visibleProjects) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `category`, `visibleProjects`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `category` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `visibleProjects` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Its connection to the rest of the file matters as much as its local code. It works with `escapeHtml`. Those calls show that `categorySection` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `categorySection`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `categorySection`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function ensureSectionDialog()

The best entry point for `ensureSectionDialog` is its responsibility in the surrounding workflow.

`ensureSectionDialog` creates and wires the reusable full-window dialog used for project sections. It builds the top controls, back/forward behavior, close behavior, content container, and event delegation for child cards. It creates the window once and then later calls `reuse` it. The function exists so every project section window behaves consistently. If each click created its own markup by hand, close buttons, back buttons, keyboard behavior, and nested navigation would drift apart.

The syntax of the function is:

```
function ensureSectionDialog() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: serializes structured data before sending or saving it; creates DOM elements instead of relying only on static

HTML; writes generated markup into the page, which is why escaping and sanitizing helpers around it matter; toggles CSS classes to express UI state; attaches event handlers that turn user actions into behavior.

Its connection to the rest of the file matters as much as its local code. It works with `closeSectionDialog`, `clearSectionRouteInHistory`, `updateSectionDialogMinimize`, `pathFromString`, `sectionForwardStack`, `pathToString`, `openParsedSection`. Those calls show that `ensureSectionDialog` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `ensureSectionDialog`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The easiest way to follow the body of `ensureSectionDialog` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

dialog: `dialog` stores a DOM element reference. It is initialized from `document.querySelector("#section-view-dialog")`. It anchors UI behavior to a specific browser element or window state. Within the function it is returned to the caller; accesses properties/methods `addEventListener`, `classList`, `className`, `dataset`, `id`, `innerHTML`, `querySelector`; passed into `append`, `closeSectionDialog`, `createElement`, `openParsedSection`, `pathFromString`, `querySelector`, `remove`; assigned or updated later in the function.

returnTarget: `returnTarget` stores a DOM element reference. It is initialized from `document.querySelector()`. Within the function it is assigned or updated later in the function.

path: `path` stores a resolved filesystem or route value. It is initialized from `pathFromString(dialog.dataset.resourcePath || "")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it accesses properties/methods `length`, `pop`; passed into `closest`, `openParsedSection`, `push`; assigned or updated later in the function.

stack: `stack` stores a computed value for runtime state or cache. It is initialized from `sectionForwardStack(dialog)`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `pop`, `push`; passed into `stringify`; assigned or updated later in the function.

stack: `stack` stores a computed value for runtime state or cache. It is initialized from `sectionForwardStack(dialog)`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `pop`, `push`; passed into `stringify`; assigned or updated later in the function.

nextPath: `nextPath` stores a computed value for filesystem location. It is initialized from `stack.pop()`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `openParsedSection`; assigned or updated later in the function.

nodeButton: `nodeButton` stores a computed value for DOM reference or UI state. It is initialized from `event.target.closest("[data-resource-path]")`. It anchors UI behavior to a specific browser element or window state. Within the function it accesses properties/methods `dataset`; passed into `openParsedSection`; assigned or updated later in the function.

function `sectionForwardStack(dialog)`

Begin with the role of `sectionForwardStack`. `sectionForwardStack` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function sectionForwardStack(dialog) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `dialog`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates. A close reading of the body shows these implementation details: parses JSON rather than treating incoming text as already trusted data.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `sectionForwardStack` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `sectionForwardStack`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `sectionForwardStack`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function updateSectionNavigation(dialog, path)

Begin with the role of `updateSectionNavigation`. `updateSectionNavigation` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function updateSectionNavigation(dialog, path) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `dialog`, `path`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. `path` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `sectionForwardStack`. Those calls show that `updateSectionNavigation` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `updateSectionNavigation`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local state inside `updateSectionNavigation` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

back: back stores a DOM element reference. It is initialized from `dialog.querySelector(".section-view-back")`. Within the function it is accesses properties/methods `hidden`, `setAttribute`, `title`; passed into `querySelector`; assigned or updated later in the function.

function closeSectionDialog(dialog)

The best entry point for `closeSectionDialog` is its responsibility in the surrounding workflow. `closeSectionDialog` reverses an open window or route state. It does more than hide markup: it must also clean navigation state, restore the previous view when appropriate, and avoid trapping the visitor inside a dialog.

The syntax of the function is:

```
function closeSectionDialog(dialog) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `dialog`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Its connection to the rest of the file matters as much as its local code. It works with `clearSectionRouteInHistory`. Those calls show that `closeSectionDialog` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `closeSectionDialog`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Inside `closeSectionDialog`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function closeOrStepBackSectionDialog(dialog)

To understand `closeOrStepBackSectionDialog`, start with the problem it owns. `closeOrStepBackSectionDialog` reverses an open window or route state. It does more than hide markup: it must also clean navigation state, restore the previous view when appropriate, and avoid trapping the visitor inside a dialog.

The syntax of the function is:

```
function closeOrStepBackSectionDialog(dialog) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `dialog`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: serializes structured data before sending or saving it.

Next, trace the helper calls that surround the function. It works with `pathFromString`, `sectionForwardStack`, `pathToString`, `openParsedSection`, `closeSectionDialog`. Those calls show that `closeOrStepBackSectionDialog` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `closeOrStepBackSectionDialog`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

After the main flow, the working values inside `closeOrStepBackSectionDialog` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

path: `path` stores a resolved filesystem or route value. It is initialized from `pathFromString(dialog.dataset.resourcePath || "")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it accesses properties/methods `length`, `pop`; passed into `openParsedSection`, `push`; assigned or updated later in the function.

stack: `stack` stores a computed value for runtime state or cache. It is initialized from `sectionForwardStack(dialog)`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `push`; passed into `stringify`; assigned or updated later in the function.

function `openParsedSection(projectId, sectionIndex, resourcePath = "", options = {})`

Read `openParsedSection` first as a named idea, not as syntax. `openParsedSection` opens a project section or nested subsection as a full-window public view. It finds the project, finds the section, walks the requested subsection path, applies the selected appearance template, renders the overview and child sections, updates navigation buttons, and syncs browser history. This function is the heart of the website's project-window behavior. Without it, clicking Design, Simulation, Documentation, or any user-created section would either scroll the page awkwardly or render content in the wrong place.

The syntax of the function is:

```
function openParsedSection(projectId, sectionIndex, resourcePath = "", options = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `projectId`, `sectionIndex`, `resourcePath`, `options`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `resourcePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter; toggles CSS classes to express UI state.

The function also has to be read through the helpers it calls. It works with `sectionHasRenderableContent`, `pathFromString`, `nodeAtPath`, `nodeHasRenderableContent`, `ensureSectionDialog`, `applyProjectTemplateToElement`, `pathToString`, `syncSectionRouteToHistory`. Those calls show that `openParsedSection` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `openParsedSection`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local objects and variables inside `openParsedSection` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

project: `project` stores a function or callback value for portfolio data state. It is initialized from `projects.find((item) => item.id === projectId)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `category`; passed into `applyProjectTemplateToElement`; assigned or updated later in the function.

sections: `sections` stores a function or callback value for portfolio data state. It is initialized from `(project?.portfolioView?.sections || []).filter((section) => section.id !== "brief" && sectionHasRenderableContent(section))`. Within the function it is assigned or updated later in the function.

section: `section` stores a computed value for portfolio data state. It is initialized from `sections[Number(sectionIndex)]`. Within the function it is accesses properties/methods `id`, `title`; passed into `displayTitle`, `filter`, `nodeAtPath`, `parsedSectionContent`, `querySelector`, `sectionHasRenderableContent`; assigned or updated later in the function.

path: `path` stores a resolved filesystem or route value. It is initialized from `pathFromString(resourcePath)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods `length`; passed into

nodeAtPath, pathToString, syncSectionRouteToHistory, updateSectionNavigation; assigned or updated later in the function.

node: node stores a resolved filesystem or route value. It is initialized from `path.length ? nodeAtPath(section, path) : section`. Within the function it is accesses properties/methods `title`; passed into `displayTitle`; assigned or updated later in the function.

dialog: dialog stores a computed value for DOM reference or UI state. It is initialized from `ensureSectionDialog()`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods `dataset`, `open`, `querySelector`, `scrollTop`, `showModal`; passed into `applyProjectTemplateToElement`, `updateSectionNavigation`; assigned or updated later in the function.

category: category stores a function or callback value for portfolio data state. It is initialized from `categories.find((item) => item.id === project.category) || {}`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `accent`; passed into `applyProjectTemplateToElement`; assigned or updated later in the function.

function closeSectionDialogForRoute()

To understand `closeSectionDialogForRoute`, start with the problem it owns. `closeSectionDialogForRoute` reverses an open window or route state. It does more than hide markup: it must also clean navigation state, restore the previous view when appropriate, and avoid trapping the visitor inside a dialog.

The syntax of the function is:

```
function closeSectionDialogForRoute() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `closeSectionDialogForRoute` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `closeSectionDialogForRoute`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

After the main flow, the working values inside `closeSectionDialogForRoute` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

dialog: dialog stores a DOM element reference. It is initialized from `document.querySelector("#section-view-dialog")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods `close`; passed into `querySelector`; assigned or updated later in the function.

function applySectionRoute(route)

Read `applySectionRoute` first as a named idea, not as syntax. `applySectionRoute` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function applySectionRoute(route) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `route`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `route` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

The function also has to be read through the helpers it calls. It works with `closeSectionDialogForRoute`, `openParsedSection`. Those calls show that `applySectionRoute` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `applySectionRoute`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The local objects and variables inside `applySectionRoute` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

opened: `opened` stores a resolved filesystem or route value. It is initialized from `openParsedSection(route.projectId, route.sectionIndex, route.resourcePath, {})`. Within the function it is assigned or updated later in the function.

function restoreSectionRouteAfterCatalogLoad()

To understand `restoreSectionRouteAfterCatalogLoad`, start with the problem it owns. `restoreSectionRouteAfterCatalogLoad` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function restoreSectionRouteAfterCatalogLoad() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser route/history. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: attaches event handlers that turn user actions into behavior; updates browser history so Back, Forward, and refresh can follow the same window state.

Next, trace the helper calls that surround the function. It works with `initializeSectionHistoryState`, `applySectionRoute`, `sectionRouteFromLocation`, `sectionRouteFromState`, `ensureSearchPanel`, `goToSearchResult`, `clearSearchHighlights`, `closeSectionDialogForRoute`. Those calls show that `restoreSectionRouteAfterCatalogLoad` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `restoreSectionRouteAfterCatalogLoad`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

After the main flow, the working values inside `restoreSectionRouteAfterCatalogLoad` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

topLink: `topLink` stores a computed value for temporary calculation. It is initialized from `event.target.closest('a[href="#top"]')`. Within the function it is assigned or updated later in the function.

question: `question` stores a computed value for temporary calculation. It is initialized from `aiAssistantInput?.value || ""`. Within the function it is passed into `answerAssistantQuestion`; assigned or updated later in the function.

button: `button` stores a computed value for DOM reference or UI state. It is initialized from `event.target.closest("[data-ai-source-id]")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods `dataset`; passed into `get`; assigned or updated later in the function.

source: `source` stores a computed value for lookup collection. It is initialized from `assistantSourceMap.get(button.dataset.aiSourceId)`. Within the function it is passed into `closest`, `goToSearchResult`; assigned or updated later in the function.

sectionButton: `sectionButton` stores a computed value for portfolio data state. It is initialized from `event.target.closest("[data-section-project]")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods `dataset`; passed into `openParsedSection`; assigned or updated later in the function.

function renderInlineSectionFiles(items = [])

The best entry point for `renderInlineSectionFiles` is its responsibility in the surrounding workflow. `renderInlineSectionFiles` turns saved data for `InlineSectionFiles` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

The syntax of the function is:

```
function renderInlineSectionFiles(items = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `items`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `items` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Its connection to the rest of the file matters as much as its local code. It works with `uniqueDownloads`, `normalizeDownloadAsset`, `escapeHtml`, `normalizeLinkTarget`, `isWebsiteLinkItem`, `linkAttributes`, `downloadAttribute`, `fileNameFromUrl`. Those calls show that `renderInlineSectionFiles` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `renderInlineSectionFiles`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

The easiest way to follow the body of `renderInlineSectionFiles` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

downloads: `downloads` stores a function or callback value for lookup collection. It is initialized from `uniqueDownloads(items.map((item) => normalizeDownloadAsset(item, "File")).filter(Boolean))`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; assigned or updated later in the function.

Resources and links

The functions in this group all support resources and links. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function normalizeLinkTarget(target = "", options = {})

Read `normalizeLinkTarget` first as a named idea, not as syntax. `normalizeLinkTarget` standardizes `LinkTarget` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeLinkTarget(target = "", options = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `target`, `options`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `target` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The function also has to be read through the helpers it calls. It works with `looksLikeBareWebAddress`. Those calls show that `normalizeLinkTarget` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `normalizeLinkTarget`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside `normalizeLinkTarget` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(target || "").trim()`. Within the function it is returned to the caller; assigned or updated later in the function.

function linkAttributes(url, item = {})

To understand `linkAttributes`, start with the problem it owns. `linkAttributes` belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function linkAttributes(url, item = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `url`, `item`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `normalizeLinkTarget`, `isWebsiteLinkItem`. Those calls show that `linkAttributes` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `linkAttributes`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

After the main flow, the working values inside `linkAttributes` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

target: `target` stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(item, url) })`. Within the function it is passed into `test`; assigned or updated later in the function.

function isLocalDownloadTarget(target = "")

Read `isLocalDownloadTarget` first as a named idea, not as syntax. `isLocalDownloadTarget` is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

The syntax of the function is:

```
function isLocalDownloadTarget(target = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `target`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `target` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. It works with `normalizeLinkTarget`. Those calls show that `isLocalDownloadTarget` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `isLocalDownloadTarget`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside `isLocalDownloadTarget` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

value: value stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(target, { assumeWeb: true })`. Within the function it is passed into `Boolean, test`; assigned or updated later in the function.

function downloadAttribute(target = "", item = {})

Begin with the role of `downloadAttribute`. `downloadAttribute` belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function downloadAttribute(target = "", item = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `target`, `item`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `target` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `normalizeLinkTarget`, `isWebsiteLinkItem`, `isLocalDownloadTarget`. Those calls show that `downloadAttribute` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `downloadAttribute`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `downloadAttribute` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

value: value stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(target, { assumeWeb: isWebsiteLinkItem(item, target) })`. Within the function it is passed into `isLocalDownloadTarget`; assigned or updated later in the function.

function fileNameFromUrl(url = "")

Begin with the role of `fileNameFromUrl`. `fileNameFromUrl` belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function fileNameFromUrl(url = "") { ... }
```


There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `url`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `fileNameFromUrl` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `fileNameFromUrl`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `fileNameFromUrl` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(url || "").split(/[?#]/)[0].split("/").pop() || "Download file"`. Within the function it is returned to the caller; passed into `decodeURIComponent`; assigned or updated later in the function.

function resourceLink(item = {}, label = item.label || item.title || item.name || fileNameFromUrl(item.url || item.artifact))

Read `resourceLink` first as a named idea, not as syntax. `resourceLink` belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function resourceLink(item = {}, label = item.label || item.title || item.name ||
  fileNameFromUrl(item.url || item.artifact)) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `item`, `label`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `label` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The function also has to be read through the helpers it calls. It works with `normalizeLinkTarget`, `isWebsiteLinkItem`, `escapeHtml`, `linkAttributes`, `downloadAttribute`, `fileNameFromUrl`. Those calls show that `resourceLink` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `resourceLink`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside `resourceLink` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

rawTarget: `rawTarget` stores a computed value for temporary calculation. It is initialized from `item.url || item.artifact || item.href || item.file || item.path || item.src || ""`. Within the function it is passed into `normalizeLinkTarget`; assigned or updated later in the function.

target: `target` stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(rawTarget, { assumeWeb: isWebsiteLinkItem(item, rawTarget) })`. Within the function it is passed into `downloadAttribute`, `escapeHtml`, `linkAttributes`; assigned or updated later in the function.

function itemUrl(item = {})

Read `itemUrl` first as a named idea, not as syntax. `itemUrl` belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function itemUrl(item = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `item`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `itemUrl` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `itemUrl`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `itemUrl`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function normalizeDownloadAsset(item = {}, fallbackType = "File")`

To understand `normalizeDownloadAsset`, start with the problem it owns. `normalizeDownloadAsset` standardizes `DownloadAsset` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeDownloadAsset(item = {}, fallbackType = "File") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `item`, `fallbackType`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `fallbackType` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Next, trace the helper calls that surround the function. It works with `itemUrl`, `itemLabel`. Those calls show that `normalizeDownloadAsset` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `normalizeDownloadAsset`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

After the main flow, the working values inside `normalizeDownloadAsset` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

url: url stores a computed value for network or routing value. It is initialized from `itemUrl(item)`. Within the function it is assigned or updated later in the function.

function collectDownloadsFromValue(value, fallbackType = "File", output = [])

Read `collectDownloadsFromValue` first as a named idea, not as syntax. `collectDownloadsFromValue` gathers scattered nested data into a shape that another function can loop over. This is important because portfolio data is hierarchical while rendering, search, and AI context often need a flat list.

The syntax of the function is:

```
function collectDownloadsFromValue(value, fallbackType = "File", output = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`, `fallbackType`, `output`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `fallbackType` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `output` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. It works with `normalizeDownloadAsset`, `collectRichDownloads`. Those calls show that `collectDownloadsFromValue` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `collectDownloadsFromValue`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside `collectDownloadsFromValue` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

asset: `asset` stores a computed value for lookup collection. It is initialized from `normalizeDownloadAsset(value, fallbackType)`. Within the function it is passed into `push`; assigned or updated later in the function.

function uniqueDownloads(items = [])

Read `uniqueDownloads` first as a named idea, not as syntax. `uniqueDownloads` belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function uniqueDownloads(items = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `items`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `items` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `uniqueDownloads` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `uniqueDownloads`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside `uniqueDownloads` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

seen: `seen` stores a constructed object for lookup collection. It is initialized from `new Set()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `add`, `has`; assigned or updated later in the function.

key: `key` stores a string value for temporary calculation. It is initialized from ``${item.url}::${item.title}``. Within the function it is passed into `add`; assigned or updated later in the function.

Appearance and responsive presentation

The functions in this group all support appearance and responsive presentation. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function cleanFontFamily(value = "")

Begin with the role of `cleanFontFamily`. `cleanFontFamily` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function cleanFontFamily(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and

generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `cleanFontFamily` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `cleanFontFamily`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `cleanFontFamily`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function normalizeFontPx(value)

Begin with the role of `normalizeFontPx`. `normalizeFontPx` standardizes `FontPx` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeFontPx(value) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `normalizeFontPx` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `normalizeFontPx`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `normalizeFontPx` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

size: size stores a computed value for temporary calculation. It is initialized from `Number(value)`. Within the function it is passed into `String`, `isFinite`; assigned or updated later in the function.

function normalizeTextColor(value = "")

To understand `normalizeTextColor`, start with the problem it owns. `normalizeTextColor` standardizes `TextColor` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeTextColor(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `normalizeTextColor` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `normalizeTextColor`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

After the main flow, the working values inside `normalizeTextColor` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

color: color stores a computed value for appearance configuration. It is initialized from `String(value || "").trim()`. Within the function it is passed into `test`; assigned or updated later in the function.

function rgbColorToHex(value = "")

Read `rgbColorToHex` first as a named idea, not as syntax. `rgbColorToHex` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function rgbColorToHex(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract

between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `rgbColorToHex` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `rgbColorToHex`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside `rgbColorToHex` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

color: `color` stores a computed value for appearance configuration. It is initialized from `String(value || "").trim()`. Within the function it is returned to the caller; accesses properties/methods `match`, `toLowerCase`; assigned or updated later in the function.

shortHex: `shortHex` stores a computed value for temporary calculation. It is initialized from `color.match(/^#([0-9a-f])([0-9a-f])([0-9a-f])$/i)`. Within the function it is assigned or updated later in the function.

rgb: `rgb` stores a computed value for temporary calculation. It is initialized from `color.match(/^rgba?(\d+),\s*(\d+),\s*(\d+)/i)`. Within the function it is assigned or updated later in the function.

function normalizePlainFieldStyle(style = {})

Read `normalizePlainFieldStyle` first as a named idea, not as syntax. `normalizePlainFieldStyle` standardizes `PlainFieldStyle` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizePlainFieldStyle(style = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `style`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `style` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. It works with `cleanFontFamily`, `normalizeFontPx`, `normalizeTextColor`, `rgbColorToHex`. Those calls show that `normalizePlainFieldStyle` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `normalizePlainFieldStyle`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside `normalizePlainFieldStyle` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

normalized: `normalized` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; accesses properties/methods `bold`, `color`, `fontFamily`, `fontPx`, `italic`, `underline`; assigned or updated later in the function.

fontFamily: `fontFamily` stores a computed value for temporary calculation. It is initialized from `cleanFontFamily(style.fontFamily || "")`. Within the function it is passed into `cleanFontFamily`; assigned or updated later in the function.

fontPx: `fontPx` stores a computed value for temporary calculation. It is initialized from `normalizeFontPx(style.fontPx || style.fontSize || "")`. Within the function it is passed into `normalizeFontPx`; assigned or updated later in the function.

color: `color` stores a computed value for appearance configuration. It is initialized from `normalizeTextColor(style.color || "")`. Within the function it is passed into `normalizeTextColor`, `rgbColorToHex`; assigned or updated later in the function.

function normalizeFieldStyles(styles = {})

To understand `normalizeFieldStyles`, start with the problem it owns. `normalizeFieldStyles` standardizes `FieldStyles` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeFieldStyles(styles = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `styles`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `styles` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `normalizePlainFieldStyle`. Those calls show that `normalizeFieldStyles` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `normalizeFieldStyles`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `normalizeFieldStyles`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function plainFieldStyleToCss(style = {})

To understand `plainFieldStyleToCss`, start with the problem it owns. `plainFieldStyleToCss` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function plainFieldStyleToCss(style = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `style`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `style` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `normalizePlainFieldStyle`. Those calls show that `plainFieldStyleToCss` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `plainFieldStyleToCss`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

After the main flow, the working values inside `plainFieldStyleToCss` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

normalized: normalized stores a computed value for temporary calculation. It is initialized from `normalizePlainFieldStyle(style)`. Within the function it is accesses properties/methods `bold`, `color`, `fontFamily`, `fontPx`, `italic`, `underline`; passed into `push`; assigned or updated later in the function.

rules: rules stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

function applyPlainFieldStyle(element, fieldId = "")

The best entry point for `applyPlainFieldStyle` is its responsibility in the surrounding workflow. `applyPlainFieldStyle` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function applyPlainFieldStyle(element, fieldId = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `element`, `fieldId`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `element` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. `fieldId` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Its connection to the rest of the file matters as much as its local code. It works with `normalizePlainFieldStyle`. Those calls show that `applyPlainFieldStyle` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `applyPlainFieldStyle`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The easiest way to follow the body of `applyPlainFieldStyle` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

style: style stores a computed value for appearance configuration. It is initialized from `normalizePlainFieldStyle(fieldStyles[fieldId] || {})`. Within the function it is accesses properties/methods `bold`, `color`, `fontFamily`, `fontPx`, `fontSize`, `fontStyle`, `fontWeight`, `italic`; assigned or updated later in the function.

function plainFieldStyleAttribute(fieldId = "")

To understand `plainFieldStyleAttribute`, start with the problem it owns. `plainFieldStyleAttribute` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function plainFieldStyleAttribute(fieldId = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `fieldId`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `fieldId` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `plainFieldStyleToCss`, `escapeHtml`. Those calls show that `plainFieldStyleAttribute` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `plainFieldStyleAttribute`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

After the main flow, the working values inside `plainFieldStyleAttribute` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

css: `css` stores a computed value for temporary calculation. It is initialized from `plainFieldStyleToCss(fieldStyles[fieldId] || {})`. Within the function it is returned to the caller; passed into `escapeHtml`; assigned or updated later in the function.

function normalizeCropAspect(value = "")

To understand `normalizeCropAspect`, start with the problem it owns. `normalizeCropAspect` standardizes `CropAspect` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeCropAspect(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `normalizeCropAspect` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `normalizeCropAspect`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `normalizeCropAspect`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function normalizeCropZoom(value = 1)

Read `normalizeCropZoom` first as a named idea, not as syntax. `normalizeCropZoom` standardizes `CropZoom` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeCropZoom(value = 1) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `normalizeCropZoom` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `normalizeCropZoom`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside `normalizeCropZoom` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

zoom: zoom stores a computed value for temporary calculation. It is initialized from Number(value). Within the function it is passed into isFinite, min; assigned or updated later in the function.

function normalizeCropPosition(value = 50)

Read normalizeCropPosition first as a named idea, not as syntax. normalizeCropPosition standardizes CropPosition data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeCropPosition(value = 50) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: value. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means normalizeCropPosition performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without normalizeCropPosition, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside normalizeCropPosition show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

position: position stores a computed value for temporary calculation. It is initialized from Number(value). Within the function it is passed into isFinite, min; assigned or updated later in the function.

function canonicalTemplateId(id)

To understand canonicalTemplateId, start with the problem it owns. canonicalTemplateId is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

The syntax of the function is:

```
function canonicalTemplateId(id) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: id. Read those names as the contract

between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `id` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `canonicalTemplateId` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `canonicalTemplateId`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `canonicalTemplateId`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function projectTemplateId(project)

Begin with the role of `projectTemplateId`. `projectTemplateId` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

The syntax of the function is:

```
function projectTemplateId(project) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `project`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `canonicalTemplateId`. Those calls show that `projectTemplateId` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `projectTemplateId`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `projectTemplateId`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function projectTemplateClass(project)

To understand `projectTemplateClass`, start with the problem it owns. `projectTemplateClass` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

The syntax of the function is:

```
function projectTemplateClass(project) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `project`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `projectTemplateId`. Those calls show that `projectTemplateClass` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `projectTemplateClass`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

After the main flow, the working values inside `projectTemplateClass` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

templateId: `templateId` stores a computed value for appearance configuration. It is initialized from `projectTemplateId(project)`. Within the function it is returned to the caller; assigned or updated later in the function.

function responsiveClassName(value, fallback)

To understand `responsiveClassName`, start with the problem it owns. `responsiveClassName` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function responsiveClassName(value, fallback) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`, `fallback`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. fallback is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means responsiveClassName performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without responsiveClassName, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside responsiveClassName, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function projectResponsiveClass(project)

The best entry point for projectResponsiveClass is its responsibility in the surrounding workflow. projectResponsiveClass handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

The syntax of the function is:

```
function projectResponsiveClass(project) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: project. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Its connection to the rest of the file matters as much as its local code. It works with projectResponsiveProfile, responsiveClassName. Those calls show that projectResponsiveClass is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without projectResponsiveClass, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The easiest way to follow the body of `projectResponsiveClass` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

profile: profile stores a computed value for portfolio data state. It is initialized from `projectResponsiveProfile(project)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `cardLayout`, `density`; passed into `responsiveClassName`; assigned or updated later in the function.

function responsiveStyleValues(project)

The best entry point for `responsiveStyleValues` is its responsibility in the surrounding workflow. `responsiveStyleValues` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function responsiveStyleValues(project) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `project`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Its connection to the rest of the file matters as much as its local code. It works with `projectResponsiveProfile`. Those calls show that `responsiveStyleValues` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `responsiveStyleValues`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The easiest way to follow the body of `responsiveStyleValues` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

profile: profile stores a computed value for portfolio data state. It is initialized from `projectResponsiveProfile(project)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `contentMax`, `mediaMax`, `sectionCardMin`, `touchTarget`; assigned or updated later in the function.

function hasPublicTemplate(project)

The best entry point for `hasPublicTemplate` is its responsibility in the surrounding workflow. `hasPublicTemplate` is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

The syntax of the function is:

```
function hasPublicTemplate(project) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `project`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. It works with `projectTemplateId`. Those calls show that `hasPublicTemplate` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `hasPublicTemplate`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `hasPublicTemplate`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function projectTemplateVisual(project)

The best entry point for `projectTemplateVisual` is its responsibility in the surrounding workflow. `projectTemplateVisual` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

The syntax of the function is:

```
function projectTemplateVisual(project) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `project`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `projectTemplateVisual` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `projectTemplateVisual`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `projectTemplateVisual`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function projectTemplateStyle(project, accent)

Begin with the role of `projectTemplateStyle`. `projectTemplateStyle` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

The syntax of the function is:

```
function projectTemplateStyle(project, accent) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `project`, `accent`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `accent` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `projectTemplateVisual`, `responsiveStyleValues`. Those calls show that `projectTemplateStyle` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `projectTemplateStyle`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local state inside `projectTemplateStyle` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

visual: `visual` stores a computed value for temporary calculation. It is initialized from `projectTemplateVisual(project) || {}`. Within the function it is accesses properties/methods `accent`, `background`, `click`, `clickText`, `hover`, `line`, `panel`, `text`; assigned or updated later in the function.

values: `values` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is passed into entries; assigned or updated later in the function.

function applyProjectTemplateToElement(element, project, accent)

Read `applyProjectTemplateToElement` first as a named idea, not as syntax. `applyProjectTemplateToElement` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function applyProjectTemplateToElement(element, project, accent) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `element`, `project`, `accent`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `element` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `accent` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

The function also has to be read through the helpers it calls. It works with `projectTemplateClass`, `projectResponsiveClass`, `projectTemplateVisual`, `responsiveStyleValues`. Those calls show that `applyProjectTemplateToElement` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `applyProjectTemplateToElement`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside `applyProjectTemplateToElement` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

visual: `visual` stores a computed value for temporary calculation. It is initialized from `projectTemplateVisual(project) || {}`. Within the function it is accesses properties/methods `accent`, `background`, `click`, `clickText`, `hover`, `line`, `panel`, `text`; assigned or updated later in the function.

values: `values` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is passed into entries; assigned or updated later in the function.

function responsiveChildren(node)

The best entry point for `responsiveChildren` is its responsibility in the surrounding workflow. `responsiveChildren` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function responsiveChildren(node) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `node`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `responsiveChildren` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `responsiveChildren`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Inside `responsiveChildren`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function normalizeTextColor(value = "")

To understand `normalizeTextColor`, start with the problem it owns. `normalizeTextColor` standardizes `TextColor` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeTextColor(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `normalizeTextColor` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `normalizeTextColor`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

After the main flow, the working values inside `normalizeTextColor` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

color: `color` stores a computed value for appearance configuration. It is initialized from `String(value || "").trim()`. Within the function it is passed into `test`; assigned or updated later in the function.

Project directory

The functions in this group all support project directory. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function `flattenProject(project)`

To understand `flattenProject`, start with the problem it owns. `flattenProject` gathers scattered nested data into a shape that another function can loop over. This is important because portfolio data is hierarchical while rendering, search, and AI context often need a flat list.

The syntax of the function is:

```
function flattenProject(project) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `project`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Next, trace the helper calls that surround the function. It works with `slugLabel`. Those calls show that `flattenProject` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `flattenProject`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

After the main flow, the working values inside `flattenProject` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

categoryLabel: `categoryLabel` stores a computed value for portfolio data state. It is initialized from `slugLabel(project.category)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into `electronicsSearchKeywords`; assigned or updated later in the function.

electronicsKeywords: `electronicsKeywords` stores a computed value for temporary calculation. It is initialized from `window.electronicsSearchKeywords`. Within the function it is assigned or updated later in the function.

richSummaryTerms: richSummaryTerms stores a function or callback value for lookup collection. It is initialized from (project.summaryRich?.blocks || []).flatMap((block) => [. Within the function it is assigned or updated later in the function.

parsedChildTerms: parsedChildTerms stores a function or callback value for lookup collection. It is initialized from (project.portfolioView?.sections || []).flatMap((section) =>. Within the function it is assigned or updated later in the function.

design: design stores a computed value for temporary calculation. It is initialized from project.design || {}. Within the function it is accesses properties/methods brief, documentation, simulation; assigned or updated later in the function.

designTerms: designTerms stores a list for temporary calculation. It is initialized from [. Within the function it is assigned or updated later in the function.

function projectVisible(project, query)

To understand projectVisible, start with the problem it owns. projectVisible handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

The syntax of the function is:

```
function projectVisible(project, query) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: project, query. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. query is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with projectMatches. Those calls show that projectVisible is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without projectVisible, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

After the main flow, the working values inside projectVisible reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

categoryMatch: categoryMatch stores a computed value for portfolio data state. It is initialized from activeFilter === "all" || project.category === activeFilter. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is returned to the caller; assigned or updated later in the function.

function normalizeCategory(category = {})

The best entry point for `normalizeCategory` is its responsibility in the surrounding workflow. `normalizeCategory` standardizes `Category` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

The syntax of the function is:

```
function normalizeCategory(category = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `category`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `category` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Its connection to the rest of the file matters as much as its local code. It works with `normalizeTextColor`. Those calls show that `normalizeCategory` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `normalizeCategory`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The easiest way to follow the body of `normalizeCategory` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

label: `label` stores a computed value for temporary calculation. It is initialized from `String(category.label || category.title || category.id || "Project category").trim() || "Project category"`. Within the function it accesses properties/methods `toLowerCase`; passed into `String`; assigned or updated later in the function.

function hydrateProjectCategories(rawCategories = [], rawProjects = [])

Read `hydrateProjectCategories` first as a named idea, not as syntax. `hydrateProjectCategories` belongs to project directory. This function normalizes, flattens, filters, or renders projects and categories. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function hydrateProjectCategories(rawCategories = [], rawProjects = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `rawCategories`, `rawProjects`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `rawCategories` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `rawProjects` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. It works with `normalizeCategory`. Those calls show that `hydrateProjectCategories` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `hydrateProjectCategories`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside `hydrateProjectCategories` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

normalized: `normalized` stores a computed value for lookup collection. It is initialized from `rawCategories.map(normalizeCategory)`. Within the function it is returned to the caller; iterated or transformed as a collection; mutated as a collection or DOM container; accesses properties/methods `map`, `push`; passed into `Set`.

knownIds: `knownIds` stores a constructed object for lookup collection. It is initialized from `new Set(normalized.map((category) => category.id))`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `add`, `has`; assigned or updated later in the function.

id: `id` stores a computed value for temporary calculation. It is initialized from `String(project?.category || "").trim()`. Within the function it is passed into `add`, `push`; assigned or updated later in the function.

label: `label` stores a computed value for temporary calculation. It is initialized from `id`. Within the function it is passed into `push`; assigned or updated later in the function.

function projectCard(project)

To understand `projectCard`, start with the problem it owns. `projectCard` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

The syntax of the function is:

```
function projectCard(project) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `project`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Next, trace the helper calls that surround the function. It works with `sectionHasRenderableContent`, `projectTemplateClass`, `projectResponsiveClass`, `projectTemplateStyle`, `renderParsedBriefBlock`, `parsedSection`, `renderMultilineInlineText`, `detailBlock`. Those calls show that `projectCard` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `projectCard`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

After the main flow, the working values inside `projectCard` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

category: `category` stores a function or callback value for portfolio data state. It is initialized from `categories.find((item) => item.id === project.category) || {}`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `accent`; assigned or updated later in the function.

accent: `accent` stores a computed value for temporary calculation. It is initialized from `category.accent || "#117c7a"`. Within the function it is passed into `projectTemplateStyle`; assigned or updated later in the function.

sections: `sections` stores a computed value for portfolio data state. It is initialized from `project.portfolioView.sections || []`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `find`; assigned or updated later in the function.

briefSection: `briefSection` stores a function or callback value for portfolio data state. It is initialized from `sections.find((section) => section.id === "brief")`. Within the function it is passed into `renderParsedBriefBlock`; assigned or updated later in the function.

otherSections: `otherSections` stores a function or callback value for portfolio data state. It is initialized from `sections.filter((section) => section.id !== "brief" && sectionHasRenderableContent(section))`. Within the function it is iterated or transformed as a collection; accesses properties/methods `map`; assigned or updated later in the function.

category: `category` stores a function or callback value for portfolio data state. It is initialized from `categories.find((item) => item.id === project.category) || {}`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `accent`; assigned or updated later in the function.

accent: `accent` stores a computed value for temporary calculation. It is initialized from `category.accent || "#117c7a"`. Within the function it is passed into `projectTemplateStyle`; assigned or updated later in the function.

function loadProjectCatalog()

Read `loadProjectCatalog` first as a named idea, not as syntax. `loadProjectCatalog` is the public website startup pipeline. It chooses embedded preview data when the builder is previewing, otherwise it fetches `projects.json` from the published site. After that it hydrates the major runtime state: categories, projects, profile, site content, fun facts, custom sections, text styles, and rich text variants. Then it renders the visible page and restores any deep-linked project window. This function is async because fetching `projects.json` can take time and can fail. It is the reason the static HTML shell becomes a real portfolio. Without it, the page would show placeholders but no saved projects, profile details, resume, fun facts, project categories, search index, or assistant context.

The syntax of the function is:

```
function loadProjectCatalog() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it calls a network resource and updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails; writes generated markup into the page, which is why escaping and sanitizing helpers around it matter; performs a fetch and therefore must handle network delay and failure.

The function also has to be read through the helpers it calls. It works with `hydrateProjectCategories`, `normalizeFieldStyles`, `normalizeSiteContent`, `normalizeProfile`, `normalizeFunFacts`, `renderProfile`, `renderSiteContent`, `renderFunFacts`. Those calls show that `loadProjectCatalog` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `loadProjectCatalog`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

The local objects and variables inside `loadProjectCatalog` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

catalogUrl: `catalogUrl` stores a constructed object for network or routing value. It is initialized from `new URL("projects.json", window.location.href)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accessed properties/methods `href`, `searchParams`; passed into `fetch`; assigned or updated later in the function.

Search and filtering

The functions in this group all support search and filtering. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function projectMatches(project, query)

Begin with the role of projectMatches. projectMatches handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

The syntax of the function is:

```
function projectMatches(project, query) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `project`, `query`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `query` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `flattenProject`. Those calls show that `projectMatches` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `projectMatches`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Inside `projectMatches`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function setActiveFilter(filter = "all")

Read `setActiveFilter` first as a named idea, not as syntax. `setActiveFilter` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function setActiveFilter(filter = "all") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `filter`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `filter` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM,

writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `setActiveFilter` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `setActiveFilter`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Inside `setActiveFilter`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function bindFilterButtons()

Read `bindFilterButtons` first as a named idea, not as syntax. `bindFilterButtons` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function bindFilterButtons() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: attaches event handlers that turn user actions into behavior.

The function also has to be read through the helpers it calls. It works with `setActiveFilter`, `renderProjects`, `updateSearchDropdown`. Those calls show that `bindFilterButtons` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `bindFilterButtons`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Inside `bindFilterButtons`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderCategoryFilters()

The best entry point for `renderCategoryFilters` is its responsibility in the surrounding workflow.

`renderCategoryFilters` turns saved data for `CategoryFilters` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

The syntax of the function is:

```
function renderCategoryFilters() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Its connection to the rest of the file matters as much as its local code. It works with `escapeHtml`, `bindFilterButtons`, `setActiveFilter`. Those calls show that `renderCategoryFilters` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `renderCategoryFilters`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Inside `renderCategoryFilters`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function flattenSearchText(values = [])

To understand `flattenSearchText`, start with the problem it owns. `flattenSearchText` gathers scattered nested data into a shape that another function can loop over. This is important because portfolio data is hierarchical while rendering, search, and AI context often need a flat list.

The syntax of the function is:

```
function flattenSearchText(values = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `values`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `values` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point

is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: serializes structured data before sending or saving it.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `flattenSearchText` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `flattenSearchText`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Inside `flattenSearchText`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function addSearchEntry(entries, entry)

To understand `addSearchEntry`, start with the problem it owns. `addSearchEntry` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function addSearchEntry(entries, entry) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `entries`, `entry`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `entries` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `entry` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `flattenSearchText`, `normalize`. Those calls show that `addSearchEntry` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `addSearchEntry`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

After the main flow, the working values inside `addSearchEntry` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

text: `text` stores a computed value for temporary calculation. It is initialized from `flattenSearchText([entry.title, entry.type, entry.context, entry.text, entry.url])`. Within the function it is passed into `flattenSearchText`, `normalize`; assigned or updated later in the function.

storedEntry: storedEntry stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; passed into push; assigned or updated later in the function.

function searchSnippet(text = "", query = "")

To understand searchSnippet, start with the problem it owns. searchSnippet belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function searchSnippet(text = "", query = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: text, query. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. text is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. query is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Next, trace the helper calls that surround the function. It works with `normalize`. Those calls show that searchSnippet is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without searchSnippet, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

After the main flow, the working values inside searchSnippet reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(text || "").replace(/s+/g, " ").trim()`. Within the function it is returned to the caller; accesses properties/methods length, slice; passed into min, normalize; assigned or updated later in the function.

index: index stores a computed value for temporary calculation. It is initialized from `normalize(clean).indexOf(normalize(query))`. Within the function it is passed into max, min; assigned or updated later in the function.

start: start stores a computed value for temporary calculation. It is initialized from `Math.max(0, index - 54)`. Within the function it is passed into slice; assigned or updated later in the function.

end: end stores a computed value for temporary calculation. It is initialized from `Math.min(clean.length, index + query.length + 80)`. Within the function it is passed into slice; assigned or updated later in the function.

function entryScore(entry, query)

Read `entryScore` first as a named idea, not as syntax. `entryScore` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function entryScore(entry, query) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `entry`, `query`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `entry` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `query` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. It works with `normalize`. Those calls show that `entryScore` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `entryScore`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

The local objects and variables inside `entryScore` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

normalizedQuery: `normalizedQuery` stores a computed value for temporary calculation. It is initialized from `normalize(query).trim()`. Within the function it is accesses properties/methods `split`; assigned or updated later in the function.

title: `title` stores a computed value for temporary calculation. It is initialized from `normalize(entry.title)`. Within the function it is accesses properties/methods `includes`, `startsWith`; passed into `normalize`; assigned or updated later in the function.

type: `type` stores a computed value for temporary calculation. It is initialized from `normalize(entry.type)`. Within the function it is accesses properties/methods `includes`; passed into `normalize`; assigned or updated later in the function.

context: `context` stores a computed value for temporary calculation. It is initialized from `normalize(entry.context)`. Within the function it is accesses properties/methods `includes`; passed into `normalize`; assigned or updated later in the function.

text: `text` stores a computed value for temporary calculation. It is initialized from `entry.normalizedText || ""`. Within the function it is accesses properties/methods `includes`; assigned or updated later in the function.

words: words stores a computed value for temporary calculation. It is initialized from `normalizedQuery.split(/\s+/).filter(Boolean)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

score: score stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is returned to the caller; assigned or updated later in the function.

function entryMatches(entry, query)

The best entry point for `entryMatches` is its responsibility in the surrounding workflow. `entryMatches` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function entryMatches(entry, query) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `entry`, `query`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `entry` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `query` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. It works with `entryScore`. Those calls show that `entryMatches` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `entryMatches`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Inside `entryMatches`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function fileIsBrowserSearchable(url = "")

Read `fileIsBrowserSearchable` first as a named idea, not as syntax. `fileIsBrowserSearchable` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function fileIsBrowserSearchable(url = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `url`. Read those names as the contract

between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `filesBrowserSearchable` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without `filesBrowserSearchable`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

The local objects and variables inside `filesBrowserSearchable` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(url || "").split(/[?#]/)[0].toLowerCase()`. Within the function it is passed into `test`; assigned or updated later in the function.

async function indexSearchableFileText(entry)

The best entry point for `indexSearchableFileText` is its responsibility in the surrounding workflow. `indexSearchableFileText` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function indexSearchableFileText(entry) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `entry`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `entry` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and calls a network resource. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. Because it is `async`, callers must await it or handle its `Promise`; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: performs a fetch and therefore must handle network delay and failure.

Its connection to the rest of the file matters as much as its local code. It works with `filesBrowserSearchable`, `flattenSearchText`, `normalize`, `updateSearchDropdown`. Those calls show that `indexSearchableFileText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `indexSearchableFileText`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

The easiest way to follow the body of `indexSearchableFileText` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

response: `response` stores data returned from a network call. It is initialized from `await fetch(entry.url)`. Within the function it is accessed properties/methods `ok`, `text`; assigned or updated later in the function.

text: `text` stores the completed result of an awaited operation. It is initialized from `await response.text()`. Within the function it is accessed properties/methods `slice`; passed into `flattenSearchText`, `normalize`; assigned or updated later in the function.

function addProjectSearchEntries(entries, project)

Begin with the role of `addProjectSearchEntries`. `addProjectSearchEntries` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function addProjectSearchEntries(entries, project) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `entries`, `project`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `entries` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `slugLabel`, `richTextTerms`, `addSearchEntry`, `sectionHasRenderableContent`, `displayTitle`, `parsedItemTerms`, `fileNameFromUrl`, `itemUrl`. Those calls show that `addProjectSearchEntries` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `addProjectSearchEntries`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

The local state inside `addProjectSearchEntries` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

categoryLabel: `categoryLabel` stores a computed value for portfolio data state. It is initialized from `slugLabel(project.category)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into `addSearchEntry`, `electronicsSearchKeywords`; assigned or updated later in the function.

baseContext: `baseContext` stores a string value for temporary calculation. It is initialized from ``${categoryLabel} / ${project.title}``. Within the function it is passed into `addSearchEntry`; assigned or updated later in the function.

projectTerms: `projectTerms` stores a list for portfolio data state. It is initialized from `[]`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into `addSearchEntry`; assigned or updated later in the function.

sections: `sections` stores a function or callback value for portfolio data state. It is initialized from `(project.portfolioView?.sections || []).filter((section) => section.id !== "brief" && sectionHasRenderableContent(section))`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

walkItems: `walkItems` stores a function or callback value for temporary calculation. It is initialized from `(items = [], path = [], parentTitles = []) => {}`. Within the function it is assigned or updated later in the function.

nextPath: `nextPath` stores a list for filesystem location. It is initialized from `[...path, index]`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `pathToString`; assigned or updated later in the function.

title: `title` stores a computed value for temporary calculation. It is initialized from `displayTitle(item.title || item.name || item.label || fileNameFromUrl(item.url || ""), "Project item")`. Within the function it is passed into `addSearchEntry`, `displayTitle`; assigned or updated later in the function.

url: `url` stores a computed value for network or routing value. It is initialized from `itemUrl(item)`. Within the function it is passed into `displayTitle`, `fileNameFromUrl`; assigned or updated later in the function.

context: `context` stores a list for temporary calculation. It is initialized from `[baseContext, displayTitle(section.title, "Section"), ...parentTitles].filter(Boolean).join(" / ")`. Within the function it is passed into `addSearchEntry`; assigned or updated later in the function.

entry: `entry` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is passed into `addSearchEntry`; assigned or updated later in the function.

storedEntry: `storedEntry` stores a computed value for temporary calculation. It is initialized from `addSearchEntry(entries, entry)`. Within the function it is passed into `indexSearchableFileText`; assigned or updated later in the function.

entry: `entry` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is passed into `addSearchEntry`; assigned or updated later in the function.

storedEntry: `storedEntry` stores a computed value for temporary calculation. It is initialized from `addSearchEntry(entries, entry)`. Within the function it is passed into `indexSearchableFileText`; assigned or updated later in the function.

function addSiteSectionSearchEntries(entries)

The best entry point for addSiteSectionSearchEntries is its responsibility in the surrounding workflow. addSiteSectionSearchEntries belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function addSiteSectionSearchEntries(entries) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: entries. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. entries is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Its connection to the rest of the file matters as much as its local code. It works with normalizeProfile, addSearchEntry, indexSearchableFileText, richTextTerms. Those calls show that addSiteSectionSearchEntries is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without addSiteSectionSearchEntries, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

The easiest way to follow the body of addSiteSectionSearchEntries is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

current: current stores a computed value for temporary calculation. It is initialized from normalizeProfile(profile || {}). Within the function it is accesses properties/methods contactIntro, displayName, email, githubUrl, linkedinUrl, phone, portfolioLabel, resumeUrl; passed into addSearchEntry; assigned or updated later in the function.

pageSections: pageSections stores a list for portfolio data state. It is initialized from [. Within the function it is iterated or transformed as a collection; accesses properties/methods forEach, splice; assigned or updated later in the function.

resumeTextEntry: resumeTextEntry stores a computed value for temporary calculation. It is initialized from addSearchEntry(entries, {. Within the function it is passed into indexSearchableFileText; assigned or updated later in the function.

function rebuildSearchIndex()

To understand rebuildSearchIndex, start with the problem it owns. rebuildSearchIndex belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function rebuildSearchIndex() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Next, trace the helper calls that surround the function. It works with `addSiteSectionSearchEntries`, `addProjectSearchEntries`. Those calls show that `rebuildSearchIndex` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `rebuildSearchIndex`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

After the main flow, the working values inside `rebuildSearchIndex` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

entries: `entries` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is passed into `addProjectSearchEntries`, `addSiteSectionSearchEntries`; assigned or updated later in the function.

function searchResultsFor(query)

The best entry point for `searchResultsFor` is its responsibility in the surrounding workflow. `searchResultsFor` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function searchResultsFor(query) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `query`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `query` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller

can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Its connection to the rest of the file matters as much as its local code. It works with `normalize`, `entryScore`. Those calls show that `searchResultsFor` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `searchResultsFor`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

The easiest way to follow the body of `searchResultsFor` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

normalizedQuery: `normalizedQuery` stores a computed value for temporary calculation. It is initialized from `normalize(query).trim()`. Within the function it is passed into `entryScore`; assigned or updated later in the function.

seen: `seen` stores a constructed object for lookup collection. It is initialized from `new Set()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `add`, `has`; assigned or updated later in the function.

key: `key` stores a list for temporary calculation. It is initialized from `[entry.kind, entry.projectId, entry.sectionIndex, entry.resourcePath, entry.domTarget, entry.url, entry.title].join("|")`. Within the function it is passed into `add`; assigned or updated later in the function.

function highlightQueryHtml(value = "", query = "")

The best entry point for `highlightQueryHtml` is its responsibility in the surrounding workflow. `highlightQueryHtml` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function highlightQueryHtml(value = "", query = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`, `query`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `query` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. It works with `escapeHtml`, `escapeRegExp`. Those calls show that `highlightQueryHtml` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `highlightQueryHtml`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

The easiest way to follow the body of `highlightQueryHtml` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

escaped: `escaped` stores a computed value for temporary calculation. It is initialized from `escapeHtml(value)`. Within the function it is returned to the caller; accesses properties/methods `replace`; assigned or updated later in the function.

cleanQuery: `cleanQuery` stores a computed value for temporary calculation. It is initialized from `String(query || "").trim()`. Within the function it is passed into `RegExp`; assigned or updated later in the function.

pattern: `pattern` stores a constructed object for temporary calculation. It is initialized from `new RegExp(`${escapeRegExp(escapeHtml(cleanQuery))}`, "ig")`. Within the function it is passed into `replace`; assigned or updated later in the function.

function clearSearchHighlights()

Begin with the role of `clearSearchHighlights`. `clearSearchHighlights` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function clearSearchHighlights() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `clearSearchHighlights` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This is why the function is worth separating instead of burying the logic somewhere else. Without `clearSearchHighlights`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Inside `clearSearchHighlights`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `textNodeSearchTargets()`

The best entry point for `textNodeSearchTargets` is its responsibility in the surrounding workflow. `textNodeSearchTargets` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function textNodeSearchTargets() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it updates browser DOM. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `textNodeSearchTargets` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `textNodeSearchTargets`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Inside `textNodeSearchTargets`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `applySearchHighlights(query)`

The best entry point for `applySearchHighlights` is its responsibility in the surrounding workflow. `applySearchHighlights` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function applySearchHighlights(query) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `query`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. query is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. It works with clearSearchHighlights, normalize, textNodeSearchTargets. Those calls show that applySearchHighlights is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without applySearchHighlights, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

The easiest way to follow the body of applySearchHighlights is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

cleanQuery: cleanQuery stores a computed value for temporary calculation. It is initialized from String(query || "").trim(). Within the function it is accesses properties/methods length; passed into indexOf, normalize, setEnd; assigned or updated later in the function.

ranges: ranges stores a list for temporary calculation. It is initialized from []. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods length, push; passed into set; assigned or updated later in the function.

needle: needle stores a computed value for temporary calculation. It is initialized from normalize(cleanQuery). Within the function it is passed into includes, indexOf; assigned or updated later in the function.

walker: walker stores a DOM element reference. It is initialized from document.createTreeWalker(root, NodeFilter.SHOW_TEXT, {. Within the function it is accesses properties/methods nextNode; assigned or updated later in the function.

node: node stores a computed value for temporary calculation. It is initialized from walker.nextNode(). Within the function it is accesses properties/methods parentElement, textContent; passed into createTreeWalker, normalize, setEnd, setStart; assigned or updated later in the function.

haystack: haystack stores a computed value for runtime state or cache. It is initialized from normalize(node.textContent). Within the function it is accesses properties/methods indexOf; assigned or updated later in the function.

index: index stores a computed value for temporary calculation. It is initialized from haystack.indexOf(needle). Within the function it is passed into indexOf, setEnd, setStart; assigned or updated later in the function.

range: range stores a DOM element reference. It is initialized from document.createRange(). Within the function it is accesses properties/methods setEnd, setStart; passed into push; assigned or updated later in the function.

function queueSearchHighlights()

Read `queueSearchHighlights` first as a named idea, not as syntax. `queueSearchHighlights` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function queueSearchHighlights() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it uses a timer to avoid blocking or to wait for another process. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

The function also has to be read through the helpers it calls. It works with `applySearchHighlights`. Those calls show that `queueSearchHighlights` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `queueSearchHighlights`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Inside `queueSearchHighlights`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function ensureSearchPanel()

Begin with the role of `ensureSearchPanel`. `ensureSearchPanel` creates or repairs something only when it is needed. This pattern avoids duplicate DOM nodes, duplicate repositories, duplicate handlers, or missing folders while still letting later code assume the resource exists.

The syntax of the function is:

```
function ensureSearchPanel() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details:

creates DOM elements instead of relying only on static HTML; writes generated markup into the page, which is why escaping and sanitizing helpers around it matter; attaches event handlers that turn user actions into behavior.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `updateSearchDropdown`, `goToSearchResult`. Those calls show that `ensureSearchPanel` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `ensureSearchPanel`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

The local state inside `ensureSearchPanel` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

shell: `shell` stores a DOM element reference. It is initialized from `document.createElement("div")`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `append`, `className`; passed into `replaceWith`; assigned or updated later in the function.

button: `button` stores a computed value for DOM reference or UI state. It is initialized from `event.target.closest("[data-search-result-index]")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods `dataset`; passed into `Number`; assigned or updated later in the function.

result: `result` stores a computed value for temporary calculation. It is initialized from `currentSearchResults[Number(button.dataset.searchResultIndex)]`. Within the function it is passed into `closest`, `goToSearchResult`, `querySelector`; assigned or updated later in the function.

function setSearchMessage(message, state = "neutral")

The best entry point for `setSearchMessage` is its responsibility in the surrounding workflow. `setSearchMessage` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function setSearchMessage(message, state = "neutral") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `message`, `state`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `message` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `state` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `setSearchMessage` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `setSearchMessage`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Inside `setSearchMessage`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderSearchResults(query, visibleCount)

Begin with the role of `renderSearchResults`. `renderSearchResults` turns saved data for `SearchResults` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

The syntax of the function is:

```
function renderSearchResults(query, visibleCount) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `query`, `visibleCount`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `query` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. `visibleCount` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `ensureSearchPanel`, `searchResultsFor`, `setSearchMessage`, `highlightQueryHtml`, `escapeHtml`, `searchSnippet`. Those calls show that `renderSearchResults` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `renderSearchResults`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

The local state inside `renderSearchResults` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

list: `list` stores a DOM element reference. It is initialized from `searchPanel.querySelector(".search-results-list")`. Within the function it is accesses properties/methods `innerHTML`; passed into `querySelector`; assigned or updated later in the function.

cleanQuery: cleanQuery stores a computed value for temporary calculation. It is initialized from `String(query || "").trim()`. Within the function it is passed into `highlightQueryHtml`, `searchResultsFor`, `searchSnippet`, `setAttribute`, `setSearchMessage`; assigned or updated later in the function.

limitedResults: limitedResults stores a computed value for temporary calculation. It is initialized from `currentSearchResults.slice(0, searchResultLimit)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; passed into `setSearchMessage`; assigned or updated later in the function.

visibleText: visibleText stores a computed value for temporary calculation. It is initialized from `visibleCount`. Within the function it is passed into `setSearchMessage`; assigned or updated later in the function.

function updateSearchDropdown()

Read `updateSearchDropdown` first as a named idea, not as syntax. `updateSearchDropdown` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function updateSearchDropdown() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

The function also has to be read through the helpers it calls. It works with `applySearchHighlights`, `renderSearchResults`. Those calls show that `updateSearchDropdown` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `updateSearchDropdown`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

The local objects and variables inside `updateSearchDropdown` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

query: query stores a computed value for temporary calculation. It is initialized from `searchInput?.value || ""`. Within the function it is passed into `applySearchHighlights`, `renderSearchResults`; assigned or updated later in the function.

visibleCount: visibleCount stores a computed value for temporary calculation. It is initialized from `applySearchHighlights(query)`. Within the function it is passed into `renderSearchResults`; assigned or updated later in the function.

function showSearchPanelIfNeeded()

The best entry point for `showSearchPanelIfNeeded` is its responsibility in the surrounding workflow. `showSearchPanelIfNeeded` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function showSearchPanelIfNeeded() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Its connection to the rest of the file matters as much as its local code. It works with `updateSearchDropdown`. Those calls show that `showSearchPanelIfNeeded` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `showSearchPanelIfNeeded`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Inside `showSearchPanelIfNeeded`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function goToSearchResult(result)

Read `goToSearchResult` first as a named idea, not as syntax. `goToSearchResult` turns a search dropdown item into navigation. Depending on what the result represents, it may open a project window, jump to a main page section, highlight text on the current page, or open a file/link. This makes search feel like a search engine instead of a passive list. The function is necessary because search results can point to many different kinds of things: project titles, section overviews, files, custom sections, contact links, or page text. One navigation rule would not work for all of them.

The syntax of the function is:

```
function goToSearchResult(result) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `result`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. result is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

The function also has to be read through the helpers it calls. It works with `scrollToDomTarget`, `openParsedSection`, `queueSearchHighlights`, `setActiveFilter`, `renderProjects`. Those calls show that `goToSearchResult` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `goToSearchResult`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

The local objects and variables inside `goToSearchResult` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

target: target stores a DOM element reference. It is initialized from `document.getElementById(result.projectId)`. Within the function it is assigned or updated later in the function.

function handleSearchInput()

Read `handleSearchInput` first as a named idea, not as syntax. `handleSearchInput` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function handleSearchInput() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

The function also has to be read through the helpers it calls. It works with `renderProjects`, `updateSearchDropdown`. Those calls show that `handleSearchInput` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `handleSearchInput`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Inside `handleSearchInput`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

Portfolio AI frontend

The functions in this group all support portfolio ai frontend. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function assistantEndpoint()

The best entry point for `assistantEndpoint` is its responsibility in the surrounding workflow. `assistantEndpoint` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantEndpoint() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it updates browser DOM. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `assistantEndpoint` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantEndpoint`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The easiest way to follow the body of `assistantEndpoint` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

metaEndpoint: `metaEndpoint` stores a DOM element reference. It is initialized from `document.querySelector('meta[name="portfolio-ai-endpoint"]')?.content || ""`. Within the function it is accesses properties/methods `trim`; passed into `String`; assigned or updated later in the function.

function assistantSourceLabel(result = {})

The best entry point for `assistantSourceLabel` is its responsibility in the surrounding workflow. `assistantSourceLabel` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantSourceLabel(result = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `result`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `result` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `assistantSourceLabel` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantSourceLabel`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The easiest way to follow the body of `assistantSourceLabel` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

title: `title` stores a computed value for temporary calculation. It is initialized from `result.title || "Portfolio item"`. Within the function it is returned to the caller; assigned or updated later in the function.

contextParts: `contextParts` stores a computed value for temporary calculation. It is initialized from `String(result.context || "")`. Within the function it is accesses properties/methods slice; assigned or updated later in the function.

specificContext: `specificContext` stores a resolved filesystem or route value. It is initialized from `contextParts.slice(-2).join(" / ")`. Within the function it is assigned or updated later in the function.

function assistantProjectSummary(project = {})

The best entry point for `assistantProjectSummary` is its responsibility in the surrounding workflow. `assistantProjectSummary` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantProjectSummary(project = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `project`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one

portfolio project instead of letting work spill across unrelated projects. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Its connection to the rest of the file matters as much as its local code. It works with `slugLabel`, `sectionHasRenderableContent`, `displayTitle`. Those calls show that `assistantProjectSummary` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantProjectSummary`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The easiest way to follow the body of `assistantProjectSummary` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

categoryLabel: `categoryLabel` stores a computed value for portfolio data state. It is initialized from `slugLabel(project.category)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

sections: `sections` stores a function or callback value for portfolio data state. It is initialized from `(project.portfolioView?.sections || [])`. Within the function it is assigned or updated later in the function.

function assistantProjectTitleAcronyms(title = "")

The best entry point for `assistantProjectTitleAcronyms` is its responsibility in the surrounding workflow. `assistantProjectTitleAcronyms` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantProjectTitleAcronyms(title = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `title`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `title` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Its connection to the rest of the file matters as much as its local code. It works with `normalize`. Those calls show that `assistantProjectTitleAcronyms` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantProjectTitleAcronyms`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The easiest way to follow the body of `assistantProjectTitleAcronyms` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

words: `words` stores a computed value for temporary calculation. It is initialized from `normalize(title)`. Within the function it is accesses properties/methods `length`, `slice`; passed into `add`; assigned or updated later in the function.

acronyms: `acronyms` stores a constructed object for lookup collection. It is initialized from `new Set()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `add`; assigned or updated later in the function.

start: `start` stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is passed into `add`; assigned or updated later in the function.

length: `length` stores a numeric value for temporary calculation. It is initialized from 2. Within the function it is passed into `add`; assigned or updated later in the function.

function `assistantProjectIsNamedInQuestion(question = "", project = {})`

To understand `assistantProjectIsNamedInQuestion`, start with the problem it owns.

`assistantProjectIsNamedInQuestion` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantProjectIsNamedInQuestion(question = "", project = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`, `project`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `normalize`, `assistantProjectTitleAcronyms`. Those calls show that `assistantProjectIsNamedInQuestion` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `assistantProjectIsNamedInQuestion`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

After the main flow, the working values inside `assistantProjectIsNamedInQuestion` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is accesses properties/methods `includes`, `split`; passed into `Set`; assigned or updated later in the function.

fillerWords: `fillerWords` stores a constructed object for lookup collection. It is initialized from `new Set(["with", "from", "into", "using", "project", "design", "system", "systems"])`. Within the function it is accesses properties/methods `has`; assigned or updated later in the function.

rawTitle: `rawTitle` stores a computed value for temporary calculation. It is initialized from `project.portfolioView?.title || project.title || project.id`. Within the function it is passed into `assistantProjectTitleAcronyms`, `normalize`; assigned or updated later in the function.

title: `title` stores a computed value for temporary calculation. It is initialized from `normalize(rawTitle)`. Within the function it is accesses properties/methods `split`; assigned or updated later in the function.

cleanWords: `cleanWords` stores a constructed object for lookup collection. It is initialized from `new Set(clean.split(/\s+/))`. Within the function it is accesses properties/methods `has`; assigned or updated later in the function.

acronymMatch: `acronymMatch` stores a computed value for temporary calculation. It is initialized from `assistantProjectTitleAcronyms(rawTitle)`. Within the function it is assigned or updated later in the function.

words: `words` stores a function or callback value for temporary calculation. It is initialized from `title.split(/\s+/).filter((word) => word.length > 3 && !fillerWords.has(word))`. Within the function it is returned to the caller; iterated or transformed as a collection; accesses properties/methods `filter`, `length`; passed into `min`; assigned or updated later in the function.

matches: `matches` stores a function or callback value for temporary calculation. It is initialized from `words.filter((word) => clean.includes(word)).length`. Within the function it is assigned or updated later in the function.

function assistantNamedProjectIds(question = "")

Read `assistantNamedProjectIds` first as a named idea, not as syntax. `assistantNamedProjectIds` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantNamedProjectIds(question = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`. Read those names as the contract

between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. It works with `assistantProjectIdsNamedInQuestion`. Those calls show that `assistantNamedProjectIds` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `assistantNamedProjectIds`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Inside `assistantNamedProjectIds`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function assistantProjectTitleMatches(question = "")

Begin with the role of `assistantProjectTitleMatches`. `assistantProjectTitleMatches` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantProjectTitleMatches(question = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `assistantNamedProjectIds`. Those calls show that `assistantProjectTitleMatches` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `assistantProjectTitleMatches`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Inside `assistantProjectTitleMatches`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function assistantUrlKind(url = "")

The best entry point for `assistantUrlKind` is its responsibility in the surrounding workflow. `assistantUrlKind` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantUrlKind(url = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `url`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Its connection to the rest of the file matters as much as its local code. It works with `filesBrowserSearchable`. Those calls show that `assistantUrlKind` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantUrlKind`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The easiest way to follow the body of `assistantUrlKind` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(url || "").split(/[?#]/)[0].toLowerCase()`. Within the function it is passed into `test`; assigned or updated later in the function.

function assistantAbsoluteUrl(url = "")

Begin with the role of `assistantAbsoluteUrl`. `assistantAbsoluteUrl` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantAbsoluteUrl(url = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `url`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `normalizeLinkTarget`, `looksLikeBareWebAddress`. Those calls show that `assistantAbsoluteUrl` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `assistantAbsoluteUrl`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local state inside `assistantAbsoluteUrl` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

target: `target` stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(url, { assumeWeb: looksLikeBareWebAddress(url) || /^https?:\W/i.test(url) })`. Within the function it is returned to the caller; passed into URL; assigned or updated later in the function.

function assistantLinkRecordsFromValue(value, context = "")

The best entry point for `assistantLinkRecordsFromValue` is its responsibility in the surrounding workflow. `assistantLinkRecordsFromValue` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantLinkRecordsFromValue(value, context = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`, `context`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `context` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. It works with `uniqueDownloads`, `collectDownloadsFromValue`, `assistantUrlKind`, `fileNameFromUrl`, `assistantAbsoluteUrl`. Those calls show that

assistantLinkRecordsFromValue is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without assistantLinkRecordsFromValue, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The easiest way to follow the body of assistantLinkRecordsFromValue is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

records: records stores a list for temporary calculation. It is initialized from []. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods push; assigned or updated later in the function.

downloads: downloads stores a computed value for temporary calculation. It is initialized from uniqueDownloads(collectDownloadsFromValue(value, "Portfolio asset", [])). Within the function it is iterated or transformed as a collection; accesses properties/methods forEach; assigned or updated later in the function.

rawUrl: rawUrl stores a computed value for network or routing value. It is initialized from download.url || download.href || download.path || "". Within the function it is passed into assistantAbsoluteUrl, fileNameFromUrl, push; assigned or updated later in the function.

function assistantUrlsFromTextValue(value, context = "")

Read assistantUrlsFromTextValue first as a named idea, not as syntax. assistantUrlsFromTextValue is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantUrlsFromTextValue(value, context = "") { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: value, context. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. context is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

The function also has to be read through the helpers it calls. It works with flattenSearchText, assistantUrlKind, fileNameFromUrl, assistantAbsoluteUrl. Those calls show that assistantUrlsFromTextValue is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `assistantUrlsFromTextValue`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local objects and variables inside `assistantUrlsFromTextValue` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

text: text stores a computed value for temporary calculation. It is initialized from `flattenSearchText([value])`. Within the function it is accesses properties/methods `match`; assigned or updated later in the function.

matches: matches stores a computed value for temporary calculation. It is initialized from `text.match(/b(?:https?:VV|www\.)[^s<>"])+/gi) || []`. Within the function it is iterated or transformed as a collection; accesses properties/methods `map`; passed into `Set`; assigned or updated later in the function.

function assistantPublicProfileLinks()

The best entry point for `assistantPublicProfileLinks` is its responsibility in the surrounding workflow. `assistantPublicProfileLinks` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantPublicProfileLinks() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. It works with `fileNameFromUrl`, `assistantUrlKind`, `assistantAbsoluteUrl`, `normalizeProfile`. Those calls show that `assistantPublicProfileLinks` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantPublicProfileLinks`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The easiest way to follow the body of `assistantPublicProfileLinks` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

records: records stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; iterated or transformed as a collection; mutated as a collection or DOM container; accesses properties/methods `filter`, `push`; assigned or updated later in the function.

add: add stores a function or callback value for temporary calculation. It is initialized from (item = {}, context = "") => {}. Within the function it is assigned or updated later in the function.

rawUrl: rawUrl stores a computed value for network or routing value. It is initialized from item.url || item.href || item.path || "". Within the function it is passed into assistantAbsoluteUrl, fileNameFromUrl, push; assigned or updated later in the function.

label: label stores a computed value for temporary calculation. It is initialized from item.label || item.title || fileNameFromUrl(rawUrl). Within the function it is passed into add; assigned or updated later in the function.

current: current stores a computed value for temporary calculation. It is initialized from normalizeProfile(profile || {}). Within the function it is accesses properties/methods githubUrl, linkedinUrl, resumeUrl, websiteUrl; passed into add; assigned or updated later in the function.

function assistantProjectEvidence(project = {})

Begin with the role of assistantProjectEvidence. assistantProjectEvidence is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantProjectEvidence(project = {}) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: project. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

The surrounding workflow becomes clearer when you notice its collaborators. It works with uniqueDownloads, collectDownloadsFromValue, assistantUrlKind, fileNameFromUrl, assistantAbsoluteUrl, assistantUrlsFromTextValue. Those calls show that assistantProjectEvidence is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without assistantProjectEvidence, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local state inside assistantProjectEvidence explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

downloads: downloads stores a computed value for temporary calculation. It is initialized from uniqueDownloads(collectDownloadsFromValue(project, "Project asset", [])). Within the function it is accesses properties/methods slice; assigned or updated later in the function.

downloadRecords: downloadRecords stores a function or callback value for lookup collection. It is initialized from `downloads.slice(0, 26).map((download) => {`. Within the function it is assigned or updated later in the function.

rawUrl: rawUrl stores a computed value for network or routing value. It is initialized from `download.url || ""`. Within the function it is passed into `assistantAbsoluteUrl`, `assistantUrlKind`, `fileNameFromUrl`; assigned or updated later in the function.

textLinkRecords: textLinkRecords stores a computed value for temporary calculation. It is initialized from `assistantUrlsFromTextValue([project.summary, project.links, project.portfolioView], project.portfolioView?.title || project.title)`. Within the function it is assigned or updated later in the function.

function assistantQuestionIsCasual(question = "")

The best entry point for `assistantQuestionIsCasual` is its responsibility in the surrounding workflow. `assistantQuestionIsCasual` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantQuestionIsCasual(question = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. It works with `normalize`. Those calls show that `assistantQuestionIsCasual` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantQuestionIsCasual`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The easiest way to follow the body of `assistantQuestionIsCasual` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

clean: clean stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

function assistantQuestionHasPortfolioIntent(question = "")

Read `assistantQuestionHasPortfolioIntent` first as a named idea, not as syntax.

`assistantQuestionHasPortfolioIntent` is one piece of the Ask My Portfolio conversation system. It helps decide

what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantQuestionHasPortfolioIntent(question = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. It works with `normalize`, `assistantProjectTitleMatches`. Those calls show that `assistantQuestionHasPortfolioIntent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `assistantQuestionHasPortfolioIntent`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local objects and variables inside `assistantQuestionHasPortfolioIntent` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

function assistantConversationSuggestsPortfolioContext(history = [])

To understand `assistantConversationSuggestsPortfolioContext`, start with the problem it owns.

`assistantConversationSuggestsPortfolioContext` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantConversationSuggestsPortfolioContext(history = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `history`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `history` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting

it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `normalize`. Those calls show that `assistantConversationSuggestsPortfolioContext` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `assistantConversationSuggestsPortfolioContext`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Inside `assistantConversationSuggestsPortfolioContext`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function assistantQuestionAllowsPublicLookup(question = "", intent = assistantQuestionIntent(question))`

To understand `assistantQuestionAllowsPublicLookup`, start with the problem it owns.

`assistantQuestionAllowsPublicLookup` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantQuestionAllowsPublicLookup(question = "", intent =
  assistantQuestionIntent(question)) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`, `intent`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `intent` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `normalize`. Those calls show that `assistantQuestionAllowsPublicLookup` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `assistantQuestionAllowsPublicLookup`, the assistant would either answer with less context, display unrelated

links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

After the main flow, the working values inside `assistantQuestionAllowsPublicLookup` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

function assistantQuestionLooksConceptual(question = "")

Begin with the role of `assistantQuestionLooksConceptual`. `assistantQuestionLooksConceptual` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantQuestionLooksConceptual(question = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `normalize`, `assistantQuestionIsEngineeringRelated`. Those calls show that `assistantQuestionLooksConceptual` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `assistantQuestionLooksConceptual`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local state inside `assistantQuestionLooksConceptual` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

function assistantQuestionIntent(question = "", history = assistantChatHistory)

Begin with the role of `assistantQuestionIntent`. `assistantQuestionIntent` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantQuestionIntent(question = "", history = assistantChatHistory) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`, `history`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `history` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `assistantQuestionIsCasual`, `normalize`, `assistantQuestionHasPortfolioIntent`, `assistantConversationSuggestsPortfolioContext`, `assistantQuestionIsEngineeringRelated`, `assistantQuestionLooksConceptual`. Those calls show that `assistantQuestionIntent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `assistantQuestionIntent`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local state inside `assistantQuestionIntent` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

hasPortfolioIntent: `hasPortfolioIntent` stores a computed value for temporary calculation. It is initialized from `assistantQuestionHasPortfolioIntent(question)`. Within the function it is assigned or updated later in the function.

```
function assistantQuestionTokens(question = "")
```

Read `assistantQuestionTokens` first as a named idea, not as syntax. `assistantQuestionTokens` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantQuestionTokens(question = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. It works with `normalize`. Those calls show that `assistantQuestionTokens` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `assistantQuestionTokens`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local objects and variables inside `assistantQuestionTokens` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

stopWords: `stopWords` stores a constructed object for lookup collection. It is initialized from `new Set([`. Within the function it is accessed via `properties/methods` `has`; assigned or updated later in the function.

function `assistantSpecificSourceTokens(question = "")`

To understand `assistantSpecificSourceTokens`, start with the problem it owns. `assistantSpecificSourceTokens` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantSpecificSourceTokens(question = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `assistantQuestionTokens`. Those calls show that `assistantSpecificSourceTokens` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `assistantSpecificSourceTokens`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

After the main flow, the working values inside `assistantSpecificSourceTokens` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

genericTokens: genericTokens stores a constructed object for lookup collection. It is initialized from new Set([. Within the function it is accesses properties/methods has; assigned or updated later in the function.

function assistantPromptTargetsProjectLanding(question = "")

Read assistantPromptTargetsProjectLanding first as a named idea, not as syntax.

assistantPromptTargetsProjectLanding is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantPromptTargetsProjectLanding(question = "") { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: question. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. It works with normalize. Those calls show that assistantPromptTargetsProjectLanding is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without assistantPromptTargetsProjectLanding, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local objects and variables inside assistantPromptTargetsProjectLanding show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: clean stores a computed value for temporary calculation. It is initialized from normalize(question). Within the function it is passed into test; assigned or updated later in the function.

function assistantEntryRelationScore(result = {}, question = "")

Read assistantEntryRelationScore first as a named idea, not as syntax. assistantEntryRelationScore is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantEntryRelationScore(result = {}, question = "") { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: result, question. Read those names as the

contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `result` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. It works with `assistantSpecificSourceTokens`, `normalize`. Those calls show that `assistantEntryRelationScore` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `assistantEntryRelationScore`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local objects and variables inside `assistantEntryRelationScore` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

tokens: `tokens` stores a computed value for temporary calculation. It is initialized from `assistantSpecificSourceTokens(question)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `length`; assigned or updated later in the function.

haystack: `haystack` stores a resolved filesystem or route value. It is initialized from `normalize([result.title, result.context, result.type, result.text, result.url].filter(Boolean).join(" "))`. Within the function it is accesses properties/methods `includes`; assigned or updated later in the function.

tokenHits: `tokenHits` stores a function or callback value for temporary calculation. It is initialized from `tokens.filter((token) => haystack.includes(token)).length`. Within the function it is assigned or updated later in the function.

relation: `relation` stores a computed value for temporary calculation. It is initialized from `tokenHits * 30`. Within the function it is returned to the caller; assigned or updated later in the function.

function `assistantNamedProjectTopicTokens(question = "", namedIds = assistantNamedProjectIds(question))`

To understand `assistantNamedProjectTopicTokens`, start with the problem it owns.

`assistantNamedProjectTopicTokens` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantNamedProjectTopicTokens(question = "", namedIds =
assistantNamedProjectIds(question)) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`, `namedIds`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `namedIds` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `normalize`, `assistantProjectTitleAcronyms`, `assistantSpecificSourceTokens`. Those calls show that `assistantNamedProjectTopicTokens` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `assistantNamedProjectTopicTokens`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

After the main flow, the working values inside `assistantNamedProjectTopicTokens` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

projectVocabulary: `projectVocabulary` stores a constructed object for portfolio data state. It is initialized from `new Set()`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is mutated as a collection or DOM container; accesses properties/methods `add`, `has`; assigned or updated later in the function.

title: `title` stores a computed value for temporary calculation. It is initialized from `project.portfolioView?.title || project.title || project.id`. Within the function it is passed into `assistantProjectTitleAcronyms`, `normalize`; assigned or updated later in the function.

function `assistantSourcesForDisplay(question = "", results = [], intent = assistantQuestionIntent(question))`

The best entry point for `assistantSourcesForDisplay` is its responsibility in the surrounding workflow. `assistantSourcesForDisplay` chooses which portfolio links appear under an assistant answer. The user complained earlier that related links were generic; this function is the fix-point for that behavior. It scores source relation to the prompt, respects named projects and topic tokens, and avoids showing source buttons that do not belong to the answer context. The function matters because sources are a promise. A displayed link should help verify or explore the answer. If this function is too loose, the assistant looks random even when the text answer is good.

The syntax of the function is:

```
function assistantSourcesForDisplay(question = "", results = [], intent =
assistantQuestionIntent(question)) { ... }
```


There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`, `results`, `intent`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `results` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `intent` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Its connection to the rest of the file matters as much as its local code. It works with `assistantSpecificSourceTokens`, `assistantNamedProjectIds`, `assistantNamedProjectTopicTokens`, `normalize`, `assistantPromptTargetsProjectLanding`, `assistantEntryRelationScore`. Those calls show that `assistantSourcesForDisplay` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantSourcesForDisplay`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The easiest way to follow the body of `assistantSourcesForDisplay` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

tokens: `tokens` stores a computed value for temporary calculation. It is initialized from `assistantSpecificSourceTokens(question)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `length`; assigned or updated later in the function.

namedProjectIdList: `namedProjectIdList` stores a computed value for portfolio data state. It is initialized from `assistantNamedProjectIds(question)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into `Set`, `assistantNamedProjectTopicTokens`; assigned or updated later in the function.

namedProjectIds: `namedProjectIds` stores a constructed object for portfolio data state. It is initialized from `new Set(namedProjectIdList)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it accesses properties/methods `has`, `size`; assigned or updated later in the function.

namedProjectTopicTokens: `namedProjectTopicTokens` stores a computed value for portfolio data state. It is initialized from `assistantNamedProjectTopicTokens(question, namedProjectIdList)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `length`; assigned or updated later in the function.

asksForLinks: asksForLinks stores a computed value for temporary calculation. It is initialized from `\b(open|show|where|link|links|github|linkedin|resume|download|file|files|repo|repository|source\s+code)\b/.test(normalize(question))`. Within the function it is passed into slice; assigned or updated later in the function.

asksForProjectLanding: asksForProjectLanding stores a computed value for portfolio data state. It is initialized from `assistantPromptTargetsProjectLanding(question)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

pageLinkIntent: pageLinkIntent stores a computed value for temporary calculation. It is initialized from `\b(resume|github|linkedin|contact|email|phone)\b/.test(normalize(question))`. Within the function it is assigned or updated later in the function.

asksForSpecificArtifact: asksForSpecificArtifact stores a computed value for temporary calculation. It is initialized from `\b(code|source|repo|repository|github|file|files|document|documents|test|tests|result|results|pcb|schematic|simulation|tool|tools)\b/.test(normalize(question))`. Within the function it is assigned or updated later in the function.

filtered: filtered stores a computed value for temporary calculation. It is initialized from results. Within the function it is returned to the caller; assigned or updated later in the function.

haystack: haystack stores a resolved filesystem or route value. It is initialized from `normalize([result.title, result.context, result.type, result.text, result.url].filter(Boolean).join(" "))`. Within the function it is accesses properties/methods includes; assigned or updated later in the function.

tokenHits: tokenHits stores a function or callback value for temporary calculation. It is initialized from `tokens.filter((token) => haystack.includes(token)).length`. Within the function it is assigned or updated later in the function.

topicHits: topicHits stores a function or callback value for temporary calculation. It is initialized from `namedProjectTopicTokens.filter((token) => haystack.includes(token)).length`. Within the function it is assigned or updated later in the function.

function assistantContextForQuestion(question = "", intent = assistantQuestionIntent(question), knownResults = null)

Read `assistantContextForQuestion` first as a named idea, not as syntax. `assistantContextForQuestion` builds the packet of portfolio evidence sent to the AI backend. It uses the question intent, search results, named project matches, profile links, project summaries, and available evidence to decide what the model should see. It is selective by design so a generic greeting does not receive a pile of unrelated project context. Without this function, the assistant would either know too little about the portfolio or always overload the model with the same generic links. This is the function that makes the AI choose context based on the conversation.

The syntax of the function is:

```
function assistantContextForQuestion(question = "", intent = assistantQuestionIntent(question),
knownResults = null) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`, `intent`, `knownResults`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing,

or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. intent is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. knownResults is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

The function also has to be read through the helpers it calls. It works with searchResultsFor, assistantNamedProjectIds, normalize, assistantPublicProfileLinks, assistantUrlKind, assistantAbsoluteUrl, assistantProjectEvidence, normalizeProfile. Those calls show that assistantContextForQuestion is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without assistantContextForQuestion, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local objects and variables inside assistantContextForQuestion show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

directResults: directResults stores a list for filesystem location. It is initialized from ["general_engineering", "general_knowledge", "general_conversation"].includes(intent). It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is iterated or transformed as a collection; accesses properties/methods map; passed into Set; assigned or updated later in the function.

projectIds: projectIds stores a list for portfolio data state. It is initialized from [...new Set(. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

matchedProjects: matchedProjects stores a computed value for portfolio data state. It is initialized from projectIds. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is iterated or transformed as a collection; accesses properties/methods flatMap, map; assigned or updated later in the function.

includePortfolioLinkContext: includePortfolioLinkContext stores a computed value for temporary calculation. It is initialized from intent === "portfolio_specific". Within the function it is assigned or updated later in the function.

wantsPublicCode: wantsPublicCode stores a computed value for temporary calculation. It is initialized from `\b(github|repo|repository|repositories|source\s+code|code|snippet|firmware|verilog|systemverilog|python|javascript)\b/.test(normalize(question))`. Within the function it is assigned or updated later in the function.

publicProfileFetches: publicProfileFetches stores a computed value for portfolio data state. It is initialized from includePortfolioLinkContext. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

aGithub: aGithub stores a computed value for Git publishing and authorization state. It is initialized from `/github/i.test(`${a.kind} || "" ${a.url} || ""`) ? 1 : 0`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is assigned or updated later in the function.

bGithub: bGithub stores a computed value for Git publishing and authorization state. It is initialized from `/github/i.test(`${b.kind} || "" ${b.url} || ""`) ? 1 : 0`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is returned to the caller; assigned or updated later in the function.

resultFetches: resultFetches stores a function or callback value for lookup collection. It is initialized from `directResults.map((result) => {`. Within the function it is assigned or updated later in the function.

projectFetches: projectFetches stores a function or callback value for portfolio data state. It is initialized from `matchedProjects.flatMap((project) => assistantProjectEvidence(project)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

sourceFetches: sourceFetches stores a list for temporary calculation. It is initialized from `[`. Within the function it is iterated or transformed as a collection; accesses properties/methods filter; assigned or updated later in the function.

uniqueSourceFetches: uniqueSourceFetches stores a function or callback value for temporary calculation. It is initialized from `sourceFetches.filter((item, index, array) => (`. Within the function it is assigned or updated later in the function.

function assistantRemember(role, content = "")

To understand assistantRemember, start with the problem it owns. assistantRemember is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantRemember(role, content = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `role`, `content`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `role` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `content` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means assistantRemember performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `assistantRemember`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

After the main flow, the working values inside `assistantRemember` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

cleanRole: `cleanRole` stores a computed value for temporary calculation. It is initialized from `role === "assistant" ? "assistant" : "user"`. Within the function it is passed into `push`; assigned or updated later in the function.

cleanContent: `cleanContent` stores a computed value for temporary calculation. It is initialized from `String(content || "").trim()`. Within the function it is accesses properties/methods slice; passed into `push`; assigned or updated later in the function.

function assistantConversationForRequest(currentQuestion = "", priorConversation = assistantChatHistory)

To understand `assistantConversationForRequest`, start with the problem it owns.

`assistantConversationForRequest` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantConversationForRequest(currentQuestion = "", priorConversation =
assistantChatHistory) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `currentQuestion`, `priorConversation`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `currentQuestion` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `priorConversation` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser route/history. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `assistantConversationForRequest` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `assistantConversationForRequest`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

After the main flow, the working values inside `assistantConversationForRequest` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

history: history stores a function or callback value for runtime state or cache. It is initialized from `priorConversation.slice(-8).map((item) => ({`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `push`; assigned or updated later in the function.

function assistantFallbackResults(question = "", intent = assistantQuestionIntent(question))

Begin with the role of `assistantFallbackResults`. `assistantFallbackResults` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantFallbackResults(question = "", intent = assistantQuestionIntent(question))
{ ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`, `intent`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `intent` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `normalize`, `assistantNamedProjectIds`, `searchResultsFor`, `slugLabel`, `flattenSearchText`. Those calls show that `assistantFallbackResults` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `assistantFallbackResults`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local state inside `assistantFallbackResults` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question).trim()`. Within the function it is accesses properties/methods `includes`; passed into `searchResultsFor`; assigned or updated later in the function.

namedProjectIds: `namedProjectIds` stores a computed value for portfolio data state. It is initialized from `assistantNamedProjectIds(question)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is returned to the caller; accesses properties/methods `includes`, `length`; assigned or updated later in the function.

results: `results` stores a computed value for temporary calculation. It is initialized from `searchResultsFor(clean)`. Within the function it is returned to the caller; iterated or transformed as a collection; accesses properties/methods `filter`, `length`, `slice`; assigned or updated later in the function.

focusedResults: focusedResults stores a function or callback value for temporary calculation. It is initialized from `results.filter((result) => namedProjectIds.includes(result.projectId))`. Within the function it is returned to the caller; accesses properties/methods `length`, `slice`; assigned or updated later in the function.

category: category stores a function or callback value for portfolio data state. It is initialized from `categories.find((item) => clean.includes(normalize(item.label)) || clean.includes(normalize(item.id)))`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `id`, `label`; passed into `slugLabel`; assigned or updated later in the function.

function assistantQuestionIsEngineeringRelated(question = "")

Read `assistantQuestionIsEngineeringRelated` first as a named idea, not as syntax.

`assistantQuestionIsEngineeringRelated` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantQuestionIsEngineeringRelated(question = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

The function also has to be read through the helpers it calls. It works with `normalize`. Those calls show that `assistantQuestionIsEngineeringRelated` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `assistantQuestionIsEngineeringRelated`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local objects and variables inside `assistantQuestionIsEngineeringRelated` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

function assistantGeneralEngineeringAnswer(question = "")

The best entry point for `assistantGeneralEngineeringAnswer` is its responsibility in the surrounding workflow.

`assistantGeneralEngineeringAnswer` is one piece of the Ask My Portfolio conversation system. It helps decide

what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantGeneralEngineeringAnswer(question = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Its connection to the rest of the file matters as much as its local code. It works with `normalize`, `assistantQuestionIsEngineeringRelated`. Those calls show that `assistantGeneralEngineeringAnswer` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantGeneralEngineeringAnswer`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The easiest way to follow the body of `assistantGeneralEngineeringAnswer` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

function `assistantLocalAnswer(question = "", results = [], intent = assistantQuestionIntent(question))`

To understand `assistantLocalAnswer`, start with the problem it owns. `assistantLocalAnswer` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantLocalAnswer(question = "", results = [], intent =
assistantQuestionIntent(question)) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`, `results`, `intent`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing,

or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. results is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. intent is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Next, trace the helper calls that surround the function. It works with `normalizeProfile`, `normalize`, `assistantGeneralEngineeringAnswer`, `searchSnippet`. Those calls show that `assistantLocalAnswer` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `assistantLocalAnswer`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

After the main flow, the working values inside `assistantLocalAnswer` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

ownerName: `ownerName` stores a computed value for temporary calculation. It is initialized from `normalizeProfile(profile || {}).displayName || "the portfolio owner"`. Within the function it is assigned or updated later in the function.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

generalAnswer: `generalAnswer` stores a computed value for temporary calculation. It is initialized from `assistantGeneralEngineeringAnswer(question)`. Within the function it is returned to the caller; assigned or updated later in the function.

generalAnswer: `generalAnswer` stores a computed value for temporary calculation. It is initialized from `assistantGeneralEngineeringAnswer(question)`. Within the function it is returned to the caller; assigned or updated later in the function.

projectIds: `projectIds` stores a list for portfolio data state. It is initialized from `[...new Set(results.map((item) => item.projectId).filter(Boolean))]`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

matchedProjects: `matchedProjects` stores a computed value for portfolio data state. It is initialized from `projectIds`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is iterated or transformed as a collection; accesses properties/methods `map`; assigned or updated later in the function.

projectNames: `projectNames` stores a function or callback value for portfolio data state. It is initialized from `matchedProjects.map((project) => project.portfolioView?.title || project.title).slice(0, 3)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `join`, `length`; passed into `push`; assigned or updated later in the function.

fileCount: fileCount stores a function or callback value for temporary calculation. It is initialized from `results.filter((item) => item.kind === "file").length`. Within the function it is passed into push; assigned or updated later in the function.

sectionCount: sectionCount stores a function or callback value for portfolio data state. It is initialized from `results.filter((item) => item.kind === "section" || item.kind === "subsection").length`. Within the function it is passed into push; assigned or updated later in the function.

pageCount: pageCount stores a function or callback value for temporary calculation. It is initialized from `results.filter((item) => item.kind === "page").length`. Within the function it is passed into push; assigned or updated later in the function.

parts: parts stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods join, push; assigned or updated later in the function.

firstSnippet: firstSnippet stores a computed value for temporary calculation. It is initialized from `searchSnippet(results[0].text, question)`. Within the function it is passed into push; assigned or updated later in the function.

function assistantSourcesMarkup(results = [])

Read assistantSourcesMarkup first as a named idea, not as syntax. assistantSourcesMarkup is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantSourcesMarkup(results = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `results`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `results` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

The function also has to be read through the helpers it calls. It works with `escapeHtml`, `assistantSourceLabel`. Those calls show that assistantSourcesMarkup is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without assistantSourcesMarkup, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local objects and variables inside assistantSourcesMarkup show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

sources: sources stores a computed value for temporary calculation. It is initialized from results.slice(0, 5). Within the function it is iterated or transformed as a collection; accesses properties/methods length, map; assigned or updated later in the function.

sourceButtons: sourceButtons stores a function or callback value for DOM reference or UI state. It is initialized from sources.map((source) => {). It anchors UI behavior to a specific browser element or window state. Within the function it is assigned or updated later in the function.

id: id stores a computed value for temporary calculation. It is initialized from String(assistantSourceCounter++). Within the function it is passed into set; assigned or updated later in the function.

function setAssistantStatus(message, state = "ready")

Read setAssistantStatus first as a named idea, not as syntax. setAssistantStatus belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function setAssistantStatus(message, state = "ready") { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: message, state. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. message is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. state is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means setAssistantStatus performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without setAssistantStatus, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Inside setAssistantStatus, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function assistantDesktopDockedHeight()

The best entry point for assistantDesktopDockedHeight is its responsibility in the surrounding workflow. assistantDesktopDockedHeight is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantDesktopDockedHeight() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `assistantDesktopDockedHeight` performs its rule directly from parameters, module state, standard APIs, or local calculations.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantDesktopDockedHeight`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The easiest way to follow the body of `assistantDesktopDockedHeight` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

value: value stores a computed value for temporary calculation. It is initialized from `getComputedStyle(aiAssistantPanel).getPropertyValue("--ai-console-rest-height").trim()`. Within the function it is accesses properties/methods endsWith; passed into `parseFloat`; assigned or updated later in the function.

function updateAssistantPanelGrowth()

Begin with the role of `updateAssistantPanelGrowth`. `updateAssistantPanelGrowth` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function updateAssistantPanelGrowth() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `assistantDesktopDockedHeight`. Those calls show that `updateAssistantPanelGrowth` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `updateAssistantPanelGrowth`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local state inside `updateAssistantPanelGrowth` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

isDesktop: `isDesktop` stores a computed value for temporary calculation. It is initialized from `window.matchMedia("(min-width: 901px)").matches`. Within the function it is assigned or updated later in the function.

dockedHeight: `dockedHeight` stores a computed value for temporary calculation. It is initialized from `assistantDesktopDockedHeight()`. Within the function it is passed into `round`, `toggle`; assigned or updated later in the function.

formHeight: `formHeight` stores a computed value for temporary calculation. It is initialized from `aiAssistantForm.getBoundingClientRect().height || 46`. Within the function it is assigned or updated later in the function.

clearHeight: `clearHeight` stores a computed value for temporary calculation. It is initialized from `aiClearChatButton?.getBoundingClientRect().height || 38`. Within the function it is assigned or updated later in the function.

titleHeight: `titleHeight` stores a DOM element reference. It is initialized from `document.querySelector("#ai-assistant-title")?.getBoundingClientRect().height || 34`. Within the function it is assigned or updated later in the function.

chromeHeight: `chromeHeight` stores a computed value for temporary calculation. It is initialized from `titleHeight + formHeight + clearHeight + 92`. Within the function it is assigned or updated later in the function.

messages: `messages` stores a list for temporary calculation. It is initialized from `[...aiAssistantLog.querySelectorAll(".ai-message")]`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `reduce`; assigned or updated later in the function.

messageGap: `messageGap` stores a computed value for temporary calculation. It is initialized from `messages.length > 1 ? (messages.length - 1) * 10 : 0`. Within the function it is assigned or updated later in the function.

messageHeight: `messageHeight` stores a function or callback value for temporary calculation. It is initialized from `messages.reduce((sum, message) => sum + message.getBoundingClientRect().height, 0) + messageGap`. Within the function it is assigned or updated later in the function.

contentHeight: `contentHeight` stores a computed value for temporary calculation. It is initialized from `messageHeight + chromeHeight`. Within the function it is passed into `round`; assigned or updated later in the function.

maxHeight: `maxHeight` stores a computed value for temporary calculation. It is initialized from `Math.min(window.innerHeight * 0.76, 720)`. Within the function it is passed into `round`; assigned or updated later in the function.

nextHeight: nextHeight stores a computed value for temporary calculation. It is initialized from `Math.round(Math.max(dockedHeight, Math.min(contentHeight, maxHeight)))`. Within the function it is passed into `setProperty, toggle`; assigned or updated later in the function.

function renderAssistantInline(value = "")

To understand `renderAssistantInline`, start with the problem it owns. `renderAssistantInline` turns saved data for `AssistantInline` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

The syntax of the function is:

```
function renderAssistantInline(value = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `value`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `renderInlineMath`. Those calls show that `renderAssistantInline` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `renderAssistantInline`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Inside `renderAssistantInline`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderAssistantAnswerContent(content = "")

To understand `renderAssistantAnswerContent`, start with the problem it owns. `renderAssistantAnswerContent` turns saved data for `AssistantAnswerContent` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

The syntax of the function is:

```
function renderAssistantAnswerContent(content = "") { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `content`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `content` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Next, trace the helper calls that surround the function. It works with `renderAssistantInline`, `escapeHtml`. Those calls show that `renderAssistantAnswerContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `renderAssistantAnswerContent`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

After the main flow, the working values inside `renderAssistantAnswerContent` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

lines: `lines` stores a computed value for temporary calculation. It is initialized from `String(content || "").replace(/r\n?/g, "\n").split("\n")`. Within the function it is iterated or transformed as a collection; mutated as a collection or DOM container; accesses properties/methods `forEach`, `join`, `push`; passed into `push`; assigned or updated later in the function.

html: `html` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

bulletItems: `bulletItems` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is iterated or transformed as a collection; mutated as a collection or DOM container; accesses properties/methods `length`, `map`, `push`; passed into `push`; assigned or updated later in the function.

codeBlock: `codeBlock` stores a computed value for temporary calculation. It is initialized from `null`. Within the function it is accesses properties/methods `language`, `lines`; passed into `escapeHtml`, `push`; assigned or updated later in the function.

flushBullets: `flushBullets` stores a function or callback value for temporary calculation. It is initialized from `() => {}`. Within the function it is assigned or updated later in the function.

flushCode: `flushCode` stores a function or callback value for temporary calculation. It is initialized from `() => {}`. Within the function it is assigned or updated later in the function.

language: `language` stores a computed value for compiler or language configuration. It is initialized from `codeBlock.language ? `data-language="${escapeHtml(codeBlock.language)}"` : ""`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `escapeHtml`, `push`; assigned or updated later in the function.

trimmed: `trimmed` stores a computed value for temporary calculation. It is initialized from `line.trim()`. Within the function it is accesses properties/methods `match`, `slice`; passed into `push`; assigned or updated later in the function.

fence: fence stores a computed value for temporary calculation. It is initialized from `trimmed.match(/^``([a-z0-9_+.#-]*)s*$/i)`. Within the function it is assigned or updated later in the function.

bullet: bullet stores a computed value for temporary calculation. It is initialized from `trimmed.match(/^[-*\u2022]\s+(.+)/u)`. Within the function it is passed into `push`; assigned or updated later in the function.

function `appendAssistantMessage(role, content, sources = [])`

The best entry point for `appendAssistantMessage` is its responsibility in the surrounding workflow.

`appendAssistantMessage` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function appendAssistantMessage(role, content, sources = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `role`, `content`, `sources`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `role` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `content` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `sources` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: creates DOM elements instead of relying only on static HTML; writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Its connection to the rest of the file matters as much as its local code. It works with `renderAssistantAnswerContent`, `assistantSourcesMarkup`, `renderMultilineInlineText`, `updateAssistantPanelGrowth`. Those calls show that `appendAssistantMessage` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `appendAssistantMessage`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The easiest way to follow the body of `appendAssistantMessage` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

article: article stores a DOM element reference. It is initialized from `document.createElement("article")`. Within the function it is returned to the caller; accesses properties/methods `className`, `innerHTML`; passed into `append`, `createElement`; assigned or updated later in the function.

function scrollAssistantAnswerToStart(article)

Read scrollAssistantAnswerToStart first as a named idea, not as syntax. scrollAssistantAnswerToStart belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function scrollAssistantAnswerToStart(article) { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: article. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. article is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it uses a timer to avoid blocking or to wait for another process. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means scrollAssistantAnswerToStart performs its rule directly from parameters, module state, standard APIs, or local calculations.

The application keeps this behavior isolated because other features rely on it being consistent. Without scrollAssistantAnswerToStart, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local objects and variables inside scrollAssistantAnswerToStart show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

scrollToAnswer: scrollToAnswer stores a function or callback value for temporary calculation. It is initialized from () => {}. Within the function it is passed into requestAnimationFrame, setTimeout; assigned or updated later in the function.

function appendAssistantPendingMessage()

Begin with the role of appendAssistantPendingMessage. appendAssistantPendingMessage belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function appendAssistantPendingMessage() { ... }
```

There is no async keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

The surrounding workflow becomes clearer when you notice its collaborators. It works with `appendAssistantMessage`. Those calls show that `appendAssistantPendingMessage` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This is why the function is worth separating instead of burying the logic somewhere else. Without `appendAssistantPendingMessage`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local state inside `appendAssistantPendingMessage` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

article: article stores a computed value for temporary calculation. It is initialized from `appendAssistantMessage("assistant", "Thinking", [])`. Within the function it is returned to the caller; assigned or updated later in the function.

function replaceAssistantMessage(article, content, sources = [])

The best entry point for `replaceAssistantMessage` is its responsibility in the surrounding workflow. `replaceAssistantMessage` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function replaceAssistantMessage(article, content, sources = []) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `article`, `content`, `sources`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `article` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `content` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `sources` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Its connection to the rest of the file matters as much as its local code. It works with `appendAssistantMessage`, `renderAssistantAnswerContent`, `assistantSourcesMarkup`, `updateAssistantPanelGrowth`, `scrollAssistantAnswerToStart`. Those calls show that `replaceAssistantMessage` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `replaceAssistantMessage`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Inside `replaceAssistantMessage`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function setAssistantBusy(isBusy)

To understand `setAssistantBusy`, start with the problem it owns. `setAssistantBusy` belongs to `portfolio ai frontend`. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function setAssistantBusy(isBusy) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `isBusy`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `isBusy` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `setAssistantBusy` performs its rule directly from parameters, module state, standard APIs, or local calculations.

The practical importance of the function is easiest to see by imagining it removed. Without `setAssistantBusy`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

After the main flow, the working values inside `setAssistantBusy` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

submitButton: `submitButton` stores a DOM element reference. It is initialized from `aiAssistantForm?.querySelector('button[type="submit"]')`. It anchors UI behavior to a specific browser element or window state. Within the function it is accessed properties/methods disabled; assigned or updated later in the function.

function clearAssistantChat()

To understand `clearAssistantChat`, start with the problem it owns. `clearAssistantChat` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
function clearAssistantChat() { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The empty parameter list means the function works from module-level state, browser state, local constants, or values it creates internally. That is common for UI helpers that operate on known page elements and backend helpers that read already configured paths.

Those syntax pieces become practical once you connect them to the data being passed in. This function accepts no explicit parameters. It reads controlled module state, browser state, local configuration, or constants that are already available in the file.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Next, trace the helper calls that surround the function. It works with `setAssistantStatus`, `updateAssistantPanelGrowth`. Those calls show that `clearAssistantChat` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `clearAssistantChat`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Inside `clearAssistantChat`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function assistantRemoteTimeoutMs(question = "", context = {}, options = {})

To understand `assistantRemoteTimeoutMs`, start with the problem it owns. `assistantRemoteTimeoutMs` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

The syntax of the function is:

```
function assistantRemoteTimeoutMs(question = "", context = {}, options = {}) { ... }
```

There is no `async` keyword, so the function completes synchronously unless it calls another asynchronous helper indirectly. The parameter list tells you what the caller must provide: `question`, `context`, `options`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `context` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated

decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Next, trace the helper calls that surround the function. It works with `normalize`. Those calls show that `assistantRemoteTimeoutMs` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `assistantRemoteTimeoutMs`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

After the main flow, the working values inside `assistantRemoteTimeoutMs` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

sourceFetches: `sourceFetches` stores a computed value for temporary calculation. It is initialized from `Array.isArray(context.sourceFetches) ? context.sourceFetches : []`. Within the function it is accessed properties/methods `some`; passed into `isArray`; assigned or updated later in the function.

usesGitHub: `usesGitHub` stores a function or callback value for Git publishing and authorization state. It is initialized from `sourceFetches.some((source) => String(source.kind || "").toLowerCase() === "github" || /github\.com\/i.test(source.url || ""))`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is assigned or updated later in the function.

async function askRemoteAssistant(question, context, options = {})

To understand `askRemoteAssistant`, start with the problem it owns. `askRemoteAssistant` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

The syntax of the function is:

```
async function askRemoteAssistant(question, context, options = {}) { ... }
```

The `async` keyword is important here: this function returns a `Promise`, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `question`, `context`, `options`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. `context` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work, calls a network resource and uses a timer to avoid blocking or to wait for another process. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. Because it is async, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished. A close reading of the body shows these implementation details: serializes structured data before sending or saving it; throws explicit errors when a required safety or compatibility condition fails; performs a fetch and therefore must handle network delay and failure.

Next, trace the helper calls that surround the function. It works with `assistantEndpoint`, `assistantRemoteTimeoutMs`. Those calls show that `askRemoteAssistant` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The practical importance of the function is easiest to see by imagining it removed. Without `askRemoteAssistant`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

After the main flow, the working values inside `askRemoteAssistant` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

endpoint: `endpoint` stores a computed value for network or routing value. It is initialized from `assistantEndpoint()`. Within the function it is passed into `fetch`; assigned or updated later in the function.

controller: `controller` stores a constructed object for temporary calculation. It is initialized from `new AbortController()`. Within the function it is accesses properties/methods `abort`, `signal`; passed into `fetch`; assigned or updated later in the function.

didTimeout: `didTimeout` stores a boolean value for temporary calculation. It is initialized from `false`. Within the function it is assigned or updated later in the function.

timer: `timer` stores a function or callback value for temporary calculation. It is initialized from `setTimeout(() => {`. Within the function it is passed into `clearTimeout`; assigned or updated later in the function.

response: `response` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `json`, `ok`; assigned or updated later in the function.

data: `data` stores the completed result of an awaited operation. It is initialized from `await response.json()`. Within the function it is accesses properties/methods `answer`, `message`, `model`, `usedWebSearch`; passed into `Boolean`, `String`; assigned or updated later in the function.

answer: `answer` stores a computed value for temporary calculation. It is initialized from `String(data.answer || data.message || "").trim()`. Within the function it is returned to the caller; passed into `String`; assigned or updated later in the function.

async function answerAssistantQuestion(question = "")

Read `answerAssistantQuestion` first as a named idea, not as syntax. `answerAssistantQuestion` is the public chatbot orchestrator. It receives the user's typed question, appends it to chat history, classifies the intent, gathers relevant portfolio context, creates a pending assistant answer, calls the backend AI when appropriate, falls back to local answers when needed, renders sources only when relevant, and restores focus to the input. This function is async because the AI backend may need network time and because context gathering can include indexed file text. It is the reason Ask My Portfolio behaves like a conversation instead of a static search form.

The syntax of the function is:

```
async function answerAssistantQuestion(question = "") { ... }
```

The `async` keyword is important here: this function returns a Promise, so callers must wait for it before trusting the result or assuming the side effect has completed. The parameter list tells you what the caller must provide: `question`. Read those names as the contract between this function and the code that calls it. The body should make sense only after those inputs are understood.

Those syntax pieces become practical once you connect them to the data being passed in. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. Because it is `async`, callers must await it or handle its Promise; otherwise the UI or backend could move on before files, network calls, Git commands, compiler runs, or model responses have finished.

The function also has to be read through the helpers it calls. It works with `appendAssistantMessage`, `assistantRemember`, `appendAssistantPendingMessage`, `setAssistantBusy`, `setAssistantStatus`, `assistantEndpoint`, `assistantQuestionIntent`, `assistantFallbackResults`. Those calls show that `answerAssistantQuestion` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

The application keeps this behavior isolated because other features rely on it being consistent. Without `answerAssistantQuestion`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

The local objects and variables inside `answerAssistantQuestion` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

cleanQuestion: `cleanQuestion` stores a computed value for temporary calculation. It is initialized from `String(question || "").trim()`. Within the function it is passed into `appendAssistantMessage`, `askRemoteAssistant`, `assistantContextForQuestion`, `assistantConversationForRequest`, `assistantFallbackResults`, `assistantLocalAnswer`, `assistantQuestionIntent`, `assistantRemember`; assigned or updated later in the function.

priorConversation: `priorConversation` stores a computed value for temporary calculation. It is initialized from `assistantChatHistory.slice()`. Within the function it is passed into `assistantConversationForRequest`, `assistantQuestionIntent`; assigned or updated later in the function.

pendingMessage: `pendingMessage` stores a computed value for temporary calculation. It is initialized from `appendAssistantPendingMessage()`. Within the function it is passed into `replaceAssistantMessage`; assigned or updated later in the function.

intent: `intent` stores a computed value for temporary calculation. It is initialized from `assistantQuestionIntent(cleanQuestion, priorConversation)`. Within the function it is passed into `askRemoteAssistant`, `assistantContextForQuestion`, `assistantFallbackResults`, `assistantLocalAnswer`, `assistantSourcesForDisplay`; assigned or updated later in the function.

rawSources: `rawSources` stores a computed value for temporary calculation. It is initialized from `assistantFallbackResults(cleanQuestion, intent)`. Within the function it is passed into `assistantContextForQuestion`, `assistantSourcesForDisplay`; assigned or updated later in the function.

sources: sources stores a computed value for temporary calculation. It is initialized from `assistantSourcesForDisplay(cleanQuestion, rawSources, intent)`. Within the function it is passed into `assistantLocalAnswer`, `replaceAssistantMessage`; assigned or updated later in the function.

context: context stores a computed value for temporary calculation. It is initialized from `assistantContextForQuestion(cleanQuestion, intent, rawSources)`. Within the function it is passed into `askRemoteAssistant`; assigned or updated later in the function.

conversation: conversation stores a computed value for temporary calculation. It is initialized from `assistantConversationForRequest(cleanQuestion, priorConversation)`. Within the function it is assigned or updated later in the function.

remoteAnswer: remoteAnswer stores the completed result of an awaited operation. It is initialized from `await askRemoteAssistant(cleanQuestion, context, {`. Within the function it is accesses properties/methods `answer`, `model`; passed into `assistantRemember`, `replaceAssistantMessage`, `setAssistantStatus`; assigned or updated later in the function.

fallbackAnswer: fallbackAnswer stores a computed value for temporary calculation. It is initialized from `assistantLocalAnswer(cleanQuestion, sources, intent)`. Within the function it is passed into `assistantRemember`, `replaceAssistantMessage`; assigned or updated later in the function.

errorAnswer: errorAnswer stores a string value for temporary calculation. It is initialized from `"I had trouble processing that question. Try rephrasing it with a project name, tool name, or electronics topic."`. Within the function it is passed into `assistantRemember`, `replaceAssistantMessage`; assigned or updated later in the function.

Curated File Guide: index.html

The public portfolio shell: metadata, semantic page structure, runtime hooks, visible sections, and script/style entry points.

Before The Code: File Overview

Abstract

index.html is the public website shell. It is the first file a recruiter or visitor receives from the static host. It defines the document metadata, search-engine hints, social-card metadata, favicon and app icons, structured data, visible page sections, semantic landmarks, and the fixed DOM ids that script.js later fills with real portfolio content. The file is intentionally light on final personal content because the builder writes most custom data into projects.json and script.js renders it dynamically.

The most important idea is that index.html is an interface contract. It tells the browser, search engines, social media previews, accessibility tools, styles.css, and script.js where things are and what each major region means. If script.js is the behavior layer, index.html is the skeleton that gives that behavior safe attachment points.

Introduction

The document begins with standard HTML setup: language, character set, viewport, description, robots instructions, author, and the AI endpoint metadata. These are not decorative tags. They affect how the page is indexed, how it scales on phones, how icons appear, and where the AI chat sends requests. After the metadata, index.html defines the visible page from top to bottom: header, hero, fun facts, builder download, summary metrics, projects, resume, dynamic sections, process, contact/AI, and footer.

Most of the actual portfolio content is not typed directly in this file. That is deliberate. The builder is supposed to let users change names, images, resumes, project categories, links, and rich text without editing HTML manually. Therefore index.html supplies stable placeholders such as #hero-title, #project-grid, #resume, #dynamic-sections, #contact, and #ask-ai. script.js finds those placeholders and fills them from the catalog.

Background Information

A static portfolio can be hosted cheaply and reliably if the page can be built from HTML, CSS, JavaScript, and JSON. index.html is designed for that model. It avoids server-only assumptions and includes assets with relative paths. It loads electronics-search.js before script.js because script.js can use the electronics search vocabulary when building project search terms.

The file also contains schema.org JSON-LD. That structured data gives search engines a machine-readable hint that the page is a ProfilePage about a Person. The builder later updates the visible content, but this baseline makes the page understandable even before dynamic content fully loads.

The page uses semantic sections and labels because public portfolio viewers include recruiters, mobile users, browser accessibility tools, and search crawlers. A good static shell gives all of them landmarks. For example, the projects section is labeled by projects-title, the resume section is hidden until a resume URL exists, and the AI assistant has a labelled panel so it can be reached from navigation and screen readers.

The file is intentionally not a finished personal biography. It is a reusable public shell. A fresh install should not hardcode Maurice-specific content, while Maurice's own built portfolio can still publish his profile through the catalog. That is why placeholder labels such as Portfolio Owner, Portfolio, and generic engineering copy exist here. The builder-generated catalog is the layer that specializes the site.

The HTML also controls load order. CSS loads before the page renders so layout and colors are available immediately. `electronics-search.js` loads before `script.js` because `script.js` may call the `electronics` keyword helper while building the search index. `script.js` loads at the end of the body so the DOM elements already exist by the time it queries them. If `script.js` loaded too early without deferring, many `document.querySelector` calls would return null.

Purpose Of The File

The purpose of `index.html` is to guarantee that the public portfolio has a stable starting structure. Without it, `script.js` would have nowhere reliable to render projects, no header to navigate from, no AI panel to attach events to, no resume viewer to populate, no contact band to update, and no metadata to help browsers or search engines understand the page.

It also separates permanent page structure from changeable portfolio content. The structure changes slowly; the content changes whenever the builder saves and publishes. That separation is what allows the builder to generate portfolio updates without rewriting the entire website architecture each time.

The file is also the public accessibility foundation. Section headings, labels, aria attributes, hidden states, object fallback text, and semantic main/header/footer landmarks make the site more understandable to assistive technology. `script.js` can only enhance that foundation; it cannot rescue a shell that has no stable names or landmarks.

Another purpose is failure tolerance. If `projects.json` fails to load, the page should still show a header, hero, search area, process section, and footer. If the resume is not configured, the resume section remains hidden. If the profile image is missing, the image elements stay hidden. This is what keeps a partially configured portfolio from looking broken.

Primary Responsibilities

The first responsibility is document identity. The head contains title, description, canonical URL, social tags, icons, AI endpoint, and structured data. These values determine how the page appears to crawlers, shared links, browser tabs, and mobile install surfaces.

The second responsibility is visible layout landmarks. The body defines the header, main content regions, sections, and footer. `styles.css` controls how they look, and `script.js` controls most dynamic filling.

The third responsibility is runtime hooks. Every id and class that `script.js` queries is part of a contract. Removing `#project-grid`, `#project-search`, `#ask-ai`, `#contact`, `#resume`, or `#dynamic-sections` would break the corresponding rendering logic.

The fourth responsibility is graceful fallback. The page contains placeholder text and hidden sections so the website has a sensible shape even before `projects.json` finishes loading. If a profile image or resume does not exist, those areas remain hidden instead of showing broken controls.

The fifth responsibility is hosting compatibility. The document uses ordinary relative paths and static assets. There is no server-side template language, no database call, and no build step required at request time. That is why the same page can be served from GitHub Pages, Cloudflare, or a local preview server.

The sixth responsibility is public trust. The page exposes recruiter-facing areas only: projects, resume, process, contact, download builder, and AI assistant. It does not include local builder controls, Git publishing buttons, compiler controls, or editing widgets. That separation protects the private builder workflow from public visitors.

Important Elements And Why They Matter

The header is the main navigation surface. Its brand link returns to the top, the nav links jump to AI, projects, resume, process, and contact, and the avatar link can take the visitor to contact information. The brand image and text are later customized by script.js.

The hero section is the first impression. It contains a hidden hero image, overlay shade, eyebrow, title, copy, project/resume buttons, GitHub button, and Ask AI jump link. These are all visible near the top because recruiter-facing content should not require deep scrolling.

The project section is the core portfolio directory. It includes a search input, filter controls, and the empty project grid that script.js fills. The search input is deliberately placed in the section heading because search is a primary navigation method, not an afterthought.

The contact band is last. It holds contact information and the AI assistant panel. The AI lives in this area because it is a helper for exploring the portfolio, while the contact section remains the final page region.

The hidden attributes are part of the contract. They do not mean the feature is dead; they mean script.js will reveal the feature only when the catalog contains the required data. This prevents empty resume viewers, empty avatars, and profile images without sources from appearing to visitors.

The IDs are also part of the contract. #project-grid, #project-search, #project-filters, #resume, #dynamic-sections, #contact, #ask-ai, #ai-assistant-form, #ai-assistant-input, and #ai-assistant-log are the anchors script.js expects. Renaming them would not only change HTML; it would break runtime behavior.

Code Walkthrough

This file is markup rather than a JavaScript function module, so the walkthrough is procedural. Each section explains a structural object in the document and why the public runtime depends on it.

Head metadata and search identity

The head makes the page understandable before JavaScript renders anything.

charset, viewport, description, robots, author, and canonical link

The charset tells the browser to interpret the file as UTF-8. The viewport makes the page scale correctly on phones. The description gives search engines a concise page summary. robots tells crawlers that indexing is allowed. author is a generic placeholder updated by published content or metadata later. The canonical link is empty in the template because the final domain can vary.

portfolio-ai-endpoint meta tag

This meta tag gives script.js a backend URL for the portfolio AI assistant. assistantEndpoint reads it before falling back to /api/portfolio-ai. Keeping the endpoint in metadata lets the same JavaScript bundle run locally and publicly while pointing to different AI backends.

Open Graph and Twitter card tags

These tags control how the portfolio looks when shared on social platforms or messaging apps. The defaults are generic because the builder can later fill real owner, title, image, and description values.

favicon and touch-icon links

These links tell browsers and mobile operating systems which icons to show in tabs, bookmarks, install prompts, and home-screen shortcuts. They point at the OMB asset set so the portfolio has a recognizable identity.

ProfilePage JSON-LD script

This structured data gives search engines a machine-readable summary of the page as a profile page about a person. The values are conservative placeholders. The important part is the shape: Person, name, image, URL, sameAs, and knowsAbout. It gives crawlers context even though the visible profile is populated dynamically.

Header and navigation contract

The header gives visitors immediate navigation and gives script.js stable places to insert brand/profile visuals.

header#top and brand link

The header has id top so Back to top links can target it. The brand anchor points to #top and contains the brand image, OMB engraving, title, and subtitle. script.js later replaces brand text and image from profile data. If these classes disappeared, renderProfile would lose its brand hooks.

site-nav links

The navigation links jump to Ask AI, Projects, Resume, Process, and Contact. These anchors match section ids later in the file. They are simple hash links so the page works on static hosting and does not need a router.

header-avatar

The avatar link starts hidden because not every portfolio has a profile image. renderProfile reveals it and sets the image when profile data provides one. Its href points to contact so the image acts as a fast route to contact information.

Hero, fun facts, download, and metrics

These are top-of-page sections that help a visitor orient quickly.

hero section

The hero contains a hidden image, shade overlay, eyebrow, h1 title, copy, action buttons, optional GitHub button, and Ask AI jump link. script.js updates the text and images. The section is labelled by hero-title, which improves accessibility and gives the document a clear main heading.

fun-facts-callout

This hidden section becomes visible only when the catalog includes fun facts or rich fun fact blocks. It sits near the top so personality appears before deep project scrolling.

builder-download-section

This section exposes the latest Windows installer link. It is public because visitors may want to clone/install the builder app. The href points to the stable latest release asset. The download attribute suggests download behavior, but the browser and GitHub ultimately decide how the file is handled.

summary-band metrics

The metrics display project count, category count, and artifact promise. script.js updates the first two after loading projects.json. The third is static because the builder portfolio consistently supports source, diagrams, and reports.

Projects, resume, dynamic sections, process, contact, and AI

These sections are the public portfolio body. Most content is populated dynamically.

projects section

This is the main project directory. It has a heading, search input, filter bar, and empty project-grid. script.js builds filter buttons, project categories, project cards, parsed section buttons, search results, and nested section windows from this area.

resume section

The resume section starts hidden because a fresh portfolio may not have a resume. renderProfile reveals it, sets the PDF object data, and updates Open/Download links when profile.resumeUrl exists. The object element gives an in-page PDF viewer when the browser supports it.

dynamic-sections container

This div is where user-created main portfolio sections render. The builder can add personal life, professional profile, social links, hobbies, or other lighter sections without editing index.html.

process section

The process section is a stable explanatory band describing problem/constraints, build artifacts, and results/reflection. It gives the page a professional engineering narrative even before specific custom sections are added.

contact-band and ask-ai panel

The contact band is intentionally last. It holds profile photo, contact details, external links, and the AI assistant. The AI form, log, input, status, and clear button have fixed ids because script.js attaches the chatbot behavior directly to them.

footer and script loading

The footer provides Back to top and copyright year. The scripts load electronics-search.js before script.js so the search vocabulary is available when script.js builds the index. Version query strings help browsers fetch updated CSS and JS after publishing.